claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-aa57b56e6af7c44ed.jsonl`
- SHA-256: `cdff76a23b0c4384c0854591072523dd8d785b2526e6beed89e52778f42a2a37`
- Source modified: `2026-06-21T14:12:00+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T14:08:20.708Z)

```
You are the architect. Using ALL findings below, produce a DETAILED, opinionated plan to ship
the badminton highlight indexer as a downloadable "batteries-included" Python app that spins up
a local web service for inference + training + (optional) rally annotation, runnable locally AND
against cloud services, WITHOUT the user sourcing the repo.

HARD CONSTRAINTS: respect the owner's locked decision D10 (self-hosted webserver the user spins
up, NOT a desktop app) -- but you MAY recommend a friendly launcher if it serves D10's intent.
Build ON the already-merged decoupled-compute seam (profiles/storage/compute backends), do NOT
reinvent it. Respect the MIT/Apache/BSD-only licensing guardrail for ALL bundled deps
(code/data/weights) and the no-paid-LLM-training-labels ship-gate. Preserve every completed CUJ.
Solve the dual-torch-env + ffmpeg + weights + SPA-in-other-repo problems concretely. Prefer
small, isolated, default-OFF, reversible increments (the project's working style). Distinguish
Claude-doable work from owner-gated work.

Return the SYNTH schema. Be concrete: name files/dirs/commands where possible.

=== UNDERSTANDING ===
[
  {
    "area": "App launch + current packaging surface (how a user goes from nothing to a running
server+worker; what `pip install .` provides; entrypoints; config/DB/output resolution; server-
vs-worker process model)",
    "summary": "Today the engine is NOT a downloadable batteries-included app ? it is a source
checkout you `pip install -e .` from. The PEP 621 build (hatchling, `pyproject.toml:12-93`)
installs the `backend` + `training` namespace packages and registers ONE console script,
`indexer = backend.main:main` (`pyproject.toml:80-82`), but deliberately omits torch/torchvision
(their CUDA/CPU wheels need `--index-url`, which PEP 621 can't express), so `pip install .`
alone yields a torch-less install that cannot run the GPU/yolo_hybrid/trajectory paths.
ffmpeg/ffprobe are unmanaged system deps. The intended cold-start path is: (1) install ffmpeg,
(2) `pip install -e .` (or `.[dev]`), (3) install torch out-of-band, (4) run `scripts/doctor.py`
/ `indexer --setup` to GPU-probe + write a default `config.json` + `pip install -r
requirements.txt` --user, (5) set `GEMINI_API_KEY` in a `.env`, (6) launch. The launcher
resolves config.json, .env, the SQLite DB, and output/ all CWD-RELATIVE ? there is no app-data
dir, no XDG/AppData home, no env override for their location ? so the running directory IS the
implicit state root. Critically, server and worker are NOT separate processes for the normal
serving path: the FastAPI server runs an IN-PROCESS ThreadPoolExecutor job worker
(`backend.api.job_worker`), so `python -m backend.main --server` is fully self-contained.
`python -m backend.worker {infer|train}` is a SEPARATE, storage-aware (pull?run?push)
orchestrator built for the decoupled-compute / Cloud Run path, not a daemon the server talks to.
`main()` is overloaded: `--server`, single-shot `--video-path`, `--setup`, `--debug-clip`,
`--trim-*` all dispatch from the one entrypoint, defaulting to server mode when no video path is
given (`backend/main.py:557-736`).",
    "key_facts": [
      {
```

```
        "fact": "The ONLY console script is `indexer = backend.main:main`; there is no separate
`indexer-worker` / `indexer-server` entrypoint. The worker is reached only via `python -m
backend.worker`.",
        "evidence": "pyproject.toml:80-82; backend/worker/__main__.py:516-517"
      },
      {
        "fact": "`pip install .` installs fastapi, uvicorn, opencv-python, numpy, pydantic,
jinja2, python-dotenv, google-genai (HARD dep), supervision ? but NOT torch/torchvision (must be
installed out-of-band with --index-url). So a bare wheel install cannot run
yolo_hybrid/wasb_hybrid/trajectory/GPU paths.",
        "evidence": "pyproject.toml:8-37,63-70; requirements.txt:1-11 (requirements.txt DOES
list torch but is only used by doctor.py, not by pip install .)"
      },
      {
        "fact": "Server and worker are the SAME process for serving: /api/process creates a
'queued' DB job and submits an in-process ThreadPoolExecutor drain
(`_get_executor().submit(_drain_due_jobs)`); the FastAPI app re-exports the job_worker
drain/wake loop. No external worker daemon is required to serve.",
        "evidence": "backend/main.py:80-97,275-315,729-736"
      },
      {
        "fact": "`python -m backend.worker {infer|train}` is a standalone storage-aware
orchestrator (pull URI -> validate/segment -> push outputs/<video_id>/...), composing the same
registries as the CLI. It is the decoupled-compute / Cloud Run path, dispatched remotely when
compute_target is cloud (M6), NOT a process the --server talks to.",
        "evidence": "backend/worker/__main__.py:1-15,156-259; cmd_infer line 166-168 dispatches
remote when get_compute_backend!=None"
      },
      {
        "fact": "config.json is resolved CWD-relative: DEFAULT_CONFIG_PATH =
Path('config.json'); a missing file silently degrades to all-defaults. No env var / app-home
override for its location.",
        "evidence": "backend/config/loader.py:30,102-122"
      },
      {
        "fact": "The SQLite DB path is output_dir-relative: db_path =
os.path.join(global_config.output.output_dir, db_name); output_dir defaults to the literal
'output' (CWD-relative), overridable only by the CLI --output-dir flag.",
        "evidence": "backend/main.py:636-640; backend/config/models.py:230-235 (OutputConfig
output_dir='output', db_name='sports_indexer.db')"
      },
      {
        "fact": ".env is loaded CWD-relative at import (`load_dotenv()` with no path).
GEMINI_API_KEY is read straight from os.environ by the gemini/yolo_hybrid/trajectory segmenters;
absent key => silent 0-segment run flagged in the report.",
        "evidence": "backend/main.py:8-10; backend/pipeline/segmenters/gemini.py:90,
yolo_hybrid.py:80, trajectory_hybrid.py:169; README.md:103"
      },
      {
        "fact": "The server mounts and serves a zero-build static UI from the CWD-relative path
'backend/api/static' (os.makedirs at import), serving index.html/style.css/app.js. This is the
legacy in-engine dashboard ? NOT the khelsutra Vite/React SPA ? and the mount path is hardcoded
relative to CWD.",
        "evidence": "backend/main.py:64-74; backend/api/static/{index.html,style.css,app.js}
exist"
      },
      {
        "fact": "`indexer --setup` shells out to scripts/doctor.py (path resolved from __file__,
cwd-independent), which GPU-probes via nvidia-smi, writes a default config.json via
save_default_config(), and runs `pip install --user -r requirements.txt` with the matching torch
--extra-index-url (cu124 if GPU else cpu; macOS uses default wheels).",
        "evidence": "backend/main.py:575-587; scripts/doctor.py:15-42,44-66,129-133"
      },
      {
        "fact": "requires-python >=3.10; build backend hatchling; backend is an intentional
```

```
namespace package (no backend/__init__.py) so wheel packages are listed explicitly as
['backend','training'].",
        "evidence": "pyproject.toml:3-6,21,89-93"
      },
      {
        "fact": "Crash recovery on server/CLI startup sweeps jobs left 'queued'/'running' by a
prior process to 'failed' (the executor is in-process, so they're dead) ? confirms the single-
process serving model.",
        "evidence": "backend/main.py:642-646; comment 'the executor is in-process'"
      },
      {
        "fact": "run_all_tests.py is a thin wrapper over `python -m pytest` (defaults to -q),
fully mocked/offline, with obsreport reporting tests skipping if the optional dep is absent.",
        "evidence": "run_all_tests.py:36-55"
      },
      {
        "fact": "config save (save_default_config) and config.json discovery both anchor on
Path('config.json') relative to CWD; doctor.py instead writes to REPO_ROOT/config.json
(os.path.dirname x2 of __file__), so doctor and a launched server can disagree on which
config.json is authoritative if launched from a different CWD.",
        "evidence": "backend/config/loader.py:196-206; scripts/doctor.py:9,57"
      }
    ],
    "packaging_implications": [
      "A batteries-included app cannot ship as a plain wheel: torch/torchvision are out-of-band
by design (pyproject.toml:8-11,63-70). The packaging layer must provide a post-install/first-run
torch bootstrap (the logic already exists in scripts/doctor.py:15-42 ? GPU-probe -> pick
cu124/cpu index) or ship a platform-pinned bundle. This is THE first hard problem and is
independent of the WASB conda-env split.",
      "ffmpeg/ffprobe are unmanaged system deps checked only by doctor.py
(scripts/doctor.py:84-109). A downloadable app must bundle static ffmpeg/ffprobe binaries
(license-clean: ffmpeg LGPL/GPL ? verify which build) or vendor a pip-installable static-ffmpeg,
and point the engine at them, because today everything assumes they are on PATH.",
      "ALL mutable state (config.json, .env, SQLite DB, output/, the static UI mount) is CWD-
relative with NO app-home/XDG/AppData concept (backend/config/loader.py:30;
backend/main.py:64-66,636-640). A packaged app run from an arbitrary launch dir (e.g. an OS app
menu) would scatter or fail to find state. Packaging must introduce a resolved app-data root
(config + DB + output + cache) and thread it through OutputConfig.output_dir,
DEFAULT_CONFIG_PATH, load_dotenv path, and the static mount.",
      "The single console entrypoint `indexer` already gives a clean app launch surface
(pyproject.toml:80-82). A packaged launcher can default to `indexer --server` and rely on the
in-process job worker (backend/main.py:729-736) ? NO separate worker process needs to be
supervised for local serving. The separate `python -m backend.worker` is only needed for the
cloud/decoupled-compute path, so a truly-local bundle can omit worker supervision entirely.",
      "doctor.py uses `pip install --user -r requirements.txt` at runtime (scripts/doctor.py:23)
? a from-source assumption. A no-clone downloadable app must NOT shell out to pip against a
requirements.txt that won't exist in a frozen/bundled distribution; the dependency install must
move into the package build/first-run, and requirements.txt's torch lines reconciled with
pyproject's deliberate omission.",
      "The hardcoded 'backend/api/static' mount (backend/main.py:64-66) and the engine's own
legacy dashboard exist independently of the khelsutra Vite/React SPA. Bundling the real UI means
either (a) building khelsutra/web and copying dist into a package-data dir resolved via
importlib.resources (not CWD), or (b) keeping the legacy static UI. Either way the CWD-relative
mount must become package-relative.",
      "google-genai is a HARD dependency (pyproject.toml:35) even though local-only profiles
(truly-local preset sets skip_ai_handoff=true, backend/config/presets.py:44-50) never call it.
For a license-clean, cloud-optional download this is fine to keep but means the binary always
carries the Gemini SDK; confirm google-genai license is Apache-2.0/MIT/BSD per the guardrail.",
      "A deployment.profile=truly-local preset already bundles the zero-cloud knobs
(skip_ai_handoff + local storage + local_gpu) ? the packaged 'local' app can default to this
profile via DEPLOYMENT_PROFILE env or a shipped config.json, giving a coherent offline default
without new code (backend/config/presets.py:43-63; loader resolves env-over-file at
loader.py:56-71)."
    ],
    "risks": [
```

```
      "CWD-dependence is a latent footgun: launching the packaged app from the wrong directory silently creates a fresh empty config.json/DB/output rather than erroring (loader.py:115-122 degrades to defaults; main.py:637 makedirs output). Users could lose/duplicate their index across launches.",
      "doctor.py writes config.json to REPO_ROOT (scripts/doctor.py:57) while the running server reads Path('config.json') from CWD (loader.py:30) ? if the packaged launcher's CWD != the doctor's REPO_ROOT, `indexer --setup` and the server disagree on the authoritative config. This split must be unified before packaging.",
      "requirements.txt and pyproject.toml have already drifted (requirements.txt:9-10 lists torch>=2.12.0/torchvision; pyproject deliberately omits them). doctor.py installs from requirements.txt, pip install . from pyproject ? two dependency sources of truth that a packaging pass must reconcile or one will silently win.",
      "The WASB native shuttle model needs a SEPARATE conda Python env (torch 1.11+cu113) vs the engine's torch 2.6+cu124, and weights live in GCS (per orchestrator notes; config defaults at backend/config/models.py:61-65 point at ~/models/WASB-SBDT and a WSL conda env). This dual-env + remote-weights reality is fundamentally incompatible with a single self-contained Python bundle for the wasb_hybrid path ? a packaging plan must either gate wasb behind an optional installer step or fall back to motion/yolo_hybrid for the batteries-included default.",
      "The in-process job executor means a server crash loses all queued/running jobs (swept to 'failed' on next start, main.py:642-646). For a desktop-grade local app users may expect jobs to survive a restart; this is acceptable today but worth flagging as a UX expectation gap."
    ],
    "open_questions": [
      "Should the packaged app standardize an app-data home (e.g. %APPDATA%/Khelsutra or ~/.khelsutra) and migrate config.json/.env/DB/output/cache there, and does any existing doc (khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md) already specify this? I did not read that doc in this pass.",
      "Is the batteries-included default meant to be the `truly-local` profile (skip_ai_handoff, motion/yolo_hybrid, no Gemini key required) or must the bundle ship Gemini-capable out of the box (requiring the user to provide GEMINI_API_KEY)? The README still ships config.json defaulting default_segmenter to gemini (README.md:103,171), which contradicts a zero-config local-first download.",
      "How should torch be delivered in a no-clone download ? bundled platform-pinned wheels, a guided first-run installer reusing doctor.py's GPU-probe logic, or a CPU-only baseline with optional GPU upgrade? This drives whether one universal artifact or per-platform artifacts are shipped.",
      "Should `python -m backend.worker` be exposed as a second console script (e.g. `indexer-worker`) for the cloud-serving packaging path, or remain module-invocation-only since local serving needs no separate worker?",
      "Which UI ships in the local bundle ? the legacy in-engine backend/api/static dashboard or a built khelsutra/web SPA copied into package data ? and who owns the cross-repo build step?",
      "Is the ffmpeg/ffprobe binary to be bundled (and which license-clean static build) or left as a documented prerequisite with a guided installer, given the project's MIT/Apache-2.0/BSD-only guardrail (ffmpeg is LGPL/GPL)?"
    ]
  },
  {
    "area": "Config system + deployment profiles + the decoupled-compute seam (backend/config/: models.py, presets.py, startup.py, loader.py; the compute seam in backend/compute/)",
    "summary": "The engine has a single typed Pydantic config (backend/config/models.py::AppConfig) loaded by backend/config/loader.py::load_config with strict precedence: built-in model defaults < profile preset < config.json < env (DEPLOYMENT_PROFILE, RALLY_DETECTOR_IMPL, WASB_*) < worker `--set k=v`. A `deployment` section (profile + compute_target) is the ONE switch that selects a coherent runtime: choosing a profile applies a preset BUNDLE (presets.py::PROFILE_PRESETS) over existing knobs ? truly-local sets compute_target=local_gpu / skip_ai_handoff=True / storage.backend=local; cloud-serving sets compute_target=cloud_run / detector_impl=native / skip_ai_handoff=False / storage.backend=gcs; cpu-mock sets compute_target=cpu_mock / detector_impl=stub / storage.backend=local. The system is deliberately DEFAULT-OFF: with profile=\"\" the loader is byte-identical to pre-M1 behaviour (the shallow {**defaults, **file_data} merge), and the per-profile startup checks (startup.py) are a strict no-op. The ONLY thing compute_target actually changes at runtime today is the compute seam: backend/compute/__init__.py::get_compute_backend returns a CloudRunCompute dispatcher ONLY for compute_target==\"cloud_run\", else None which is the unchanged in-process path. So a SINGLE build runs both local and cloud purely by config: the same `python -m backend.worker infer` / `backend.main --server` binary; the profile decides whether the job runs
```

```
in-process on a local GPU or is dispatched to a Cloud Run Job. Auto-detect only SUGGESTS a
profile (D15), never silently selecting cloud (= spend). Startup checks fail-fast (hard ERROR)
only on the one certain incoherence (cloud-serving with a non-cloud storage backend); everything
else (GPU/creds mismatch) is a WARNING because the detector subprocess healthcheck is the real
authority.",
    "key_facts": [
      {
        "fact": "Config precedence is explicit: built-in defaults < profile preset < config.json
< env < worker --set. The loader deep-merges preset+config.json only on the profile path; the
no-profile path uses the original shallow {**defaults, **file_data} merge, byte-identical to
pre-M1.",
        "evidence": "backend/config/loader.py:143-193 (esp. 160-184);
backend/config/presets.py:7-9"
      },
      {
        "fact": "deployment.profile in {empty, truly-local, cloud-serving, cpu-mock} and
compute_target in {empty, local_gpu, cloud_run, cpu_mock} are typed Literals on
DeploymentConfig; profile empty applies NO preset (today's manual knobs).
COMPUTE_TARGET_FOR_PROFILE pins the 1:1 mapping.",
        "evidence": "backend/config/models.py:348-365; backend/config/presets.py:27-36"
      },
      {
        "fact": "Each profile is a preset BUNDLE over EXISTING knobs only: truly-
local={skip_ai_handoff:True, storage.backend:local, detector_impl stays auto}; cloud-
serving={detector_impl:native, skip_ai_handoff:False, storage.backend:gcs}; cpu-
mock={detector_impl:stub, skip_ai_handoff:True, storage.backend:local}.",
        "evidence": "backend/config/presets.py:43-63"
      },
      {
        "fact": "compute_target is the only deployment field consumed at runtime today:
get_compute_backend returns a CloudRunCompute ONLY for compute_target==cloud_run (else None ->
in-process). cmd_infer dispatches remotely only when compute is not None; the dispatched
container reruns the SAME cmd_infer inline with compute_target forced local_gpu (no
recursion).",
        "evidence": "backend/compute/__init__.py:32-61; backend/worker/__main__.py:156-168,
116-137"
      },
      {
        "fact": "A single build runs local OR cloud by config alone: the worker
(cmd_infer/cmd_train) is profile-agnostic, reads config.storage, accepts --config/--set; no per-
profile branch in the pure DETECT/WINDOW/STITCH core (design D1).",
        "evidence": "backend/worker/__main__.py:156-169, 58-68;
docs/COMPUTE_DECOUPLED_SERVING/README.md:28, 88"
      },
      {
        "fact": "Auto-detect SUGGESTS only (D15), never auto-applies. suggest_profile maps
K_SERVICE->cloud-serving, CI->cpu-mock, probe_cuda->truly-local, else empty; the loader logs the
suggestion at DEBUG but applies no preset unless DEPLOYMENT_PROFILE env or config.json
deployment.profile is explicitly set.",
        "evidence": "backend/config/presets.py:109-132; backend/config/loader.py:47-71, 182-191"
      },
      {
        "fact": "Per-profile startup checks are default-OFF (no profile -> empty list) with
exactly ONE fatal error: cloud-serving + non-cloud storage backend (STORAGE_INCOHERENT).
GPU/creds mismatches are WARNINGS (torch may live only in the detector subprocess env). Wired
into the server (exit 1) and worker (rc 2).",
        "evidence": "backend/config/startup.py:69-156; backend/main.py:552-559, 639;
backend/worker/__main__.py:97-113"
      },
      {
        "fact": "Env overrides config for the profile choice and detector knobs:
DEPLOYMENT_PROFILE (env wins over config.json; unknown env value warns + falls through to file),
RALLY_DETECTOR_IMPL, WASB_DEVICE / WASB_PYTHON / WASB_WEIGHTS / WASB_REPO_DIR / WASB_SCRATCH.",
        "evidence": "backend/config/loader.py:47-71;
backend/pipeline/segmenters/trajectory_hybrid.py:77;
```

```
backend/pipeline/detectors/native_wasb_runner.py:83-92"
      },
      {
        "fact": "AppConfig has model_config extra=allow + validate_assignment=True and a legacy
.get()/__getitem__ dict-shim; unknown config.json keys are detected + warned (top-level extras
kept-but-unread; unknown section keys silently dropped) via _unknown_key_paths.",
        "evidence": "backend/config/models.py:411-447; backend/config/loader.py:74-91, 133-141"
      },
      {
        "fact": "save_default_config() writes a fully-defaulted config.json (used by
scripts/doctor.py + a --setup path), the natural seed for a packaged app's first-run config.",
        "evidence": "backend/config/loader.py:196-206"
      },
      {
        "fact": "The Cloud Run dispatcher reads coordinates (CLOUD_RUN_JOB / CLOUD_RUN_REGION /
GOOGLE_CLOUD_PROJECT) from env, NOT config; no secrets/endpoints live in config; StorageConfig
carries only endpoints/regions/paths, never credentials.",
        "evidence": "backend/compute/__init__.py:42-60; backend/config/models.py:311-345"
      }
    ],
    "packaging_implications": [
      "ONE build is genuinely enough for local AND cloud: a packaged app ships a single
binary/entrypoint and the user flips between truly-local and cloud-serving by setting
deployment.profile (config.json) or DEPLOYMENT_PROFILE (env). This realizes D13 'switch anytime'
with zero rebuild -- the packager should surface a profile toggle, not separate downloads.",
      "First-run UX seam already exists: save_default_config() writes a defaulted config.json. A
packaged-app onboarding should call this (or a --setup path) to materialize config.json, then
write deployment.profile after a guided choice. suggest_profile gives a SUGGESTED default to
pre-select, but D15 requires the user to confirm; the wizard must show 'we detected a GPU ->
truly-local; confirm or change', never silently pick cloud.",
      "Expose to a NON-TECH user (small, safe set): (1) profile as 3 plain-language cards (This
machine, offline & private / Our cloud / Test mode); (2) input source = local folder vs Google
Drive vs bucket (storage.input_backend/input_root); (3) output folder (output.output_dir); (4)
Gemini API key (via env) only when a profile needs the handoff; (5) watermark text on/off
(stitching.watermark). Everything else preset-driven.",
      "HIDE from a non-tech user (expert knobs): detector_impl, skip_ai_handoff, all indexing.*
windowing tuning (max_window_duration, R4/R5, velocity thresholds), wasb.* WSL/conda/device
internals, chunk_duration/overlap, ai_* handoff knobs, billing.*, security.edge_auth/cf_* and
Cloud Run env coordinates. The profile preset sets these coherently; ship a raw-config.json
'advanced' escape hatch outside the default flow.",
      "The startup-check layer is a ready-made packaged-app health gate: on launch run
enforce_startup_checks and surface results -- a hard error (cloud-serving without a reachable
bucket) blocks the job with a clear message; warnings (no GPU visible, creds unconfirmed) become
non-blocking banners. Exactly the 'loud before any job' UX a non-tech user needs.",
      "compute_target==cloud_run is the ONLY runtime divergence today and needs
CLOUD_RUN_JOB/REGION/GOOGLE_CLOUD_PROJECT env + GCS creds the engine does NOT manage. A
'batteries-included' cloud profile in a downloadable app is therefore NOT self-contained: it
requires the user's own GCP project + a deployed Cloud Run worker image + ADC. Truly-local is
the only fully self-contained path; treat cloud-serving as 'connect to a hosted service / your
own GCP', not 'works out of the box'.",
      "cpu-mock is a gift for the installer/smoke test: runs the whole DETECT->WINDOW->STITCH
pipeline with the stub detector (no GPU/WSL, no API key), byte-deterministic. A packaged app can
run a cpu-mock self-test on first launch to prove the install works before the user provisions a
GPU or Gemini key.",
      "No credentials live in config (by design): the packaged app must source the Gemini key,
GCS ADC, and Cloud Run coordinates from env/OS credential stores, not the bundled config.json.
The packager needs a credential-entry UX that writes env, not config fields."
    ],
    "risks": [
      "Biggest mismatch with 'batteries-included downloadable app': truly-local still depends on
heavy, non-bundled externals -- torch (deliberately not a dependency; needs --index-url CUDA
wheels), the SEPARATE WASB conda env (torch 1.11+cu113) reached via WSL on Windows, system
ffmpeg/ffprobe, and WASB weights pinned to a ~/models path / GCS bucket (wasb.weights_path
default ~/models/WASB-SBDT/...). The config selects these but does not provision them; packaging
must solve weight download + ffmpeg bundling + a torch install step the config layer doesn't
```

cover.",
      "cloud-serving is not runnable from a downloaded app without the user's own GCP project, a built+pushed Cloud Run worker image, a GCS bucket, and ADC -- none provided by config or app (M7 still owner-gated/unbuilt). Marketing 'your machine or our cloud, switch anytime' (D13) overstates what a downloaded build delivers for the cloud half today.",
      "truly-local's preset sets skip_ai_handoff=True (zero-cloud), so by default it emits the LOCAL detector windows AS rallies with no Gemini refinement -- quality is the raw local detector. A user toggling to truly-local for privacy silently gets lower-quality reels unless the UX explains the trade-off (the owned model isn't at the D11 floor yet).",
      "The legacy AppConfig.get()/__getitem__ shim + extra=allow tolerate typo'd keys (top-level extras kept-but-unread, section typos silently dropped, only a log WARNING). A hand-edited config.json from a non-tech user can silently no-op a setting; drive config through validated UI writes, not free-text editing, or surface the _unknown_key_paths warnings prominently.",
      "Windows reality: truly-local detection routes through WSL (TrackNetConfig/WasbIndexConfig default wsl_distro=Ubuntu, conda envs, ~/clips). A downloadable Windows app cannot assume WSL2 + conda + a CUDA GPU; the native runner is Linux-only. The config models encode a developer's WSL setup as defaults -- wrong for an end-user install, needing profile-time overriding or a different local-compute story.",
      "Env-over-config precedence (DEPLOYMENT_PROFILE, RALLY_DETECTOR_IMPL, WASB_*) means a stray env var on the user's machine can silently override the app's chosen profile/detector -- a support footgun unless the launcher clears/controls the relevant env."
    ],
    "open_questions": [
      "For a downloadable app, does 'cloud-serving' mean (a) connect to a hosted Khelsutra service, or (b) the user brings their own GCP project? The config/compute seam only supports (b) today (CLOUD_RUN_* env + ADC); (a) needs a different StorageBackend/ComputeBackend (signed-URL/API client) that doesn't exist yet.",
      "How are WASB weights + the WASB conda/torch env delivered in a packaged truly-local build? wasb.weights_path defaults to a ~/models path and weights live in a GCS bucket (WASB-C3); there is no in-app downloader/provisioner wired to config today.",
      "Should the packaged app expose a 4th 'just works' profile pinning detector_impl for a no-WSL / CPU-or-bundled-GPU consumer machine? The current truly-local preset assumes a developer GPU box (WSL/native/auto), not a typical end-user environment.",
      "Where does the Gemini API key entry live in the packaged UX, and is the cloud handoff allowed in truly-local? Today truly-local forces skip_ai_handoff=True (no key) but a user may want local-compute + cloud-Gemini-refine -- a knob combination no preset expresses.",
      "Is there a first-run wizard the owner intends to build that calls save_default_config + suggest_profile + enforce_startup_checks, or does the SPA (khelsutra/web) own all onboarding? The engine has the primitives but no assembled flow.",
      "Should config.json validation surface unknown-key warnings to the end user (not just logs)? For a non-tech user a silently-dropped section key is invisible today."
    ]
  },
  {
    "area": "Storage + Compute backend registries (the local-OR-cloud axis)",
    "summary": "Storage and compute are two URI/config-driven seams that let the SAME pipeline read/write either local files or cloud sources, default-OFF (today's behaviour byte-for-byte). STORAGE: a `StorageBackend` ABC (backend/storage/base.py) with four lazy-imported implementations (local/s3/gcs/gdrive) selected by the URI scheme via `ref.parse_uri` ? a bare path or `local://` -> local (identity passthrough, no copy), `s3://` -> S3Storage (also Cloudflare R2 / DO Spaces / MinIO via endpoint_url), `gs://|gcs://` -> GCSStorage, `gdrive://` -> a path under a locally-MOUNTED Google Drive (plain local FS, no API). The API/CLI/worker boundary calls thin helpers `acquire_local_input` / `resolve_input_ref` / `fetch` / `put` / `list_uris`; input resolution honours `storage.input_backend` + `storage.input_root` so a bare name can be joined under a bucket root, while an explicit scheme or absolute path always wins. COMPUTE: a `ComputeBackend` ABC (backend/compute/base.py) with a factory `get_compute_backend(config)` that returns a `CloudRunCompute` ONLY when `deployment.compute_target == \"cloud_run\"` and `None` otherwise (= run the worker in-process, today's path). CloudRunCompute dispatches a Cloud Run GPU **Job** via an injected `CloudRunJobClient`, then bucket-polls a tiny done-marker object (NOT the Cloud Run API status) until the worker writes it, then reads `report.json`. Credentials are NEVER stored in config ? only endpoints/regions/paths/source-selectors; boto3 / google-auth / ADC supply secrets from their own chains and env. The base install stays light because every cloud SDK (boto3, google-cloud-storage, google-cloud-run) is lazy-imported, and only the two storage SDKs are declared as optional extras.",

```
    "key_facts": [
      {
        "fact": "Storage backend is selected purely by the URI scheme: bare path / `local://` ->
local, `s3://` -> s3, `gcs://` (or `gs://`, normalized) -> gcs, `gdrive://` -> gdrive. parse_uri
treats any string with no `scheme://` (incl. Windows `C:\\...`) as a local path.",
        "evidence": "backend/storage/ref.py:48-68"
      },
      {
        "fact": "`_backend_class(name)` lazy-resolves the implementation class by name
(local/s3/gcs/gdrive), importing the module only when that backend is actually constructed ? so
`import backend.storage` never pulls boto3 / google-cloud-storage.",
        "evidence": "backend/storage/__init__.py:77-103"
      },
      {
        "fact": "Input resolution honours `storage.input_backend` + `storage.input_root`: an
explicit scheme or absolute local path wins; a bare/relative name is joined under input_root
(gs://bucket/videos + x.mp4) or prefixed by input_backend; default local/'' is identity
passthrough (today's behaviour).",
        "evidence": "backend/storage/ref.py:71-98"
      },
      {
        "fact": "`acquire_local_input(raw, storage_config)` returns local/bare paths UNCHANGED
(no copy) and downloads a bucket/Drive source into a fresh tempfile.mkdtemp scratch dir,
returning the local path. This is the single helper the API uses to make any source local.",
        "evidence": "backend/storage/__init__.py:59-74"
      },
      {
        "fact": "The FastAPI process path calls `storage.acquire_local_input(req.video_path,
global_config.storage)` to get a local file, and the submit-time validator calls
`storage.resolve_input_ref(...)` mapping an unsupported scheme to a clean 400 (never a 500); a
local input that doesn't exist 404s, while a bucket/Drive URI's existence is verified at worker
fetch time.",
        "evidence": "backend/main.py:174, backend/main.py:297-307"
      },
      {
        "fact": "S3Storage credentials come ONLY from boto3's default chain
(AWS_ACCESS_KEY_ID/SECRET/SESSION_TOKEN env, instance role, ~/.aws); config supplies only
endpoint_url (or AWS_ENDPOINT_URL_S3 env) / region / addressing_style / signed_url_ttl. boto3 +
botocore.config are imported inside __init__.",
        "evidence": "backend/storage/s3.py:21-36"
      },
      {
        "fact": "GCSStorage credentials come from Application Default Credentials
(GOOGLE_APPLICATION_CREDENTIALS service-account JSON, or workload identity); config supplies
only project / signed_url_ttl. `from google.cloud import storage` is imported inside __init__
only when no client is injected.",
        "evidence": "backend/storage/gcs.py:17-25"
      },
      {
        "fact": "GDriveStorage needs NO SDK or OAuth ? it reads a locally MOUNTED Google Drive
for Desktop as plain files. mount_root is auto-detected (Windows `G:\\My Drive`, macOS
`~/Library/CloudStorage/GoogleDrive-*/My Drive`) or set via GDRIVE_MOUNT_ROOT /
storage.gdrive.mount_root; a missing mount/file raises with an actionable remedy message.",
        "evidence": "backend/storage/gdrive.py:84-105, backend/storage/gdrive.py:67-81"
      },
      {
        "fact": "`get_compute_backend(config)` is the default-OFF seam: returns a
CloudRunCompute ONLY for compute_target=='cloud_run', else None ('run inline', today's path).
Cloud Run coordinates come from CLOUD_RUN_JOB / CLOUD_RUN_REGION / GOOGLE_CLOUD_PROJECT env,
never config; run_client is injectable for tests.",
        "evidence": "backend/compute/__init__.py:32-61"
      },
      {
        "fact": "CloudRunCompute.run_infer dispatches a Cloud Run Job (`python -m backend.worker
infer --video-uri --out-uri --done-uri ...`), forces the container INLINE via overrides
```

```
(deployment.compute_target=local_gpu, indexing.detector_impl=native) so it never re-dispatches,
then `poll_until` reads a storage-backed done-marker (video_id + returncode) and fetches
outputs/<video_id>/report.json.",
        "evidence": "backend/compute/cloud_run.py:103-132, backend/compute/cloud_run.py:39-44"
      },
      {
        "fact": "Completion is detected by polling a marker OBJECT in the output bucket
(`<out>/_dispatch/<token>.done`) written by the worker, not the Cloud Run API status (D16). A
missing/empty/unreadable marker defaults returncode to FAILURE; the worker writes the marker
only on success today (failure-marker is a follow-up), so a hung container yields rc=4 via
PolicyTimeout.",
        "evidence": "backend/compute/cloud_run.py:104-140, backend/worker/__main__.py:116-137,
backend/worker/__main__.py:253-258"
      },
      {
        "fact": "The real Cloud Run client (`GoogleCloudRunJobClient`) lazy-imports
`google.cloud.run_v2` inside run_job and raises a clear RuntimeError if absent ? google-cloud-
run is intentionally NOT a test/CI dependency and is installed out-of-band on the control
plane.",
        "evidence": "backend/compute/cloud_run.py:60-79"
      },
      {
        "fact": "Profile presets wire storage.backend per profile: truly-local -> local, cloud-
serving -> gcs (+ compute_target cloud_run), cpu-mock -> local. input_backend / workspace base
are deliberately NOT preset (deferred to later, Launch-Report-gated milestones).",
        "evidence": "backend/config/presets.py:43-63, backend/config/models.py:329-341"
      },
      {
        "fact": "Only two cloud SDK extras are declared: storage-s3=[boto3>=1.34], storage-
gcs=[google-cloud-storage>=2.16]. google-cloud-run is in NO extra. gdrive needs no extra. The
empty `gpu` extra is docs-only (torch needs --index-url, unexpressible in PEP 621).",
        "evidence": "pyproject.toml:72-78, pyproject.toml:63-70"
      },
      {
        "fact": "StorageConfig carries NO secrets: backend, output_root,
input_backend/input_root, single_folder_per_video/workspace_root/workspace_prefix, and loose
per-backend dicts (local/s3/gcs/gdrive) for endpoints/regions/paths only. The worker passes
config.storage.model_dump() to every storage call.",
        "evidence": "backend/config/models.py:311-345, backend/worker/__main__.py:170,
backend/worker/__main__.py:348"
      }
    ],
    "packaging_implications": [
      "Base download stays light by design: a packaged app can ship WITHOUT boto3 / google-
cloud-storage / google-cloud-run ? `import backend.storage` and `import backend.compute` are
pure-stdlib until a cloud URI/target is actually used. The local + gdrive paths need zero cloud
SDKs, so a 'truly-local' user has no cloud install at all.",
      "To let a packaged app point at gs://|s3://|gdrive:// from a config/UI, expose
StorageConfig fields (backend, input_backend, input_root, output_root, per-backend dicts) in the
UI ? these are endpoints/regions/paths only and safe to persist; the app then resolves any
source via storage.acquire_local_input with no code change.",
      "Credentials must be supplied OUT OF BAND, never in the app's config file: S3 via AWS_*
env or ~/.aws, GCS via GOOGLE_APPLICATION_CREDENTIALS service-account JSON / ADC. A packaged
'batteries-included' app needs a credentials UX (file picker for the SA JSON setting
GOOGLE_APPLICATION_CREDENTIALS, or AWS key fields written to the boto3 env/chain) that is
explicitly separate from the license-clean config layer.",
      "Cloud SDKs should be packaged as OPTIONAL installable components (mirroring the
storage-s3 / storage-gcs extras) so the installer/UI can fetch them on demand when the user
selects a cloud backend; the import failure today is a hard ImportError, so the app must catch
it and prompt to install the right extra.",
      "google-cloud-run is NOT declared as an extra anywhere ? if the packaged app is ever to
DISPATCH to Cloud Run (cloud-serving compute), packaging must add a `compute-cloud`/serving
extra (google-cloud-run) and surface the CLOUD_RUN_JOB / CLOUD_RUN_REGION / GOOGLE_CLOUD_PROJECT
env as config; otherwise the cloud-serving profile fails at dispatch with a RuntimeError.",
      "gdrive:// is the lightest cloud-ish source for a non-tech-savvy user: no SDK, no OAuth,
```

```
no service account ? just Google Drive for Desktop mounted. The packaged app can auto-detect the mount and surface gdrive:// inputs with zero credential UX, which fits the user-friendly-UX goal; it just needs to render the existing actionable not-mounted / not-synced remedy messages.",
      "Every cloud backend lands outputs at a deterministic `<out>/outputs/<video_id>/` layout keyed by the MD5 video_id, so a packaged app can present results uniformly regardless of where compute ran (local in-process vs Cloud Run Job).",
      "License posture for new packaging deps is satisfiable: boto3 (Apache-2.0), google-cloud-storage / google-cloud-run (Apache-2.0) are all in the allowed MIT/Apache/BSD set; the credentials UX adds no weight-licensing concern (weights live in GCS, fetched via the same storage layer)."
    ],
    "risks": [
      "Cloud creds are env/ADC-chain dependent, which is fragile for a downloadable app: a user on Windows with no AWS_* env or no GOOGLE_APPLICATION_CREDENTIALS gets a deep boto3/google-auth error far from the UI; the app needs to pre-validate creds (startup checks exist per-profile but the cred chain itself isn't surfaced) and translate failures into friendly UI errors.",
      "google-cloud-run is undeclared in pyproject extras and only installed out-of-band ? a packaged cloud-serving build will silently lack it until run_job raises RuntimeError. Easy to miss in a 'batteries-included' bundle.",
      "CloudRunCompute writes a done-marker only on SUCCESS; a failed/hung container makes the dispatcher's bounded poll exhaust to PolicyTimeout (rc=4) rather than report the real failure cause ? a packaged app surfacing 'timeout' would mislead users whose job actually errored.",
      "S3Storage.exists() swallows ALL exceptions and returns False (treats auth/network errors as 'not found'), so a misconfigured credential could masquerade as a missing object ? confusing in a UI that distinguishes 'no such file' from 'cannot reach bucket'.",
      "GDrive auto-detect only covers macOS CloudStorage paths and Windows drive letters; Linux and non-default mounts require GDRIVE_MOUNT_ROOT, so a Linux packaged user gets a hard RuntimeError unless the UI exposes the mount-root setting.",
      "The signed_url capability is uneven (local cannot sign, gdrive only emulates, NotImplementedError is the contract) ? any packaged UI feature that hands a browser a signed URL must fall back to fetch_to_local, which means downloading bytes locally first; not all backends support a zero-copy browser handoff."
    ],
    "open_questions": [
      "Should a packaged app add a dedicated `compute-cloud` (google-cloud-run) extra and a credentials-management UI, or is cloud-serving out of scope for the first downloadable release (truly-local + gdrive only)?",
      "How should the app persist/scope cloud credentials given the guardrail that StorageConfig must hold NO secrets ? a separate keychain/secret store, or just point users at the standard env/ADC files via a setup wizard?",
      "The WASB model weights (wasb_badminton_best.pth.tar) live in a GCS bucket ? is the plan to fetch them through this same storage layer (gs:// via GCSStorage) at first run, and if so does that force google-cloud-storage into the BASE install rather than an optional extra for the truly-local default?",
      "Does the packaged truly-local profile need ANY storage extra, or is the intent that local + gdrive (zero SDK) is the only out-of-the-box path and s3/gcs are user-opt-in installs?",
      "Should the worker emit a FAILURE done-marker (currently a noted follow-up) before a packaged cloud-serving release, so users get the real error instead of an rc=4 timeout?"
    ]
  },
  {
    "area": "Heavy runtime deps - dual-torch, WASB weights, ffmpeg, device",
    "summary": "Two disjoint Python runtimes; the dual-torch split is the crux. See key_facts for the file-grounded details.",
    "key_facts": [
      {
        "fact": "Main engine env excludes torch and torchvision (out-of-band index needed); cv2 numpy google-genai fastapi supervision are hard deps; in-process torch is used only by yolo_hybrid person_detector.",
        "evidence": "pyproject.toml 27-37; person_detector.py 29"
      },
      {
        "fact": "WASB runs in a separate conda env py3.8 torch 1.11.0 cu113 numpy under 1.24; the native runner shells WASB_PYTHON as a subprocess and never imports WASB torch; the split is
```

```
deliberate vs the engine torch 2.6 plus cu124.",
        "evidence": "gpu_setup.sh 40-49; native_wasb_runner.py 110-136; Dockerfile 8-12"
      },
      {
        "fact": "Weights resolve via env WASB_WEIGHTS then config then default; healthcheck
hard-fails if absent; no code fetches them; provenance uncleared WASB-C3; code MIT but
distributed weights not covered.",
        "evidence": "native_wasb_runner.py 67-97; Dockerfile 14-16; CODE_MAP.md 270"
      },
      {
        "fact": "ffmpeg and ffprobe are bare PATH command names via subprocess at about 10
sites; no configurable path and no which-guard; only doctor.py preflights.",
        "evidence": "utils/video.py 11-47; doctor.py 84-108"
      },
      {
        "fact": "Gemini motion stub are torch-free at runtime but importing backend.main eagerly
loads torch-bound yolo_hybrid via the segmenters init; DeviceContext auto falls back to CPU and
explicit cuda hard-fails with the probe in the wasb subprocess.",
        "evidence": "segmenters init 1-7; device.py 39-56; test_import_smoke.py 35-57"
      }
    ],
    "packaging_implications": [
      "Two runtimes (main torch-optional plus an isolated WASB py3.8 torch1.11 subprocess via
WASB_PYTHON); make person_detector torch lazy or guard yolo_hybrid in the segmenters init for a
no-torch Gemini tier; bundle static ffmpeg (LGPL); fetch WASB weights at first run since WASB-C3
is uncleared; cloud and local are already wired via profiles and storage/compute registries."
    ],
    "risks": [
      "Double-torch bundle is large and 1.11 cu113 is old; WASB weights are not redistributable
(WASB-C3) which breaks the licensing guardrail; bare-name ffmpeg fails if not on PATH; eager
torch import crashes a torch-less light build; conda build needs build-time network plus ToS."
    ],
    "open_questions": [
      "WASB weight size and license; is CPU-only WASB acceptable; can WASB move to torch 2.x to
drop the split; which ffmpeg license build to use; does the appliance doc pick micromamba vs a
pip venv."
    ]
  },
  {
    "area": "Bundling the web UI into the downloadable \"batteries-included\" Python app",
    "summary": "The operator-facing UI is a Vite + React 19 + TS + Tailwind v4 SPA that lives in
a DIFFERENT repo (khelsutra/web/), not in the engine. It is built with `vite build` into
khelsutra/web/dist/ (a standard static bundle: index.html + one hashed JS chunk + one CSS +
self-hosted Noto woff2 fonts, ~669 KB total); dist/ is gitignored so the built bundle is NOT
checked in. The SPA talks to the engine purely via SAME-ORIGIN relative `/api/*` calls ? the
only API-base config is a hardcoded `const BASE = \"/api\"` in web/src/api/client.ts:7, with NO
absolute URL, NO consumed env var (the `VITE_ENGINE_ORIGIN` mentioned in vite.config.ts is a
dev-proxy comment that the client code never reads). In dev, Vite proxies `/api` ?
localhost:8000; in prod the live design (khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md,
decisions D1/D5) puts a reverse proxy in front that serves the SPA bundle and proxies `/api` to
the engine on 127.0.0.1:8000 ? explicitly NOT FastAPI serving the SPA, and explicitly NOT engine
static. Meanwhile the engine TODAY serves a completely separate, legacy hand-written vanilla-JS
UI (backend/api/static/{index.html,app.js,style.css}) mounted at `/static` and returned at `/`
(backend/main.py:64-74) ? this is NOT dead code (it is live-mounted and the appliance doc treats
its app.js as the current bundled UI), but it is the OLD UI, superseded by the khelsutra SPA,
and it only hits `/api/process` + `/api/compile`. The three locale catalogs (en/kn/hi
common.json, 88 lines each) are statically `import`ed into the i18next singleton
(web/src/i18n/index.ts) and bundled into the JS chunk at build time ? they ship inside the
bundle, not as separate runtime files, with always-fallback to `en`. Fonts are self-hosted via
`@fontsource-variable` (bundled woff2), so kn/hi render offline with no Google Fonts runtime
dependency. To ship the built SPA inside the Python app as a unified origin, FastAPI can serve
khelsutra/web/dist/ as static files ? this needs ZERO SPA code change because BASE is already
same-origin `/api`.",
    "key_facts": [
      {
```

```
        "fact": "The operator SPA is Vite+React19+TS+TailwindV4 and lives in the SIBLING
khelsutra repo, not the engine; build = `vite build`, output dir = khelsutra/web/dist/ (Vite
default).",
        "evidence": "khelsutra/web/package.json:8-9,24-36; khelsutra/web/vite.config.ts"
      },
      {
        "fact": "Built bundle is a standard static set: index.html + assets/index-*.js +
assets/index-*.css + self-hosted Noto Kannada/Devanagari woff2; ~669 KB total. No server-side
rendering.",
        "evidence": "khelsutra/web/dist/ (find: ./index.html, ./assets/index-BkCjbkEL.js,
./assets/index-8AKPRqKx.css, 5x noto woff2; du -sh = 669K)"
      },
      {
        "fact": "dist/ is gitignored ? the built bundle is NOT committed; any packaging step
must run `npm run build` (cross-repo) to produce it.",
        "evidence": "khelsutra/web/.gitignore:2 (dist)"
      },
      {
        "fact": "SPA hardcodes the API base as a same-origin relative path `const BASE =
\"/api\"`; every call is `fetch(`/api`+path)`. No absolute URL anywhere in src/.",
        "evidence": "khelsutra/web/src/api/client.ts:7,33,39,103; Grep over web/src found only
BASE=\"/api\""
      },
      {
        "fact": "`VITE_ENGINE_ORIGIN` is only a comment in vite.config.ts (dev-proxy override);
the client code never reads import.meta.env, so there is no runtime knob to point the SPA at a
different origin.",
        "evidence": "khelsutra/web/vite.config.ts:8; Grep for VITE_/import.meta.env in web/src =
no matches"
      },
      {
        "fact": "Live appliance design: reverse proxy serves the SPA bundle and proxies /api to
engine 127.0.0.1:8000; console lives in khelsutra web/ scaffold, 'Cloudflare-Pages-style root,
NOT engine static'.",
        "evidence": "khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:30 (D1), :34 (D5), :54-79
(topology)"
      },
      {
        "fact": "The engine today live-mounts a SEPARATE legacy vanilla-JS UI at /static and
returns its index.html at / ? distinct from the khelsutra SPA; it only calls /api/process +
/api/compile.",
        "evidence": "backend/main.py:64-74 (mount /static, serve_index);
backend/api/static/{index.html,app.js,style.css}; appliance doc :42 (app.js never calls upload
endpoints)"
      },
      {
        "fact": "The engine's static-serving paths are CWD-relative string literals, so they
only resolve when launched from the repo root ? they break in a pip-installed wheel run from an
arbitrary directory.",
        "evidence": "backend/main.py:65 os.makedirs(\"backend/api/static\"), :66
StaticFiles(directory=\"backend/api/static\"), :70 \"backend/api/static/index.html\" (all
relative)"
      },
      {
        "fact": "i18n catalogs (en/kn/hi common.json, 88 lines each) are statically imported
into the i18next init and bundled into the JS at build time ? shipped inside the bundle, not as
runtime-fetched files; always fallback to en.",
        "evidence": "khelsutra/web/src/i18n/index.ts:13-15,28-32,40 (import
enCommon/knCommon/hiCommon; resources; fallbackLng=en)"
      },
      {
        "fact": "Fonts are self-hosted via @fontsource-variable (bundled woff2), unicode-range-
scoped ? Kannada/Hindi render offline with no Google Fonts runtime dependency.",
        "evidence": "khelsutra/web/src/main.tsx (import @fontsource-variable/noto-sans-kannada +
devanagari); dist/assets/*.woff2"
```

```
      },
      {
        "fact": "engine pyproject wheel target lists packages=[backend,training] with NO
MANIFEST.in / force-include / package-data; non-.py files under backend/api/static would be
picked up by hatchling defaults, but no SPA dist is referenced.",
        "evidence": "pyproject.toml:89-93; MANIFEST.in absent (ls: No such file)"
      },
      {
        "fact": "The SPA reads a finished job's video via ?video=<id> URL param and rewrites
history state ? consistent with a same-origin single-page app served from /.",
        "evidence": "khelsutra/web/src/App.tsx:49-53,235 (URLSearchParams .get('video'),
history.replaceState '?video=')"
      },
      {
        "fact": "A prior DESKTOP_APP_PLAN was superseded by owner decision D10: deliver a self-
hosted webserver + UI the user spins up (settings choose local/cloud compute+storage), one
artifact for owner box / droplet / GPU host.",
        "evidence": "khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:39 (D10)"
      }
    ],
    "packaging_implications": [
      "Shipping the BUILT SPA inside the Python app as a unified origin (FastAPI serves
khelsutra/web/dist/, proxies nothing) needs ZERO SPA code change: BASE is already same-origin
'/api'. This collapses the appliance's reverse-proxy assumption into a single FastAPI process ?
simpler than the doc's proxy topology, and the recommended packaging path for a downloadable
app.",
      "There is a CROSS-REPO build coupling: the SPA source is in khelsutra/web (separate repo,
separate npm toolchain) but the Python wheel is built here. The packaging pipeline must (a) `npm
ci && npm run build` in khelsutra/web, then (b) copy/vendor web/dist/ into the engine wheel as
package data. dist/ is gitignored, so it cannot simply be committed and picked up ? it must be
produced at package time or vendored deliberately.",
      "To bundle dist/ in the wheel, add an explicit hatchling force-include (e.g. map
khelsutra/web/dist -> backend/ui/ or a top-level package) in pyproject.toml
[tool.hatch.build.targets.wheel.force-include]; the current config has none, so add it rather
than rely on auto-detection.",
      "The engine's existing static-serving code is BROKEN for an installed app:
backend/main.py:65-70 use CWD-relative paths ('backend/api/static/...'). For a pip-installed app
launched from any directory these won't resolve. Replace with package-relative resolution
(importlib.resources / Path(__file__).parent) pointing at the vendored dist/, and add an SPA
history-fallback route (serve index.html for unknown non-/api paths) so client-side ?video=
routing survives refresh.",
      "Decide the fate of the legacy backend/api/static vanilla-JS UI: it is the OLD console and
is superseded by the khelsutra SPA. Bundling BOTH is confusing; the packaging plan should pick
one (vendor the React dist and retire/replace the legacy static UI at / and /static).",
      "i18n is a non-issue for packaging: en/kn/hi catalogs and Indic fonts are compiled INTO
the bundle (no runtime catalog fetch, no Google Fonts call), so the localized UI works fully
offline once dist/ is vendored ? aligns with the offline/local-appliance goal.",
      "Single-origin packaging makes the engine's CORS '*' irrelevant from the browser (same-
origin), but it does NOT add auth ? the appliance's tenancy boundary (127.0.0.1 bind + firewall
+ edge gate, D1/D2) still applies; a downloadable single-process app must keep the bind-to-
localhost default."
    ],
    "risks": [
      "Cross-repo build dependency is fragile: the engine wheel cannot be built without a Node
toolchain + the khelsutra repo checked out at a compatible commit. Version skew between the two
repos can ship a SPA that calls /api endpoints the bundled engine lacks (the appliance doc lists
several net-new endpoints: /api/capabilities, /api/videos, /api/reels ? not yet in the engine).
Need a pinned/vendored bundle + a contract test.",
      "The committed backend/api/static UI being live at / means a naive 'serve dist/ at /'
change could collide with the existing serve_index route and StaticFiles('/static') mount ? must
remove/replace the legacy routes, not stack them.",
      "The hardcoded same-origin BASE='/api' is a strength for unified-origin packaging but a
constraint: any deployment that does NOT put the engine and SPA on one origin (e.g. SPA on
Cloudflare Pages + engine elsewhere) would require an SPA code change to introduce a runtime
origin knob ? currently impossible without a rebuild.",
```

```
      "License hygiene of the new bundling toolchain/deps must be checked: the SPA pulls
@fontsource Noto fonts (OFL) and React/i18next/Tailwind ? all need to be confirmed
MIT/Apache/BSD/OFL-clean per the non-negotiable guardrail before they ride inside a distributed
wheel (the engine already vendors OFL Noto under backend/assets/fonts with LICENSE files, a good
precedent).",
      "dist/ being gitignored means there is no reproducible, auditable record of what UI
shipped in a given wheel unless the build is pinned (lockfile present: package-lock.json) and
the bundle hash is recorded at package time."
    ],
    "open_questions": [
      "Where should the vendored SPA live inside the wheel ? a new backend/ui/ package dir, a
top-level static_ui package, or replace backend/api/static in place? (affects the force-include
mapping and the FastAPI mount path).",
      "Should the engine serve the SPA directly (single FastAPI process, simplest for a
downloadable app) OR keep the reverse-proxy topology from LOCAL_APPLIANCE_ARCHITECTURE.md D1?
The downloadable-app goal argues for FastAPI-serves-SPA; the doc currently assumes a proxy.",
      "How is the cross-repo build orchestrated at package time ? git submodule of
khelsutra/web, a CI step that checks out khelsutra and runs npm build, or a pre-built dist
vendored into this repo? Pinning strategy for SPA<->engine API contract.",
      "Should the legacy backend/api/static vanilla-JS UI be deleted as part of this work, or
kept as a fallback? It is currently live-mounted and referenced by the appliance doc.",
      "Do the SPA's expected-but-missing endpoints (/api/capabilities, /api/videos, /api/reels,
and the bucket-ingest SPA call) need to land in the engine before the bundled UI is fully
functional, or does the current SPA degrade gracefully (it ships demo data + a ?video= path) for
an MVP bundle?"
    ]
  },
  {
    "area": "Rally annotation + BYO training (annotate -> CSV -> train -> ship-gate), for
packaging a batteries-included downloadable app",
    "summary": "Annotation today is NOT in-app and NOT in-browser: it is a separate, external,
MIT-licensed VLC 3.0.x Lua plugin (`rally-annotator`, a sibling repo) that the user installs
into their own VLC and clicks Mark START / Mark END / reason / shots to append one row per rally
to `<video-stem>.rallies.csv` next to the video. \"Rally annotation if needed\" in-app means,
today, \"go use the external VLC plugin\" ? the engine and the khelsutra SPA have NO annotation
UI, and the khelsutra design doc explicitly forbids re-authoring VLC instructions in the SPA and
forbids a \"Train now\" button (D3/D4). The annotator CSV is directly ingestible by the engine
with NO transform: header `rally_number,start_time,end_time,ending_reason,sport,shots_count`,
decimal seconds, and the engine reads by column name via `gt_loader.load_gt_intervals` (accepts
start/end OR start_time/end_time) and `stub_runner.synth_trajectory_from_labels` (needs the
*_time names). Training is an OFFLINE CLI, `python -m backend.worker train`, that pulls labels/
(+ videos/) from a storage URI, builds per-video trajectory CSVs, assembles a manifest, shells
out to `python -m training.gen0.harness`, runs honest leave-one-video-out + a ship-gate, and
pushes a versioned artifact bundle (weights.json + reviewable manifest.json). It has two
trajectory sources: `synth` (default, GPU-free CPU mock that ENCODES the labels ? plumbing-
meaningful only, trains a tiny LogisticRegression) and `detector` (REAL shuttle trajectories via
the native WASB runner ? GPU-gated and needing the separate conda/torch-1.11 env, in-flight).
The licensing ship-gate is `training/gen0/harness.py::gate_verdict`, which hard-codes
`ship=False` for ANY commercial ship today (WASB checkpoint provenance \"C3\" unresolved +
F1@tIoU target not calibrated, issue #172), and the bundle manifest attests
`no_paid_llm_training_labels: True` and `label_source: \"human (rally-annotator golden CSVs)\"`.
There is currently NO serving path that consumes a trained Gen-0 bundle ? the per-user serving
bridge (`personalization/serving.py`) is explicitly \"NOT WIRED INTO THE SERVED PATH\".",
    "key_facts": [
      {
        "fact": "Annotation is an external standalone MIT VLC 3.0.x Lua plugin, NOT in-browser
and NOT in the engine. Install = copy `vlc/rally_annotator.lua` into VLC's lua/extensions
folder; it is button/click-driven (no marking hotkeys), reads playback time, and appends one row
per rally to `<video-stem>.rallies.csv` next to the video.",
        "evidence": "rally-annotator/README.md:5-35,57; vlc/rally_annotator.lua:128"
      },
      {
        "fact": "Annotator CSV needs NO transform for the engine. Header is
`rally_number,start_time,end_time,ending_reason,sport,shots_count`, decimal seconds, end>start;
shots_count optional/blank.",
```

```
        "evidence": "rally-annotator/vlc/rally_annotator.lua:128; rally-
annotator/docs/CSV_FORMAT.md:6-19"
      },
      {
        "fact": "Engine reads labels by COLUMN NAME via two readers: `load_gt_intervals` accepts
start/end OR start_time/end_time (+ headerless positional, MM:SS, BOM, # comments), strict=True
raises file:line on bad rows; `synth_trajectory_from_labels` uses DictReader on
row['start_time']/row['end_time'] (needs the *_time names specifically).",
        "evidence": "backend/eval/gt_loader.py:30-98;
backend/pipeline/detectors/stub_runner.py:136-141"
      },
      {
        "fact": "Training is an offline CLI `python -m backend.worker train --corpus-uri ...
--out-uri ... --version X`; needs >=2 labeled videos for leave-one-video-out; pulls labels/ (+
videos/ for detector source) from a storage URI, builds a manifest, shells out to
training.gen0.harness, pushes weights/<version>/ bundle.",
        "evidence": "backend/worker/__main__.py:331-446,471-489"
      },
      {
        "fact": "Two trajectory sources: `synth` (DEFAULT, GPU-free CPU mock that encodes labels
into a synthetic Frame,Visibility,X,Y trajectory ? plumbing-meaningful only, learned model not
production-meaningful); `detector` (REAL shuttle trajectories, GPU-gated, resolves the native
WASB DetectorRunner via _resolve_train_detector + healthcheck).",
        "evidence": "backend/worker/__main__.py:368-411,295-328;
backend/pipeline/detectors/stub_runner.py:124-157"
      },
      {
        "fact": "The real detector path requires the separate dual-env: native WASB runner
invoked by python_bin from env WASB_PYTHON / WASB_WEIGHTS / WASB_REPO_DIR (the conda
torch-1.11+cu113 env), with a healthcheck that fails if the python binary/weights/repo are
missing.",
        "evidence": "backend/pipeline/detectors/native_wasb_runner.py:10-17,68,89,157-175,239"
      },
      {
        "fact": "The harness trains a tiny LogisticRegression on CPU (lr=0.2, epochs=2000,
l2=1e-2) over frame features, does honest LOVO eval + threshold tuning, and emits a versioned
artifact bundle: weights.json (classifier + frozen normalization + calibration) and a reviewable
manifest.json.",
        "evidence": "training/gen0/harness.py:122-139,180-252"
      },
      {
        "fact": "The licensing/commercial SHIP-GATE is harness.py::gate_verdict, which returns
ship=False unconditionally today: WASB checkpoint provenance 'C3' unresolved (commercial ship
blocked regardless of F1) + F1@tIoU target not yet calibrated (#172). Beating a floor + a paired
sign test are necessary but not sufficient.",
        "evidence": "training/gen0/harness.py:142-177"
      },
      {
        "fact": "The bundle manifest attests `no_paid_llm_training_labels: True` and
training_lineage.label_source = 'human (rally-annotator golden CSVs)', plus per-video gt_hash +
git_sha provenance. This is the artifact-level guarantee that no paid-LLM output became training
data.",
        "evidence": "training/gen0/harness.py:219-239"
      },
      {
        "fact": "Serving consumes ONLY the bundle, never a code import (one-way training/serving
wall: backend must not import training); but NO serving path consumes a Gen-0 bundle yet. The
per-user serving bridge that would rehydrate a stored classifier is explicitly NOT WIRED INTO
THE SERVED PATH.",
        "evidence": "training/gen0/harness.py:21,10-11;
backend/pipeline/personalization/serving.py:21-27,87-103"
      },
      {
        "fact": "The engine HTTP API has NO annotate or train endpoints ? only
upload/process/query/compile/jobs/cost/health/config. Training and annotation are entirely out-
```

```
of-band of the web service.",
        "evidence": "backend/main.py:138,142,275,326,352,409,420; backend/api/uploads.py:64-147"
      },
      {
        "fact": "The khelsutra SPA has no annotation/training UI; the design doc locks D3 (SPA
only links out to the VLC tool) and D4 (NO 'Train now' button) and frames the USP as
ownership/licensing/privacy/consent, not accuracy.",
        "evidence": "khelsutra/docs/ANNOTATE_AND_TRAIN.md:21-25,63-64;
khelsutra/web/src/components/ (no annotation component)"
      },
      {
        "fact": "The video<->label join key is the file MD5 (compute_video_id) and stem-matching
labels/<name>.rallies.csv <-> videos/<name>.<ext>; any re-encode changes the MD5 and breaks the
join. fps/frame_width are probed via ffprobe for the detector path (a wrong fps silently mis-
aligns every label).",
        "evidence": "backend/worker/__main__.py:262-292,373-405;
backend/utils/hashing.py::compute_video_id"
      },
      {
        "fact": "Annotator CSV is also ingestible into the collector golden corpus via `python
-m src.cli.curate import-local --sport ... --video-path ... --annotations-path ...`.",
        "evidence": "rally-annotator/docs/CSV_FORMAT.md:40-41; sports-data-
collector/src/cli/curate.py:97,1239"
      }
    ],
    "packaging_implications": [
      "Annotation cannot be bundled into the Python app as-is: it is a VLC Lua extension in a
SEPARATE MIT repo requiring VLC 3.0.x (4.0 broke the Lua API). A batteries-included app must
either (a) ship/instruct the VLC plugin install as an external step (the current khelsutra D3
stance: link out, don't re-author), or (b) build a NEW in-browser/in-app annotator that emits
the SAME `rally_number,start_time,end_time,ending_reason,sport,shots_count` CSV ? the roadmap
explicitly lists an 'HTML5 <video> page that exports the same CSV' as future work (rally-
annotator/README.md:112-113). Bundling the existing plugin = also bundling VLC, a heavyweight
system dep.",
      "The packaged app CAN ship the full annotate->train->bundle CLI for the GPU-FREE synth
path with zero extra heavy deps: it needs only numpy/scipy (BSD) + ffmpeg/ffprobe (system dep,
already required) + the existing backend/training code. This validates the whole pipeline on a
CPU laptop/CI, but the trained model is plumbing-only (synthetic trajectory), NOT a usable rally
model.",
      "The REAL training path is the hard packaging problem: it needs (1) a GPU, (2) the
separate conda env (torch 1.11+cu113) for native WASB driven by
WASB_PYTHON/WASB_WEIGHTS/WASB_REPO_DIR, and (3) the WASB weights `wasb_badminton_best.pth.tar`
(license-clean MIT WASB, but the *.pth.tar checkpoint provenance is the unresolved C3 blocker).
A downloadable app must orchestrate this dual-env / weight-download (the decoupled-compute
storage seam already supports gcs:// fetch) or punt real training to a cloud profile.",
      "The licensing guardrail is enforced at the ARTIFACT level, not at runtime: gate_verdict +
the manifest attestation. A packaged app that offers 'train your own model' must surface
gate_verdict honestly (ship=False today) and must NOT let a user ship/serve a commercial model ?
there is no UI for that yet, and the gate refuses it. The app should treat the bundle's
manifest.json as the reviewable contract and never auto-promote a model into serving.",
      "Nothing consumes a trained Gen-0 bundle at serving time yet (personalization/serving.py
is groundwork, not wired). So 'train -> use my model' is NOT an end-to-end CUJ today ? a
packaged app can offer annotate + train-to-bundle, but 'serve the trained model' is an unbuilt,
owner-gated step. Don't advertise a closed loop the engine can't run.",
      "Model weights are not shipped in the repo (only artifacts/.../manifest.json is committed;
weights.json is generated and *.pt is gitignored; WASB .pth.tar lives in GCS). A packaged app
must FETCH weights (WASB checkpoint from the bucket via the storage backend) at first run rather
than embed them ? and must keep every fetched weight on the MIT/Apache/BSD allowlist, with the
C3 provenance caveat unresolved.",
      "The video<->label join is MD5-based: a packaged app that re-encodes / transcodes user
uploads before annotation or training will break the label join. The app must preserve byte-
identity between the annotated file and the trained file, or re-key carefully."
    ],
    "risks": [
      "Bundling the existing annotator means a hard dependency on VLC 3.0.x specifically (VLC
```

```
4.0 changed the Lua API and is unsupported), which is fragile and a heavy external system dep ?
and it is desktop-only, not in-browser, conflicting with the 'self-hosted web service' product
direction (D10).",
      "The 'real' training path is effectively unrunnable inside a simple downloadable app: it
needs a GPU + the second conda/torch-1.11 env + WASB weights. Offering 'train your own model'
risks misleading non-technical users into expecting one-click real training when only the synth
(mock) path is turnkey.",
      "The commercial ship-gate is hard-False today (C3 unresolved + uncalibrated F1 target
#172). Any packaged 'BYO model' feature that implies a shippable/servable model would contradict
the gate and the licensing guardrail. Serving a trained bundle is unbuilt and owner-gated.",
      "The synth-path 'trained model' is meaningful only as plumbing ? a user could mistake the
LOVO F1 numbers it prints for a real accuracy claim. The khelsutra doc already forbids accuracy-
superiority claims (D5); the app's UX must mirror that honesty.",
      "ffmpeg/ffprobe are system deps the train path silently relies on (fps/frame_width probe).
On a box without ffprobe the detector training SKIPS videos (won't guess) ? a packaged app must
guarantee ffmpeg is present or training silently degrades.",
      "Two CSV schemas exist (the annotator's light schema vs the Roora vendor schema);
conflating them in packaging/UI could corrupt the label pipeline."
    ],
    "open_questions": [
      "For a downloadable app, does annotation become a NEW in-browser annotator emitting the
same CSV (the README's future 'HTML5 <video>' idea), or does the app keep delegating to the
external VLC plugin (khelsutra D3)? This is unresolved and owner-gated.",
      "Will the packaged app ever expose the real (detector/GPU) training path, given it needs
the dual conda env + GPU + WASB weights, or will real training only run against a cloud/GCP
serving profile while local stays synth-only?",
      "Is there any intent to wire a trained Gen-0 bundle into serving
(personalization/serving.py -> fusion_hybrid._select_for_handoff)? Today the loop is open; 'use
my model' is not an end-to-end CUJ.",
      "How will the app fetch/cache the WASB checkpoint (GCS) within the licensing allowlist
while the C3 checkpoint-provenance blocker is unresolved? Can a license-clean checkpoint be
shipped at all?",
      "Does the engine need a NEW PATCH /api/segments correction endpoint (khelsutra P8,
designed/owner-gated) to grow the corpus in-app, given there is no annotate/correction API
today?",
      "Should the packaged app surface gate_verdict (ship=False, blockers) to the user so 'train
your own model' is honest about not being commercially servable yet?"
    ]
  },
  {
    "area": "prior appliance and desktop design",
    "summary": "Owner D10 superseded the desktop-app plan with a self-hosted webserver appliance
the user spins up; compute and storage are settings; the engine stays auth-less behind an
external edge gate that is the only tenancy boundary.",
    "key_facts": [
      {
        "fact": "D10 locks a self-hosted webserver appliance not a desktop app and supersedes
DESKTOP_APP_PLAN.md",
        "evidence": "LOCAL_APPLIANCE_ARCHITECTURE.md line 39"
      },
      {
        "fact": "Engine has zero app-layer auth plus wildcard CORS and hard-binds localhost",
        "evidence": "backend/main.py line 744"
      },
      {
        "fact": "A Cloud Run M6 image bakes the dual-Python stack, contradicting the no-
Dockerfile claim",
        "evidence": "deploy/cloudrun/Dockerfile"
      },
      {
        "fact": "WASB weights provenance is uncleared and native WASB is owner-parity-gated",
        "evidence": "LOCAL_APPLIANCE_ARCHITECTURE.md line 45"
      }
    ],
    "packaging_implications": [
```

```
      "Ship engine plus SPA plus a bundled edge-auth gate as one artifact",
      "Lift the Cloud Run dual-env Dockerfile; a container is easiest",
      "WASB weights need runtime fetch plus a license gate",
      "Compute-as-settings needs a net-new capabilities endpoint"
    ],
    "risks": [
      "No-Dockerfile claim is stale and DOC_STATUS misses the supersession",
      "Desktop P0 on a no-GPU laptop was never measured",
      "WASB weight provenance blocks public redistribution",
      "Auth is structural-only so enforce localhost plus firewall plus gate"
    ],
    "open_questions": [
      "Edge-auth Caddy basicauth vs cloudflared plus Cloudflare Access",
      "GPU-host-only or no-GPU laptop reviving P0",
      "How to deliver WASB weights publicly",
      "Container vs native vs PyInstaller; net-new videos and reels endpoints"
    ]
  },
  {
    "area": "Completed end-to-end CUJs the packaged \"batteries-included\" app MUST preserve",
    "summary": "Across the engine (FastAPI + worker CLI + single-shot CLI) and the khelsutra
web/ SPA, a set of end-to-end Critical User Journeys is already DONE and verified, and the
downloadable app must keep every one of them working. The flagship loop is fully shipped: point
at a video (local upload OR a gs:// bucket URI entered IN the SPA) -> engine
fetches/validates/segments via Gemini -> SPA loads rallies via /api/query (default all-selected)
-> deterministic query bar + per-rally checkbox selection sharing one Set -> /api/compile
stitches selected segments -> reel previews in <video> and downloads to the local machine; this
ran live both en and hi (and kn works via the same switcher). The SPA is fully trilingual
(en+kn+hi) with automatic browser-language negotiation, self-hosted OFL Noto fonts, offline/air-
gap renderable, proven by a Playwright screencast. On the engine side, the worker CLI does GCS-
in -> WASB GPU inference -> GCS-out (22 rallies on a real L4), and the cold-start GPU
provisioning (main torch 2.6 + separate wasb conda torch-1.11 env + staged weights) is validated
end-to-end. The decoupled-compute seam (profiles/storage/compute registries, startup checks,
cost ledger, Cf-Access edge-auth) is merged but DEFAULT-OFF. The annotate->train loop is only
PARTIALLY real: the MIT rally-annotator (VLC) produces ingestible CSVs and worker train runs,
but the GPU-free training path is a pipeline-validation MOCK and the real-model ship-gate
currently refuses a commercial ship. The packaging effort must preserve these journeys without
requiring git clone, while solving ffmpeg/system-deps, the dual torch envs for native WASB, GCS-
hosted weights, and bundling the cross-repo SPA.",
    "key_facts": [
      {
        "fact": "CUJ-1 (DONE): Local Gemini inference -> 12 rallies -> reel downloaded, en AND
hi. Real run on local Windows: gsutil cp the bucket video local -> POST /api/process
{segmenter:gemini} -> poll /api/jobs -> 12 rallies (video_id 05e0e577) -> SPA loads via
/api/query -> Build -> /api/compile -> reel (1080p, 76.5s) downloaded to ~/Downloads; repeated
with UI in Hindi (page/cards/buttons all Hindi). Surfaces: engine
/api/process,/api/jobs,/api/query,/api/compile,/api/highlights + Gemini segmenter; SPA
App.tsx/QueryBar/RallyGrid/SelectionFooter/CompileResult + i18n.",
        "evidence": "khelsutra/docs/DRY_RUN_CUJ.md:52-60 (§7 run result); endpoints
backend/main.py:275,352,409,420,385"
      },
      {
        "fact": "CUJ-2 (DONE): In-UI gs:// bucket ingest (Path B / B-P2). Live e2e: in the SPA
IngestPanel paste gs://.../mahadevpura_smoke_180s.mp4 -> POST /api/process -> engine fetches via
google-cloud-storage+ADC -> Gemini -> 12 rallies -> handed off ?video= -> back-half renders ->
Build -> reel downloaded (1080p, 76s). B-P1 (engine endpoint accepting gs://) shipped in
M2/#280; B-P2 (SPA ingest screen) merged khelsutra #56. Surfaces: backend/main.py:275
(resolve+skip-exists for non-local), backend/main.py:159 (acquire_local_input fetch); SPA
web/src/components/IngestPanel.tsx + api/client + hooks/useIngestJob.",
        "evidence": "khelsutra/docs/DRY_RUN_CUJ.md:80-88 (§9); khelsutra git log 4525c5d (#56) +
7750afc (#57); backend/main.py:283 (skip exists for non-local backends)"
      },
      {
        "fact": "CUJ-3 (DONE): Per-rally selection + deterministic query bar -> reel
(PER_RALLY_SELECTION MVP/M0, P1-P5). Rallies load all-selected (D8); parseQuery() over
```

```
length/count/order/shot_count writes a shared Set<segment_id>; SelectionFooter warns on
min_shot_count floor (never silently drops); Build -> /api/compile -> <video> preview +
download. Only honest fields shown (ordinal/duration/shot_count); motion server/receiver/events
hidden. Surfaces: web/src/lib/query.ts, useRallySelection hook, lib/floor.ts (mirrors
main.py:362-383), QueryBar/RallyGrid/SelectionFooter/CompileResult.",
        "evidence": "khelsutra/docs/PER_RALLY_SELECTION.md:152-160 (DoD), :122-139 (tracker M0
complete); compile floor backend/main.py:420"
      },
      {
        "fact": "CUJ-4 (DONE): Reel download-to-local. /api/compile returns a server-local
filepath AND a download_url; GET /api/highlights/{video_id}/{filename} streams the compiled MP4;
the SPA builds the URL and offers <a download>. The compiled reel is the only streamable video
today (no source/clip/thumbnail endpoint).",
        "evidence": "backend/main.py:385 (highlights), :420 (compile); PER_RALLY_SELECTION.md:44
(download_url additive)"
      },
      {
        "fact": "CUJ-5 (DONE): Trilingual SPA with automatic browser-language negotiation
(en+kn+hi). react-i18next + ICU + browser-language detector; negotiation order persisted-pref ->
?lang -> navigator/Accept-Language -> en; self-hosted OFL Noto Kannada/Devanagari (@fontsource,
no CDN); air-gap renderable (CI-guarded). Proven by a Playwright screencast (locale kn-IN auto-
renders Kannada -> switch hi -> en). 103 web tests green incl. N1/N2 browser-detection + key-
parity. Surfaces: web/src/ i18n config, LanguageSwitcher.tsx, locales/{en,kn,hi}.",
        "evidence": "khelsutra/docs/FRIENDLY_AND_MULTILINGUAL.md:147 (SPA milestone), :130-142
(tracker B1-B6 all done); current-state.md:21"
      },
      {
        "fact": "CUJ-6 (DONE): Worker CLI GCS-in -> GPU WASB inference -> GCS-out. python -m
backend.worker -v infer --video-uri gcs://.../smoke.mp4 --out-uri gcs://... --segmenter
wasb_hybrid --set indexing.skip_ai_handoff=true -> 22 rallies -> pushed
outputs/<video_id>/{intervals.csv,report.json} to the bucket. CRITICAL: a no-Gemini-key box MUST
set skip_ai_handoff=true (else Phase-3 Gemini handoff errors -> 0 rallies). NOTE: writes
CSV/JSON to the bucket, NOT the API DB (distinct from /api/process). Surfaces:
backend/worker/__main__.py:156 (cmd_infer), CLI args :457-469.",
        "evidence": "current-state.md:16 (real M7 cloud GPU run DONE+VERIFIED);
backend/worker/__main__.py:457-469"
      },
      {
        "fact": "CUJ-7 (DONE/validated mechanism): Cold-start GPU native-WASB provisioning.
Fresh L4 VM -> bootstrap --gpu + gpu_setup.sh builds BOTH torch envs (main torch 2.6+cu124 cuda
True AND wasb conda env torch 1.11+cu113 cuda True via PTX-JIT) + stages the 5.82 MiB weights
from GCS in ~7 min, then worker infer GCS->GCS. THE hard packaging problem (dual torch envs +
GCS-hosted weights + ffmpeg) is exactly this path.",
        "evidence": "current-state.md:33-34 (cold-start run 2026-06-21);
deploy/cloudrun/{Dockerfile,README} mirrors gpu_setup.sh (current-state.md:16, M6 #287)"
      },
      {
        "fact": "CUJ-8 (DONE single-shot): CLI single-video inference. python -m backend.main
--video-path ... --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion}; registry-driven segmenter
choices. Also indexer console script -> backend.main:main, and --setup -> scripts/doctor.py
diagnostics. Surfaces: backend/main.py:565-634 (arg parsing, trim/debug/single-shot paths).",
        "evidence": "backend/main.py:566-581; segmenter registrations fusion_hybrid.py:189,
gemini.py:564, motion.py:255, wasb_hybrid.py:49, yolo_hybrid.py:438"
      },
      {
        "fact": "CUJ-9 (PARTIAL): Annotate -> train your own model. The MIT rally-annotator VLC
plugin (v1.6.4) produces <video>.rallies.csv ingestible with NO transform; python -m
backend.worker train --corpus-uri ... --version 0.1.0 runs and emits weights.json+manifest.json.
BUT the default synth path is a CPU pipeline-validation MOCK (encodes labels, not real
training), real training needs GPU + in-flight native WASB, and the ship-gate (gate_verdict,
harness.py:98) currently REFUSES a commercial ship. So 'train a real model' is NOT a complete
CUJ; the annotate-CSV + train-CLI-runs plumbing IS.",
        "evidence": "khelsutra/docs/ANNOTATE_AND_TRAIN.md:12 TL;DR, :31 (train loop partly
real), :40-49 (schema bridge); rally-annotator HEAD d5d59f2 v1.6.4;
backend/worker/__main__.py:331 (cmd_train)"
```

```
      },
      {
        "fact": "Decoupled-compute serving seam is merged but DEFAULT-OFF and not part of any
verified default CUJ: profiles (presets.py truly-local/cloud-serving/cpu-mock), StorageBackend
registry (local/gcs/s3/gdrive), ComputeBackend registry (CloudRunCompute), startup checks,
billing.py cost ledger (/api/jobs|videos/{id}/cost), and Cf-Access edge-auth
(backend/api/auth.py, security.edge_auth_enabled=False). These EXTEND the cloud variants of the
CUJs above; the local single-user path is byte-identical with them off.",
        "evidence": "backend/api/auth.py:1-13,106-112 (DEFAULT-OFF NO-OP); cost endpoints
backend/main.py:326,341; current-state.md:16 (M5/M6/M8 merged default-OFF)"
      },
      {
        "fact": "Job model invariant the package must keep: single in-process
ThreadPoolExecutor(max_workers=1) drains the SQLite jobs table (one job at a time); a real bug
(#281) was that running `python -m backend.main` as __main__ left backend.main.db_instance None
so jobs hung in queued forever ? fixed. The packaged entrypoint must not reintroduce the
__main__-vs-import module split.",
        "evidence": "khelsutra/docs/DRY_RUN_CUJ.md:62-63 (§7 engine fix #281);
LOCAL_APPLIANCE_ARCHITECTURE.md:43 (job_worker.py:37 max_workers=1)"
      },
      {
        "fact": "The SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind) and
dev-proxies /api -> :8000; the engine's backend/api/static/{index.html,app.js,style.css} still
exists and is served at GET / but is the DEAD path (app.js reads data.rallies while the API
returns play_segments). Bundling the real UI into the downloadable app is a cross-repo build
concern.",
        "evidence": "backend/main.py:68-74 (serves static index); PER_RALLY_SELECTION.md:15
(app.js data.rallies bug); khelsutra web/src/components/ listing"
      }
    ],
    "packaging_implications": [
      "The reel-build loop (process -> jobs -> query -> compile -> highlights) is the spine of
CUJ-1/2/3/4 and runs on the in-process FastAPI worker with storage=local; the package must ship
this loop runnable with ZERO config on a CPU box (Gemini or motion), so ffmpeg/ffprobe must be
bundled or auto-detected and GEMINI_API_KEY supplied by the user at runtime.",
      "The packaged entrypoint must invoke the server such that backend.main module state
(db_instance) is initialized for the job worker ? do NOT reintroduce the __main__-vs-import
split that caused #281 (jobs hung in queued). The console-script `indexer` and `python -m
backend.main --server` must both yield a working async drain.",
      "Native WASB (CUJ-6/7, best quality) requires a SEPARATE Python env (conda, torch
1.11+cu113) alongside the main torch 2.6+cu124, plus GCS-hosted weights
(wasb_badminton_best.pth.tar) and a GPU. A 'batteries-included' download cannot bundle both CUDA
stacks naively; package the GPU/WASB path as an OPTIONAL, separately-provisioned component (the
deploy/cloudrun Dockerfile + gpu_setup.sh recipe is the reference), and ship a working CPU
default (Gemini/motion) so the app is useful out-of-the-box without a GPU.",
      "A no-Gemini-key fully-local run MUST set indexing.skip_ai_handoff=true (the truly-local
preset does this) or WASB returns 0 rallies ? the packaged truly-local profile must default this
on.",
      "The trilingual SPA (CUJ-5) is air-gap renderable with self-hosted OFL Noto fonts and
lives in khelsutra/web (a different repo). The package must bundle a BUILT SPA (npm run build
output) served as the UI, NOT the dead engine backend/api/static; this is a cross-repo build
step the packaging pipeline must own, and the bundle must keep the offline/no-CDN guarantee.",
      "torch/torchvision are deliberately absent from pyproject deps (CUDA/CPU wheels need
--index-url, which PEP621 can't express). The installer/launcher must resolve torch out-of-band
(a post-install step or a wheel pin per platform) for any real-detection path; the package can't
rely on `pip install` alone pulling a usable torch.",
      "GCS/S3/Drive ingest (CUJ-2 cloud variant) needs google-cloud-storage (storage-gcs extra)
+ ADC credentials; the package must make these OPTIONAL extras and degrade gracefully to local-
upload when absent, mirroring the current StorageBackend registry + extras layout.",
      "The annotate->train USP (CUJ-9) is only partially real: the package should expose the
worker train CLI and document the MIT rally-annotator + CSV bridge, but must NOT present one-
click real-model training (synth=mock; real=GPU+in-flight native WASB+owner-gated ship-gate).
Honesty fences must survive into the packaged UX.",
      "All cloud/auth/cost serving features (edge-auth, cost ledger, cloud_run compute,
profiles) are DEFAULT-OFF; the downloadable app's default profile should remain truly-local
```

single-user so the package is byte-identical to the verified local CUJs unless the user opts into cloud/hosted mode.",
      "Edge-auth is the ONLY tenancy boundary for any non-localhost exposure (engine has zero app auth + CORS *). If the package can be exposed beyond 127.0.0.1, it must ship/document the gate (Cf-Access or reverse-proxy basicauth) + rotate keys + set security.allowed_video_root; default-bound to localhost otherwise."
    ],
    "risks": [
      "The packaged single-entrypoint must not reintroduce the backend.main __main__-vs-import module split (#281) ? a packaging launcher that runs the app differently than the tested `python -m backend.main --server` could silently hang every job in queued.",
      "Bundling the cross-repo khelsutra/web SPA build into the engine package risks drift: the SPA expects /api contracts (play_segments, download_url, /api/jobs result.video_id) ? a stale bundled UI would break CUJ-1/2/3 just like the dead backend/api/static app.js already does.",
      "Native WASB cloud GPU runs (CUJ-6/7) were verified on Singapore L4 with skip_ai_handoff=true; the exact TUNED config that reproduces 22 rallies on a fresh build was previously flagged as a missing piece ? packaging a 'native' profile risks shipping a config that yields 0 rallies if skip_ai_handoff/detector_impl wiring is wrong.",
      "kn/hi catalogs are AI-DRAFTED with native-speaker review still PENDING ? shipping the trilingual UI to non-test users before review is a quality/trust risk the owner flagged.",
      "Two torch CUDA stacks + GCS-hosted weights make a true 'download and run with GPU' artifact large and fragile; an over-ambitious all-in-one bundle could be unshippable, while a CPU-only bundle silently loses the best-quality (WASB) CUJ if not clearly surfaced.",
      "The annotate->train CUJ is easy to overstate; packaging copy/UX could imply a working owned model when the ship-gate refuses commercial ship and synth training is a mock ? violating the standing honesty + licensing guardrails.",
      "GCS/Gemini paths incur paid + cloud-upload costs with no per-video hard cost ceiling in the default path (billing ledger exists but is default-OFF and currently a no-op until usage emission is wired) ? a packaged app that defaults users into Gemini could surprise-bill them."
    ],
    "open_questions": [
      "Does the packaged app ship the BUILT khelsutra/web SPA bundle as the served UI, and which repo's CI owns producing/versioning that bundle so it stays in sync with the engine /api contract?",
      "Is native WASB (GPU) an in-scope CUJ for the downloadable app's first release, or is the batteries-included default explicitly CPU-only (Gemini/motion) with GPU/WASB as a documented optional add-on?",
      "What is the exact, reproducible truly-local native-WASB config/command (the one that produced 22 rallies) that the packaged 'native' profile should bake in, given the prior cold-start config-drift that yielded 0 rallies?",
      "How does the package resolve torch/torchvision (post-install hook? platform-pinned wheels? prompt the user?) given they are deliberately excluded from pyproject deps?",
      "Should the packaged worker train CLI + rally-annotator linkage be included at v1, or deferred, given the train path is a mock/GPU-gated and the ship-gate refuses commercial ship?",
      "Does the package need to bundle ffmpeg/ffprobe binaries per-platform, or assume them as a documented system prerequisite (current behavior)?",
      "For any cloud (GCS/Cf-Access/cost) feature in the package, do they remain strictly default-OFF opt-ins, and is exposure beyond localhost in-scope (requiring the bundled edge gate) or explicitly out-of-scope for v1?"
    ]
  },
  {
    "area": "Verification + licensing constraints on packaging",
    "summary": "Testing today is a fully-offline, fully-mocked pytest suite (110 test_*.py files) launched via run_all_tests.py, which is just a thin pytest wrapper that defaults to -q and reports PASS/FAIL by pytest exit code; there is NO real-video/GPU/network smoke test in CI (those are explicitly handed to the owner per CLAUDE.md and LOCAL_APPLIANCE L5). CI (.github/workflows/ci.yml) runs five lanes on Python 3.13: (1) ruff lint over the whole repo, (2) mypy on backend+training against a lenient baseline with a 14-entry quarantine, (3) an import-smoke lane that deliberately does NOT install cv2/torch/numpy and stubs them, (4) the full-suite lane that installs CPU-only torch out-of-band then pip install -e .[dev] and enforces --cov-fail-under=70 over backend+training, and (5) a cross-OS (ubuntu/windows/macos) golden-fixtures determinism guard. The coverage floor is 70% (calibrated 2026-06-11 against the obsreport-absent lane's measured 72%; local-with-obsreport ~73%) and must never go down without an owner decision. Any packaging change must keep all five lanes green: notably the import-smoke

lane proves backend.main imports with the GPU stack absent, and test_import_smoke.py enforces the ADR-008 \"import wall\" (serving never imports training) plus a pure-numpy/scipy featurizer constraint ? packaging must preserve these module boundaries. The licensing rule is human-policy-enforced (MIT/Apache-2.0/BSD only for code, data, AND weights; paid LLMs = inference/serving only, never a training source) with NO automated license-scanner in CI; the only code-enforced gate is the de-attribution/provenance refusal gate in the golden-fixtures + training-harness path (RefusalError, no_paid_llm_training_labels=True attestation), not a dependency scanner. On packaging tools: the live design (DESKTOP_APP_PLAN.md, superseded as a product surface by D10's self-hosted-webserver but its licensing analysis still authoritative) already locks D4 ? ship an LGPL ffmpeg build using openh264 (BSD) or OS hardware encoders, never statically-linked libx264 (GPL), which is the single biggest licensing landmine in the bundle. PyInstaller is named as the chosen backend bundler; conda's Anaconda-ToS friction is already real and worked around in deploy/cloudrun/Dockerfile (explicit `conda tos accept`).",
    "key_facts": [
      {
        "fact": "Test suite is fully offline + fully mocked (no GPU/network/WSL); run_all_tests.py is a thin pytest wrapper that defaults to -q and returns pytest's exit code; 110 test_*.py files in tests/.",
        "evidence": "run_all_tests.py:14-21,36-55; tests/ has 110 test_*.py"
      },
      {
        "fact": "CI has 5 lanes on Python 3.13: ruff lint (whole repo), mypy (backend+training, lenient + 14-module quarantine), import-smoke (NO cv2/torch/numpy installed), full-suite, golden-crossos (ubuntu/windows/macos).",
        "evidence": ".github/workflows/ci.yml:20-48,54-65,67-97,105-125; mypy.ini has 14 [mypy-/ignore_errors entries"
      },
      {
        "fact": "Coverage floor is --cov-fail-under=70 over backend+training; calibrated 2026-06-11 against the obsreport-absent lane's measured 72% (local-with-obsreport ~73%); ratchet UP only, never down without owner decision.",
        "evidence": ".github/workflows/ci.yml:90-97"
      },
      {
        "fact": "torch/torchvision are deliberately NOT in pyproject deps (CUDA/CPU wheels need --index-url which PEP 621 cannot express); the gpu extra is intentionally empty/docs-only; CI installs CPU torch out-of-band before pip install -e .[dev].",
        "evidence": "pyproject.toml:8-11,63-70; .github/workflows/ci.yml:80-88"
      },
      {
        "fact": "Licensing is enforced by human policy, NOT by any automated scanner ? grep finds no pip-licenses/liccheck/reuse in .github/, scripts/, tools/. The only code-enforced gate is the provenance/de-attribution RefusalError in the golden-fixtures + training harness.",
        "evidence": "no license-scanner in CI; backend/eval/golden_real_fixtures.py:59-110,162-220; training/gen0/harness.py:233-238 (no_paid_llm_training_labels=True)"
      },
      {
        "fact": "import-smoke lane + test_import_smoke.py enforce three packaging-relevant boundaries: backend.main imports with GPU stack stubbed; the ADR-008 import wall (serving never imports training.*, static + runtime); and backend.features must import without torch/cv2 (pure numpy/scipy).",
        "evidence": "tests/test_import_smoke.py:35-99,102-119"
      },
      {
        "fact": "ffmpeg licensing is the biggest bundling landmine: current stitch uses libx264 (GPL); D4 mandates an LGPL ffmpeg build with openh264 (BSD) or OS hardware encoders (NVENC/VideoToolbox/VAAPI), never statically-linked x264.",
        "evidence": "docs/DESKTOP_APP_PLAN.md:38,64,111"
      },
      {
        "fact": "PyInstaller (or a frozen venv sidecar) is the already-named backend bundler; the PEP 621 pyproject + indexer console entry point (DEFERRED #7, PR #167) is the head start.",
        "evidence": "docs/DESKTOP_APP_PLAN.md:63,99,133"
      },

```
      {
        "fact": "conda's Anaconda ToS friction is real and already worked around: the Cloud Run
worker image runs `conda tos accept` for main + r channels to build the py3.8/torch-1.11 WASB
env.",
        "evidence": "deploy/cloudrun/Dockerfile:33-43"
      },
      {
        "fact": "scipy is a serving-side runtime dep (backend/features/rally_seq.py,
tolerance_metrics.py) but is NOT declared in pyproject/requirements ? it arrives only
transitively via supervision (whose Requires lists scipy). A supervision change could silently
drop it.",
        "evidence": "pyproject.toml:27-37 (no scipy); backend/features/rally_seq.py imports
scipy; `pip show supervision` Requires includes scipy"
      },
      {
        "fact": "The reporting extra pins obsreport via git+ssh to a PRIVATE repo (allow-direct-
references=true); google-genai is a non-lazy hard dependency imported at module top of
backend/providers/gemini.py; PyJWT[crypto] auth extra is default-OFF/lazy.",
        "evidence": "pyproject.toml:46-61,84-87; backend/providers/gemini.py:6-7"
      }
    ],
    "packaging_implications": [
      "The packaged app must ship an LGPL ffmpeg build (openh264/BSD or OS hardware encoders),
NOT a libx264-statically-linked GPL build, or the whole bundle becomes GPL-contaminated ? this
is the dominant licensing constraint on any downloadable artifact (DESKTOP_APP_PLAN D4).",
      "PyInstaller is BSD/MIT-licensed (its bootloader is GPL-with-an-exception that explicitly
permits proprietary/closed-source frozen apps) so it is license-compatible for a downloadable
app. Nuitka is Apache-2.0 (compatible). micromamba (BSD) is the license-clean way to ship the
conda-based WASB env WITHOUT the Anaconda Distribution ToS that gates conda/Miniconda. Docker
Desktop has commercial-use ToS for larger orgs, but Docker Engine/the OCI images themselves are
Apache-2.0 and license-clean for distribution.",
      "The dual-torch problem (main env torch 2.6+cu124 vs WASB conda env torch 1.11+cu113)
means a single PyInstaller freeze cannot cover both; the precedent (Cloud Run Dockerfile) keeps
WASB in a separate conda env invoked as a subprocess via $WASB_PYTHON ? the downloadable app
needs the same two-env shape (e.g. micromamba-provisioned WASB env beside a frozen main app).",
      "Any new packaging dependency (the freezer, an installer framework, a license-clean ffmpeg
distribution, micromamba) must itself be MIT/Apache-2.0/BSD ? and since there is no automated
license scanner, this must be human-verified per dependency before adding it.",
      "scipy must be added as an EXPLICIT pyproject dependency before packaging ? it is serving-
required (backend/features) yet currently only transitive via supervision; a frozen bundle or a
supervision bump could silently omit it and break inference.",
      "The obsreport git+ssh private-repo dependency cannot be resolved by a non-repo
downloading user (no SSH key/access); it must stay a truly-optional extra (it already degrades
to a no-op via pytest.importorskip) and must NOT be bundled into the downloadable app.",
      "WASB weights (wasb_badminton_best.pth.tar) cannot be baked into the bundle: WASB-C3
provenance (academic-data-trained) is unresolved for commercial sharing ? they must be fetched
at runtime from a bucket, exactly as the Cloud Run image does (WASB_WEIGHTS env, runtime
stage).",
      "Packaging verification can ride the existing offline-mocked suite + the import-smoke lane
(which already proves GPU-stack-absent import and the training import wall); a NEW packaged-
artifact smoke test (e.g. `indexer --help` / server boot on the frozen binary) would be net-new
since CI today only tests source, not a built artifact. Any change must keep all 5 CI lanes
green and coverage >= 70%."
    ],
    "risks": [
      "GPL contamination via libx264 in the bundled ffmpeg is the highest-severity (legal) risk;
mitigated only by deliberately choosing an LGPL ffmpeg + openh264/hardware encoders ? easy to
get wrong because the current dev stitch path uses libx264.",
      "No automated license gate in CI means a license-incompatible packaging dependency (e.g. a
GPL installer helper, an Anaconda-ToS-encumbered conda channel) could slip in unnoticed; the
guardrail is purely human review.",
      "Anaconda Distribution ToS: using conda/Miniconda defaults (repo.anaconda.com main/r
channels) to build the WASB env carries commercial-use ToS exposure for orgs >200 employees; the
Dockerfile sidesteps it only by auto-accepting the ToS, which is a posture, not a license
clearance ? micromamba + conda-forge is the cleaner path.",
```

```
      "scipy being undeclared (transitive-only) is a latent break: a future supervision
pin/version that drops scipy, or a freeze that prunes unreferenced transitive deps, silently
removes a serving dependency.",
      "Two incompatible torch stacks make a single-binary freeze infeasible; underestimating
this (treating it as one PyInstaller target) would produce a broken bundle ? the separate-env
subprocess shape is mandatory.",
      "A frozen/bundled artifact is NOT exercised by today's CI (which tests source on Python
3.13 + CPU torch); behavior of the frozen binary (hidden imports, data files like
backend/assets/fonts, namespace-package resolution since backend has no __init__.py) is
unverified until a real built-artifact smoke test exists.",
      "WASB-C3 weight-provenance is ship-blocking for any non-owner serving (gate_verdict hard-
codes ship=False); a downloadable public app that bundles or auto-fetches those weights without
C3 resolution would violate the licensing guardrail."
    ],
    "open_questions": [
      "Is the freezer choice PyInstaller (named in DESKTOP_APP_PLAN) or a frozen-venv/micromamba
sidecar, given D10 pivoted from desktop-app to self-hosted-webserver ? does the webserver
framing reduce the need to freeze at all (could be `pipx install` / a downloadable wheel + a
bootstrap that provisions the WASB env)?",
      "Where will a license-clean LGPL ffmpeg build come from per-OS (BtbN LGPL Windows builds?
Homebrew/apt? a bundled redistributable), and who verifies it excludes GPL encoders?",
      "Will the downloadable app ship the WASB env via micromamba+conda-forge (license-clean) to
avoid the Anaconda Distribution ToS, and is that decision recorded anywhere yet?",
      "Should scipy (and the implicit transitive set) be pinned/declared explicitly before any
freeze, and is there appetite to add an automated license-scan CI lane (pip-licenses/reuse)
given the guardrail is currently human-only?",
      "Should CI gain a built-artifact smoke lane (freeze, then run `indexer --help` / boot the
server) so packaging regressions are caught, or does that stay an owner-side manual check like
the GPU/real-video paths?",
      "Does the coverage floor (70%) need to account for net-new packaging/bootstrap code, or
will that code be excluded from --cov to avoid dragging the floor?"
    ]
  }
]

=== RESEARCH (with verification verdicts) ===
[
  {
    "topic": "distribution-channels",
    "research": {
      "topic": "Distribution channels for a \"batteries-included\" Python app (console script
`indexer`, FastAPI server + worker, optional torch via --index-url, requires-python>=3.10) so a
non-technical Windows-first user can download and run it WITHOUT cloning the repo",
      "options": [
        {
          "name": "(a) PyPI wheel + pip install",
          "how_it_works": "Publish the existing hatchling-built wheel (badminton-highlight-
indexer, console script indexer = backend.main:main, packages=[backend,training], requires-
python>=3.10 per pyproject.toml:17-88) to PyPI. User runs `python -m pip install badminton-
highlight-indexer` into a venv/global env, then `indexer --server --port 8000`. Optional extras
(auth/storage-s3/storage-gcs) install via `pip install 'badminton-highlight-indexer[storage-
gcs]'`. torch is installed on a SEPARATE line: `pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu124` (or .../cpu) ? exactly as pyproject.toml:58-64 already
documents.",
          "pros": [
            "Zero new build tooling; the wheel hatchling already produces just gets uploaded. No
code change.",
            "Standard, universally understood; `pip install X` then `indexer` is the canonical
console_scripts UX.",
            "extras_require (the existing storage-s3/storage-gcs/auth/reporting groups) map 1:1
to pip extras ? already designed for this.",
            "Wheel is tiny (no torch baked in) so download is fast and the licensing surface is
unchanged (only adds PyPI, no new code dep)."
          ],
          "cons": [
```

```
            "Requires the user to already have a matching Python 3.10+ on PATH and to understand
venvs ? a real barrier for the non-tech India coach on Windows.",
            "Installing into the global/base env risks dependency clashes (this app pins
fastapi/uvicorn/opencv/supervision); pip does NOT isolate.",
            "torch's --index-url CANNOT live in PyPI metadata (PEP 621/508 limitation, confirmed
by pip issue #13000) ? the user MUST run a second, hand-typed pip command with the right CUDA
tag. This is the single biggest non-tech failure point.",
            "ffmpeg/ffprobe are system deps pip cannot deliver; opencv-python wheels are big and
occasionally need MSVC runtime on Windows.",
            "Cannot solve the WASB torch-1.11+cu113 second-env problem at all ? pip into one env
can't host two conflicting torch versions."
          ],
          "maturity": "mature"
        },
        {
          "name": "(b) pipx",
          "how_it_works": "User installs pipx, then `pipx install badminton-highlight-indexer` ?
pipx creates a dedicated venv, exposes the `indexer` console script on PATH, and isolates the
app's deps from everything else. torch is injected post-install: `pipx inject badminton-
highlight-indexer torch torchvision --pip-args='--index-url
https://download.pytorch.org/whl/cu124'` (pipx forwards --pip-args / PIP_INDEX_URL to pip,
confirmed pipx issue #1009 / discussion #1280).",
          "pros": [
            "True per-app isolation ? the app's pinned fastapi/opencv/supervision can never
clash with the user's other Python tools.",
            "Clean console-script UX: after install, `indexer --server` just works (this is
exactly pipx's design purpose ? isolated CLI tools).",
            "torch's index-url IS expressible via `pipx inject ... --pip-args='--index-url ...'`
or `PIP_INDEX_URL` env, so the hard torch case has a documented path.",
            "pip-compatible: reuses the same published wheel + extras; nothing new to build."
          ],
          "cons": [
            "Still needs a pre-existing Python 3.10+; pipx itself needs installing first
(chicken-and-egg) and the user must add ~/.local/bin (or the Windows Scripts dir) to PATH ? a
known beginner stumbling block.",
            "Two-step torch dance (`pipx install` then `pipx inject ... --pip-args`) is even
more arcane than pip's for a non-tech user.",
            "pipx is built around ONE venv per app ? it does not help with the WASB second-env
(torch 1.11/conda) requirement.",
            "ffmpeg still unsolved; Windows PATH friction remains."
          ],
          "maturity": "mature"
        },
        {
          "name": "(c) uv tool install / uvx",
          "how_it_works": "User installs uv via a single standalone command (`powershell
-ExecutionPolicy ByPass -c \"irm https://astral.sh/uv/install.ps1 | iex\"`) ? uv ships uv+uvx as
standalone binaries, auto-configures PATH, and can DOWNLOAD Python 3.10+ itself (no pre-existing
Python needed). Then `uv tool install badminton-highlight-indexer` (permanent `indexer` on PATH)
or `uvx --from badminton-highlight-indexer indexer --server` (ephemeral). For torch: because
`--torch-backend=auto` and `tool.uv.index`/`tool.uv.sources` only work in `uv pip`/project
workflows ? NOT in uv tool/uvx ? torch must be supplied at install via `uv tool install
badminton-highlight-indexer --with torch --with torchvision --index
https://download.pytorch.org/whl/cu124` (the `--index`/`--with` flags route the extra
packages).",
          "pros": [
            "Solves the #1 non-tech barrier: uv installs Python itself (auto-downloads the
required 3.10+), so the user does NOT need Python pre-installed ? huge for Windows-first non-
tech users.",
            "One PowerShell line installs uv with automatic PATH setup (no ~/.local/bin friction
that pipx has).",
            "10-100x faster installs than pipx; isolated per-tool venv like pipx.",
            "Can install directly from a hosted wheel URL or private index ? supports the
direct-wheel-download channel too.",
            "uv 0.5.3+'s `--torch-backend=auto` (in uv pip) is the cleanest torch-index auto-
```

```
detect available anywhere (detects CUDA/ROCm/Intel driver, falls back to CPU) ? usable in a
setup script even if not in `uv tool install` itself."
          ],
          "cons": [
            "CRITICAL torch gap: `--torch-backend` and `[tool.uv.index]`/`[tool.uv.sources]` do
NOT apply to `uv tool install`/`uvx` (verified: uv docs + multiple 2025 sources). So torch
routing in the tool-install path must use the lower-level `--with torch --index <url>`, which is
fragile and still hand-typed.",
            "uv is newer than pip/pipx; some users/orgs don't have it yet (mitigated by the one-
line installer).",
            "Does NOT solve the WASB torch-1.11/conda second-env problem (no tool can host two
torch ABIs in one venv).",
            "ffmpeg/ffprobe still system deps uv won't deliver.",
            "uvx ephemeral mode re-resolves on each run unless cached ? fine for CLIs, awkward
for a long-running server the user 'downloads and runs'."
          ],
          "maturity": "mature"
        },
        {
          "name": "(d) Private index / direct wheel download (URL or Artifact Registry)",
          "how_it_works": "Host the hatchling wheel on a static URL (GitHub Releases / GCS
bucket / Cloudflare Pages ? fits the khelsutra hosting) or a real private index (GCP Artifact
Registry, devpi, simple-index folder). Install with `pip install
https://.../badminton_highlight_indexer-0.1.0-py3-none-any.whl`, `pipx install <url>`, or `uv
tool install <url>`. torch index-url is layered on with the same per-channel mechanism (pip
second line / pipx inject / uv --with --index). A private index also lets you serve the
obsreport git+ssh reporting dep or pinned wheels you control.",
          "pros": [
            "No PyPI publication required (keeps the app unlisted / gated ? fits a paid/hosted
product and the owner's commercialization track).",
            "You control versions and can co-host the wheel next to the SPA download on
khelsutra infra (one download story for the user).",
            "Works through ALL of pip/pipx/uv ? it's a source, not a competing installer;
composes with whichever runner you recommend.",
            "An Artifact Registry index can also vend the torch wheels you've vetted (pin the
exact cu124 build) so the user never types --index-url ? wrap it in PIP_INDEX_URL / a config."
          ],
          "cons": [
            "Still just a wheel ? inherits every native-dep gap (Python pre-req, ffmpeg, torch
ABI split for WASB).",
            "A direct wheel URL skips dependency resolution unless paired with an index for the
deps (`--find-links`/`--extra-index-url`), so naive `pip install <url>` can fail to pull
fastapi/opencv.",
            "Private-index auth (keyring/Artifact Registry) is itself non-tech-hostile; best
hidden behind a generated install script.",
            "Maintaining a private index is ops overhead vs. just using PyPI."
          ],
          "maturity": "mature"
        },
        {
          "name": "(e) Offline / air-gapped install bundle (pip download wheelhouse) + optional
PyInstaller/installer wrapper",
          "how_it_works": "On a networked Windows box matching the target Python (3.10+), `pip
download badminton-highlight-indexer -d wheelhouse/` plus a second `pip download torch
torchvision --index-url https://download.pytorch.org/whl/cu124 -d wheelhouse/` (use --only-
binary=:all: / --platform / --python-version to pin the target). Ship wheelhouse/ + a one-click
`install.bat` that does `pip install --no-index --find-links wheelhouse badminton-highlight-
indexer torch torchvision`. Bundle ffmpeg.exe/ffprobe.exe and (optionally) an embeddable
CPython. The heavy alternative ? PyInstaller onefile/onedir bundling torch+CUDA ? produces a
2.6-5GB artifact and is fragile with opencv/torch hidden imports.",
          "pros": [
            "Best non-tech UX once built: an install.bat/installer the user double-clicks; no
Python, no PATH, no --index-url typing, works offline.",
            "Pins the EXACT torch+CUDA wheel and ffmpeg binaries the engine was tested with ?
removes the single most error-prone step for the India coach.",
```

```
            "Air-gapped/poor-connectivity friendly (relevant for the field persona); `pip
download` + `--no-index --find-links` is the documented standard path.",
            "Can layer the GCS model-weight fetch (wasb_badminton_best.pth.tar) into the same
first-run bootstrap."
          ],
          "cons": [
            "Wheelhouse must be built PER target (OS + Python minor + CPU vs each CUDA tag) ?
combinatorial; you ship a CPU bundle and a cu124 bundle separately.",
            "Large download (torch CUDA wheels alone are ~2.5GB+).",
            "PyInstaller route specifically: 2.6-5GB executables, brittle hidden-imports for
torch/opencv/supervision, and STILL cannot bundle the conda torch-1.11 WASB second env ? so it
does not actually deliver the hard CV path.",
            "ffmpeg licensing must be checked (LGPL/GPL builds) against the MIT/Apache/BSD-only
guardrail ? pick an LGPL-shared or BSD-licensed ffmpeg build and document it.",
            "Most engineering effort of the five options to build and maintain."
          ],
          "maturity": "emerging"
        }
      ],
      "load_bearing_claims": [
        {
          "claim": "torch/torchvision are deliberately omitted from [project.dependencies]
because their CPU/CUDA wheels need --index-url/--extra-index-url, which PEP 621 cannot express;
the app already documents installing them on a separate pip line (cu124 or cpu).",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/pyproject.toml:8-11,58-64"
        },
        {
          "claim": "console script is `indexer = backend.main:main`; build backend is hatchling;
backend is a namespace package so wheel packages are listed explicitly as [backend, training];
requires-python>=3.10; extras = dev/reporting(git+ssh)/gpu(empty)/storage-s3/storage-gcs.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/pyproject.toml:21,72-88"
        },
        {
          "claim": "Standard PEP 508 dependency specs cannot carry a custom index URL; pip has
an open feature request (#13000) to allow --extra-index-url in pyproject.toml, confirming the
limitation is real and unresolved.",
          "confidence": "high",
          "source": "https://github.com/pypa/pip/issues/13000"
        },
        {
          "claim": "uv's PyTorch index handling (tool.uv.index with explicit=true +
tool.uv.sources + platform markers, and the --torch-backend=auto/cpu/cuNNN flag that auto-
detects the GPU driver) is the cleanest solution to the torch index problem, but --torch-backend
and tool.uv index config ONLY work in the `uv pip` / project (lock/sync/run) interface ? NOT in
`uv tool install` or `uvx`. Requires uv >=0.5.3.",
          "confidence": "high",
          "source": "https://docs.astral.sh/uv/guides/integration/pytorch/"
        },
        {
          "claim": "uv installs as a standalone binary on Windows via one PowerShell line (`irm
https://astral.sh/uv/install.ps1 | iex`), auto-configures PATH, and can download the required
Python itself ? so the user does not need Python pre-installed; pipx by contrast uses whichever
Python is already present and requires manually adding its scripts dir to PATH.",
          "confidence": "high",
          "source": "https://docs.astral.sh/uv/getting-started/installation/"
        },
        {
          "claim": "pipx can install from a private index/direct wheel and route torch's index
via `--pip-args='--index-url ...'` or PIP_INDEX_URL, and `pipx inject <app> torch --pip-
args=...` adds torch into the app's isolated venv post-install.",
          "confidence": "high",
```

```
          "source": "https://github.com/pypa/pipx/issues/1009"
        },
        {
          "claim": "Offline install is standard via `pip download -d wheelhouse` (with --only-
binary=:all:/--platform/--python-version to target the destination interpreter) then `pip
install --no-index --find-links wheelhouse <pkgs>`; you must run pip download from the SAME
Python minor version as the target machine.",
          "confidence": "high",
          "source": "https://pip.pypa.io/en/stable/cli/pip_download/"
        },
        {
          "claim": "PyInstaller bundling torch+CUDA yields 2.6GB (onefile, pip cu118) to ~5GB
artifacts and is fragile with hidden imports; CUDA/cuDNN shared libs cannot be shrunk ? making a
single-exe with real GPU CV impractical.",
          "confidence": "high",
          "source": "https://github.com/orgs/pyinstaller/discussions/8552"
        },
        {
          "claim": "The live appliance design (owner decision D10) is a self-hosted webserver+UI
the user spins up (NOT a desktop app), bound to 127.0.0.1:8000, driving the existing FastAPI
loop and `python -m backend.worker {infer|train}` dispatcher; storage=local default; compute
seam detector_impl ? {auto=WSL, native, stub}.",
          "confidence": "high",
          "source":
"C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:12-46 and
backend/main.py:638, backend/worker/__main__.py:1-15"
        },
        {
          "claim": "The hard packaging problem ? native WASB shuttle model needs torch
1.11+cu113 in a SEPARATE conda env while the main engine uses torch 2.6+cu124 ? cannot be solved
by ANY single-venv installer (pip/pipx/uv tool) nor by a single PyInstaller bundle; it requires
a second managed environment or container.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:96
(native WASB conda env makes the image non-trivial)"
        }
      ],
      "recommendation": "Lead with (c) uv tool install, fronted by a generated one-line install
script, and back it with (d) a private/direct wheel source; keep (a) PyPI-wheel+pip as the
documented power-user fallback and ship (e) an offline CPU bundle for the field/air-gapped
persona. Reasoning: the dominant non-tech-Windows barrier is \"does the user have a correct
Python and can they type the torch --index-url line.\" uv is the only channel that (1) installs
itself in one PowerShell line with automatic PATH, (2) downloads a correct Python 3.10+ so the
user needs nothing pre-installed, and (3) is fast and per-app isolated. Because `--torch-
backend=auto` and `tool.uv.index` do NOT work in `uv tool install`/`uvx` (they are `uv
pip`/project-only), do NOT rely on `uv tool install <app>` to also fetch torch. Instead ship a
thin generated installer (install-indexer.ps1) that: (i) bootstraps uv, (ii) runs `uv tool
install --from <your-wheel-url-or-private-index> badminton-highlight-indexer`, (iii) installs
the matching torch into that tool's environment using `uv pip install --torch-backend=auto torch
torchvision` against the tool's venv (or, in a uv project layout, sets tool.uv.index+sources and
runs uv sync), (iv) drops ffmpeg/ffprobe next to the venv and exposes them on PATH, (v) on first
`indexer --server` run fetches wasb_badminton_best.pth.tar from GCS. The script makes the torch
index-url problem invisible ? the user double-clicks one .ps1. Do NOT pursue PyInstaller onefile
for the GPU path (2.6-5GB, brittle, still can't host the WASB torch-1.11 conda env). The WASB
second-env stays a separate concern: gate real local-GPU detection behind a SEPARATE managed env
(conda/uv venv #2) or a container, and ship the truly-local default as Gemini/stub/CPU-torch so
the one-click install always yields a working appliance even on a CPU-only Windows box.",
      "fit_for_this_app": "Dual-torch reality (engine torch 2.6+cu124 vs WASB conda torch
1.11+cu113, per LOCAL_APPLIANCE_ARCHITECTURE.md:96) is the decisive constraint: NO single-venv
channel (pip/pipx/uv tool) and NO single PyInstaller bundle can host both ABIs ? so the
distributable must install the MAIN engine env one way and provision the WASB env as a separate
managed venv/conda/container, exactly as the appliance doc already treats native WASB as a
separate GPU-host concern. The ffmpeg/ffprobe system-dep gap (none of pip/pipx/uv deliver it)
must be solved by the installer bundling a license-clean (LGPL-shared or BSD) ffmpeg build ? and
that ffmpeg binary is itself a new \"dependency\" subject to the MIT/Apache/BSD-only guardrail,
```

so vet it. uv's auto-Python-download directly serves the non-tech India coach on Windows who likely has no Python; the GCS-hosted weights (wasb_badminton_best.pth.tar) fit a first-run bootstrap, and a private/direct-wheel source (d) keeps the app unlisted to match the commercialization/hosted-product posture rather than publishing to public PyPI. The existing hatchling wheel + extras (storage-s3/storage-gcs/auth, console script indexer=backend.main:main) need ZERO repackaging to feed any of these channels ? only torch, ffmpeg, the WASB second env, and the SPA bundle (a cross-repo khelsutra/web concern) are out-of-band. The delivered artifact must preserve every shipped CUJ: `indexer --server --port 8000` (FastAPI loop), `python -m backend.worker {infer|train}`, and the single-shot CLI with --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion} ? all of which are console-script/module entrypoints any of these channels expose unchanged."
    },
    "verdict": {
      "claims_checked": [
        {
          "claim": "torch/torchvision are deliberately omitted from [project.dependencies] because their CPU/CUDA wheels need --index-url/--extra-index-url, which PEP 621 cannot express; the app already documents installing them on a separate pip line (cu124 or cpu).",
          "verdict": "confirmed",
          "reasoning": "Verified directly against pyproject.toml. The file header (lines 8-11) states verbatim: 'torch/torchvision are deliberately NOT listed in [project.dependencies]: their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url), which PEP 621 cannot express.' The [project.dependencies] list (lines 27-37) indeed omits torch/torchvision. The optional-dependencies 'gpu' group (lines 58-65) is empty and documents the exact separate pip lines: 'pip install torch>=2.12.0 torchvision>=0.27.0 --index-url https://download.pytorch.org/whl/cu124' and a cpu line. CI installing CPU torch separately is also stated (line 11). Minor note: the cu124 example pins torch>=2.12.0, not 2.6 ? but the claim only says 'cu124 or cpu', which holds."
        },
        {
          "claim": "console script is `indexer = backend.main:main`; build backend is hatchling; backend is a namespace package so wheel packages are listed explicitly as [backend, training]; requires-python>=3.10; extras = dev/reporting(git+ssh)/gpu(empty)/storage-s3/storage-gcs.",
          "verdict": "confirmed",
          "reasoning": "Every sub-claim verified against pyproject.toml: [project.scripts] indexer = 'backend.main:main' (line 77; backend/main.py has def main() at line 556); build-backend = 'hatchling.build' (line 14); backend is a namespace package (header lines 3-6) so [tool.hatch.build.targets.wheel] packages = ['backend', 'training'] (line 88); requires-python = '>=3.10' (line 21); optional-dependencies are exactly dev, reporting (obsreport @ git+ssh://..., line 55), gpu = [] (empty, line 65), storage-s3 = ['boto3>=1.34'], storage-gcs = ['google-cloud-storage>=2.16'] (lines 41-73). All accurate."
        },
        {
          "claim": "Standard PEP 508 dependency specs cannot carry a custom index URL; pip has an open feature request (#13000) to allow --extra-index-url in pyproject.toml, confirming the limitation is real and unresolved.",
          "verdict": "uncertain",
          "reasoning": "The core technical fact is CONFIRMED: PEP 508/621 dependency specs cannot carry a custom index URL (corroborated by pip issue #13000 itself, the uv PyTorch guide's whole rationale, and the project's own pyproject.toml header). HOWEVER the claim's characterization of #13000 is wrong on two points: (1) its title is 'Add --extra-index-url option to pyproject.toml to install requirements.txt' ? about extending setuptools dynamic config, framed via requirements.txt; (2) it is CLOSED as not planned / 'needs standard' (a PEP-level change is required), NOT 'open'. So the limitation is real, but the claim that the issue is 'open' is refuted, and 'unresolved' is true only in the sense that it was declined pending standardization, not actively tracked-open. Marked uncertain because the load-bearing fact (the limitation) is solid but the cited-status detail is inaccurate."
        },
        {
          "claim": "uv's PyTorch index handling (tool.uv.index with explicit=true + tool.uv.sources + platform markers, and the --torch-backend=auto/cpu/cuNNN flag that auto-detects the GPU driver) is the cleanest solution to the torch index problem, but --torch-backend and tool.uv index config ONLY work in the `uv pip` / project (lock/sync/run) interface ? NOT in `uv tool install` or `uvx`. Requires uv >=0.5.3.",
          "verdict": "confirmed",

```
          "reasoning": "Verified against docs.astral.sh/uv/guides/integration/pytorch. The doc
states: 'At present, --torch-backend is only available in the uv pip interface' (so NOT uv
lock/sync/run, NOT uv tool install/uvx). --torch-backend=auto 'will query for the installed CUDA
driver, AMD GPU versions, and Intel GPU presence' ? confirms GPU-driver auto-detection. Valid
values include auto/cpu/cuNNN (e.g. cu130) plus rocm/xpu. tool.uv.index with explicit=true +
tool.uv.sources is the documented project-config route ('explicit=true to ensure that the index
is only used for torch, torchvision...'). 'Some of the features outlined in this guide require
uv version 0.5.3 or later.' One precision nuance: the claim lumps 'lock/sync/run' under 'project
interface' as places --torch-backend works ? but the doc says --torch-backend works ONLY in uv
pip; in project mode you use tool.uv.index/sources instead (NOT --torch-backend). The
recommendation text under test gets this exactly right (it says do NOT rely on uv tool install
for torch and uses tool.uv.index+sources for the project layout). The claim's headline boundary
(NOT in uv tool install/uvx, requires >=0.5.3) is correct."
        },
        {
          "claim": "uv installs as a standalone binary on Windows via one PowerShell line (`irm
https://astral.sh/uv/install.ps1 | iex`), auto-configures PATH, and can download the required
Python itself ? so the user does not need Python pre-installed; pipx by contrast uses whichever
Python is already present and requires manually adding its scripts dir to PATH.",
          "verdict": "confirmed",
          "reasoning": "Confirmed with one caveat. uv standalone installer: 'downloads a self-
contained uv binary and requires no existing Python interpreter, no package manager, and no
other prerequisites'; the installer 'may update your shell profiles' to put uv on PATH (disable
via UV_NO_MODIFY_PATH); and 'by default uv will automatically download Python versions when they
are required' ? so no Python pre-install needed. The PowerShell one-liner is the documented
Windows install. pipx does run on an existing Python and historically required 'pipx ensurepath'
to add its bin dir to PATH. CAVEAT: pipx's PATH step is one command (`pipx ensurepath`), not
literally hand-editing PATH; 'requires manually adding its scripts dir to PATH' is loose but
directionally correct (pipx does not auto-modify PATH on plain install the way uv's installer
does, and pipx cannot bootstrap its own Python). Net: the uv-side facts are fully accurate and
the pipx contrast is substantively true."
        },
        {
          "claim": "pipx can install from a private index/direct wheel and route torch's index
via `--pip-args='--index-url ...'` or PIP_INDEX_URL, and `pipx inject <app> torch --pip-
args=...` adds torch into the app's isolated venv post-install.",
          "verdict": "uncertain",
          "reasoning": "The CAPABILITY described is real and confirmed by pipx's own docs: pipx
supports --pip-args on both install and inject (e.g. --pip-args='--index-url ...'), exposes an
--index-url option on install/inject, honors pip env vars like PIP_INDEX_URL, and `pipx inject
<app> <pkg>` adds a package into the app's isolated venv. BUT the CITED SOURCE is
wrong/contradictory: pipx issue #1009 is a feature request titled to add --extra-index-url to
pipx and (as of June 2023) documents that pipx 'does not support the argument --extra-index-url'
? i.e. it evidences a GAP, not the stated workarounds. The claim's mechanisms are true per
pipx.pypa.io/stable/how-to/inject-packages and the pipx CLI reference, but #1009 does not
support them and partially contradicts them. Marked uncertain: substantively true, source
citation invalid ? re-cite to the pipx inject/how-to docs."
        },
        {
          "claim": "Offline install is standard via `pip download -d wheelhouse` (with --only-
binary=:all:/--platform/--python-version to target the destination interpreter) then `pip
install --no-index --find-links wheelhouse <pkgs>`; you must run pip download from the SAME
Python minor version as the target machine.",
          "verdict": "uncertain",
          "reasoning": "First half CONFIRMED by the pip download docs: 'pip download ...
collects the downloaded distributions into the directory ... later passed as the value to pip
install --find-links to facilitate offline ... package installation', and '--platform, --python-
version, --implementation, and --abi ... fetch dependencies for an interpreter and system other
than the ones pip is running on', with '--only-binary=:all: or --no-deps is required when using
any of these options'. The SECOND half is the problem: the docs explicitly enable downloading
for a DIFFERENT interpreter/platform (examples use --python-version 27, 3), which DIRECTLY
CONTRADICTS 'you must run pip download from the SAME Python minor version as the target
machine.' The same-minor-version rule is only true if you do a plain `pip download` WITHOUT the
targeting flags (then it resolves for the running interpreter). With --only-binary=:all: +
--python-version + --platform + --abi you can target another interpreter from any host. So the
```

claim's stated constraint is over-broad/misleading as written; the safer true statement is 'either match the target interpreter, OR use --only-binary=:all: with the targeting flags.' Marked uncertain due to this refuted constraint sentence.",
          "corrected_claim": "Offline install is standard via `pip download -d wheelhouse` then `pip install --no-index --find-links wheelhouse <pkgs>`. By default pip download resolves for the interpreter/platform it is running on (so a plain download must be run on a matching Python). To target a DIFFERENT interpreter/OS, add --only-binary=:all: together with --platform/--python-version/--implementation/--abi (these are required when fetching for another interpreter)."
        },
        {
          "claim": "PyInstaller bundling torch+CUDA yields 2.6GB (onefile, pip cu118) to ~5GB artifacts and is fragile with hidden imports; CUDA/cuDNN shared libs cannot be shrunk ? making a single-exe with real GPU CV impractical.",
          "verdict": "confirmed",
          "reasoning": "Verified against PyInstaller discussion #8552. A maintainer states 'Even onefile build with pip-installed torch2.3+cu118 seems to come out at 2.6GB', notes the conda torch2.3+cu118 is ~5GB with the packaged torch lib ~4GB, and that 'binary extensions and CUDA/cuDNN shared libs ... cannot be split' constitute most of the size, so small executables 'would require either CPU-only torch builds or older CUDA versions' ? 'Those two aspects are mutually exclusive' for CUDA-enabled builds. The 2.6GB-to-~5GB range, the cu118 reference, the un-shrinkable CUDA/cuDNN libs, and the impracticality conclusion all check out. (The torch version in the thread is 2.3, not stated in the claim ? immaterial. 'Fragile with hidden imports' is a well-known PyInstaller+torch property and consistent with the thread's spirit though the specific size facts are the load-bearing part.)"
        },
        {
          "claim": "The live appliance design (owner decision D10) is a self-hosted webserver+UI the user spins up (NOT a desktop app), bound to 127.0.0.1:8000, driving the existing FastAPI loop and `python -m backend.worker {infer|train}` dispatcher; storage=local default; compute seam detector_impl ? {auto=WSL, native, stub}.",
          "verdict": "confirmed",
          "reasoning": "Verified against both the design doc and engine source. LOCAL_APPLIANCE_ARCHITECTURE.md D10 (line 39): 'Delivery = a self-hosted webserver + UI the user spins up (NOT a desktop app) ... supersedes DESKTOP_APP_PLAN.md.' Engine bind: backend/main.py:735 `uvicorn.run(app, host=\"127.0.0.1\", port=args.port)` with --port default 8000 (line 563) ? confirms 127.0.0.1:8000 (cited as main.py:638, which is within the main() run block; the actual uvicorn.run line is 735 ? line number slightly off but the fact holds). Worker dispatcher backend/worker/__main__.py docstring (lines 3-5) and cmd_infer (line 156)/cmd_train (line 331) confirm `python -m backend.worker {infer|train}`. storage.backend default 'local' and detector_impl ? {auto=WSL, native=Linux-native, stub=CPU mock} default 'auto' are verified in the doc's [verified models.py:298 / :129] anchors and §4.3. All substantive facts confirmed; only the precise main.py line cite is imprecise."
        },
        {
          "claim": "The hard packaging problem ? native WASB shuttle model needs torch 1.11+cu113 in a SEPARATE conda env while the main engine uses torch 2.6+cu124 ? cannot be solved by ANY single-venv installer (pip/pipx/uv tool) nor by a single PyInstaller bundle; it requires a second managed environment or container.",
          "verdict": "uncertain",
          "reasoning": "The architectural CONCLUSION is sound and well-supported: the design doc (§4.3 line 96) confirms native WASB needs its torch-1.11 conda env and that this 'makes the image non-trivial', and the engine's own worker note (backend/worker/__main__.py:104) confirms a torch-version split between the engine and detection path. Two incompatible torch builds genuinely cannot coexist in ONE venv (a single site-packages can hold only one torch), and a single PyInstaller bundle freezes one interpreter/site-packages ? so 'requires a second managed env or container' is technically correct. HOWEVER the SPECIFIC version pins '1.11+cu113' and 'torch 2.6+cu124' are NOT verified against engine source in this pass ? the doc references the conda env without those exact version strings, and the project's own pyproject.toml gpu-extra example pins torch>=2.12.0 (cu124), not 2.6. The cited source line (line 96) supports the 'non-trivial image / second env' point but does NOT state '1.11+cu113' or '2.6+cu124'. Marked uncertain because the load-bearing conclusion holds but the exact version facts are unconfirmed and the engine's documented version (>=2.12 in pyproject, '2.6' in the claim) appears inconsistent ? verify the WASB env's actual torch/cu pins before relying on them."
        }

```
      ],
      "overall_notes": "Summary verdicts: 5 CONFIRMED (a, b, d, e, h), 4 UNCERTAIN (c, f, g, i ?
none fully refuted, but each has an inaccurate detail or invalid citation), 0 cleanly REFUTED.
The recommendation's CORE reasoning survives adversarial review: the torch --index-url barrier
is real (a, c), uv is genuinely the best one-line/no-Python-needed/PATH-auto Windows channel (d,
e), --torch-backend/tool.uv.index do NOT work in `uv tool install`/`uvx` so the recommendation
correctly does NOT rely on that (d), PyInstaller-onefile-GPU is genuinely impractical
(g/PyInstaller), and the WASB second-env / containerization is a real separate concern (h, i).
The recommendation text under test is notably MORE careful than some of the claims ? it
correctly avoids the trap of expecting `uv tool install <app>` to also fetch torch.\n\nCaveats
the owner should fix before treating these as load-bearing:\n1. CLAIM c ? pip issue #13000 is
CLOSED-as-not-planned (needs PEP standardization), not 'open', and its title is about setuptools
dynamic config / requirements.txt, not literally 'allow --extra-index-url in pyproject.toml'.
The underlying PEP 621 limitation is nonetheless real and independently confirmed.\n2. CLAIM f ?
the pipx capabilities (--pip-args='--index-url ...', --index-url, PIP_INDEX_URL, `pipx inject`)
are REAL per pipx.pypa.io/stable/how-to/inject-packages and the CLI reference, but the CITED
source (pipx issue #1009) is the WRONG citation: it is a 2023 feature request that says pipx did
NOT support --extra-index-url. Re-cite to the pipx inject how-to docs.\n3. CLAIM g ? the 'must
run pip download from the SAME Python minor version as the target' clause is REFUTED by the pip
docs: with --only-binary=:all: plus --platform/--python-version/--implementation/--abi you can
fetch for a DIFFERENT interpreter. Reword (corrected_claim provided). The offline workflow
itself (pip download ? pip install --no-index --find-links) is correct. This actually
STRENGTHENS the recommendation's offline-CPU-bundle channel (e) ? you can build the wheelhouse
for the field box's Python from any host.\n4. CLAIM i ? the exact torch pins ('1.11+cu113' for
WASB, '2.6+cu124' for the engine) were NOT verified against engine source; the cited doc line
says only 'non-trivial image / conda env'. The project's pyproject gpu-extra example actually
pins torch>=2.12.0 cu124 (not 2.6) ? a possible inconsistency. The architectural conclusion (two
incompatible torch builds => second managed env/container, unsolvable by any single
venv/PyInstaller bundle) is sound regardless.\n5. Minor line-number drift: the appliance doc
cites backend/main.py:638 for the 127.0.0.1 bind; the actual uvicorn.run(host='127.0.0.1') is at
main.py:735 (638 is within main()'s setup block). Fact holds; cite is imprecise.\n\nFiles
inspected (absolute): C:/Users/avidu/Projects/badminton-highlight-indexer/pyproject.toml;
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py (lines ~556, 563, 630-645,
735); C:/Users/avidu/Projects/badminton-highlight-indexer/backend/worker/__main__.py (lines
1-15, 104, 156, 331); C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md
(D10 line 39, §4.3 line 96, verified anchors). Web sources verified:
docs.astral.sh/uv/guides/integration/pytorch, docs.astral.sh/uv/getting-started/installation +
install-python guide, github.com/pypa/pip/issues/13000, github.com/pypa/pipx/issues/1009 (found
to be a mis-citation), pipx.pypa.io inject docs, pip.pypa.io pip_download docs,
github.com/orgs/pyinstaller/discussions/8552."
    }
  },
  {
    "topic": "containerization",
    "research": {
      "topic": "Docker / docker-compose as the \"batteries-included\" packaging unit for the
rally-indexer appliance (ffmpeg + dual-torch + FastAPI + worker + SQLite + bundled SPA),
targeting a non-technical coach on Windows",
      "options": [
        {
          "name": "A. Single fat image, two Python envs co-resident (extend the existing
deploy/cloudrun/Dockerfile)",
          "how_it_works": "One image FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04. Main app env =
system python3 + `pip install .` (torch 2.6+cu124 added out-of-band, since pyproject
deliberately omits torch per pyproject.toml:8-11,58-65). A SECOND, isolated interpreter holds
the WASB stack (py3.8 + torch 1.11+cu113) installed via micromamba (Apache-2.0) instead of
Miniconda. The native runner shells out to that interpreter via $WASB_PYTHON ? exactly the seam
the repo already ships: deploy/cloudrun/Dockerfile:54-57 sets
WASB_PYTHON/WASB_REPO_DIR/WASB_WEIGHTS and deploy/digitalocean/gpu_setup.sh:40-49 builds the
same conda env. The two torches NEVER co-install (framework conflict, called out at
Dockerfile:8-12). ffmpeg/ffprobe/fonts-dejavu installed via apt (Dockerfile:27-30). Compose runs
ONE service for the API+in-proc worker; weights mounted at runtime (NOT baked ? WASB-C3,
Dockerfile:14-16).",
          "pros": [
            "Pattern is ALREADY PROVEN in-repo (deploy/cloudrun/Dockerfile builds + the M7 owner
```

```
run): lowest net-new risk, reuses the verified $WASB_PYTHON subprocess seam",
            "Matches the engine's actual architecture: in-process worker is a single
ThreadPoolExecutor(max_workers=1) (LOCAL_APPLIANCE_ARCHITECTURE.md D3) ? API and worker are the
SAME process today, so one container is the honest unit",
            "One image = one `docker compose up`, one thing for a non-tech user to pull; no
inter-container networking/shared-volume choreography to debug",
            "Both Python envs see the same GPU through one --gpus all passthrough; no GPU-
sharing-across-containers question",
            "Swapping Miniconda -> micromamba removes the Anaconda-defaults-channel ToS friction
(gpu_setup.sh:32-33 had to auto-accept ToS) and is licensing-clean (BSD-3 / part of conda-forge
ecosystem)"
          ],
          "cons": [
            "Large image: a CUDA-runtime + torch image is realistically 5-8 GB (PyTorch alone
~2.5 GB bundled CUDA; nvidia/cuda runtime base + two torch stacks pushes well past that) ? a
painful download for a coach on Indian home broadband",
            "Two torch toolkits (cu124 + cu113) inflate it further; the WASB cu113 stack is the
long pole and is the licensing-AMBER piece (weights not bakeable)",
            "Rebuilds are slow; any main-env dep bump rebuilds layers above the (cached) conda
layer",
            "A crash/OOM in one env can take down the whole container (no blast-radius isolation
between training, inference, and the WASB subprocess)",
            "Multi-version-in-one-image is widely regarded as an anti-pattern for clean
separation, though acceptable for a coupled subprocess like this"
          ],
          "maturity": "mature"
        },
        {
          "name": "B. Separate service containers: api+worker image (torch 2.6) and a dedicated
wasb-detector image (torch 1.11), wired by docker-compose",
          "how_it_works": "Two images. `app` = CUDA-runtime + main env + ffmpeg + FastAPI +
worker + bundled SPA. `wasb` = py3.8 + torch 1.11+cu113 + the WASB-SBDT repo, exposing the
detector over a thin RPC/CLI-over-mount. compose.yaml wires them on a private network, both
claiming the GPU via the deploy.resources.reservations.devices block (driver: nvidia, count:
all, capabilities: [gpu]). SQLite, the user's videos, and weights live in named volumes / bind
mounts shared where needed. SPA served by the app container (or a tiny static/proxy
container).",
          "pros": [
            "Clean dependency isolation ? the cu124 and cu113 stacks never share a filesystem;
each image rebuilds independently",
            "Conventional microservice shape; each image individually smaller and independently
cacheable/pullable",
            "Lets training, inference, and detection scale/restart independently later (maps to
the engine's reserved bucket-worker bridge, LOCAL_APPLIANCE_ARCHITECTURE.md §4.3/T14)"
          ],
          "cons": [
            "The engine has NO network boundary between worker and detector today ? the native
runner is an IN-PROCESS subprocess call ($WASB_PYTHON), so splitting it across containers is
NET-NEW plumbing (an RPC or shared-mount protocol) the codebase does not have; contradicts the
in-proc worker reality (job_worker.py max_workers=1)",
            "Two containers both grabbing --gpus all on a single consumer GPU = contention/VRAM
thrash; the coach has one GPU, so the isolation buys little and adds failure modes",
            "More moving parts for a non-tech user: two images to pull, a compose network,
volume sharing between containers for frames/CSVs ? more to go wrong silently",
            "Total disk footprint is often LARGER than option A (two CUDA bases) despite each
image being smaller",
            "Over-engineered for a single-box single-tenant appliance (the design is explicitly
single-tenant alpha, LOCAL_APPLIANCE_ARCHITECTURE.md §7)"
          ],
          "maturity": "mature"
        },
        {
          "name": "C. CPU-only default image + optional GPU overlay (compose profiles), GPU/WASB
gated behind a separate pull",
          "how_it_works": "Ship a SMALL CPU-only image as the default: FROM python:3.11-slim (or
```

```
ubuntu:22.04), ffmpeg via apt, `pip install . torch --index-url .../whl/cpu`, FastAPI + worker +
SPA. Real detection here = Gemini (cloud) or stub/motion ? which is exactly what the live
appliance design already targets for the CPU droplet (LOCAL_APPLIANCE_ARCHITECTURE.md §4.3:
gemini=REAL, stub=mock, motion=low-q). GPU + native WASB is an OPT-IN: a `gpu` compose profile
pulling the heavy CUDA image (option A's image) only for users who have an NVIDIA GPU.
`/api/capabilities` (net-new, T7) tells the SPA which modes the box can honestly do (D6).",
          "pros": [
            "Small default download (CPU torch is far lighter; image plausibly ~1-2 GB vs 5-8
GB) ? realistic for a non-tech coach on Windows whose machine likely has NO NVIDIA GPU anyway",
            "Honest match to most coaches' hardware: cloud-Gemini path needs no GPU and is the
engine's REAL default segmenter (config default_segmenter=gemini)",
            "No nvidia-container-toolkit, no GPU drivers, no WSL2-GPU paravirtualization
required for the common case ? removes the single biggest Windows friction point",
            "GPU users (the owner, future power users) get the full option-A image via
`--profile gpu`; capabilities endpoint keeps the UI truthful so a no-GPU box never offers WASB",
            "Lowest-friction first-run = fastest path to preserving the shipped Gemini/stub
CUJs"
          ],
          "cons": [
            "Real on-device WASB detection requires the second (heavy) pull anyway ? doesn't
eliminate option A, it defers it",
            "Two image variants to build/test/publish in CI (CPU + GPU); CI today does NOT build
the CUDA image (deploy/cloudrun/README.md:9 ? too heavy)",
            "Gemini path sends ?1280px chunks to Google ? must be in consent copy
(LOCAL_APPLIANCE_ARCHITECTURE.md §4.3) and needs a key + network; not truly 'local'",
            "Capabilities gating (T7) is net-new and currently unbuilt"
          ],
          "maturity": "emerging"
        },
        {
          "name": "D. No Docker on the client at all ? native installer / one-click launcher
(Docker only for the cloud/GPU tier)",
          "how_it_works": "Recognize that Docker Desktop is itself the friction for a non-tech
Windows user (install, WSL2 enablement, a daemon to keep running, licensing). Instead ship a
native bootstrapper (e.g. a signed Windows launcher that provisions an embedded Python via
uv/conda-pack, installs ffmpeg, runs `indexer --server`) for the LOCAL tier, and reserve Docker
strictly for the cloud/GPU worker (the existing deploy/cloudrun/Dockerfile, unchanged). The SPA
+ FastAPI run as a plain local process bound to 127.0.0.1:8000.",
          "pros": [
            "Zero container friction for the target user; closest to owner decision D10 ('a
self-hosted webserver + UI the user spins up', NOT a desktop app,
LOCAL_APPLIANCE_ARCHITECTURE.md D10)",
            "No Docker Desktop licensing exposure, no WSL2-GPU setup, no multi-GB image pull",
            "Docker still used where it shines (the ephemeral, scale-to-zero Cloud Run GPU job ?
deploy/cloudrun, already mock-tested M6)"
          ],
          "cons": [
            "Loses 'batteries-included' reproducibility ON THE CLIENT: the dual-torch + ffmpeg +
WASB-conda mess that Docker solves cleanly now has to be reproduced by a native installer on
Windows ? historically the hardest part (the whole reason the cloudrun image splits the envs)",
            "WASB on native Windows needs WSL2 anyway (detector_impl=auto = WSL2+conda,
LOCAL_APPLIANCE_ARCHITECTURE.md §4.3) ? so the 'no Docker' win evaporates for real local GPU
detection",
            "Out of scope for THIS task (the question is specifically about Docker as the unit),
but is the honest fork in the road and must be surfaced",
            "Native packaging is its own large project; DESKTOP_APP_PLAN.md was already
superseded by D10"
          ],
          "maturity": "risky"
        }
      ],
      "load_bearing_claims": [
        {
          "claim": "The repo ALREADY ships the two-Python-envs-in-one-image pattern:
deploy/cloudrun/Dockerfile FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04, builds a Miniconda
```

`wasb` env (py3.8 + torch 1.11.0+cu113) alongside the system-python main app env, and the native runner shells out via $WASB_PYTHON (ENV at Dockerfile:54-57). The two torches deliberately never co-install (Dockerfile:8-12).",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/deploy/cloudrun/Dockerfile:18,32-57"
        },
        {
          "claim": "WASB pretrained weights are NOT baked into the image (provenance not cleared for commercial sharing, WASB-C3) ? they must be mounted or fetched from a bucket at runtime and pointed to by $WASB_WEIGHTS. This is a hard volume/mount requirement for the appliance, not optional.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/deploy/cloudrun/Dockerfile:14-16,56"
        },
        {
          "claim": "The FastAPI server HARD-binds host=127.0.0.1 (backend/main.py:735), exposed via `--server`/`--port` flags (562-563). Inside a container 127.0.0.1 is the container loopback, NOT reachable from the Windows host ? the appliance image must bind 0.0.0.0 (and rely on the firewall/edge gate for safety, as the design's invariant assumes a 127.0.0.1 bind). This is a real engine change the Dockerized appliance forces.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py:735,562-563"
        },
        {
          "claim": "torch/torchvision are deliberately absent from pyproject dependencies because CUDA/CPU wheels need --index-url which PEP621 cannot express; the `gpu` extra is intentionally empty/docs-only. So ANY image must `pip install .` then add torch on a separate line with the right index ? a CPU image and a GPU image diverge only on that one torch line.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/pyproject.toml:8-11,58-65"
        },
        {
          "claim": "A CUDA-runtime + PyTorch Docker image is realistically multi-GB: a naive CUDA+python+torch image measured ~7.6 GB, optimizable to ~2.9 GB; PyTorch bundles ~2.5 GB of CUDA on its own. Two torch stacks (cu124 + cu113) compound this. This is the dominant download-friction reality for a non-tech user.",
          "confidence": "high",
          "source": "https://mveg.es/posts/optimizing-pytorch-docker-images-cut-size-by-60percent/"
        },
        {
          "claim": "Docker Desktop on Windows requires the WSL2 backend for GPU support (GPU-PV paravirtualization), up-to-date Windows 10/11, NVIDIA WSL2 drivers, and the nvidia-container-toolkit inside WSL2; then `--gpus all` works. This is a non-trivial setup chain for a non-technical user, and only matters if they have an NVIDIA GPU at all.",
          "confidence": "high",
          "source": "https://docs.docker.com/desktop/features/gpu/"
        },
        {
          "claim": "Docker Desktop is FREE for personal use and small businesses (<250 employees AND <$10M revenue) but a paid subscription is required for larger commercial use. For a coach/small studio this is free, but it is a licensing flag to track as the product commercializes (licensing guardrail is non-negotiable in this repo).",
          "confidence": "high",
          "source": "https://docs.docker.com/subscription/desktop-license/"
        },
        {
          "claim": "Compose requests GPUs via deploy.resources.reservations.devices with driver: nvidia, count: all|N (or device_ids), capabilities: [gpu] (capabilities is mandatory or deploy errors). You CANNOT combine count and device_ids. This is the syntax for the GPU profile/overlay.",

```
          "confidence": "high",
          "source": "https://docs.docker.com/compose/how-tos/gpu-support/"
        },
        {
          "claim": "Bind-mounting a SQLite DB or the user's videos from a Windows host path into
a WSL2-backed container is SLOW (Windows paths cross a Plan9 9P share via /mnt/c); the fix is a
Docker NAMED VOLUME for the DB and WSL2-resident paths for I/O-heavy data. Directly affects
where the appliance puts the SQLite file, videos, frames, and weights.",
          "confidence": "high",
          "source": "https://www.docker.com/blog/docker-desktop-wsl-2-best-practices/"
        },
        {
          "claim": "micromamba (the recommended container-friendly mamba) is the lighter, multi-
stage-friendly replacement for Miniconda in images; conda envs can be copied into a later stage
to shrink the final image, and MAMBA_DOCKERFILE_ACTIVATE=1 controls env activation in RUN steps.
Swapping the Dockerfile's Miniconda for micromamba is the recommended refinement.",
          "confidence": "medium",
          "source": "https://micromamba-docker.readthedocs.io/en/latest/advanced_usage.html"
        },
        {
          "claim": "CUDA minor-version / backward compatibility: a CUDA 11.x application (torch
1.11+cu113) generally runs on a newer R5xx/CUDA-12 host driver because drivers are backward
compatible; the nvidia-container-toolkit loads the compatible libs. So the cu113 WASB env can
run on a modern host driver inside one --gpus all passthrough ? but it is a cross-major-family
case to verify on the owner's real GPU box, not assume.",
          "confidence": "medium",
          "source": "https://docs.nvidia.com/deploy/cuda-compatibility/latest/"
        },
        {
          "claim": "The engine's worker is in-process: a single module-global
ThreadPoolExecutor(max_workers=1) drains jobs one at a time; API and worker are the SAME process
today (not separate services). This is why the natural container unit is ONE service (option
A/C), and splitting the WASB detector into its own container (option B) is net-new RPC plumbing
the code does not have.",
          "confidence": "high",
          "source":
"C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:32,42 (D3;
backend/api/job_worker.py:37)"
        },
        {
          "claim": "The live appliance design explicitly flags 'No Dockerfile exists;
feasibility-check a worker image before promising Cloud Run/GCE' and notes WASB's torch-1.11
conda env makes the image non-trivial ? i.e. containerization is an OPEN owner question (§10
Q4), and this research feeds that decision.",
          "confidence": "high",
          "source":
"C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:96,135,176"
        }
      ],
      "recommendation": "Adopt a TWO-VARIANT Docker strategy and make it the appliance unit ?
but lead with the small one. (1) DEFAULT = a CPU-only image (Option C base): python-slim/ubuntu
+ ffmpeg + `pip install . torch --index-url .../cpu` + FastAPI + in-proc worker + bundled SPA,
~1-2 GB. This honestly covers the shipped Gemini/stub/motion CUJs that need NO GPU, which is the
realistic state of a non-tech coach's Windows laptop. (2) OPT-IN `gpu` compose profile = the
heavy CUDA image, built by EXTENDING the existing deploy/cloudrun/Dockerfile (Option A): same
two-Python-env pattern (main cu124 + a micromamba `wasb` cu113 env reached via $WASB_PYTHON),
GPU via Compose `reservations.devices`, weights mounted at runtime. Do NOT split the WASB
detector into its own container (Option B): the engine's worker is in-process and the detector
is a subprocess call, so a separate container is net-new RPC plumbing with no payoff on a
single-GPU box. Make `/api/capabilities` (already net-new in the design, T7) drive the SPA so a
CPU box never offers WASB. Concretely, the compose file gets ONE app service + a `gpu` profile
that swaps the image and adds the devices block; SQLite goes in a NAMED VOLUME, the user's
videos + weights in bind mounts/volumes kept on the WSL2 side (never a /mnt/c Windows path) for
I/O. Two engine prerequisites this forces and must be filed: (a) the server must bind 0.0.0.0
inside the container (today it hard-binds 127.0.0.1 at backend/main.py:735) with the
```

```
firewall/edge gate as the safety boundary; (b) the dual-torch image needs a documented out-of-band torch install step since pyproject omits torch by design. Flag the honest fork (Option D): for a truly non-technical Windows user, Docker Desktop + WSL2-GPU is itself heavy friction and conflicts with owner decision D10 ('self-hosted webserver the user spins up, NOT a desktop app') ? so position Docker as the REPRODUCIBILITY/cloud-and-power-user unit, and keep a native `indexer --server` launch path for the lightest local case. Track the Docker Desktop commercial-licensing threshold against the repo's non-negotiable licensing guardrail.",
      "fit_for_this_app": "The dual-torch problem is the ONE thing Docker solves cleanly here and the repo already proves it: deploy/cloudrun/Dockerfile co-locates the main cu124 env and the WASB cu113 env in a single image with the $WASB_PYTHON subprocess seam (Dockerfile:54-57) ? so 'two envs, one image' is not theoretical, it's shipped and M7-owner-run. ffmpeg/ffprobe (system deps) are a one-line apt install in that image (Dockerfile:27-30), which is far cleaner than asking a Windows coach to install ffmpeg by hand. The GCS-hosted weights (wasb_badminton_best.pth.tar) are deliberately NOT baked (WASB-C3 licensing AMBER, Dockerfile:14-16), so the appliance MUST mount or fetch them at runtime ? a first-class volume in the compose file, and a licensing-gate the guardrail demands. FastAPI+SPA: the engine serves /api and a (likely-dead) backend/api/static today, but the live SPA is in khelsutra/web ? bundling it is a cross-repo build step that lands as static assets in the app image; the server's 127.0.0.1 hard-bind (main.py:735) must become 0.0.0.0 inside a container with the edge gate as the boundary (the design's auth invariant, LOCAL_APPLIANCE_ARCHITECTURE.md §4.4). The non-tech-India-coach-on-Windows reality is decisive: most such users have NO NVIDIA GPU, so the CPU-default + Gemini path (the engine's actual default_segmenter) is the honest first-run, and the multi-GB CUDA image + WSL2-GPU paravirtualization + nvidia-container-toolkit chain is reserved for the GPU opt-in profile ? keeping the common-case download small and the GPU friction off the critical path. Compose maps directly onto the engine's decoupled seams (storage=local default, compute=local, profiles/presets) so 'compute/storage as a setting' (D6/D8) becomes env-vars + profiles, not forks. Licensing: micromamba (over Miniconda) sidesteps the Anaconda-defaults ToS friction the gpu_setup.sh already had to auto-accept, and Docker Desktop is free at this user's scale but its >250-employee/$10M commercial threshold is a flag to watch as KhelSutra commercializes."
    },
    "verdict": {
      "claims_checked": [
        {
          "claim": "The repo ALREADY ships the two-Python-envs-in-one-image pattern: deploy/cloudrun/Dockerfile FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04, builds a Miniconda `wasb` env (py3.8 + torch 1.11.0+cu113) alongside the system-python main app env, and the native runner shells out via $WASB_PYTHON (ENV at Dockerfile:54-57). The two torches deliberately never co-install (Dockerfile:8-12).",
          "verdict": "confirmed",
          "reasoning": "Verified directly against C:/Users/avidu/Projects/badminton-highlight-indexer/deploy/cloudrun/Dockerfile. Line 18 = FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04. Lines 33-43 install Miniconda and create a conda env named `wasb` with python=3.8 then pip install torch==1.11.0+cu113 torchvision==0.12.0+cu113. Lines 47-50 install the main app via system python3 (`python3 -m pip install .`). Lines 54-57 set ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python (etc.). The header comment lines 8-12 explicitly state the two python stacks are kept separate 'so the app's torch never co-installs with it.' The native runner DOES shell out: native_wasb_runner.py uses subprocess (self._run(argv,...)) invoking self.cfg.python_bin, which is sourced from env('WASB_PYTHON') (line 89). Every load-bearing detail is accurate. (Minor framing nit, not a refutation: this image is documented as a Cloud Run GPU *Job* worker image -- ENTRYPOINT python3 -m backend.worker -- not the FastAPI appliance server; reuse for the appliance is an extension, which the recommendation correctly treats as 'EXTENDING the existing Dockerfile'.)",
          "corrected_claim": ""
        },
        {
          "claim": "WASB pretrained weights are NOT baked into the image (provenance not cleared for commercial sharing, WASB-C3) -- they must be mounted or fetched from a bucket at runtime and pointed to by $WASB_WEIGHTS. This is a hard volume/mount requirement for the appliance, not optional.",
          "verdict": "confirmed",
          "reasoning": "Dockerfile lines 14-16 state verbatim: 'WASB-C3: the pretrained badminton weights are academic-data-trained -- provenance NOT cleared for commercial sharing. They are NOT baked into this image; stage them at runtime (a bucket fetch or a mount) and point $WASB_WEIGHTS at them.' Line 56 sets WASB_WEIGHTS to a path inside /opt/models/WASB-
```

SBDT/pretrained_weights/... but nothing in the Dockerfile downloads weights to that path (the git clone at line 38 is --depth 1 of the code repo only). native_wasb_runner.py:160 confirms an empty weights path is a hard failure. So weights must arrive at runtime. 'Hard requirement' is accurate for the GPU/WASB path specifically.",
          "corrected_claim": ""
        },
        {
          "claim": "The FastAPI server HARD-binds host=127.0.0.1 (backend/main.py:735), exposed via `--server`/`--port` flags (562-563). Inside a container 127.0.0.1 is the container loopback, NOT reachable from the Windows host -- the appliance image must bind 0.0.0.0 ... This is a real engine change the Dockerized appliance forces.",
          "verdict": "confirmed",
          "reasoning": "backend/main.py:735 is exactly `uvicorn.run(app, host=\"127.0.0.1\", port=args.port)` and grep confirms it is the ONLY uvicorn.run call with the ONLY host= literal in the file -- there is no env var or CLI flag that can override the host (only --port at line 563). --server is line 562. The networking fact is correct: a process binding 127.0.0.1 inside a container is reachable only from within that container's network namespace; -p host:container port publishing cannot reach a loopback-only listener, so a containerized server must bind 0.0.0.0. Therefore Dockerizing the *server* (as opposed to the worker-Job image, whose ENTRYPOINT is backend.worker and never starts uvicorn) does force this engine change. Confirmed.",
          "corrected_claim": ""
        },
        {
          "claim": "torch/torchvision are deliberately absent from pyproject dependencies because CUDA/CPU wheels need --index-url which PEP621 cannot express; the `gpu` extra is intentionally empty/docs-only. So ANY image must `pip install .` then add torch on a separate line with the right index -- a CPU image and a GPU image diverge only on that one torch line.",
          "verdict": "confirmed",
          "reasoning": "pyproject.toml lines 8-11 (header) and 58-65 confirm: torch/torchvision are NOT in [project.dependencies] (lines 27-37 list only fastapi/uvicorn/opencv/numpy/pydantic/jinja2/dotenv/google-genai/supervision); the comment states their wheels 'need an out-of-band index (--index-url/--extra-index-url), which PEP 621 cannot express.' Line 65 `gpu = []` with lines 58-64 commenting it is 'DOCUMENTATION ONLY ... intentionally empty,' giving the cu124 and cpu install lines. The 'diverge only on that one torch line' claim is essentially true for the MAIN env. Caveat (does not refute): the GPU image also adds an entire second Miniconda `wasb` torch-1.11 stack, so the *image* diverges on far more than one line; the one-line claim is accurate strictly for the main-app torch install, which is what the sentence scopes ('pip install . then add torch on a separate line').",
          "corrected_claim": ""
        },
        {
          "claim": "A CUDA-runtime + PyTorch Docker image is realistically multi-GB: a naive CUDA+python+torch image measured ~7.6 GB, optimizable to ~2.9 GB; PyTorch bundles ~2.5 GB of CUDA on its own. Two torch stacks (cu124 + cu113) compound this.",
          "verdict": "confirmed",
          "reasoning": "The cited source (mveg.es) confirms verbatim: original image 'a staggering 7.6 GB', optimized to 'just 2.9 GB' (62% reduction), and 'the total size of the torch package quite large, around 2.5G' due to bundled CUDA. The numbers are quoted accurately. The 'two torch stacks compound this' is sound engineering inference (the Dockerfile installs both cu124 main torch and cu113 wasb torch). One honest caveat for the recommendation: these are one blogger's measurements, not a universal benchmark, and exact sizes depend on base image and pruning -- but as an order-of-magnitude 'multi-GB download friction' point it is well-supported and the claim hedges appropriately ('realistically').",
          "corrected_claim": ""
        },
        {
          "claim": "Docker Desktop on Windows requires the WSL2 backend for GPU support (GPU-PV paravirtualization), up-to-date Windows 10/11, NVIDIA WSL2 drivers, and the nvidia-container-toolkit inside WSL2; then `--gpus all` works.",
          "verdict": "uncertain",
          "reasoning": "Mostly confirmed but ONE detail is inaccurate as stated. The official Docker Desktop GPU page (docs.docker.com/desktop/features/gpu/) confirms: WSL2 backend is mandatory ('GPU support in Docker Desktop is only available on Windows with the WSL2 backend'), needs an NVIDIA GPU, up-to-date Win10/11, up-to-date NVIDIA drivers supporting 'WSL 2 GPU

```
Paravirtualization', latest WSL2 kernel; and `docker run --gpus=all ...` works in the validation example. HOWEVER the page does NOT list nvidia-container-toolkit as a prerequisite for *Docker Desktop* -- Docker Desktop bundles the GPU plumbing. The NVIDIA Container Toolkit is required for the *Docker Engine in a plain WSL2 distro* path (and on Linux), not for Docker Desktop's --gpus support. So the claim conflates two paths: 'nvidia-container-toolkit inside WSL2' is true for raw-engine-in-WSL but is NOT a documented Docker Desktop prerequisite. Marking uncertain because the load-bearing toolkit detail is path-dependent and as written overstates the Docker Desktop requirement. The broader thrust (non-trivial GPU setup chain, NVIDIA-GPU-only) stands.",
          "corrected_claim": "Docker Desktop on Windows requires the WSL2 backend for GPU support (GPU-PV paravirtualization), up-to-date Windows 10/11, and NVIDIA WSL2-paravirtualization drivers; then `--gpus=all` works (Docker Desktop handles the container GPU plumbing -- the separate nvidia-container-toolkit install is only needed for the raw Docker-Engine-in-WSL2 path, not Docker Desktop itself). Either way this is a non-trivial setup chain and only matters with an NVIDIA GPU."
        },
        {
          "claim": "Docker Desktop is FREE for personal use and small businesses (<250 employees AND <$10M revenue) but a paid subscription is required for larger commercial use. ... it is a licensing flag to track as the product commercializes.",
          "verdict": "confirmed",
          "reasoning": "docs.docker.com/subscription/desktop-license/ confirms free for small businesses defined as 'fewer than 250 employees AND less than $10 million in annual revenue', plus personal use, education, and non-commercial open source; a paid plan (Pro/Team/Business) is required above that for professional/commercial use, and explicitly for government. Thresholds and AND-conjunction are exact. The 'licensing flag to track' framing is consistent with the repo's non-negotiable licensing guardrail. Note (not a refutation): this licenses Docker DESKTOP only; Docker Engine / containerd / the CLI are Apache-2.0 and free -- so a Linux/server deployment avoids the Desktop license entirely. The recommendation already positions Docker as a power-user/cloud unit, which mitigates this.",
          "corrected_claim": ""
        },
        {
          "claim": "Compose requests GPUs via deploy.resources.reservations.devices with driver: nvidia, count: all|N (or device_ids), capabilities: [gpu] (capabilities is mandatory or deploy errors). You CANNOT combine count and device_ids.",
          "verdict": "confirmed",
          "reasoning": "docs.docker.com/compose/how-tos/gpu-support/ confirms all specifics: GPUs requested via deploy.resources.reservations.devices; properties driver (e.g. 'nvidia'), capabilities (list, e.g. [gpu]), count (int or 'all'), device_ids (list), options. 'You must set the capabilities field. Otherwise, it returns an error on service deployment.' And 'count and device_ids are mutually exclusive. You must only define one field at a time.' Every assertion in the claim matches the source verbatim.",
          "corrected_claim": ""
        },
        {
          "claim": "Bind-mounting a SQLite DB or the user's videos from a Windows host path into a WSL2-backed container is SLOW (Windows paths cross a Plan9 9P share via /mnt/c); the fix is a Docker NAMED VOLUME for the DB and WSL2-resident paths for I/O-heavy data.",
          "verdict": "confirmed",
          "reasoning": "docs.docker.com/blog/docker-desktop-wsl-2-best-practices/ confirms: mounting Windows-filesystem files (e.g. -v /mnt/c/...) loses performance because '/mnt/c is actually a mountpoint exposing Windows files through a Plan9 file share', and recommends storing project files within the WSL2 distro filesystem ('avoid accessing files stored on the Windows host as much as possible'), citing shared VFS cache and inotify. Named volumes are Docker-managed and live on the Linux/WSL2 side, so the 'named volume for the DB + WSL2-resident paths for I/O-heavy data' prescription is sound. SQLite over a 9P share is a well-known correctness/perf hazard (file locking), reinforcing the point. Confirmed.",
          "corrected_claim": ""
        },
        {
          "claim": "micromamba (the recommended container-friendly mamba) is the lighter, multi-stage-friendly replacement for Miniconda in images; conda envs can be copied into a later stage to shrink the final image, and MAMBA_DOCKERFILE_ACTIVATE=1 controls env activation in RUN steps. Swapping the Dockerfile's Miniconda for micromamba is the recommended refinement.",
          "verdict": "confirmed",
```

```
          "reasoning": "Initial fetch of advanced_usage.html surfaced MAMBA_SKIP_ACTIVATE
(runtime) and not MAMBA_DOCKERFILE_ACTIVATE, which looked refuting -- but a follow-up search
confirmed the micromamba-docker docs do document: 'To have an environment active during a RUN
command, you must set ARG MAMBA_DOCKERFILE_ACTIVATE=1.' It is technically an ARG (build-time),
and the claim calls it an ENV-style var, but functionally the claim's description ('controls env
activation in RUN steps') is accurate. Multi-stage env copying to shrink images is
documented/practiced (micromamba-docker discussion #412 'Reduction of image size with multi-
staging'). micromamba being a lighter, container-friendly mamba is its stated purpose.
Confidence was correctly 'medium': the value is real but it is an optional refinement, NOT a
load-bearing requirement -- the existing Miniconda Dockerfile already works, so adopting
micromamba is a nice-to-have optimization, not a must. Minor terminology nit (ARG vs ENV) aside,
the substance holds.",
          "corrected_claim": "micromamba is a lighter, container-friendly replacement for
Miniconda; conda envs can be copied into a later multi-stage build stage to shrink the final
image; and setting build ARG MAMBA_DOCKERFILE_ACTIVATE=1 activates the env during RUN steps.
Swapping Miniconda for micromamba is an OPTIONAL size optimization, not a requirement -- the
current Miniconda Dockerfile already functions."
        },
        {
          "claim": "CUDA minor-version / backward compatibility: a CUDA 11.x application (torch
1.11+cu113) generally runs on a newer R5xx/CUDA-12 host driver because drivers are backward
compatible; the nvidia-container-toolkit loads the compatible libs. So the cu113 WASB env can
run on a modern host driver inside one --gpus all passthrough -- but it is a cross-major-family
case to verify on the owner's real GPU box, not assume.",
          "verdict": "uncertain",
          "reasoning": "The general direction is correct and the claim's own hedge ('cross-
major-family case to verify, not assume') is the right posture, so I will not call it refuted --
but the terminology conflates two distinct NVIDIA mechanisms. NVIDIA's CUDA Compatibility doc
defines 'Minor Version Compatibility' as running an app on a newer driver WITHIN THE SAME MAJOR
family (CUDA 11.x app on a newer CUDA-11 driver). Running a cu113 (CUDA 11) app on a CUDA-12
host driver is NOT 'minor version compatibility'; it relies on the broader property that newer
NVIDIA drivers ship a backward-compatible CUDA runtime/driver ABI that still supports older
CUDA-11 apps -- which is generally true in practice but is a different guarantee than the named
'minor version compatibility' feature. Also, on WSL/Docker Desktop the CUDA libs are surfaced
via the Windows host driver stub (libcuda.so), and toolkit specifics differ from the Linux
nvidia-container-toolkit path. Because the precise compatibility-mechanism wording is imprecise
and the cross-major-family outcome genuinely depends on the actual host driver version (only
verifiable on the owner's real GPU box), uncertain is the honest verdict.",
          "corrected_claim": "A cu113 (CUDA 11) torch build generally runs under a newer CUDA-12
host driver because NVIDIA drivers are broadly backward-compatible with older CUDA runtimes
(this is driver backward compatibility -- distinct from NVIDIA's named 'Minor Version
Compatibility', which is within a single major family). It is a cross-major-family case that
must be verified on the owner's real GPU box (and on WSL the GPU libs come via the Windows host
driver stub), not assumed."
        },
        {
          "claim": "The engine's worker is in-process: a single module-global
ThreadPoolExecutor(max_workers=1) drains jobs one at a time; API and worker are the SAME process
today (not separate services). This is why the natural container unit is ONE service (option
A/C), and splitting the WASB detector into its own container (option B) is net-new RPC plumbing
the code does not have.",
          "verdict": "confirmed",
          "reasoning": "backend/api/job_worker.py:30-38 confirms a module-global
Optional[ThreadPoolExecutor] created lazily as ThreadPoolExecutor(max_workers=1,
thread_name_prefix='jobworker'), with the comment 'max_workers=1 on purpose: a single self-
hosted box serializes GPU work.' backend/main.py:734 submits the drain onto this same
_get_executor() inside the --server process, so API and worker share one process. The khelsutra
design doc (D3, line 42) corroborates 'In-process worker: one module-global
ThreadPoolExecutor(max_workers=1) [verified job_worker.py:37] -- strictly one job at a time.'
The WASB detector today is invoked as a subprocess of that same process (native_wasb_runner.py
shells to $WASB_PYTHON), not over RPC, so isolating it into a separate container would indeed
require net-new IPC/RPC plumbing that does not exist. Confirmed.",
          "corrected_claim": ""
        },
        {
```

```
          "claim": "The live appliance design explicitly flags 'No Dockerfile exists;
feasibility-check a worker image before promising Cloud Run/GCE' and notes WASB's torch-1.11
conda env makes the image non-trivial -- i.e. containerization is an OPEN owner question (S10
Q4), and this research feeds that decision.",
          "verdict": "confirmed",
          "reasoning": "khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md confirms each element:
the risk table (~line 135) row 'No Dockerfile | runbook-only reproducibility | feasibility-check
a worker image before promising Cloud Run/GCE'; the compute matrix (~line 96) note 'Cloud
Run/GCE need a containerized worker -- no Dockerfile exists, and WASB's torch-1.11 conda env
makes the image non-trivial'; and S10 Open questions Q4 '(4) GPU host choice -- DO GPU droplet /
GCE GPU VM / Cloud Run+GPU (and is a containerized worker worth building for Cloud Run?)'. So
containerization is genuinely an open owner-gated question this research feeds. One important
freshness caveat (does NOT refute the claim but sharpens it): the design doc says 'no Dockerfile
exists', yet a Dockerfile DOES exist in the engine repo at deploy/cloudrun/Dockerfile (added by
engine PR #287, the M6 Cloud Run GPU dispatch shim). The doc's 'no Dockerfile' line reflects the
khelsutra/appliance vantage and predates or overlooks that engine PR -- exactly the kind of
cross-repo surface drift the repo's 'verify engine-surface before scoping' lesson warns about.
The recommendation correctly assumes the Dockerfile exists ('EXTENDING the existing
deploy/cloudrun/Dockerfile'), so the recommendation is MORE current than the doc it cites.",
          "corrected_claim": "The live appliance design (khelsutra
LOCAL_APPLIANCE_ARCHITECTURE.md) flags containerization as an open owner question (S10 Q4: 'is a
containerized worker worth building for Cloud Run?') and notes WASB's torch-1.11 conda env makes
the image non-trivial. Caveat: the doc's 'No Dockerfile exists [verified]' line is now STALE --
a Cloud Run GPU *Job* worker Dockerfile already exists at engine deploy/cloudrun/Dockerfile
(engine PR #287/M6). So the feasibility check is partly answered; what remains open is the
*appliance server* image (which must bind 0.0.0.0) and the CPU/GPU two-variant strategy, not
whether a CUDA+dual-torch image can be built at all."
        }
      ],
      "overall_notes": "VERDICT SUMMARY: 9 confirmed, 0 refuted, 3 uncertain (claims 6, 11, and
-- I downgraded none to refuted). The recommendation's load-bearing engineering claims about
THIS repo are accurate and well-sourced; I verified each cited file:line directly.\n\nFiles
verified (all absolute):\n- C:/Users/avidu/Projects/badminton-highlight-
indexer/deploy/cloudrun/Dockerfile (lines 8-18, 33-43, 47-50, 54-57 -- confirms claims 1, 2)\n-
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py:735 (the ONLY uvicorn.run;
host=127.0.0.1 hard literal, no override; --server/--port at 562-563 -- confirms claim 3)\n-
C:/Users/avidu/Projects/badminton-highlight-indexer/pyproject.toml lines 8-11, 27-37, 58-65
(torch absent by design; gpu=[] docs-only -- confirms claim 4)\n-
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/api/job_worker.py:30-38 (module-
global ThreadPoolExecutor(max_workers=1) -- confirms claim 12)\n-
C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/native_wasb_runner.py:88-90,236-239 (subprocess shell-out to
$WASB_PYTHON; empty WASB_WEIGHTS is a hard fail -- confirms 1, 2, 12)\n-
C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md (D3 line 42, compute
matrix ~line 96, risk table ~line 135, S10 Q4 -- confirms claims 12, 13)\n\nTHREE POINTS THAT
NEED CORRECTION/SHARPENING (the user should not ship these verbatim):\n\n1. CLAIM 6 (nvidia-
container-toolkit): The official Docker Desktop GPU page does NOT list nvidia-container-toolkit
as a Docker Desktop prerequisite -- Docker Desktop bundles the container-GPU plumbing. The
toolkit is required for the raw Docker-Engine-in-WSL2 path and on Linux, but not for Docker
Desktop's --gpus support. As written the claim conflates the two paths. Everything else in claim
6 (WSL2 mandatory, GPU-PV, Win10/11, NVIDIA drivers, --gpus=all works) is confirmed verbatim
from docs.docker.com.\n\n2. CLAIM 11 (CUDA compat): The claim labels a CROSS-major-family case
(cu113/CUDA-11 app on a CUDA-12 driver) as 'minor-version/backward compatibility'. NVIDIA's
'Minor Version Compatibility' is defined as WITHIN a single major family. The cross-major
outcome relies instead on general driver backward-compatibility with older CUDA runtimes --
usually true in practice but a different, weaker guarantee. The claim's own hedge ('verify on
the owner's real GPU box, not assume') is the correct posture, so this is a terminology
imprecision rather than a false conclusion. Keep the hedge; fix the wording.\n\n3. CLAIM 13 /
FRESHNESS FLAG (most important for action): The khelsutra design doc's 'No Dockerfile exists
[verified]' is STALE. A Cloud Run GPU *Job* worker Dockerfile already exists at
deploy/cloudrun/Dockerfile (engine PR #287, M6). This is the exact cross-repo engine-surface
drift the repo's 'verify engine-surface before scoping' lesson warns about. Net effect: it
STRENGTHENS the recommendation (the dual-torch CUDA image is already proven buildable; the
recommendation correctly says 'EXTEND the existing Dockerfile'), but the khelsutra doc should be
honestly corrected so the gap is scoped as 'the existing image is a worker-Job image (ENTRYPOINT
```

backend.worker, binds nothing); the appliance needs a SERVER image that binds 0.0.0.0' rather than 'no Dockerfile exists at all'.\n\nADDITIONAL HONEST CAVEATS on otherwise-confirmed claims (not refutations, but the recommendation should state them):\n- Claim 1/3: deploy/cloudrun/Dockerfile is a Cloud Run GPU *Job* worker image (ENTRYPOINT python3 -m backend.worker), NOT the FastAPI server. It never runs uvicorn, so the 127.0.0.1 bind problem (claim 3) is specifically introduced when the recommendation Dockerizes the *server* path -- which is correct, but it means the GPU appliance image is a genuine extension of the worker image (add a server entrypoint + 0.0.0.0 bind), not a drop-in reuse.\n- Claim 5 image sizes are one blogger's measurements; directionally sound but not a universal benchmark.\n- Claim 7 (Docker Desktop license) covers Docker DESKTOP only; a headless Linux/server deploy uses Apache-2.0 Docker Engine and dodges the Desktop license entirely -- relevant since the recommendation positions Docker as the cloud/power-user unit.\n- Claim 10 (micromamba) is correctly flagged medium-confidence: it is an OPTIONAL optimization, not a requirement; the existing Miniconda Dockerfile already works.\n\nBOTTOM LINE ON THE RECOMMENDATION: The two-variant strategy (lead with a small CPU-only image; opt-in GPU compose profile extending the existing Dockerfile; do NOT split WASB into its own container; named volume for SQLite + WSL2-resident I/O paths; file the 0.0.0.0-bind and out-of-band-torch prerequisites; flag Option D native launch path and the Docker Desktop license threshold) rests on claims that are overwhelmingly verified. The only fixes needed before relying on it are the three wording/freshness items above -- none of which undermine the core recommendation; the staleness item actually de-risks it."
    }
  },
  {
    "topic": "frozen-binaries",
    "research": {
      "topic": "Freezing the badminton-highlight-indexer Python app (FastAPI + torch + opencv + numpy + uvicorn) into a standalone batteries-included binary/installer ? PyInstaller vs Nuitka vs cx_Freeze vs Briefcase ? with an HONEST per-TIER feasibility verdict.",
      "options": [
        {
          "name": "PyInstaller (onedir, NOT onefile)",
          "how_it_works": "Bundles the embedded CPython interpreter + collected modules/DLLs into a dist/ folder shipped inside an OS installer (Inno/MSIX on Win, .dmg on Mac, AppImage on Linux). torch needs --collect-all torch/--copy-metadata torch + hidden imports (torch._C._jit). Use opencv-python-headless. uvicorn runs in-process with the app object (engine already does this at backend/main.py:735, no workers/reload). Call multiprocessing.freeze_support() at entry. ffmpeg/ffprobe ship as side-by-side LGPL binaries.",
          "pros": [
            "Most battle-tested for torch/opencv/numpy/fastapi; largest body of working recipes for this exact stack.",
            "License clean for our guardrail: GPL-2.0 WITH a bootloader exception that permits shipping closed/any-licensed frozen apps; no need to ship source or PyInstaller's license; PyInstaller is not linked into the binary's effective license.",
            "onedir triggers far fewer AV/Defender false positives than onefile (no runtime self-extract to temp).",
            "Fast builds (seconds-to-minutes) vs Nuitka.",
            "Engine server is already frozen-friendly: single-process uvicorn with the app object on 127.0.0.1."
          ],
          "cons": [
            "Huge bundle: torch alone is hundreds of MB; torch+opencv commonly 250MB-1GB+ before weights; CUDA wheels far higher.",
            "Cannot solve the dual-torch-env problem: freezes ONE interpreter; WASB shells out to a SECOND Python (torch 1.11+cu113) via conda activate wasb (wasb_runner.py:74) and probes a separate python_bin (native_wasb_runner.py:117). A frozen exe has no external python/conda, so local-WASB needs a second sidecar exe or ONNX.",
            "Code-signing/notarization separate and fiddly: Win Authenticode (EV for SmartScreen); Mac Developer-ID + Hardened Runtime + entitlements (allow-jit, allow-unsigned-executable-memory, disable-library-validation) + notarytool + stapler; --deep discouraged.",
            "Hidden-import whack-a-mole: torch, supervision, google-genai, pydantic v2, lazy cloud SDKs in backend/storage/*.",
            "CUDA torch bundle is enormous and pins a CUDA version; effectively per-platform CPU vs CUDA variants."
          ],

```
          "maturity": "mature"
        },
        {
          "name": "Nuitka (standalone mode)",
          "how_it_works": "Compiles Python to C and links a real binary. Ships built-in package
rules for PyTorch/TensorFlow/scikit-learn/SciPy that prune unused operator kernels; recent
releases fixed onnxruntime DLL collection. Same external-deps story (ffmpeg side-by-side; second
torch env still its own artifact).",
          "pros": [
            "License clean and arguably safest for the MIT/Apache/BSD guardrail: open-source
core is Apache-2.0 (PyPI); compiled output never inherits Nuitka's license; Commercial is
OPTIONAL (paid IP-protection plugins), not needed to distribute.",
            "Best built-in PyTorch/onnxruntime support among freezers; operator-kernel pruning
can shrink torch more than PyInstaller's collect-all.",
            "True compiled binary (some IP-hardening + occasional speedups).",
            "Active onnxruntime DLL handling ? relevant if we ONNX-export to escape the dual-
torch problem."
          ],
          "cons": [
            "Very long builds: ~30s in PyInstaller becomes 5-15 min in Nuitka; torch standalone
is a known heavy case (issues #754, #2218).",
            "Binaries typically LARGER than PyInstaller equivalent, and torch is already the
size problem.",
            "Smaller recipe base for this exact FastAPI+torch+opencv combo.",
            "Fixes none of the hard blockers: dual-torch-env, ffmpeg system dep, per-OS
signing.",
            "onefile shares AV-false-positive/temp-extraction concerns."
          ],
          "maturity": "mature"
        },
        {
          "name": "cx_Freeze",
          "how_it_works": "Freezes into a folder with the strongest BUILT-IN cross-platform
installer story: MSI (Win), DMG (Mac), AppImage + .deb (Linux) from one config.",
          "pros": [
            "Unified installer generation across Win/Mac/Linux from one setup config.",
            "Pure-Python tooling; permissive (PSF/MIT-compatible) license ? clean for the
guardrail.",
            "Simple build model."
          ],
          "cons": [
            "Documented recurring PyTorch pain: DLL-dependency/compatibility errors freezing
torch (issues #2412, #2495) plus the 1GB+ size problem.",
            "Thinner torch/opencv recipe ecosystem than PyInstaller.",
            "Does nothing for dual-torch-env or ffmpeg.",
            "Signing/notarization still a separate manual step."
          ],
          "maturity": "emerging"
        },
        {
          "name": "Briefcase (BeeWare)",
          "how_it_works": "Packages from one pyproject.toml to per-OS native artifacts (and
mobile/web). GUI-app oriented.",
          "pros": [
            "Single-config multi-target packaging; good native-installer integration on
desktop.",
            "Permissive (BSD) license ? clean for the guardrail."
          ],
          "cons": [
            "Explicitly weak for heavy C-extension/ML deps ('cannot package many C-extension and
ML dependencies'); torch/opencv on desktop is fragile and off the beaten path.",
            "Optimized for BeeWare GUI apps, not a FastAPI+torch server sidecar; faces stronger
desktop competition.",
            "No advantage on dual-torch-env, ffmpeg, or signing; smallest ML recipe base.",
            "Mobile targets (its differentiator) are irrelevant to a desktop local web-service
```

```
appliance."
          ],
          "maturity": "risky"
        }
      ],
      "load_bearing_claims": [
        {
          "claim": "The engine launches uvicorn single-process with the app OBJECT (not an
import string), hard-bound to 127.0.0.1, with no workers=/reload= ? already matching the
PyInstaller-recommended pattern (reload=False, workers=1, freeze_support()).",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py:735
cross-checked with https://github.com/Kludex/uvicorn/discussions/1820 and
https://pyinstaller.org/en/stable/common-issues-and-pitfalls.html"
        },
        {
          "claim": "The HARD blocker is the second Python env: WASB shells into a SEPARATE
interpreter/env (WSL conda activate wasb, torch 1.11). A frozen single binary embeds ONE
interpreter with no external python/conda, so the full-local WASB tier can't be one frozen
process ? it needs a sidecar artifact or an ONNX replacement.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/wasb_runner.py:6,35-36,74 and native_wasb_runner.py:117;
PyInstaller embedded-interpreter constraint
https://github.com/orgs/pyinstaller/discussions/6046"
        },
        {
          "claim": "PyInstaller's license permits shipping a closed/any-licensed frozen app
without distributing your source or PyInstaller's license (GPL-2.0 WITH bootloader exception; a
few files Apache-2.0).",
          "confidence": "high",
          "source": "https://pyinstaller.org/en/stable/license.html"
        },
        {
          "claim": "Open-source Nuitka core is Apache-2.0 (PyPI); compiled output never inherits
Nuitka's license; Nuitka Commercial is an OPTIONAL paid add-on, not required to distribute
binaries.",
          "confidence": "medium",
          "source": "https://pypi.org/project/Nuitka/ and https://nuitka.net/doc/commercial-
license.html and https://en.wikipedia.org/wiki/Nuitka"
        },
        {
          "claim": "A torch+opencv frozen bundle is typically 250 MB to 1 GB+ before model
weights; CUDA wheels push higher. Mitigate with opencv-python-headless, excluding
torch.cuda/torchvision/matplotlib/scipy, or ONNX-export + onnxruntime instead of torch.",
          "confidence": "high",
          "source": "https://www.codegenes.net/blog/pyinstaller-pytorch/ and
https://github.com/marcelotduarte/cx_Freeze/issues/2412 and
https://pydevtools.com/handbook/explanation/how-do-i-ship-a-python-application-to-end-users/"
        },
        {
          "claim": "PyInstaller --onefile commonly triggers AV/Windows-Defender false positives
(runtime self-extract to temp = packer-like); --onedir avoids extraction and trips far fewer
heuristics. Code-signing whitelists your cert and dramatically reduces (not eliminates) flags.",
          "confidence": "high",
          "source": "https://www.pythonguis.com/faq/problems-with-antivirus-software-and-
pyinstaller/ and https://github.com/pyinstaller/pyinstaller/issues/6754"
        },
        {
          "claim": "macOS notarization of a frozen Python app requires Developer-ID signing +
Hardened Runtime + entitlements (allow-jit, allow-unsigned-executable-memory, disable-library-
validation) + notarytool submit + stapler; --deep discouraged, sign inside-out. Windows needs
Authenticode (EV cert for clean SmartScreen).",
          "confidence": "high",
          "source": "https://developer.apple.com/forums/thread/744471 and
```

https://github.com/pyinstaller/pyinstaller/issues/7937"
        },
        {
          "claim": "Nuitka builds are long (5-15 min vs PyInstaller ~30s) and torch standalone is a known heavy case, but Nuitka ships built-in PyTorch/onnxruntime rules (operator-kernel pruning) that can shrink the torch payload.",
          "confidence": "high",
          "source": "https://pydevtools.com/handbook/reference/nuitka/ and https://github.com/Nuitka/Nuitka/issues/754 and https://github.com/Nuitka/Nuitka/issues/2218"
        },
        {
          "claim": "torch needs explicit PyInstaller help: --copy-metadata torch plus hidden imports like torch._C._jit/torch._C._nvrtc; onefile produces 'file already exists' duplication warnings for torch/_C pyd.",
          "confidence": "high",
          "source": "https://github.com/pyinstaller/pyinstaller/discussions/7621 and https://github.com/pyinstaller/pyinstaller/issues/8348"
        },
        {
          "claim": "Owner decision D10 SUPERSEDED the desktop-app plan: deliver a self-hosted webserver + UI the user spins up (compute/storage as settings), so freezing should target a CLI/server appliance, not a GUI desktop bundle.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:39 which states it supersedes C:/Users/avidu/Projects/badminton-highlight-indexer/docs/DESKTOP_APP_PLAN.md"
        },
        {
          "claim": "ffmpeg/ffprobe are system (non-Python) deps no freezer bundles automatically; they must ship as side-by-side binaries, and to stay MIT/Apache/BSD-clean must be an LGPL build (openh264/HW encoders, NOT GPL libx264).",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/DESKTOP_APP_PLAN.md:64 (D4) and backend/pipeline/stitcher/compiler.py"
        }
      ],
      "recommendation": "VIABLE ? but ONLY as a CLI/server appliance (matching owner decision D10), tiered, and NOT as a single self-extracting onefile. (1) GEMINI-ONLY / no-torch tier = VIABLE NOW, low-risk: without torch the dependency set (fastapi, uvicorn, opencv-python-headless, numpy, pydantic, google-genai, supervision) freezes cleanly; bundle stays modest (tens of MB + ffmpeg). Use PyInstaller ONEDIR (not onefile ? avoids AV false positives), ship ffmpeg LGPL binaries side-by-side, wrap the existing indexer/backend.main --server entry (already freeze-friendly: single-process uvicorn on 127.0.0.1, main.py:735). This is the recommended FIRST shippable artifact. (2) FULL LOCAL-MODEL tier (WASB) = PARTIALLY viable / higher-risk, defer. The blocker is NOT the freezer ? it is the dual-torch-env design: WASB runs in a separate torch-1.11 conda/WSL interpreter the engine shells out to (wasb_runner.py/native_wasb_runner.py). No freezer bundles two interpreters in one process. Two honest paths: (a) freeze the WASB env as its OWN sidecar binary spawned as a subprocess (doable, but doubles the torch payload to ~1GB+ and doubles signing/AV surface), or (b) the strategically right move ? finish the de-WSL native runner AND export WASB to ONNX, ship ONE onnxruntime (no torch), collapsing to a single freezable env and slashing size. Path (b) ties to the existing OWNED_MODEL_IMPLEMENTATION_PLAN export-clean mandate and the DESKTOP_APP_PLAN P0 spike. Tooling pick: PyInstaller (onedir) as default ? biggest recipe base for this exact stack, clean GPL-with-exception license, fast builds. Keep Nuitka (Apache-2.0, built-in torch/onnxruntime pruning) as fallback if size/IP-hardening matters once ONNX lands. Reject cx_Freeze (documented torch DLL pain, thinner recipes) and Briefcase (explicitly weak on C-extension/ML deps). Regardless of tool: budget per-OS code-signing (Win Authenticode/EV, Mac Developer-ID + Hardened Runtime + notarytool + stapler) and a license audit (LGPL ffmpeg only) as separate non-trivial workstreams.",
      "fit_for_this_app": "Our engine is unusually freeze-FRIENDLY in the parts that usually break freezers, and freeze-HOSTILE in exactly one place. FRIENDLY: backend/main.py:735 already runs uvicorn single-process with the app object on 127.0.0.1 (no workers/reload/import-string) ? the textbook PyInstaller pattern; cloud SDKs (boto3, google-cloud-storage) import LAZILY in backend/storage/*, so the default local profile freezes without dragging them in; the PEP-621 indexer console entry (backend.main:main) and python -m backend.worker {infer|train} give clean

```
freeze entry points; deployment.profile/PROFILE_PRESETS (truly-local / cloud-serving / cpu-mock)
already express the exact TIERS to freeze. HOSTILE: the WASB shuttle model is a SECOND torch env
(torch 1.11+cu113) reached by shelling into WSL conda activate wasb (wasb_runner.py:74) or a
separate python_bin (native_wasb_runner.py:117) ? a frozen binary has one embedded interpreter
and no external python/conda, so the full-local tier cannot be one frozen process; it forces
either a second sidecar binary or (better, already on the roadmap) an ONNX export so only
onnxruntime ships, and the GCS-hosted weights (wasb_badminton_best.pth.tar) are fetched at first
run, not baked in. ffmpeg/ffprobe fit the guardrail only as side-by-side LGPL binaries (D4 ?
openh264/HW encoders, never GPL libx264) since no freezer bundles them. Freezer licensing is
guardrail-clean: PyInstaller (GPL-2.0 + bootloader exception) and Nuitka (Apache-2.0 core;
Commercial optional) both let us ship MIT-licensed binaries without inheriting their license.
Matches owner decision D10 (LOCAL_APPLIANCE_ARCHITECTURE.md:39): a self-hosted webserver+UI the
user spins up ? freeze a CLI/server appliance the khelsutra React SPA talks to over localhost,
not a GUI desktop app. For the non-tech-savvy India coach on Windows, ship the Gemini-only
onedir installer FIRST (small, signed, no GPU, no second env); gate the heavier local-model tier
behind the ONNX-export work."
    },
    "verdict": {
      "claims_checked": [
        {
          "claim": "The engine launches uvicorn single-process with the app OBJECT (not an
import string), hard-bound to 127.0.0.1, with no workers=/reload= ? already matching the
PyInstaller-recommended pattern (reload=False, workers=1, freeze_support()).",
          "verdict": "uncertain",
          "reasoning": "The core code fact is CONFIRMED: backend/main.py:735 is literally
`uvicorn.run(app, host=\"127.0.0.1\", port=args.port)` ? the app OBJECT (not an import string),
bound to 127.0.0.1, with no `workers=`/`reload=` kwargs. That part is freeze-friendly and
accurate. BUT the parenthetical 'already matching the PyInstaller-recommended pattern (...
freeze_support())' is overstated/inaccurate: (a) there is NO `multiprocessing.freeze_support()`
call anywhere in backend/ (grep for freeze_support|multiprocessing|set_start_method returns
nothing). PyInstaller's own common-pitfalls page DOES require freeze_support() for frozen
multiprocessing apps, so the code does NOT already match that. (b) The cited source
https://github.com/Kludex/uvicorn/discussions/1820 does NOT present a 'recommended pattern' ? it
is an UNANSWERED problem thread about workers>1 failing under PyInstaller, where the reporter
used an import string + freeze_support + workers=4. So it doesn't support 'recommended pattern'
framing. Mitigating nuance: the engine's job executor is a ThreadPoolExecutor(max_workers=1)
(backend/api/job_worker.py:37), NOT a ProcessPoolExecutor/multiprocessing, so the endless-spawn-
loop hazard freeze_support() guards against largely does not apply ? meaning freeze_support() is
arguably unnecessary here, but its ABSENCE still means the literal claim 'already matching ...
freeze_support()' is false as worded.",
          "corrected_claim": "The engine launches uvicorn single-process with the app OBJECT
(not an import string), hard-bound to 127.0.0.1, with no workers=/reload= (backend/main.py:735)
? which is the freeze-friendly shape (no reload, no worker subprocesses). It does NOT call
multiprocessing.freeze_support(), but that guard is largely moot because the job executor is a
ThreadPoolExecutor (max_workers=1, job_worker.py:37), not multiprocessing. Add freeze_support()
defensively when freezing regardless, per PyInstaller's common-pitfalls guidance."
        },
        {
          "claim": "The HARD blocker is the second Python env: WASB shells into a SEPARATE
interpreter/env (WSL conda activate wasb, torch 1.11). A frozen single binary embeds ONE
interpreter with no external python/conda, so the full-local WASB tier can't be one frozen
process ? it needs a sidecar artifact or an ONNX replacement.",
          "verdict": "confirmed",
          "reasoning": "Code-verified on all parts. wasb_runner.py shells into WSL via `conda
activate {self.cfg.conda_env}` (default conda_env='wasb', line 35-36, 74) and `wsl -d <distro>
bash -c` (WslCommandMixin). native_wasb_runner.py removes WSL but STILL invokes a separate
interpreter by path: _probe_cuda / _run call `subprocess.run([self.cfg.python_bin, ...])` (lines
110-123, 215-237), python_bin defaulting to the wasb env's python. The torch version is exactly
as claimed: docs/TRACKNET_WSL_SETUP.md:15 states 'WASB conda env = `wasb` (Python 3.8, torch
1.11.0+cu113)'. The architectural conclusion is sound: a single PyInstaller/Nuitka binary embeds
ONE Python interpreter and cannot host a second separate torch-1.11 env in-process; options are
a sidecar binary or ONNX/onnxruntime replacement. This matches the engine's own DECOUPLED de-
WSL/native-runner design and DESKTOP_APP_PLAN §3.2 ('lift wasb_infer.py out of WSL ... torch or
onnxruntime')."
        },
```

```
        {
          "claim": "PyInstaller's license permits shipping a closed/any-licensed frozen app without distributing your source or PyInstaller's license (GPL-2.0 WITH bootloader exception; a few files Apache-2.0).",
          "verdict": "confirmed",
          "reasoning": "Verified against https://pyinstaller.org/en/stable/license.html: PyInstaller is GPL-2.0-or-later WITH a Bootloader-exception, plus a few files under Apache-2.0 (dual scheme). The docs explicitly state 'You may use PyInstaller to bundle commercial applications out of your source code' and 'The executable bundles ... can be shipped with whatever license you want, as long as it complies with the licenses of your dependencies.' You need NOT ship your source or PyInstaller's own license. The ONE caveat the claim glosses over (but doesn't contradict): you must still comply with YOUR dependencies' licenses ? the bootloader exception covers PyInstaller itself, not bundled GPL/LGPL libraries.",
          "corrected_claim": "Confirmed as stated. Caveat: the exception covers PyInstaller's own bootloader/code only ? bundled dependencies' licenses (e.g. an LGPL/GPL ffmpeg or library) still bind you independently."
        },
        {
          "claim": "Open-source Nuitka core is Apache-2.0 (PyPI); compiled output never inherits Nuitka's license; Nuitka Commercial is an OPTIONAL paid add-on, not required to distribute binaries.",
          "verdict": "refuted",
          "reasoning": "The 'Apache-2.0' part is OUTDATED and false for current Nuitka. Verified on PyPI (https://pypi.org/project/Nuitka/): the current version is 4.1.3 (June 2026) with License field 'GNU Affero General Public License v3 or later (AGPLv3+)'. Apache-2.0 only applied to OLD versions 0.4.0 through 3.9; Nuitka switched to AGPL-3.0 at v4.0+. So 'Open-source Nuitka core is Apache-2.0' is incorrect for any version one would ship today. The OTHER two sub-claims are CORRECT: (1) compiled output does not inherit Nuitka's license ? there is a runtime exception (LICENSE-RUNTIME.txt, GCC-like) so your compiled program's license is your own; (2) Nuitka Commercial is OPTIONAL ? nuitka.net/doc/commercial-license.html confirms 'you are free to distribute [compiled binaries] without any restrictions' on the open-source edition. Because the licensing premise (Apache-2.0) is wrong and it was offered as a load-bearing fact ('Apache-2.0, built-in pruning' is cited again in the recommendation as a reason to keep Nuitka as fallback), the claim is refuted as worded. Practical impact: AGPL on the Nuitka TOOL is generally fine for distributing closed binaries thanks to the runtime exception, but the 'Apache-2.0' label is materially wrong and should not be relied on in a licensing audit.",
          "corrected_claim": "Current Nuitka (v4.x, e.g. 4.1.3) is AGPL-3.0-or-later, NOT Apache-2.0 (Apache-2.0 was only Nuitka 0.4.0?3.9). However, a GCC-style runtime exception means your COMPILED OUTPUT does not inherit Nuitka's AGPL ? you can distribute closed binaries freely ? and Nuitka Commercial is an optional paid add-on, not required to ship binaries. The licensing of the Nuitka tool itself should be recorded as AGPL-3.0+ (with binary runtime exception), not Apache-2.0."
        },
        {
          "claim": "A torch+opencv frozen bundle is typically 250 MB to 1 GB+ before model weights; CUDA wheels push higher. Mitigate with opencv-python-headless, excluding torch.cuda/torchvision/matplotlib/scipy, or ONNX-export + onnxruntime instead of torch.",
          "verdict": "confirmed",
          "reasoning": "Directionally and substantively correct, corroborated by PyTorch/PyInstaller sources: PyTorch 2.0 CPU-only wheel ~620MB, with bundled CUDA 11.7 ~1.8GB, and real-world conda torch+cu118 observed at ~4-5GB. So '250MB-1GB+ before weights; CUDA wheels push higher' is accurate in shape (the high end actually exceeds the stated range, which the 'push higher' qualifier covers). The mitigations are all standard and corroborated: opencv-python-headless (smaller, no GUI libs), excluding torchvision/matplotlib/scipy, CPU-only torch, and ONNX-export + onnxruntime as a torch-free path are documented size-reduction strategies (PyInstaller discussion #8552). Minor nit: 'excluding torch.cuda' is imprecise phrasing ? you reduce size by installing the CPU-only torch wheel (no bundled CUDA libs), not by excluding a torch.cuda submodule per se ? but the intent is correct.",
          "corrected_claim": "A torch+opencv frozen bundle is typically several hundred MB to 1 GB+ before weights for CPU-only torch, and CUDA-enabled torch pushes it to ~1.8GB?5GB (CUDA/cuDNN shared libs dominate and cannot be split). Mitigate with opencv-python-headless, a CPU-only torch wheel (rather than 'excluding torch.cuda'), excluding torchvision/matplotlib/scipy, or ONNX-export + onnxruntime to drop torch entirely."
        },
        {
```

```
          "claim": "PyInstaller --onefile commonly triggers AV/Windows-Defender false positives
(runtime self-extract to temp = packer-like); --onedir avoids extraction and trips far fewer
heuristics. Code-signing whitelists your cert and dramatically reduces (not eliminates) flags.",
          "verdict": "confirmed",
          "reasoning": "Fully corroborated. PyInstaller issue #6754 and pythonguis.com confirm
--onefile extracts to a temp folder at runtime, an extraction-and-execute pattern that mirrors
malware packers, so AV heuristics (incl. Windows Defender) flag it; the PyInstaller bootloader
has itself landed on AV threat lists. Recommended fixes match the claim: use one-folder (onedir)
mode distributed via an installer, code-sign with a valid cert, and submit false-positive
reports. Code signing 'dramatically reduces but does not eliminate' false positives, especially
with Defender/SmartScreen ? verbatim consistent with the sources."
        },
        {
          "claim": "macOS notarization of a frozen Python app requires Developer-ID signing +
Hardened Runtime + entitlements (allow-jit, allow-unsigned-executable-memory, disable-library-
validation) + notarytool submit + stapler; --deep discouraged, sign inside-out. Windows needs
Authenticode (EV cert for clean SmartScreen).",
          "verdict": "confirmed",
          "reasoning": "Substantially confirmed. Sources confirm notarization requires the
Hardened Runtime and, for PyInstaller/Python binaries, the entitlements
com.apple.security.cs.allow-unsigned-executable-memory and com.apple.security.cs.allow-jit, plus
com.apple.security.cs.disable-library-validation, a secure --timestamp, then notarytool submit +
stapler staple, and Developer-ID signing. Windows Authenticode (EV cert for clean SmartScreen
reputation) is standard and correct. ONE nuance worth flagging: the claim says '--deep
discouraged' ? Apple's general guidance does discourage --deep, but at least one PyInstaller-
specific source states --deep is effectively required for PyInstaller .app bundles because all
bundled Python libs must be signed; the cleaner modern practice is to sign inside-out (sign
nested binaries/dylibs first, bundle last) rather than rely on --deep. So 'sign inside-out' is
the right recommendation; '--deep discouraged' is true as Apple guidance but is contested in the
PyInstaller context. Not enough to refute ? the actionable guidance is correct.",
          "corrected_claim": "Confirmed. Nuance: 'sign inside-out' (sign nested dylibs/binaries
first, then the bundle) is the correct modern practice; Apple discourages --deep generally,
though some PyInstaller workflows still rely on it because every bundled lib must be signed.
Also requires a secure --timestamp for notarization."
        },
        {
          "claim": "Nuitka builds are long (5-15 min vs PyInstaller ~30s) and torch standalone
is a known heavy case, but Nuitka ships built-in PyTorch/onnxruntime rules (operator-kernel
pruning) that can shrink the torch payload.",
          "verdict": "confirmed",
          "reasoning": "Corroborated. Nuitka compiles every module through a C compiler, so
medium apps take ~5?15 min (vs PyInstaller's bundle-only ~seconds), and 'very long compile times
for pytorch standalone build' is a tracked known-heavy case (Nuitka issue #754). Recent Nuitka
releases added built-in package-specific rules for PyTorch (and TensorFlow/scikit-learn/SciPy)
that prune unused operator kernels ? directly supporting the 'operator-kernel pruning shrinks
the torch payload' claim. The '~30s' PyInstaller figure is illustrative/order-of-magnitude (real
torch+opencv PyInstaller builds can take minutes), but the relative point (Nuitka much slower
than PyInstaller) is correct.",
          "corrected_claim": "Confirmed in substance. The '~30s PyInstaller' figure is an order-
of-magnitude illustration ? a torch+opencv PyInstaller build can take several minutes ? but
Nuitka is materially slower (5?15 min+, torch standalone being a known heavy case), and its
built-in PyTorch rules do prune unused operator kernels to shrink the payload."
        },
        {
          "claim": "torch needs explicit PyInstaller help: --copy-metadata torch plus hidden
imports like torch._C._jit/torch._C._nvrtc; onefile produces 'file already exists' duplication
warnings for torch/_C pyd.",
          "verdict": "confirmed",
          "reasoning": "Corroborated by PyInstaller discussions/issues (#7621, #8348,
codegenes.net): packaging torch needs --copy-metadata torch (for metadata-reading consumers),
hidden imports such as --hidden-import torch._C._jit and torch._C._nvrtc, and JIT source-file
collection (module_collection_mode pyz+py). The '.pyd file already exists' duplication warnings
under --onefile for torch's _C extension are a documented symptom. All sub-claims verified."
        },
        {
```

```
          "claim": "Owner decision D10 SUPERSEDED the desktop-app plan: deliver a self-hosted
webserver + UI the user spins up (compute/storage as settings), so freezing should target a
CLI/server appliance, not a GUI desktop bundle.",
          "verdict": "confirmed",
          "reasoning": "Verified verbatim against the cited source.
C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md line 39 (Decisions-locked
table, D10): 'Delivery = a self-hosted webserver + UI the user spins up (NOT a desktop app); the
user points at local storage + GPU/CPU OR cloud locations and the UI drives the tool ...
supersedes DESKTOP_APP_PLAN.md; one artifact (this appliance) runs on the owner box, a droplet,
or a GPU host ? same UI, compute/storage are settings.' Dated 2026-06-20, attributed to Owner.
The cited line number (:39) is exact. The inference that freezing should target a CLI/server
appliance rather than a GUI desktop bundle follows directly. Note: DESKTOP_APP_PLAN.md itself is
still status DRAFT/owner-gated/nothing-built, consistent with being superseded before any
build."
        }
      ],
      "overall_notes": "Net verdict on the RECOMMENDATION ('VIABLE as a CLI/server appliance,
tiered, onedir not onefile'): the recommendation's SHAPE is sound and well-supported, but it
rests on two factual defects that should be corrected before it is relied on.\n\nVERIFIED-SOLID
claims: 2 (dual-env WASB blocker, incl. exact torch 1.11.0+cu113), 3 (PyInstaller
GPL+bootloader-exception lets you ship closed binaries), 5 (onefile AV false positives / onedir
+ signing), 6 (mac notarization recipe), 7 (Nuitka slow builds + torch kernel pruning), 8 (torch
hidden imports / copy-metadata / .pyd dup warnings), 9 (owner D10 supersedes DESKTOP_APP_PLAN ?
exact line match), and the core of 1 and 4. The Gemini-only / no-torch first-tier and the WASB-
as-sidecar-or-ONNX second-tier analysis is technically correct and matches the engine's own de-
WSL/native-runner + export-clean direction.\n\nTWO DEFECTS:\n1. CLAIM 4 IS REFUTED ON LICENSE
FACTS. Current Nuitka (v4.1.3, June 2026) is AGPL-3.0+, NOT Apache-2.0; Apache-2.0 only covered
Nuitka 0.4.0?3.9. The recommendation cites 'Nuitka (Apache-2.0 ...)' as a reason to keep it as
the fallback tool ? that licensing label is wrong and must be recorded as AGPL-3.0+ in any
audit. The SAVING GRACE the recommendation can still lean on: Nuitka's GCC-style runtime
exception means your COMPILED OUTPUT does not inherit AGPL and Commercial is optional, so Nuitka
is still usable for closed binaries ? but for a repo whose guardrail is 'MIT/Apache/BSD only for
everything bundled,' note Nuitka is a build-time TOOL (not bundled into the artifact), so its
AGPL doesn't taint the shipped binary. Still, 'Apache-2.0' as stated is false.\n\n2. CLAIM 1 IS
OVERSTATED. The freeze-friendly facts (app object, 127.0.0.1, no workers/reload at
backend/main.py:735) are real and load-bearing, but (a) the engine does NOT call
multiprocessing.freeze_support() (absent from the whole backend), and (b) the cited uvicorn
discussion #1820 is an unanswered bug thread, not a 'recommended pattern.' PyInstaller's own
docs DO recommend freeze_support(); the engine's saving grace is that its job runner is a
ThreadPoolExecutor (job_worker.py:37), not multiprocessing, so the endless-spawn hazard mostly
doesn't apply. Recommend adding freeze_support() defensively when freezing regardless.\n\nMINOR:
claim 6's '--deep discouraged' is true as Apple general guidance but contested for PyInstaller
bundles (some workflows need it); 'sign inside-out' is the right call. Claim 4/size: real
torch+CUDA bundles can hit 1.8?5GB, exceeding the '250MB?1GB+' band, though the 'CUDA pushes
higher' qualifier covers it.\n\nFiles inspected (all absolute):
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py (uvicorn.run at :735; no
freeze_support); backend/pipeline/detectors/wasb_runner.py (WSL conda activate wasb);
backend/pipeline/detectors/native_wasb_runner.py (subprocess to python_bin);
backend/api/job_worker.py (:37 ThreadPoolExecutor max_workers=1);
backend/pipeline/stitcher/compiler.py (:253 -c:v libx264 ? relevant to the ffmpeg-LGPL claim in
the recommendation, confirming the current encoder is GPL libx264); docs/DESKTOP_APP_PLAN.md (D4
libx264/LGPL); docs/TRACKNET_WSL_SETUP.md (:15 wasb env torch 1.11.0+cu113);
C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md (:39 D10 supersedes
DESKTOP_APP_PLAN)."
    }
  },
  {
    "topic": "dual-python-env",
    "research": {
      "topic": "Solving the dual-Python-env problem (main torch 2.x + WASB native runner needs
torch 1.11) for a downloadable, batteries-included badminton highlight-indexer app",
      "options": [
        {
          "name": "(a) conda-pack / constructor ? pre-built relocatable env tarball",
          "how_it_works": "Build the WASB torch-1.11+cu113 conda env ONCE on a matching-OS build
```

machine, then `conda-pack` it into a relocatable tarball that ships inside the app download. On first run the app unpacks it and runs `conda-unpack` to fix prefixes. The WASB subprocess (the existing `native_wasb_runner.py` `_run([python_bin, 'wasb_infer.py', ...])` path) targets that env's python by absolute path (WASB_PYTHON). `constructor` is the same idea but produces a full installer.",
          "pros": [
            "conda-pack is New BSD licensed ? license-clean (does NOT itself drag in Anaconda ToS).",
            "Reuses the existing subprocess seam verbatim: native_wasb_runner.py already invokes the wasb env python by path (WASB_PYTHON) and shells wasb_infer.py ? zero pipeline change.",
            "Deterministic: the exact pinned WASB stack (torch==1.11.0+cu113, timm==0.6.5, hydra-core==1.2.0) ships frozen, so no resolver drift on the user's machine.",
            "No network/compiler needed on the user's box at install time."
          ],
          "cons": [
            "OS-locked: the tarball must be built per target OS (the build OS must match the target OS); a Windows build won't run on Linux/macOS.",
            "Huge: a CUDA torch env is ~4?5 GB on disk; this is a SECOND full torch env on top of the main one ? ~10 GB combined download/footprint.",
            "After conda-unpack the env cannot be relocated again; brittle if the user moves the app folder.",
            "Still needs the user to have a matching CUDA driver for the cu113 build; the GPU win only lands on NVIDIA.",
            "The env was built FROM some channel ? if built from Anaconda `defaults`, that re-introduces the 200-employee ToS trap; must be built from conda-forge only."
          ],
          "maturity": "mature"
        },
        {
          "name": "(b) micromamba ? ship a tiny static binary + create the env on first run from conda-forge",
          "how_it_works": "Bundle the ~10 MB statically-linked `micromamba` binary plus a conda-forge lock file. On first launch the app runs `micromamba create -f wasb-lock.yml -p <app>/envs/wasb` against conda-forge ONLY, materializing the torch-1.11 env locally. The WASB subprocess then targets `<app>/envs/wasb/bin/python` exactly as today.",
          "pros": [
            "micromamba is BSD-3-Clause and conda-forge has NO commercial restriction ? fully license-clean for redistribution, sidesteps the Anaconda `defaults` ToS entirely.",
            "Tiny shipped artifact (just the binary + lock file); the heavy env is fetched once at install.",
            "Cross-platform: the SAME lock-file flow works on Win/Linux/macOS (micromamba is the standard self-contained env manager).",
            "Reuses the existing WASB subprocess seam unchanged.",
            "Lock file = reproducible env; conda-forge carries cu-enabled torch builds."
          ],
          "cons": [
            "Requires network + a multi-GB download at FIRST RUN (bad for the non-tech India coach on a slow/metered uplink ? directly conflicts with the VIDEO_LOCALITY_MODEL bandwidth pain).",
            "First-run env solve can fail or stall on a flaky connection; needs a robust progress/retry UX.",
            "Two full torch envs still coexist on disk (~10 GB after install).",
            "conda-forge cu113-era torch 1.11 builds are old; availability/pinning needs verification per platform.",
            "Still NVIDIA-only for the GPU path."
          ],
          "maturity": "mature"
        },
        {
          "name": "(c) Subprocess against a separate venv/conda env (status quo, generalized)",
          "how_it_works": "Keep exactly what `native_wasb_runner.py` already does: the main engine (torch 2.x) NEVER imports WASB's torch; it shells out to a separate interpreter (WASB_PYTHON) running `wasb_infer.py`, which reads a `Frame,Visibility,X,Y` CSV back. The dual envs never share an address space, so the torch version clash is structurally impossible. Packaging = pick (a)/(b)/(e) to PRODUCE that second env; (c) is the runtime contract, not the

```
install mechanism.",
          "pros": [
            "Already built, tested, and is the de-risk design the DetectorRunner ABC was created
for (base.py:46-54; native_wasb_runner.py module docstring).",
            "Process isolation cleanly defeats the torch-version conflict ? no ABI/symbol
clashes ever.",
            "Env-first config (WASB_PYTHON / WASB_WEIGHTS / WASB_REPO_DIR) means the packager
only has to point env vars at whatever env (a)/(b)/(e) produced.",
            "Owner byte-parity already validated 2026-06-20 on a real GPU (1194/1195 frames)."
          ],
          "cons": [
            "It is a runtime mechanism, NOT a packaging solution ? it does not by itself answer
'how does the second env land on the user's disk'.",
            "Still requires the WASB-SBDT repo src/ present (wasb_infer.py is copied into it) +
the .pth.tar weights fetched from GCS.",
            "Inherits whatever footprint the chosen env-delivery option carries (~5 GB)."
          ],
          "maturity": "mature"
        },
        {
          "name": "(d) Port WASB to modern torch 2.x ? collapse to ONE env",
          "how_it_works": "Recreate the WASB model architecture under torch 2.x and
`load_state_dict()` the existing `wasb_badminton_best.pth.tar` weights into it (PyTorch
guarantees state_dict backward-compat across versions; WASB is a vanilla CNN ? HRNet-style
backbone via timm + an online tracker, no exotic ops). Drop the torch-1.11 pin entirely; WASB
runs in-process in the main torch-2.x env. Effort: re-pin timm to a 2.x-compatible release, set
torch.load(weights_only=False) for the legacy checkpoint, fix any deprecated call sites, then
re-validate against the owner's byte-parity gate.",
          "pros": [
            "ELIMINATES the dual-env problem outright ? one torch, one env, ~5 GB instead of ~10
GB.",
            "WASB is a standard CNN; state_dict load across torch versions is officially
backward-compatible and conv/pool/relu have no breaking ops ? low technical risk.",
            "Massively simplifies packaging: any single-env tool (PyInstaller, micromamba, plain
pip wheels) now suffices.",
            "Removes the WSL/conda-activate machinery for good (the standing D3 desktop
decision).",
            "License-clean: WASB-SBDT is MIT, timm is Apache-2.0."
          ],
          "cons": [
            "Requires re-validating quality: cuDNN/op-kernel differences across torch versions
can shift sub-pixel trajectory outputs ? must re-pass the owner byte-parity / golden-regression
gate on a real GPU (offline suite can't prove the numbers).",
            "timm 0.6.5 -> modern timm may rename/move the HRNet builder; needs a code touch +
pin bump.",
            "Legacy .pth.tar must be loaded with weights_only=False (trusted-source, acceptable
since it's our own weight) or re-serialized once.",
            "One-time engineering + a labelled-video verification pass (owner-side) before it
can be trusted in production."
          ],
          "maturity": "emerging"
        },
        {
          "name": "(d') Export WASB to ONNX ? run with onnxruntime, NO torch in the WASB path",
          "how_it_works": "One-time: load the torch-1.11 model + weights and
`torch.onnx.export()` the detector to a single .onnx file. At runtime the app loads it with
`onnxruntime` (CPUExecutionProvider / CUDAExecutionProvider) ? NO torch dependency at all on the
WASB path. The tracker/windowing stays pure-Python. This is literally the `ONNXRunner` the ABC
already documents (base.py:47-51: 'Load weights locally (torch or onnx) ... Write the exact same
CSV format so trajectory_to_action_windows works verbatim'). healthcheck becomes 'onnx file
exists + onnxruntime imports'.",
          "pros": [
            "Best fit for a DOWNLOADABLE app: onnxruntime is ~tens of MB vs torch's multi-GB;
ship a small artifact, not a 5 GB framework.",
            "Removes BOTH the dual-env clash AND the torch-version pin ? the WASB path no longer
```

```
needs torch 1.11 OR torch 2.x.",
            "Cross-platform CPU + GPU from one artifact: onnxruntime has CPU, CUDA, DirectML
(Windows GPU), and CoreML (Apple) execution providers ? directly answers the no-GPU laptop /
Apple-Silicon unknowns in DESKTOP_APP_PLAN P0.",
            "onnxruntime is MIT ? license-clean. ONNX file is portable and self-describing.",
            "The seam is already designed for this exact runner; CSV contract unchanged so
windowing/stitch are untouched."
          ],
          "cons": [
            "Export work + verification: must confirm every WASB op exports cleanly (HRNet ops
do; the online tracker is Python, not exported) and re-pass the owner byte-parity / golden gate
? numeric drift possible.",
            "Dynamic shapes / the sliding-window batching must be handled in the export (opset +
dynamic_axes).",
            "GPU EPs still need the matching CUDA/cuDNN or DirectML runtime present on the
box.",
            "A one-time export pipeline must be built and the .onnx artifact versioned (likely
alongside weights in GCS)."
          ],
          "maturity": "emerging"
        },
        {
          "name": "(e) Docker-only for the WASB native model",
          "how_it_works": "Ship a Dockerfile/image carrying the frozen torch-1.11+cu113 WASB env
(WASB-SBDT already provides a Dockerfile based on nvcr.io tensorrt:21.06-py3). The main engine
calls the container (docker run / a sidecar) to produce the trajectory CSV; on cloud this maps
onto the already-built CloudRunCompute dispatch + the GPU-worker image (M6).",
          "pros": [
            "Cleanest version pinning and reproducibility; the upstream Dockerfile is a ready
starting point.",
            "Aligns with the existing decoupled-compute Cloud Run + GPU-worker seam (M6) for the
CLOUD/HEAVY tier.",
            "Total isolation of the torch-1.11 stack from the host."
          ],
          "cons": [
            "Requires Docker + NVIDIA Container Toolkit on the user's machine ? a hard NO for
the non-tech India coach on Windows (the target persona). Disqualifies it as the LOCAL/default
delivery.",
            "Multi-GB image pull; GPU passthrough on Windows/WSL is finicky for non-experts.",
            "The upstream base image (nvcr.io tensorrt) needs a license/redistribution check
before bundling.",
            "Heavier ops burden than a subprocess for a single-box appliance."
          ],
          "maturity": "mature"
        },
        {
          "name": "(f) Tier split ? LIGHT = Gemini-only (no torch), HEAVY = separate WASB
download",
          "how_it_works": "The base downloadable app ships with ZERO torch: it runs the Gemini
cloud segmenter (google-genai is already a hard dep), the motion segmenter, and the stub ?
covering ingest -> rallies -> reel end-to-end with no GPU. The HEAVY/on-device WASB stack (env
via (b)/(d')/(a)) is an OPT-IN second download for users who want local, no-upload detection.",
          "pros": [
            "Tiny base install (no torch at all) ? fastest, most reliable download for the non-
tech persona; matches the appliance's 'ships on Gemini today' reality
(LOCAL_APPLIANCE_ARCHITECTURE D10).",
            "Preserves every completed CUJ: Gemini/motion/stub paths need no torch; the engine
already gates segmenters via /api/capabilities honesty.",
            "Defers the multi-GB WASB payload to users who actually have a GPU and want local
detection.",
            "Composes with any other option ? (f) is the PACKAGING TIERING, (a/b/d/d') is how
the HEAVY tier's env is delivered."
          ],
          "cons": [
            "LIGHT tier is cloud-dependent (uploads to Gemini) ? violates the L0 'nothing leaves
```

```
the machine' promise for that tier; must be honest in consent copy.",
            "Two install paths to document/support; users must understand the tier choice.",
            "Doesn't by itself solve the dual-env problem for the HEAVY tier ? it just contains
it behind an opt-in."
          ],
          "maturity": "emerging"
        }
      ],
      "load_bearing_claims": [
        {
          "claim": "WASB-SBDT pins exactly numpy==1.22.4, torch==1.11.0+cu113,
torchvision==0.12.0+cu113 (cu113 extra-index), plus hydra-core==1.2.0, timm==0.6.5,
einops==0.4.1, opencv-python==4.6.0.66 ? i.e. the second env is genuinely a
torch-1.11/CUDA-11.3-era stack, distinct from the engine's modern torch.",
          "confidence": "high",
          "source": "https://github.com/nttcom/WASB-SBDT/blob/main/Dockerfile (read via gh api:
`pip3 install numpy==1.22.4 torch==1.11.0+cu113 torchvision==0.12.0+cu113`)"
        },
        {
          "claim": "WASB-SBDT is MIT licensed (Copyright 2023 NTT Communications) ? code is
commercial-clean under the MIT/Apache/BSD guardrail.",
          "confidence": "high",
          "source": "https://github.com/nttcom/WASB-SBDT/blob/main/LICENSE.md"
        },
        {
          "claim": "The WASB model is a standard CNN (HRNet-style backbone via timm + an online
tracker), no exotic ops ? src/ has models/, detectors/, trackers/, dataloaders/; timm==0.6.5 is
the backbone source. This is what makes both torch-2.x port (d) and ONNX export (d') low-op-
risk.",
          "confidence": "high",
          "source": "github gh api repos/nttcom/WASB-SBDT/contents/src (configs, detectors,
models, trackers) + Dockerfile timm==0.6.5"
        },
        {
          "claim": "The Anaconda ToS trap is the `defaults` channel (repo.anaconda.com), which
requires a paid license for orgs with 200+ employees ? NOT conda the tool. conda-forge /
miniforge / micromamba are BSD-3-Clause and free for ALL use including commercial
redistribution.",
          "confidence": "high",
          "source": "https://github.com/conda-forge/miniforge/blob/main/LICENSE +
https://pydevtools.com/handbook/explanation/is-conda-actually-free/ +
https://ubinfie.github.io/2024/10/15/anaconda-defaults.html"
        },
        {
          "claim": "conda-pack is New BSD licensed and creates relocatable conda envs, but the
build OS must match the target OS and an env cannot be relocated again after conda-unpack.",
          "confidence": "high",
          "source": "https://conda.github.io/conda-pack/ + https://github.com/conda/conda-pack"
        },
        {
          "claim": "micromamba is a fully static, self-contained ~single-binary env manager
(BSD) ideal for bundling; it can create envs from a conda-forge lock file with no conda install
present.",
          "confidence": "high",
          "source": "https://mamba.readthedocs.io/en/stable/user_guide/micromamba.html +
https://github.com/mamba-org/mamba"
        },
        {
          "claim": "Bundling CUDA torch is ~4?5 GB on disk per env (torch GPU wheel ~583 MB,
grows to ~4 GB bundled with CUDA/cuDNN libs) ? so two coexisting torch envs is roughly a 10 GB
footprint; this is the core reason to prefer collapsing to one env or to ONNX.",
          "confidence": "high",
          "source": "https://github.com/orgs/pyinstaller/discussions/8552 +
https://github.com/pytorch/pytorch/issues/17621"
        },
```

```
        {
          "claim": "PyTorch officially guarantees state_dict backward-compat: a state_dict saved
on 1.11 loads into a 2.x model built with the same architecture; standard CNN ops
(conv/pool/relu) have no breaking changes across versions. (Loading a whole pickled model is NOT
guaranteed ? use state_dict.)",
          "confidence": "high",
          "source": "https://docs.pytorch.org/docs/2.11/generated/torch.nn.Module.html
(backward-compat note) + pytorch.org/get-started/pytorch-2-x"
        },
        {
          "claim": "torch.onnx.export produces a self-contained graph that runs with onnxruntime
WITHOUT the torch framework; onnxruntime offers CPU, CUDA, DirectML (Windows GPU) and CoreML
execution providers and is MIT-licensed ? a small (tens-of-MB) cross-platform runtime.",
          "confidence": "high",
          "source": "https://onnxruntime.ai/blogs/pytorch-on-the-edge +
https://onnxruntime.ai/docs/tutorials/export-pytorch-model.html"
        },
        {
          "claim": "The engine ALREADY ships the subprocess seam (c) and the ABC explicitly
documents an ONNXRunner (d') de-risk path: native_wasb_runner.py invokes WASB_PYTHON by path and
shells wasb_infer.py to a Frame,Visibility,X,Y CSV; base.py:46-54 names a 'LinuxNativeRunner (or
ONNXRunner)' that loads weights 'torch or onnx' and writes 'the exact same CSV format so
trajectory_to_action_windows works verbatim'. Owner byte-parity validated 2026-06-20 (1194/1195
frames).",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/native_wasb_runner.py:52,110-113,210-237 + base.py:46-54"
        },
        {
          "claim": "The engine's main env is modern torch 2.x (requirements.txt pins
torch>=2.12.0 / torchvision>=0.27.0; pyproject keeps torch OUT of deps because CUDA wheels need
--index-url) ? confirming the version clash with WASB's 1.11 is real and unavoidable in a single
shared interpreter.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/requirements.txt
(torch>=2.12.0) + pyproject.toml:8-11,58-65"
        },
        {
          "claim": "Delivery is a self-hosted webserver+UI the user spins up (NOT a desktop
app), targeting non-tech recreational players on commodity (often no-GPU) hardware; D3 already
commits to dropping WSL and running the detector natively per-OS; D4 requires MIT/Apache/BSD-
only bundled deps incl. weights.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md D10
+ C:/Users/avidu/Projects/badminton-highlight-indexer/docs/DESKTOP_APP_PLAN.md D3,D4 §3.1"
        }
      ],
      "recommendation": "Layer two decisions, do NOT pick a single bullet.\n\n1) PACKAGING
TIERING = (f). Ship a torch-FREE LIGHT tier as the default download: google-genai is already a
hard dep, so Gemini + motion + stub deliver the full ingest -> rallies -> reel CUJ with zero
torch, a tiny install, and a reliable download for the non-tech India-coach persona on a slow
uplink. The on-device WASB detector is an OPT-IN HEAVY add-on. This preserves every completed
CUJ and matches the appliance's already-shipped 'runs on Gemini today' reality
(LOCAL_APPLIANCE_ARCHITECTURE D10).\n\n2) For the HEAVY tier, the cleanest long-term answer is
(d') EXPORT WASB TO ONNX + onnxruntime ? which the codebase was explicitly designed for (base.py
names the ONNXRunner; native_wasb_runner.py is the sibling that already shells out to the same
CSV contract). It DISSOLVES the dual-env problem entirely: the WASB path needs neither torch
1.11 nor torch 2.x, ships a ~tens-of-MB runtime instead of a 5 GB framework, and gives CPU +
NVIDIA(CUDA) + Windows(DirectML) + Apple(CoreML) from one artifact ? directly answering the no-
GPU-laptop and Apple-Silicon unknowns the DESKTOP_APP_PLAN P0 spike was created to
resolve.\n\n3) BRIDGE / fallback while ONNX export is validated: collapse to one env via (d)
torch-2.x port if the timm/HRNet load is quick, OR ship the second env via (b) micromamba-from-
conda-forge (license-clean, sidesteps the Anaconda `defaults` ToS) reusing the EXISTING
subprocess seam (c) unchanged ? pointing WASB_PYTHON at the micromamba env. Use conda-pack (a)
only as a build-time convenience for an offline-friendly first-run (no network), but mind its
```

OS-lock + no-relocate caveat. AVOID (e) Docker as the LOCAL default ? it requires Docker + NVIDIA toolkit, a non-starter for the target user (keep Docker only for the CLOUD/HEAVY worker, where it aligns with the M6 Cloud Run + GPU seam).\n\nLICENSING NON-NEGOTIABLE: whatever env path is chosen, build/solve from conda-forge ONLY (never the Anaconda `defaults` channel) ? micromamba/miniforge/conda-pack tools are all BSD; the ToS trap is the channel, not the tool. WASB-SBDT (MIT), timm (Apache), onnxruntime (MIT) are all clean.\n\nBoth ONNX (d') and torch-port (d) MUST re-pass the owner byte-parity / golden-regression gate on a real GPU before production ? numeric drift across torch versions / the ONNX op set is the one real risk, and the offline mocked suite cannot prove the numbers (hand to owner with the labelled corpus).",
      "fit_for_this_app": "Our engine is uniquely well-positioned: the dual-env split is already isolated behind a SUBPROCESS seam (native_wasb_runner.py shells WASB_PYTHON -> wasb_infer.py -> a Frame,Visibility,X,Y CSV; windowing/stitch are model-agnostic), and the DetectorRunner ABC (base.py:46-54) explicitly pre-designed an ONNXRunner that 'loads weights torch or onnx' and 'writes the exact same CSV format so trajectory_to_action_windows works verbatim'. So (d') ONNX or (d) torch-port slot in WITHOUT touching the pipeline, the FastAPI loop, or the decoupled compute/storage seams (M1-M8). FFmpeg/ffprobe stay system deps (LGPL build per D4); the wasb_badminton_best.pth.tar weights stay in GCS and download on demand (or convert to a versioned .onnx alongside them) ? fitting the existing StorageBackend(gcs) + presets profiles. For the non-tech India coach on Windows, the LIGHT torch-free tier (Gemini, already a hard dep) is the reliable default, and the appliance's compute-as-a-setting + /api/capabilities honesty already lets the UI offer only what the box can truthfully do ? so an opt-in ONNX HEAVY tier surfaces cleanly. This also retires the WSL2 requirement (D3) that the current owner-only path depends on, and the licensing guardrail is satisfied end-to-end (WASB MIT, timm Apache, onnxruntime/micromamba/conda-pack all MIT/BSD, conda-forge-only channel)."
    },
    "verdict": {
      "claims_checked": [
        {
          "claim": "WASB-SBDT pins exactly numpy==1.22.4, torch==1.11.0+cu113, torchvision==0.12.0+cu113 (cu113 extra-index), plus hydra-core==1.2.0, timm==0.6.5, einops==0.4.1, opencv-python==4.6.0.66 ? a torch-1.11/CUDA-11.3-era stack distinct from the engine's modern torch.",
          "verdict": "confirmed",
          "reasoning": "Fetched nttcom/WASB-SBDT/main/Dockerfile via gh api. Lines verbatim: `pip3 install numpy==1.22.4 torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url https://download.pytorch.org/whl/cu113` and `pip3 install hydra-core==1.2.0 ... opencv-python==4.6.0.66 ... einops==0.4.1 timm==0.6.5 ...`. Every pin in the claim matches exactly. Base image is nvcr.io/nvidia/tensorrt:21.06-py3. This is unambiguously a torch-1.11/cu113 stack, vs the engine's torch>=2.12.0 (requirements.txt:9). Confirmed."
        },
        {
          "claim": "WASB-SBDT is MIT licensed (Copyright 2023 NTT Communications) ? commercial-clean under the MIT/Apache/BSD guardrail.",
          "verdict": "confirmed",
          "reasoning": "Fetched LICENSE.md via gh api: `MIT License / Copyright (c) 2023 NTT Communications Corporation` followed by the standard MIT grant text. MIT satisfies the MIT/Apache-2.0/BSD-only licensing guardrail. The claim's 'NTT Communications' is the holder 'NTT Communications Corporation' ? same entity. Confirmed. (Caveat orthogonal to this claim: MIT covers the CODE; the redistributable WEIGHTS' license is asserted elsewhere in the repo's notes as MIT but was not re-verified here.)"
        },
        {
          "claim": "The WASB model is a standard CNN (HRNet-style backbone via timm + an online tracker), no exotic ops; src/ has models/, detectors/, trackers/, dataloaders/; timm==0.6.5 is the backbone source ? making torch-2.x port (d) and ONNX export (d') low-op-risk.",
          "verdict": "confirmed",
          "reasoning": "MOSTLY TRUE but one sub-claim is WRONG on mechanism. CONFIRMED: src/ contains configs, dataloaders, datasets, detectors, losses, models, optimizers, runners, trackers, utils. The shipped default model is HRNet ? eval.yaml defaults to `model: wasb` -> model/wasb.yaml `name: hrnet`, tracker `online`, detector `tracknetv2`. models/hrnet.py is a self-contained HRNet built from a YAML spec of standard STEM/STAGE conv blocks (BOTTLENECK/BASIC, FUSE SUM, FINAL_CONV_KERNEL 1) ? only `import torch`/`torch.nn`, plain conv/pool/bn/relu/deconv ops. So 'standard CNN, no exotic ops -> low-op-risk for torch-2.x and ONNX' is well-supported (arguably strengthened). REFUTED sub-claim: 'HRNet-style backbone VIA timm' / 'timm is the backbone source' is incorrect. GitHub code-search for `timm` in the repo

returns total_count=1, the only file being the Dockerfile. timm is a declared (and effectively unused/transitive) dependency, NOT imported by hrnet.py or the shipped model path; HRNet is hand-implemented in src/models/hrnet.py.",
          "corrected_claim": "The shipped WASB model is a self-contained, hand-implemented HRNet (src/models/hrnet.py, standard conv/bottleneck/basic blocks, torch-only imports) selected via configs/model/wasb.yaml (name: hrnet) plus an 'online' tracker; src/ has models/, detectors/, trackers/, dataloaders/. timm==0.6.5 is pinned in the Dockerfile but is NOT the backbone source (it is not imported anywhere in src/ ? code-search hits only the Dockerfile). The 'standard CNN, no exotic ops' premise still holds and makes both the torch-2.x port and ONNX export low-op-risk."
        },
        {
          "claim": "The Anaconda ToS trap is the `defaults` channel (repo.anaconda.com) requiring a paid license for orgs with 200+ employees ? NOT conda the tool. conda-forge / miniforge / micromamba are BSD-3-Clause and free for ALL use including commercial redistribution.",
          "verdict": "confirmed",
          "reasoning": "Web search corroborates: Anaconda's paid-license requirement attaches to the 'defaults' channel (Anaconda Repository, repo.anaconda.com) for organizations >=200 employees/contractors; conda the tool (BSD) and community channels like conda-forge are free regardless of org size when used via Miniforge/non-Anaconda installers. Nuance worth flagging (does not refute the claim): a March 2024 ToS update removed prior nonprofit/government/academic exemptions, so 'free for ALL non-defaults use' is right for the channel question but the broader Anaconda ToS has tightened around defaults usage. miniforge LICENSE is BSD-3-Clause; micromamba is BSD-3-Clause (confirmed separately in claim 6's search). Confirmed."
        },
        {
          "claim": "conda-pack is New BSD licensed, creates relocatable conda envs, but the build OS must match the target OS and an env cannot be relocated again after conda-unpack.",
          "verdict": "confirmed",
          "reasoning": "conda-pack docs/README confirm all three facts: (1) offered under a New BSD license; (2) builds relocatable envs for deployment where python/conda isn't installed; (3) 'The OS where the environment was built must match the OS of the target' (Windows env can't go to Linux) and 'Once an environment is unpacked and conda-unpack has been executed, it cannot be relocated.' Confirmed."
        },
        {
          "claim": "micromamba is a fully static, self-contained ~single-binary env manager (BSD), ideal for bundling; it can create envs from a conda-forge lock file with no conda install present.",
          "verdict": "confirmed",
          "reasoning": "mamba docs confirm micromamba is a single-file statically-linked executable, BSD-3-Clause licensed, and can create environments from lock files (`micromamba create -n my-env -f conda-lock.yml`) without installing conda-lock/conda ? when a lockfile is used it skips channel queries and the solver. Matches the claim. Confirmed."
        },
        {
          "claim": "Bundling CUDA torch is ~4-5 GB on disk per env (GPU wheel ~583 MB, grows to ~4 GB bundled with CUDA/cuDNN) ? so two coexisting torch envs is ~10 GB; the core reason to prefer one env or ONNX.",
          "verdict": "confirmed",
          "reasoning": "The orders of magnitude are right. pytorch/pytorch#17621 confirms the GPU wheel is ~582-583 MB; a typical PyTorch-bearing image is 2-4 GB with torch+CUDA+cuDNN occupying a large share. A fully-installed CUDA torch env (torch + nvidia-cu* CUDA/cuDNN runtime wheels) commonly lands in the ~4-5 GB range, so two coexisting envs ~10 GB is a fair estimate. Mild caveat: the exact 4-5 GB/env and 10 GB total are reasonable approximations, not a single authoritative figure ? they vary with CUDA version and whether CUDA libs are bundled vs shared. The directional argument (two torch envs are very large -> prefer collapsing to one or ONNX) is sound. Confirmed (approximate figures)."
        },
        {
          "claim": "PyTorch officially guarantees state_dict backward-compat: a state_dict saved on 1.11 loads into a 2.x model built with the same architecture; standard CNN ops have no breaking changes across versions. Loading a whole pickled model is NOT guaranteed ? use state_dict.",

```
          "verdict": "confirmed",
          "reasoning": "PyTorch's official save/load tutorial confirms the operative
recommendation: saving the state_dict is 'the recommended method'; saving the whole model with
pickle binds the data 'to the specific classes and the exact directory structure used when the
model is saved,' so 'your code can break in various ways when used in other projects or after
refactors.' A state_dict is just a dict of tensors keyed by parameter name, so it loads into an
identically-architected module independent of version. Nuance on wording: PyTorch does not
publish a formal versioned 'backward-compatibility GUARANTEE' for state_dicts the way the claim
phrases it; the practical robustness (and the explicit steer away from whole-model pickling) is
what's documented. The claim's actionable conclusion (use state_dict, not pickled whole model;
standard conv/pool/relu carry over) is correct.",
          "corrected_claim": "PyTorch officially RECOMMENDS state_dict save/load over pickling
the whole model (the pickled-model path binds to class paths/dir structure and breaks across
refactors). A state_dict (tensors keyed by parameter name) loads into an identically-architected
module regardless of the torch version, and standard CNN ops (conv/pool/relu) are stable across
versions ? so a 1.11-saved WASB HRNet state_dict is expected to load into a 2.x rebuild of the
same architecture. (This is documented best-practice + the nature of state_dicts, not a formally
published cross-version 'guarantee'; numeric parity must still be re-validated.)"
        },
        {
          "claim": "torch.onnx.export produces a self-contained graph that runs with onnxruntime
WITHOUT torch; onnxruntime offers CPU, CUDA, DirectML (Windows GPU), CoreML execution providers
and is MIT-licensed ? a small (tens-of-MB) cross-platform runtime.",
          "verdict": "confirmed",
          "reasoning": "onnxruntime docs confirm it is an independent inference framework that
runs ONNX model files standalone (no PyTorch needed) and supports execution providers including
CPU (default/fallback), NVIDIA CUDA, Windows DirectML, and Apple CoreML (plus TensorRT,
OpenVINO, QNN, NNAPI, ROCm, etc.). License verified directly via gh api
repos/microsoft/onnxruntime/license -> SPDX 'MIT'. The 'tens-of-MB' size is right for the
CPU/base runtime wheel (the CUDA EP package is larger, hundreds of MB, but the framework itself
is far smaller than a ~4 GB torch+CUDA env). torch.onnx.export producing a self-contained graph
is standard PyTorch behavior. Confirmed."
        },
        {
          "claim": "The engine ALREADY ships the subprocess seam (c) and the ABC documents an
ONNXRunner (d') de-risk path: native_wasb_runner.py invokes WASB_PYTHON by path and shells
wasb_infer.py to a Frame,Visibility,X,Y CSV; base.py:46-54 names a 'LinuxNativeRunner (or
ONNXRunner)' that loads weights 'torch or onnx' and writes 'the exact same CSV format so
trajectory_to_action_windows works verbatim.' Owner byte-parity validated 2026-06-20 (1194/1195
frames).",
          "verdict": "confirmed",
          "reasoning": "Verified against the actual files. native_wasb_runner.py: resolves
python_bin from env WASB_PYTHON (from_indexing_cfg, line ~89) and invokes it by path via
subprocess.run (_run, lines 110-113); builds argv shelling `wasb_infer.py` (line 215) and writes
`{key}_wasb.csv` (line 208); module docstring + wasb_infer.py:8 confirm the
`Frame,Visibility,X,Y` header. base.py lines 46-54 verbatim: 'Optional Linux-native path (the
main de-risk goal): A LinuxNativeRunner (or ONNXRunner) would: ... Load weights locally (torch
or onnx) ... Write the exact same CSV format so trajectory_to_action_windows works verbatim.'
Byte-parity note in native_wasb_runner.py:30-32: 'Validated 2026-06-20 on an RTX 2070 (via WSL):
native streaming vs cached WSL-disk CSV matched 1194/1195 frames ... two native runs were byte-
identical.' All cited line numbers match. Confirmed."
        },
        {
          "claim": "The engine's main env is modern torch 2.x (requirements.txt pins
torch>=2.12.0 / torchvision>=0.27.0; pyproject keeps torch OUT of deps because CUDA wheels need
--index-url), confirming the clash with WASB's 1.11 is real and unavoidable in a single shared
interpreter.",
          "verdict": "confirmed",
          "reasoning": "requirements.txt lines 9-10: `torch>=2.12.0` and `torchvision>=0.27.0`.
pyproject.toml header (lines 8-11) and [project.optional-dependencies] gpu (lines 58-65) state
explicitly that torch/torchvision are deliberately NOT in [project.dependencies] because 'their
CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url), which PEP 621
cannot express,' with the gpu extra intentionally empty and install instructions out-of-band.
WASB requires torch==1.11.0; a single interpreter cannot hold both torch 1.11 and torch>=2.12
simultaneously, so the dual-env (or port/ONNX) need is real. Confirmed."
```

```
        },
        {
          "claim": "Delivery is a self-hosted webserver+UI the user spins up (NOT a desktop app), for non-tech recreational players on commodity (often no-GPU) hardware; D3 commits to dropping WSL and running the detector natively per-OS; D4 requires MIT/Apache/BSD-only bundled deps incl. weights.",
          "verdict": "confirmed",
          "reasoning": "Verified both cited docs. khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md D10 (row, 2026-06-20): 'Delivery = a self-hosted webserver + UI the user spins up (NOT a desktop app) ... supersedes DESKTOP_APP_PLAN.md' ? exact match for the delivery framing. DESKTOP_APP_PLAN.md D3: 'Drop the WSL2 dependency; run the detector natively per-OS' (Windows-native, Linux+NVIDIA, macOS MPS/CPU). D4: 'Commercial-clean licensing for everything bundled: MIT / Apache-2.0 / BSD only (code, weights)', with the ffmpeg LGPL-build caveat. §3.1 confirms the no-discrete-GPU commodity-hardware target. One tension to surface (does not refute): the claim cites BOTH D10 and DESKTOP_APP_PLAN as if mutually consistent, but D10 explicitly SUPERSEDES DESKTOP_APP_PLAN.md ? the appliance (webserver+UI) is the current delivery model and the desktop-app doc is superseded; D3/D4's native-detector + licensing decisions still carry forward, but anyone citing the desktop-app doc should note its superseded status. The 'runs on Gemini today' reality is confirmed (LOCAL_APPLIANCE lines 16/23/39). Confirmed."
        }
      ],
      "overall_notes": "11 of 12 claims confirmed; 1 (claim 3) confirmed in its load-bearing conclusion but REFUTED on a sub-mechanism. The recommendation's foundation is sound but two corrections matter:\n\n1) CLAIM 3 timm error (the only real refutation). The shipped WASB detector is NOT a 'timm backbone' ? it is a hand-implemented HRNet in src/models/hrnet.py built from a YAML block-spec, importing only torch. GitHub code-search for `timm` across nttcom/WASB-SBDT returns exactly ONE hit: the Dockerfile (a declared, effectively unused/transitive dep). This does NOT weaken ? it STRENGTHENS ? the recommendation: the model is plain conv/bottleneck/basic-block CNN with standard ops (FINAL_CONV_KERNEL=1, FUSE SUM, optional deconv), which is exactly what makes both the torch-2.x port (d) and ONNX export (d') genuinely low-op-risk. When scoping the ONNX/port work, do not assume a timm dependency exists to port; there is none to worry about.\n\n2) WORDING nuances on claims 6 & 7 (confirmed, not refuted). (a) The '~4-5 GB/env, ~10 GB for two envs' figures are fair approximations, not a single authoritative number ? they swing with CUDA version and whether CUDA libs are bundled. (b) PyTorch does not publish a formal versioned 'state_dict backward-compatibility GUARANTEE'; what's documented is the strong RECOMMENDATION to use state_dict over whole-model pickling (which binds to class paths and breaks on refactor). The actionable conclusion (use state_dict; standard CNN ops carry across versions) is correct either way.\n\n3) DOC-CITATION tension on claim 11 (confirmed, flag only). The claim cites LOCAL_APPLIANCE D10 AND DESKTOP_APP_PLAN D3/D4 together, but D10 explicitly states it SUPERSEDES DESKTOP_APP_PLAN.md. The current delivery model is the self-hosted webserver+UI appliance (runs on Gemini today, native-WASB on a GPU host later); the desktop-app doc is superseded though its native-detector and MIT/Apache/BSD-licensing decisions still carry forward. Cite the appliance doc as authoritative for delivery shape.\n\nVERIFICATION SCOPE: External facts (WASB pins/license/src layout, Anaconda ToS, conda-pack, micromamba, torch wheel size, onnxruntime EPs+MIT, PyTorch save/load guidance) verified via gh api + web + direct license API. Local code/doc citations (native_wasb_runner.py line refs, base.py:46-54, requirements.txt, pyproject.toml, DESKTOP_APP_PLAN D3/D4/§3.1, LOCAL_APPLIANCE D10) verified by reading the files. NOT verifiable locally and correctly deferred by the recommendation itself: the numeric byte-parity / golden-regression gate for the torch-2.x port AND the ONNX export ? drift across torch versions / the ONNX opset is the one genuine risk and must be re-run by the owner on a real GPU with the labelled corpus. The offline mocked suite asserts the CSV contract, not the numbers. The recommendation already calls this out; it is the correct gate. Relevant files: C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/native_wasb_runner.py, .../base.py, .../wasb_infer.py, .../requirements.txt, .../pyproject.toml, .../docs/DESKTOP_APP_PLAN.md, C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md."
    }
  },
  {
    "topic": "weights-and-ffmpeg-bundling",
    "research": {
      "topic": "Bundling or obtaining MODEL WEIGHTS (WASB shuttle .pth.tar) and FFMPEG plus FFPROBE for a downloadable batteries-included Python app for a non-technical Windows user",
      "options": [
        {
```

```
          "name": "WEIGHTS-A: Download-on-first-run from a public GCS or CDN bucket we control
(RECOMMENDED for weights)",
          "how_it_works": "First run downloads the WASB shuttle weight from a public read-only
GCS bucket over HTTPS into a per-user cache, verifies a pinned SHA-256, then sets WASB_WEIGHTS
to the cached path. Reuses the existing GCS mirror, GCSStorage.fetch_to_local at
backend/storage/gcs.py line 30, and the WASB_WEIGHTS env override at native_wasb_runner.py line
90. A public-read fetch can be a plain HTTPS GET needing no google-cloud-storage dependency or
credentials.",
          "pros": [
            "Reuses the existing GCS mirror plus fetch_to_local plus WASB_WEIGHTS seam, so
minimal new code",
            "Public-read fetch needs only stdlib or requests, no new license dependency, no
credentials for the user",
            "Keeps the wheel and installer tiny and dodges the PyPI 100 MiB per-file limit",
            "Full control of versioning and egress; frontable with a CDN for India latency",
            "Checksum-verify plus cache-once gives offline reruns and tamper detection"
          ],
          "cons": [
            "We pay egress and must keep the bucket alive indefinitely",
            "Needs network on first run; must be loud, resumable, and progress-barred, not a
silent hang",
            "Open download surface, mitigated by read-only, checksum pinning, and a CDN or rate
limit",
            "We must self-host the weight lawfully, which is fine because WASB-SBDT is MIT"
          ],
          "maturity": "mature"
        },
        {
          "name": "WEIGHTS-B: Download-on-first-run from Hugging Face Hub via huggingface_hub",
          "how_it_works": "Mirror the weight into an HF model repo we own and call
hf_hub_download with the repo id and filename on first run. The library manages a content-hashed
cache, resumable downloads, integrity, and an offline switch via HF_HUB_OFFLINE or
local_files_only. Point WASB_WEIGHTS at the returned cached path.",
          "pros": [
            "Batteries-included downloader with resume, cache, content-hash verify, retry, and
progress bars",
            "huggingface_hub is Apache-2.0 and license-clean, a single well-maintained
dependency",
            "Built-in offline mode plus a stable cache for rerun and offline UX",
            "HF hosts the bytes and a CDN, so no egress bill or availability burden on us",
            "Easy public hosting with a model card noting the MIT origin"
          ],
          "cons": [
            "Adds huggingface_hub plus transitive deps, all permissive but more to license-
audit",
            "Third party we do not control, with gating, rename, or rate-limit risk and outages
hitting first-run",
            "Must preserve the MIT license and attribution in the re-hosted repo",
            "Corporate or institutional proxies sometimes block huggingface.co"
          ],
          "maturity": "mature"
        },
        {
          "name": "WEIGHTS-C: Vendor the weight file inside the wheel or installer",
          "how_it_works": "Ship the weight as package data inside the wheel or the app installer
so it is on disk with zero network, resolved via importlib.resources and fed to WASB_WEIGHTS.",
          "pros": [
            "Truly offline from second one, the best non-tech UX with no download failure mode",
            "No bucket or HF runtime dependency; deterministic and air-gap friendly",
            "Simplest runtime code path"
          ],
          "cons": [
            "PyPI hard-caps one file at 100 MiB and a project at 10 GiB, viable only if weight
plus wheel stay under 100 MB and risky as more sports or weights are added",
            "Bloats every download even for Gemini-only users who need no weights",
```

```
            "A new weight version means re-cutting the whole wheel or installer",
            "Couples model cadence to release cadence and creates large CI artifacts"
          ],
          "maturity": "mature"
        },
        {
          "name": "FFMPEG-A: System-install via winget plus PATH discovery with a configurable
override (RECOMMENDED for ffmpeg)",
          "how_it_works": "Route every ffmpeg and ffprobe call through one new resolver that
checks a configured path or the FFMPEG_BINARY env var, then shutil.which, then a known install
dir, otherwise failing loud with the exact winget command. First-run setup extends
scripts/doctor.py to detect a missing binary and offer or auto-run the winget install of
Gyan.FFmpeg. Because the user installs ffmpeg, we never redistribute it and the GPL build
copyleft never reaches our MIT app.",
          "pros": [
            "Zero redistribution by us, so the GPL versus LGPL landmine is sidestepped
entirely",
            "winget ships on Windows 10 and 11; one command gets a full GPL build with libx264,
libfreetype, and ffprobe, so the engine stitch, watermark, and duration probe work unchanged",
            "Fixes the imageio-ffmpeg ffprobe gap because full builds ship ffprobe",
            "doctor.py already checks PATH and prints the winget command, a small step to a
clean resolver plus auto-offer",
            "shutil.which plus a configurable path is standard and debuggable and works for
users who already have ffmpeg"
          ],
          "cons": [
            "Not fully batteries-included; a one-time install step on first run, mitigated by
auto-invoking winget with consent, progress, and a re-check",
            "winget can be absent or disabled on locked-down boxes, needing a documented manual
fallback to unzip a static build and set the config path",
            "Needs the small refactor of the bare ffmpeg and ffprobe subprocess strings in
video.py and compiler.py to go through the resolver"
          ],
          "maturity": "mature"
        },
        {
          "name": "FFMPEG-B: Bundle imageio-ffmpeg (BSD-2 wrapper, ffmpeg binary in the wheel)",
          "how_it_works": "Add imageio-ffmpeg; its platform wheel bundles an ffmpeg executable
around 60 MiB. Use get_ffmpeg_exe to locate it and set IMAGEIO_FFMPEG_EXE or our resolver to
that path.",
          "pros": [
            "Zero-touch ffmpeg; pip install pulls the binary with no winget or PATH step",
            "The Python wrapper is BSD-2-Clause and clean",
            "Cross-platform wheels from one dependency"
          ],
          "cons": [
            "BLOCKER: bundles only ffmpeg, not ffprobe, but the engine requires ffprobe in
video.py and compiler.py, so ffprobe must come from elsewhere anyway",
            "LICENSE LANDMINE: its binary uses libx264 for mp4 which requires --enable-gpl, so
it is effectively a GPL build and redistributing it in our installer pulls GPL onto the bundle,
violating the MIT, Apache, BSD-only guardrail",
            "Minimal build; drawtext, overlay watermark, and specific encoders not guaranteed
across versions",
            "Adds about 60 MiB for users who may only use the Gemini path"
          ],
          "maturity": "mature"
        },
        {
          "name": "FFMPEG-C: Redistribute a static LGPL ffmpeg plus ffprobe build (BtbN lgpl
variant)",
          "how_it_works": "Ship BtbN LGPL static build of ffmpeg plus ffprobe beside the app and
point the resolver at them. Per the desktop-plan D4 decision, use LGPL only and switch to
libopenh264 (BSD), the native LGPL H.264 encoder, or OS hardware encoders like NVENC instead of
libx264; the engine c:v libx264 default must change to an LGPL-clean encoder.",
          "pros": [
```

```
            "Batteries-included and ships ffprobe, unlike imageio-ffmpeg, with both binaries
present offline",
            "LGPL omits GPL x264, so redistribution is permissible if re-link obligations are
met",
            "We control the exact build and version and it aligns with the ratified D4"
          ],
          "cons": [
            "LGPL still imposes redistributor obligations such as an offer to re-link and
shipping the LGPL license plus an ffmpeg source pointer",
            "The LGPL build excludes libx264, so the engine c:v libx264 at compiler.py line 253
fails and must swap to libopenh264, native, or NVENC, plus confirm libfreetype for drawtext",
            "We must host and track the build plus its source for the LGPL source offer, and
per-OS binaries bloat the installer",
            "More moving parts than telling winget to install it"
          ],
          "maturity": "emerging"
        }
      ],
      "load_bearing_claims": [
        {
          "claim": "PyPI default limits a single uploaded file to 100 MiB and a whole project to
10 GiB; larger files require a per-project limit-increase request after a sub-limit release
exists. This caps the vendor-weights-in-wheel option.",
          "confidence": "high",
          "source": "https://docs.pypi.org/project-management/storage-limits/"
        },
        {
          "claim": "imageio-ffmpeg bundled ffmpeg binary is effectively a GPL build because it
uses libx264 as the default mp4 codec and libx264 requires enable-gpl; redistributing it in our
installer would impose GPL on the bundle.",
          "confidence": "high",
          "source": "https://github.com/imageio/imageio-ffmpeg +
https://www.ffmpeg.org/legal.html"
        },
        {
          "claim": "imageio-ffmpeg bundles only ffmpeg, not ffprobe. The engine hard-requires
ffprobe for video duration at backend/utils/video.py line 11 and compiler.py line 113, so
imageio-ffmpeg alone cannot satisfy the engine.",
          "confidence": "high",
          "source": "https://github.com/imageio/imageio-ffmpeg/issues/23"
        },
        {
          "claim": "The imageio-ffmpeg wrapper is BSD-2-Clause; binary resolution order is the
IMAGEIO_FFMPEG_EXE env var, then the bundled binary, then conda, then system PATH, and the env
var is used directly without validation.",
          "confidence": "high",
          "source": "https://github.com/imageio/imageio-
ffmpeg/blob/main/imageio_ffmpeg/_utils.py"
        },
        {
          "claim": "gyan.dev and BtbN GPL static Windows builds installed by winget Gyan.FFmpeg
or BtbN.FFmpeg.GPL are GPLv3 and include libx264, libx265, and ffprobe. Fine to use when user-
installed but not to redistribute in an MIT, Apache, BSD-only bundle.",
          "confidence": "high",
          "source": "https://www.gyan.dev/ffmpeg/builds/ + https://github.com/BtbN/FFmpeg-
Builds/issues/29"
        },
        {
          "claim": "BtbN LGPL builds omit GPL components and so exclude libx264; H.264 in an
LGPL build needs the native LGPL encoder or libopenh264 which is BSD. Shipping an LGPL ffmpeg
therefore requires changing the engine c:v libx264 default.",
          "confidence": "high",
          "source": "https://github.com/BtbN/FFmpeg-Builds + https://www.ffmpeg.org/legal.html"
        },
        {
```

```
          "claim": "Embedding ffmpeg built with enable-gpl makes the whole build GPL and forbids
embedding it in a non-GPL redistributed bundle; LGPL builds allow redistribution subject to re-
link or source-offer obligations. The clean escape is to not redistribute ffmpeg and have the
user system-install it.",
          "confidence": "high",
          "source": "https://www.ffmpeg.org/legal.html"
        },
        {
          "claim": "huggingface_hub is Apache-2.0 and provides hf_hub_download with a content-
hashed cache, resumable downloads, and an offline switch via HF_HUB_OFFLINE or local_files_only,
giving a batteries-included first-run downloader.",
          "confidence": "high",
          "source": "https://pypi.org/project/huggingface-hub/ +
https://huggingface.co/docs/huggingface_hub/en/package_reference/file_download"
        },
        {
          "claim": "WASB-SBDT from nttcom is MIT-licensed, so its pretrained shuttle weights are
redistributable and we can self-host them on our GCS bucket or on HF. Verify each weight file
actual host and provenance before mirroring, since the README points at MODEL_ZOO.md and some
academic weights are Google-Drive-hosted.",
          "confidence": "medium",
          "source": "https://github.com/nttcom/WASB-SBDT"
        },
        {
          "claim": "The engine already has the seams for download-on-first-run:
GCSStorage.fetch_to_local is overwrite-aware and does mkdir plus download, and a WASB_WEIGHTS
env override is consumed by NativeWasbConfig.from_indexing_cfg with env winning over config, so
weight-fetch can set WASB_WEIGHTS with no detector code change.",
          "confidence": "high",
          "source": "backend/storage/gcs.py line 30 ;
backend/pipeline/detectors/native_wasb_runner.py lines 80-97"
        },
        {
          "claim": "ffmpeg and ffprobe are invoked as bare PATH strings via subprocess:
backend/utils/video.py uses ffprobe at line 11 and ffmpeg at line 33; the stitcher uses ffprobe
at compiler.py line 113, c:v libx264 at line 253, and drawtext or overlay watermark at lines
179-185. A single resolver helper is the minimal change for a configurable or bundled binary.",
          "confidence": "high",
          "source": "backend/utils/video.py ; backend/pipeline/stitcher/compiler.py"
        },
        {
          "claim": "scripts/doctor.py already does first-run checks: shutil.which for ffmpeg and
ffprobe, GPU detection, torch out-of-band index selection cu124 or cpu, and on Windows it prints
exactly the winget install command for Gyan.FFmpeg. The recommended first-run setup extends this
rather than adding new tooling.",
          "confidence": "high",
          "source": "scripts/doctor.py lines 84-108 and 15-34"
        },
        {
          "claim": "PyInstaller-freezing with CUDA torch yields about 2.6 to 4 GB bundles
because CUDA and cuDNN libs cannot be split; small bundles require CPU-only torch. This argues
against a frozen all-in-one binary and for pip or installer plus on-demand weights plus system
ffmpeg.",
          "confidence": "high",
          "source": "https://github.com/orgs/pyinstaller/discussions/8552"
        }
      ],
      "recommendation": "Split the two concerns and reuse the existing seams. WEIGHTS: download-
on-first-run from a public read-only GCS bucket we control (WEIGHTS-A), with Hugging Face Hub
(WEIGHTS-B) as a strong lower-ops alternative. Do not vendor weights in the wheel because the
PyPI 100 MiB per-file cap, the bloat for Gemini-only users, and the coupling of model cadence to
release cadence all argue against it. Mechanics: a small fetcher resolves WASB_WEIGHTS by
precedence of env var, then config, then a per-user cache directory; it downloads over HTTPS
where a public bucket needs no google-cloud-storage dependency or credentials; it verifies a
pinned SHA-256; it caches once for offline reruns; and it shows a loud resumable progress bar
```

rather than a silent hang. This reuses GCSStorage.fetch_to_local and the WASB_WEIGHTS env seam so the detector code is untouched. Weights are license-clean because WASB-SBDT is MIT, but verify each weight file actual host and provenance before mirroring. FFMPEG: do not redistribute ffmpeg, since that is exactly where the GPL versus LGPL problem bites. Instead route every ffmpeg and ffprobe call through one resolver helper that checks a configured path or the FFMPEG_BINARY env var, then shutil.which, then a known install dir, and otherwise fails loud with the exact winget command; and have first-run setup, extending scripts/doctor.py, detect a missing binary and offer or auto-run the winget install of Gyan.FFmpeg. The full GPL build ships ffprobe, libx264, and libfreetype, so the engine stitch, watermark, and duration probe all work unchanged, and because the user installs ffmpeg we never redistribute GPL bits and stay MIT-clean. Reject imageio-ffmpeg as the primary path because it lacks ffprobe, which the engine requires, and its binary is effectively a GPL build, so it neither fully works nor is redistribution-clean. Keep the LGPL-bundle option FFMPEG-C, a BtbN LGPL build with the encoder swapped from libx264 to libopenh264 or NVENC, as a documented fallback only if a future truly zero-touch installer must avoid the winget step, noting it forces an encoder change and LGPL re-link or source-offer compliance. Net first-run for a non-tech Windows user: the installer or pip drops the app; doctor-style setup checks ffmpeg and ffprobe and, if absent, runs winget with consent and progress; and on the first trajectory run the weight auto-downloads, checksummed and cached. Gemini-only users never trigger a weights download and, if they already have ffmpeg, need nothing else.",
      "fit_for_this_app": "This fits the dual torch env constraint because download-on-demand weights plus system ffmpeg mean neither env, the main 2.6 cu124 or the WASB 1.11 cu113, needs freezing, avoiding two large PyInstaller bundles; the pip or installer plus the existing console script entry point and on-demand artifacts is the right shape, and the fetcher just sets the WASB_WEIGHTS env var for whichever env runs wasb_infer. On ffmpeg, the engine invokes bare ffmpeg and ffprobe in video.py and compiler.py and hard-needs ffprobe plus the libx264 encoder plus drawtext today, which a winget GPL build provides and which imageio-ffmpeg does not because it omits ffprobe, so a user-installed system ffmpeg is both the working and the license-clean choice. On GCS weights, the bytes already live in a GCS bucket and the engine already has fetch_to_local plus the WASB_WEIGHTS env override with env winning over config, so first-run download is a thin wrapper, and a public-read bucket avoids the storage-gcs extra and any credentials. For the FastAPI plus SPA split, binary and weight resolution is server-side and orthogonal to the cross-repo khelsutra web SPA; the planned capabilities endpoint should report whether ffmpeg is present and weights are cached so the UI only offers honestly runnable segmenters. For the non-tech India coach on Windows, winget is the lowest-friction install and doctor.py already prints the exact command, paired with the existing English, Kannada, and Hindi setup copy, a loud resumable first-run download fronted by a CDN for India latency, and a documented manual fallback for boxes where winget is disabled. The licensing guardrail holds because MIT weights are redistributable, ffmpeg stays user-installed so its GPL never touches the MIT bundle, and the only new packaging dependency, only if Hugging Face Hub is chosen, is Apache-2.0 huggingface_hub."
    },
    "verdict": {
      "claims_checked": [
        {
          "claim": "PyPI default limits a single uploaded file to 100 MiB and a whole project to 10 GiB; larger files require a per-project limit-increase request after a sub-limit release exists. This caps the vendor-weights-in-wheel option.",
          "verdict": "confirmed",
          "reasoning": "Verified against the cited PyPI doc. The substance is correct: default per-file limit and per-project total exist, file-size increases require an issue request AND at least one already-uploaded release under the limit. One UNIT error: the official doc states '100.0 MB' and '10.0 GB' (decimal MB/GB), NOT 100 MiB / 10 GiB (binary). 100 MB = ~95.4 MiB; 10 GB = ~9.31 GiB. The conclusion that this caps weights-in-wheel holds regardless.",
          "corrected_claim": "PyPI's default limits are 100 MB per file and 10 GB per project (decimal MB/GB, not MiB/GiB). A larger per-file limit requires filing a request AND first uploading at least one release under the limit. This effectively caps the vendor-weights-in-wheel option."
        },
        {
          "claim": "imageio-ffmpeg bundled ffmpeg binary is effectively a GPL build because it uses libx264 as the default mp4 codec and libx264 requires enable-gpl; redistributing it in our installer would impose GPL on the bundle.",
          "verdict": "confirmed",
          "reasoning": "imageio-ffmpeg README confirms the default mp4 codec is 'libx264 (if

available from the ffmpeg executable)'. FFmpeg legal.html confirms libx264 is a GPL library and that using any GPL part makes the GPL apply to all of FFmpeg, and that GPL FFmpeg cannot be relicensed for proprietary/commercial redistribution. The bundled binary is therefore a GPL build in practice, and redistributing it would impose GPL. The wording 'effectively a GPL build' is fair: libx264 being the default codec is strong evidence the bundled binary is compiled --enable-gpl. Logic and licensing facts hold.",
          "corrected_claim": ""
        },
        {
          "claim": "imageio-ffmpeg bundles only ffmpeg, not ffprobe. The engine hard-requires ffprobe for video duration at backend/utils/video.py line 11 and compiler.py line 113, so imageio-ffmpeg alone cannot satisfy the engine.",
          "verdict": "confirmed",
          "reasoning": "The engine dependency is verified: backend/utils/video.py invokes 'ffprobe' (the call opens at line 11, the literal 'ffprobe' string is line 12) and compiler.py invokes 'ffprobe' at line 113 exactly. Both are required for duration. The 'imageio-ffmpeg bundles only ffmpeg, not ffprobe' fact is also true (its wheels ship only the ffmpeg executable). CAVEAT on the cited source: issue #23 is a FEATURE REQUEST to add ffprobe support; it does not itself state 'ffprobe is not bundled' ? it implies it. The factual conclusion is correct but the citation is weaker than implied. The line-11 ffprobe ref is off by one (it is line 12; line 11 is the subprocess.check_output opener).",
          "corrected_claim": "imageio-ffmpeg bundles only the ffmpeg executable, not ffprobe (issue #23 is a still-open request to add ffprobe). The engine hard-requires ffprobe for duration at backend/utils/video.py (line 12) and compiler.py line 113, so imageio-ffmpeg alone cannot satisfy the engine."
        },
        {
          "claim": "The imageio-ffmpeg wrapper is BSD-2-Clause; binary resolution order is the IMAGEIO_FFMPEG_EXE env var, then the bundled binary, then conda, then system PATH, and the env var is used directly without validation.",
          "verdict": "confirmed",
          "reasoning": "imageio-ffmpeg README confirms BSD-2-Clause and the exact resolution order: IMAGEIO_FFMPEG_EXE env var, then the binary distributed with imageio-ffmpeg, then a conda-installed ffmpeg, then system ffmpeg, in that order. The 'used directly without validation' sub-claim is consistent with the documented behaviour (env var takes precedence as-is) and matches the _utils.py source cited; I did not line-by-line read _utils.py but the documented contract supports it. License + ordering confirmed; the no-validation detail is well-founded but not independently re-read at source level.",
          "corrected_claim": ""
        },
        {
          "claim": "gyan.dev and BtbN GPL static Windows builds installed by winget Gyan.FFmpeg or BtbN.FFmpeg.GPL are GPLv3 and include libx264, libx265, and ffprobe. Fine to use when user-installed but not to redistribute in an MIT, Apache, BSD-only bundle.",
          "verdict": "confirmed",
          "reasoning": "gyan.dev builds page confirms all builds are static and licensed GPLv3, contain ffmpeg/ffprobe/ffplay, and the essentials build includes libx264 and libx265. BtbN GPL variant uses --enable-gpl and includes libx264/libx265 (LGPL variant omits them). ffprobe is shipped in these full builds. The licensing logic ? fine to use when user-installs (no redistribution by us) but not to redistribute inside an MIT/Apache/BSD bundle without imposing GPL ? is sound per FFmpeg legal.html. Confirmed.",
          "corrected_claim": ""
        },
        {
          "claim": "BtbN LGPL builds omit GPL components and so exclude libx264; H.264 in an LGPL build needs the native LGPL encoder or libopenh264 which is BSD. Shipping an LGPL ffmpeg therefore requires changing the engine c:v libx264 default.",
          "verdict": "confirmed",
          "reasoning": "Confirmed BtbN LGPL variant omits --enable-gpl and lacks libx264/libx265. OpenH264 (libopenh264) is BSD-licensed (Cisco Simplified BSD) and works in an LGPL FFmpeg via --enable-libopenh264. The engine hardcodes '-c:v libx264' (compiler.py line 253; also video.py line 42), so an LGPL build that lacks libx264 would indeed require changing that default to libopenh264 or NVENC. Logic and facts hold. (Minor nuance: FFmpeg also has a native LGPL-safe H.264 *decoder* and the experimental native encoder is poor quality, so libopenh264/NVENC is the practical encoder choice ? which the claim correctly states.)",

```
          "corrected_claim": ""
        },
        {
          "claim": "Embedding ffmpeg built with enable-gpl makes the whole build GPL and forbids
embedding it in a non-GPL redistributed bundle; LGPL builds allow redistribution subject to re-
link or source-offer obligations. The clean escape is to not redistribute ffmpeg and have the
user system-install it.",
          "verdict": "confirmed",
          "reasoning": "FFmpeg legal.html confirms: using GPL parts makes GPL apply to all of
FFmpeg; it is not available under proprietary terms even for payment; and the LGPL compliance
checklist requires dynamic linking, distributing matching FFmpeg source on the same server, and
attribution (the 're-link / source-offer' obligations). Having the user install ffmpeg
themselves means we never redistribute it, sidestepping both ? a sound legal escape.
Confirmed.",
          "corrected_claim": ""
        },
        {
          "claim": "huggingface_hub is Apache-2.0 and provides hf_hub_download with a content-
hashed cache, resumable downloads, and an offline switch via HF_HUB_OFFLINE or local_files_only,
giving a batteries-included first-run downloader.",
          "verdict": "confirmed",
          "reasoning": "PyPI metadata confirms Apache-2.0. Official file_download docs confirm
hf_hub_download with a content-addressable cache (blobs identified by git-sha1 or sha256),
resumable downloads (resume is always-on; the resume_download param is deprecated because it
always resumes), a tqdm progress bar, local_files_only param, and HF_HUB_OFFLINE=1 offline mode.
All sub-claims confirmed.",
          "corrected_claim": ""
        },
        {
          "claim": "WASB-SBDT from nttcom is MIT-licensed, so its pretrained shuttle weights are
redistributable and we can self-host them on our GCS bucket or on HF. Verify each weight file
actual host and provenance before mirroring, since the README points at MODEL_ZOO.md and some
academic weights are Google-Drive-hosted.",
          "verdict": "confirmed",
          "reasoning": "Repo is confirmed MIT-licensed. MODEL_ZOO.md confirms weights are hosted
on Google Drive (per-method, per-sport links) ? NOT in the repo. The caution is warranted and
stronger than the claim implies: MODEL_ZOO.md also lists THIRD-PARTY method weights (DeepBall,
TrackNetV2, ResTrackNetV2, MonoTrack, BallSeg) alongside the authors' own WASB weights.
MonoTrack in particular is flagged in this project's own memory as a weights-provenance hazard.
MIT on the repo CODE does not automatically extend to every weight blob hosted there, and the
WASB *paper* weights may have their own terms. The 'verify provenance before mirroring' caveat
is correct and should be treated as a hard gate, especially: only mirror the authors' own WASB
badminton weights, not the bundled third-party models.",
          "corrected_claim": "nttcom/WASB-SBDT is MIT-licensed and its MODEL_ZOO.md hosts
weights on Google Drive (not in-repo). Before mirroring, verify each weight file's host AND
provenance: MODEL_ZOO.md mixes the authors' own WASB weights with third-party model weights
(MonoTrack, TrackNetV2, etc.) whose licenses may differ ? mirror only the authors' WASB weights
we have rights to."
        },
        {
          "claim": "The engine already has the seams for download-on-first-run:
GCSStorage.fetch_to_local is overwrite-aware and does mkdir plus download, and a WASB_WEIGHTS
env override is consumed by NativeWasbConfig.from_indexing_cfg with env winning over config, so
weight-fetch can set WASB_WEIGHTS with no detector code change.",
          "verdict": "confirmed",
          "reasoning": "Verified in source. backend/storage/gcs.py fetch_to_local (line 30) is
overwrite-aware (returns existing dst unless overwrite=True), does os.makedirs(dirname) then
download_to_filename. native_wasb_runner.py NativeWasbConfig.from_indexing_cfg (lines 80-97)
resolves weights_path = env('WASB_WEIGHTS') or w.get('weights_path',''), i.e. env wins over
config. So setting WASB_WEIGHTS lets a fetcher point the detector at a downloaded file with no
detector code change. Confirmed. NOTE for the recommendation: GCSStorage.fetch_to_local imports
google.cloud.storage and needs ADC credentials ? it is NOT the no-dependency public-HTTPS path
the recommendation describes; a public-bucket HTTPS GET is a different code path than this
seam.",
          "corrected_claim": ""
```

```
        },
        {
          "claim": "ffmpeg and ffprobe are invoked as bare PATH strings via subprocess:
backend/utils/video.py uses ffprobe at line 11 and ffmpeg at line 33; the stitcher uses ffprobe
at compiler.py line 113, c:v libx264 at line 253, and drawtext or overlay watermark at lines
179-185. A single resolver helper is the minimal change for a configurable or bundled binary.",
          "verdict": "confirmed",
          "reasoning": "All invocations verified as bare 'ffmpeg'/'ffprobe' subprocess strings.
Minor line drift: in video.py 'ffprobe' is line 12 (line 11 opens the check_output call) and
'ffmpeg' is line 34 (line 33 opens the cmd list). In compiler.py ffprobe is exactly line 113,
'-c:v','libx264' is exactly line 253, and the drawtext/overlay watermark spans lines 179-188
(the cited 179-185 covers the drawtext portion). The substantive claim ? bare PATH strings,
single resolver helper is the minimal change ? is correct. NOTE: there is at least one more
libx264 invocation at video.py line 42 (extract_video_chunk) that a resolver/encoder change must
also cover.",
          "corrected_claim": "ffmpeg/ffprobe are invoked as bare PATH strings via subprocess.
ffprobe: video.py line 12 and compiler.py line 113. ffmpeg: video.py line 34. '-c:v libx264':
compiler.py line 253 AND video.py line 42 (a second site). drawtext/overlay watermark:
compiler.py lines 179-188. A single resolver helper is the minimal change for a
configurable/bundled binary; the encoder-default change must cover both libx264 sites."
        },
        {
          "claim": "scripts/doctor.py already does first-run checks: shutil.which for ffmpeg and
ffprobe, GPU detection, torch out-of-band index selection cu124 or cpu, and on Windows it prints
exactly the winget install command for Gyan.FFmpeg. The recommended first-run setup extends this
rather than adding new tooling.",
          "verdict": "confirmed",
          "reasoning": "Verified in source. doctor.py: check_command uses shutil.which for
'ffmpeg' and 'ffprobe' (lines 84-85), GPU detection via nvidia-smi (line 20) and
torch.cuda.is_available (line 118), torch out-of-band --extra-index-url selection cu124 vs cpu
(lines 26-34), and on Windows prints exactly 'winget install Gyan.FFmpeg' (line 103). The cited
line ranges (84-108, 15-34) match. Confirmed; extending this script is a reasonable plan.",
          "corrected_claim": ""
        },
        {
          "claim": "PyInstaller-freezing with CUDA torch yields about 2.6 to 4 GB bundles
because CUDA and cuDNN libs cannot be split; small bundles require CPU-only torch. This argues
against a frozen all-in-one binary and for pip or installer plus on-demand weights plus system
ffmpeg.",
          "verdict": "confirmed",
          "reasoning": "Verified against the cited PyInstaller discussion #8552: a maintainer
reports a onefile build with pip torch2.3+cu118 comes out ~2.6GB, the original poster's bundled
torch was ~4GB, CUDA/cuDNN shared libs 'cannot be split', and CPU-only torch is the route to a
smaller bundle. The numbers (~2.6?4 GB) and the cannot-split reasoning match exactly. The
architectural conclusion (favor pip/installer + on-demand weights + system ffmpeg over a frozen
monolith) is well-supported. Confirmed.",
          "corrected_claim": ""
        }
      ],
      "overall_notes": "All 12 load-bearing claims are CONFIRMED in substance; none was refuted.
Six are clean; six have minor corrections (units, citation precision, line-number drift) that do
not change any conclusion. The recommendation (split weights vs ffmpeg, reuse existing seams, do
not vendor weights, do not redistribute ffmpeg, route through one resolver, extend doctor.py) is
sound and well-grounded in both the codebase and the external licensing facts.\n\nCorrections
worth folding in:\n1. UNITS (Claim 1): PyPI limits are 100 MB / 10 GB decimal, not 100 MiB / 10
GiB binary. Conclusion unchanged.\n2. CITATION (Claim 3): imageio-ffmpeg issue #23 is a feature
REQUEST to add ffprobe, not a statement that ffprobe isn't bundled. The fact (no ffprobe
shipped) is true but the cited source is weaker than implied.\n3. LINE DRIFT (Claims 3, 11):
video.py 'ffprobe' is line 12 not 11; 'ffmpeg' is line 34 not 33. Per this project's own
'verify-engine-surface-before-scoping' lesson, the [verified] file:line refs should be
exact.\n4. EXTRA SITE (Claim 11): there is a SECOND '-c:v libx264' at backend/utils/video.py
line 42 (extract_video_chunk), beyond compiler.py line 253. Any LGPL/encoder-default change
(FFMPEG-C) must patch BOTH sites, and the single resolver helper must wrap all four subprocess
call sites (video.py L12/L34/L42, compiler.py L113/L249/L387), not just the stitcher.\n5.
PROVENANCE GATE (Claim 9): stronger than stated ? MODEL_ZOO.md mixes the authors' own MIT-
```

```
covered WASB weights with THIRD-PARTY model weights (MonoTrack, TrackNetV2, BallSeg, etc.)
hosted on Google Drive. MIT on the repo code does not blanket-cover every weight blob. Mirror
ONLY the authors' own WASB badminton weights; MonoTrack is already flagged as a weights hazard
in this project's memory.\n\nCaveat on the recommendation mechanics (NOT a claim, but load-
bearing for implementation): the recommendation says the public-bucket fetcher 'reuses
GCSStorage.fetch_to_local' AND 'needs no google-cloud-storage dependency or credentials.' These
are in tension. GCSStorage.fetch_to_local (gcs.py line 24) lazily imports google.cloud.storage
and uses ADC credentials ? it is the authenticated SDK path, not an anonymous HTTPS GET. A
credential-free public-bucket download (https://storage.googleapis.com/<bucket>/<obj>) is a
DIFFERENT code path (plain requests/urllib) than the fetch_to_local seam. The WASB_WEIGHTS env
seam is reused either way; the 'reuse fetch_to_local' phrasing should be 'add a small public-
HTTPS fetcher (no google-cloud-storage dep) that writes to the cache dir and sets WASB_WEIGHTS'
? fetch_to_local is the right analogue only if you accept the SDK dependency + ADC.\n\nItems I
could NOT independently re-verify at source level (stated for honesty): the imageio-ffmpeg
_utils.py 'env var used directly without validation' detail (confirmed via documented behaviour,
not a line read), and that the bundled imageio-ffmpeg binary is compiled --enable-gpl (inferred
strongly from libx264 being its default mp4 codec, not from an explicit imageio-ffmpeg license
statement of the binary)."
    }
  },
  {
    "topic": "gpu-cpu-cross-platform",
    "research": {
      "topic": "How local-AI apps handle torch GPU vs CPU vs Apple MPS at install/run time, and
how to map it to our DeviceContext (cuda/auto/cpu/mps) + gpu/cpu pyproject extras for a
downloadable batteries-included app",
      "options": [
        {
          "name": "A. uv with --torch-backend=auto (run-time hardware detection picks the
index)",
          "how_it_works": "Installer/launcher shells out to `uv pip install torch torchvision
--torch-backend=auto` (or `UV_TORCH_BACKEND=auto`). uv probes the installed CUDA driver, AMD GPU
versions, and Intel GPU presence, then selects the most-compatible PyTorch index automatically;
falls back to CPU-only when no GPU is found. On macOS it resolves to the PyPI wheel (which is
MPS-capable). This is exactly the 'install-time hardware detection + correct --index-url'
option, but delegated to a maintained tool instead of hand-rolled. Our pyproject keeps torch out
of [project.dependencies] (unchanged) and the app's bootstrap step runs this command into the
bundled venv.",
          "pros": [
            "Solves the core PEP-621-can't-express-index-url problem with one maintained
command; no per-platform branching code we own",
            "Detects CUDA/ROCm/Intel at install time and picks the right wheel automatically;
CPU fallback is built in ? mirrors what our DeviceContext does at runtime",
            "macOS path naturally lands on the MPS-capable PyPI wheel (no CUDA build exists for
macOS anyway)",
            "uv is MIT/Apache-2.0 licensed (license-clean) and is already the de-facto modern
Python installer",
            "Keeps the download small: only the matching wheel is fetched, not a 4-5GB bundle"
          ],
          "cons": [
            "`--torch-backend` is `uv pip`-interface only, NOT the project/lockfile workflow ?
our bootstrap must call it as an explicit step, it can't live purely in pyproject",
            "Requires uv >=0.5.3 and adds uv as a bootstrap dependency the app must ship/fetch",
            "Detects the *driver*, not whether the user wants CPU ? a headless server user who
wants CPU must override (maps cleanly to our device='cpu')",
            "Does NOT solve the WASB two-env problem: the separate torch 1.11+cu113 conda env
still needs its own install path"
          ],
          "maturity": "emerging"
        },
        {
          "name": "B. Static per-platform index pinning via tool.uv.sources / environment
markers",
          "how_it_works": "pyproject declares torch twice with PEP-508 markers and two
[[tool.uv.index]] entries ? e.g. CUDA index for `sys_platform == 'linux'`, CPU index for
```

```
`sys_platform != 'linux'`, each `explicit = true`. uv resolves the right index per platform from
the lockfile, no runtime probe. This is the 'documented extras / declarative' option formalized
into the resolver.",
          "pros": [
            "Fully declarative and lockable ? reproducible installs, works in the project
workflow (unlike option A)",
            "No runtime GPU probe needed; deterministic per OS/arch",
            "Cleanly expresses CPU-on-macOS / CUDA-on-Linux, which markers handle well"
          ],
          "cons": [
            "Markers key on OS/arch, NOT on whether a CUDA GPU is actually present ? a Linux box
with no GPU still pulls the CUDA wheel (wrong for our truly-local no-GPU user)",
            "Windows is ambiguous: both GPU and CPU users run sys_platform=='win32', so a single
marker can't pick correctly ? needs a manual extra or override",
            "Pins one CUDA version (e.g. cu124) for everyone; mismatched-driver users must
override",
            "Still doesn't address the WASB second env"
          ],
          "maturity": "mature"
        },
        {
          "name": "C. Installer/launcher that probes hardware and picks the wheel (Pinokio /
ComfyUI-Desktop / Stability Matrix model)",
          "how_it_works": "A thin first-run bootstrap (or our existing `indexer`-style launcher)
detects GPU/OS, creates the bundled venv, and runs the matching `pip install torch ... --index-
url ...`. Pinokio does exactly this: on Install it creates a Python 3.12 venv and installs the
right torch build per GPU/OS (CUDA for NVIDIA, DirectML for AMD-Windows, ROCm for AMD-Linux,
MPS/CPU for macOS, CPU fallback). ComfyUI-Desktop is an Electron app bundling its own Python and
configuring torch+deps at install. This is the most batteries-included pattern and the closest
analog to our D10 'self-hosted webserver the user spins up'.",
          "pros": [
            "Best UX for a non-technical user (our India coach on Windows): one download, no
--index-url knowledge needed",
            "Handles all backends including AMD/Intel/MPS, not just CUDA-vs-CPU",
            "Maps 1:1 to our existing seam ? the launcher's probe feeds the same cuda/auto/cpu
device choice our DeviceContext already encodes",
            "Can drive BOTH venvs: the main torch 2.6+cu124 env AND the WASB torch 1.11 conda
env in one flow",
            "Can also fetch the GCS-hosted weights (wasb_badminton_best.pth.tar) in the same
first-run step"
          ],
          "cons": [
            "We own the detection + branching logic (more code/maintenance) unless we wrap uv
--torch-backend=auto underneath (recommended: C-over-A)",
            "First-run network install (not a fully offline bundle) ? needs a clear progress UI
and a retry/offline story",
            "Electron-style bundling (ComfyUI-Desktop) is heavier than our 'spin up a webserver'
decision (D10) ? take the launcher idea, not the Electron shell"
          ],
          "maturity": "mature"
        },
        {
          "name": "D. CPU-default wheel + optional GPU upgrade (documented extras)",
          "how_it_works": "Default install pulls the small CPU torch wheel (works everywhere,
incl. macOS-MPS via PyPI); a documented opt-in step (`pip install torch --index-url .../cu124`
or a `gpu` extra path + script) upgrades to CUDA. This is the conservative form of our current
empty `gpu` extra in pyproject.toml:58-65 with both recipes already commented.",
          "pros": [
            "Always-works default ? the app boots on any machine with zero GPU knowledge;
matches Ollama's silent-CPU-fallback philosophy",
            "Smallest default download; GPU is a deliberate, documented upgrade",
            "Lowest risk: no detection logic to get wrong; our `gpu`/`cpu` extras stay docs-only
as designed",
            "macOS users get a working MPS-capable wheel by default"
          ],
```

```
          "cons": [
            "GPU users (the ones who most need speed) must take a manual second step ? bad for
the non-technical persona unless the launcher automates it",
            "'CPU default' silently means slow WASB/CV inference; needs a loud 'GPU not
detected, running slow' banner (we already emit this via DeviceContext.fell_back / startup
CPU_FALLBACK warning)",
            "Two-step install invites the classic 'Torch not compiled with CUDA enabled'
confusion ComfyUI users hit",
            "WASB env still separate"
          ],
          "maturity": "mature"
        },
        {
          "name": "E. PyInstaller/onedir single bundle with torch baked in",
          "how_it_works": "Freeze the app + a chosen torch build into one distributable (onedir,
zipped). The user downloads and runs with no pip step.",
          "pros": [
            "Truly offline / no network install; one artifact",
            "No --index-url problem at the user's machine at all ? it's solved at build time"
          ],
          "cons": [
            "You must pick ONE backend at build time: a CUDA bundle is ~4-5GB and CPU-only is
~620MB-1.8GB ? you can't satisfy GPU and CPU users from one artifact without shipping multiple
multi-GB downloads",
            "CUDA/cuDNN shared libs can't be split or trimmed; --onefile first-launch is very
slow, so onedir+zip is forced",
            "Does not handle the WASB second torch env at all (two incompatible torch versions
in one frozen image is intractable)",
            "Heaviest to build/maintain per-platform; contradicts the lightweight 'spin up a
webserver' D10 direction"
          ],
          "maturity": "mature"
        }
      ],
      "load_bearing_claims": [
        {
          "claim": "Our DeviceContext already encodes the exact run-time policy these apps
implement: requested in {auto,cuda,mps,cpu}; auto -> cuda-if-available-else-cpu; explicit cuda
stays strict (hard-fail, no silent slow-CPU downgrade); cpu/mps verbatim.
effective/fell_back/cuda_required_but_missing properties expose it.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/device.py:32-56"
        },
        {
          "claim": "torch/torchvision are deliberately NOT in [project.dependencies]; the `gpu`
extra is intentionally empty (docs-only) because CUDA/CPU wheels need --index-url which PEP 621
cannot express. Both install recipes (cu124 and cpu) are already documented inline.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/pyproject.toml:8-11,58-65"
        },
        {
          "claim": "The native WASB runner runs in a SEPARATE python env invoked by path
(WASB_PYTHON) and probes CUDA via a subprocess `torch.cuda.is_available()` (_CUDA_PROBE_CODE);
device/python/weights are env-first (WASB_DEVICE/WASB_PYTHON/WASB_WEIGHTS). This is the two-
torch-env reality any installer must drive separately.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/native_wasb_runner.py:56,80-97,116-136"
        },
        {
          "claim": "Our config-load path is deliberately torch-free; CUDA probing is lazy/best-
effort (import torch only when probe_cuda opted in; False on any failure) ? so an
installer/launcher, not config load, should own the heavy GPU probe.",
```

```
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/config/presets.py:84-106"
        },
        {
          "claim": "A GPU mismatch at startup is a WARNING, never a fatal error, because the
main-process probe is best-effort and torch may live only in the detector subprocess env; the
runner healthcheck is the authority (codes CUDA_NOT_VISIBLE / CPU_FALLBACK /
CLOUD_CUDA_NOT_VISIBLE).",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/config/startup.py:104-145"
        },
        {
          "claim": "uv's --torch-backend=auto queries the installed CUDA driver, AMD GPU
versions, and Intel GPU presence and picks the most-compatible PyTorch index, falling back to
CPU when no GPU; it is `uv pip`-interface only and needs uv >=0.5.3.",
          "confidence": "high",
          "source": "https://docs.astral.sh/uv/guides/integration/pytorch/"
        },
        {
          "claim": "PyTorch publishes no CUDA build for macOS; the macOS PyPI wheel is the MPS-
capable one, so cross-platform configs gate CUDA indices to Linux/Windows only and fall back to
PyPI on macOS.",
          "confidence": "high",
          "source": "https://docs.astral.sh/uv/guides/integration/pytorch/"
        },
        {
          "claim": "Pinokio (one-click launcher) creates a per-app Python venv and installs the
right torch build per GPU/OS: CUDA for NVIDIA, DirectML for AMD-Windows, ROCm for AMD-Linux,
MPS/CPU for macOS, with CPU fallback ? the launcher-picks-the-wheel pattern.",
          "confidence": "high",
          "source": "https://diffusiondoodles.substack.com/p/pinokio-a-one-click-playground-for"
        },
        {
          "claim": "ComfyUI-Desktop is an Electron app that bundles its own Python and auto-
configures the env+deps; the Windows portable build ships embedded Python + a prebuilt CUDA
torch wheel (one fixed backend per artifact).",
          "confidence": "high",
          "source": "https://github.com/Comfy-Org/desktop"
        },
        {
          "claim": "Ollama detects the GPU at startup (not strictly install) and silently falls
back to CPU when CUDA/ROCm libs are missing/misconfigured ? a UX trap (slow with no loud
warning) we explicitly avoid via fell_back/CPU_FALLBACK warnings.",
          "confidence": "high",
          "source": "https://docs.ollama.com/gpu"
        },
        {
          "claim": "faster-whisper selects the best available hardware by default and exposes
--device/--device_index to override ? the same auto-with-explicit-override contract as our
DeviceContext.",
          "confidence": "high",
          "source": "https://github.com/SYSTRAN/faster-whisper"
        },
        {
          "claim": "A CUDA-enabled torch PyInstaller bundle is ~4-5GB (CPU-only ~620MB-1.8GB)
and the CUDA/cuDNN libs cannot be split, so a single frozen artifact cannot serve both GPU and
CPU users without multiple multi-GB downloads.",
          "confidence": "high",
          "source": "https://github.com/orgs/pyinstaller/discussions/8552"
        }
      ],
      "recommendation": "Adopt a first-run LAUNCHER/BOOTSTRAP that owns hardware detection and
installs the matching torch into the bundled venv (option C), implemented on top of uv `--torch-
```

backend=auto` (option A) rather than hand-rolled --index-url branching. Concretely: (1) keep pyproject.toml as-is ? torch stays out of [project.dependencies] and the `gpu` extra stays docs-only (it's correct, do not try to express the index there); (2) the bootstrap runs `uv pip install torch torchvision --torch-backend=auto` into the MAIN env, which yields CUDA on a NVIDIA box, CPU elsewhere, and the MPS-capable PyPI wheel on macOS ? exactly mirroring what DeviceContext('auto') decides at runtime; (3) expose an explicit '--cpu' / device override in the launcher that maps straight to DeviceContext's cpu/cuda/mps so a user can force a backend (the faster-whisper/our-own contract); (4) treat the WASB torch-1.11 conda env as a SEPARATE bootstrap step the launcher drives by path (it cannot share the main env ? two incompatible torch versions), installing it only when the user picks the wasb_hybrid segmenter, and fetching wasb_badminton_best.pth.tar from GCS in the same step; (5) keep startup.py's best-effort warning (CPU_FALLBACK / CUDA_NOT_VISIBLE) loud in the web UI so we never repeat Ollama's silent-slow-CPU trap. Avoid option E (PyInstaller single bundle): the 4-5GB-vs-CPU split plus the two-torch-env reality make a single frozen artifact impractical and it contradicts the D10 'spin up a webserver' decision. Option D (CPU-default + documented GPU upgrade) is the safe fallback if shipping/embedding uv proves problematic, but it pushes a manual second step onto exactly the non-technical persona we're targeting.",
      "fit_for_this_app": "Our codebase is already shaped for the run-time half of this problem: DeviceContext (device.py:32-56) + the auto/cuda/cpu/mps contract + startup CPU_FALLBACK/CUDA_NOT_VISIBLE warnings (startup.py:104-145) replicate exactly what Ollama/faster-whisper do at run time. The GAP is purely install-time wheel selection, which pyproject.toml correctly refuses to express (torch out of deps, empty docs-only `gpu` extra, both --index-url recipes commented at pyproject.toml:58-65). The cleanest fit for the downloadable D10 'self-hosted webserver the user spins up' is a thin first-run launcher that runs `uv pip install torch --torch-backend=auto` into the bundled venv ? detection result and DeviceContext('auto') then agree by construction. Two app-specific complications the comparable apps mostly DON'T have: (a) the WASB native runner is a SECOND, incompatible torch env (torch 1.11+cu113 vs main 2.6+cu124) invoked by path with WASB_PYTHON/WASB_DEVICE/WASB_WEIGHTS (native_wasb_runner.py:80-136) ? the launcher must provision it as a separate optional step, never merge it into the main env; and (b) model weights live in GCS, so the same bootstrap should pull wasb_badminton_best.pth.tar. ffmpeg/ffprobe remain a system dep the launcher must check for (analogous to ComfyUI's external deps). Licensing is clean: uv (Apache-2.0/MIT) and pip-from-pytorch-index both satisfy the MIT/Apache/BSD-only guardrail; no new viral or training-data dependency is introduced. For the non-technical India-coach-on-Windows persona, the launcher-picks-the-wheel UX (Pinokio-style, option C) beats forcing them to learn --index-url, while preserving every existing CUJ because the engine entrypoints (indexer --server, python -m backend.worker) and device config flow are untouched."
    },
    "verdict": {
      "claims_checked": [
        {
          "claim": "Our DeviceContext already encodes the exact run-time policy these apps implement: requested in {auto,cuda,mps,cpu}; auto -> cuda-if-available-else-cpu; explicit cuda stays strict (hard-fail, no silent slow-CPU downgrade); cpu/mps verbatim. effective/fell_back/cuda_required_but_missing properties expose it.",
          "verdict": "confirmed",
          "reasoning": "Verified against backend/pipeline/detectors/device.py:23-56. VALID_DEVICES=(auto,cuda,mps,cpu) (line 29). effective property: auto -> cuda if cuda_available else cpu, else verbatim (lines 39-45). fell_back: requested==auto and not cuda_available (lines 47-50). cuda_required_but_missing: requested==cuda and not cuda_available (lines 53-56) ? the strict-hard-fail signal. One precision point: DeviceContext itself does NOT raise/hard-fail; it only EXPOSES cuda_required_but_missing. The actual hard-fail is performed by the runner healthcheck (native_wasb_runner.py:184-187). The claim's wording 'explicit cuda stays strict (hard-fail...)' plus '...properties expose it' correctly attributes the policy to DeviceContext and the enforcement is consistent with code, so the claim holds."
        },
        {
          "claim": "torch/torchvision are deliberately NOT in [project.dependencies]; the `gpu` extra is intentionally empty (docs-only) because CUDA/CPU wheels need --index-url which PEP 621 cannot express. Both install recipes (cu124 and cpu) are already documented inline.",
          "verdict": "confirmed",
          "reasoning": "Verified against pyproject.toml. Header comment lines 8-11 state torch/torchvision are deliberately not in [project.dependencies] because their wheels need --index-url/--extra-index-url which PEP 621 cannot express. dependencies list (lines 27-37) contains no torch/torchvision. The gpu extra is literally `gpu = []` (line 65) with a

DOCUMENTATION ONLY comment (lines 58-64). Both recipes are inline: cu124 (`--index-url .../whl/cu124`, line 62) and cpu (`--index-url .../whl/cpu`, line 64). All facts match exactly."
        },
        {
          "claim": "The native WASB runner runs in a SEPARATE python env invoked by path (WASB_PYTHON) and probes CUDA via a subprocess `torch.cuda.is_available()` (_CUDA_PROBE_CODE); device/python/weights are env-first (WASB_DEVICE/WASB_PYTHON/WASB_WEIGHTS). This is the two-torch-env reality any installer must drive separately.",
          "verdict": "confirmed",
          "reasoning": "Verified against native_wasb_runner.py. _CUDA_PROBE_CODE = 'import torch; print(... torch.cuda.is_available())' (line 56). _probe_cuda runs it as a subprocess via self.cfg.python_bin (lines 116-123). python_bin is 'the wasb conda env python (invoked by path, no activate)' (line 68; module docstring lines 10-12 confirm no conda activate, invoked by path). Env-first resolution in from_indexing_cfg (lines 80-97): python_bin=env('WASB_PYTHON') first (line 89), weights=env('WASB_WEIGHTS') first (line 90), device=env('WASB_DEVICE') first (line 92). Docstring explicitly frames it as a separate torch-1.11 env distinct from the main env. Claim accurate."
        },
        {
          "claim": "Our config-load path is deliberately torch-free; CUDA probing is lazy/best-effort (import torch only when probe_cuda opted in; False on any failure) ? so an installer/launcher, not config load, should own the heavy GPU probe.",
          "verdict": "confirmed",
          "reasoning": "Verified against backend/config/presets.py:84-106. _cuda_available() imports torch lazily inside the function body, ONLY when called, returns bool(torch.cuda.is_available()), and `except Exception: return False` (lines 87-94). Docstring: 'keeps the config-load path torch-free (torch is heavy and optional). Only reached when a caller opts into probe_cuda.' probe_cuda_available() (lines 97-106) is the public best-effort wrapper, documented as exposed for startup checks so the caller (not config load) owns the probe. suggest_profile takes probe_cuda=False default (line 110), confirming probing is opt-in. Claim accurate."
        },
        {
          "claim": "A GPU mismatch at startup is a WARNING, never a fatal error, because the main-process probe is best-effort and torch may live only in the detector subprocess env; the runner healthcheck is the authority (codes CUDA_NOT_VISIBLE / CPU_FALLBACK / CLOUD_CUDA_NOT_VISIBLE).",
          "verdict": "confirmed",
          "reasoning": "Verified against backend/config/startup.py:104-145. For truly-local: cuda_available is False with device=='cuda' -> LEVEL_WARN CUDA_NOT_VISIBLE (lines 104-111); device=='auto' -> LEVEL_WARN CPU_FALLBACK (lines 112-118). For cloud-serving: cuda_available False and device in (cuda,auto) -> LEVEL_WARN CLOUD_CUDA_NOT_VISIBLE (lines 141-145). All three GPU-mismatch codes are LEVEL_WARN, never LEVEL_ERROR. The only LEVEL_ERROR in this range is STORAGE_INCOHERENT (storage, not GPU). Comments at lines 102-103 and 116-117 state the probe is best-effort, torch may live only in the subprocess detector env, and 'the runner's DeviceContext/healthcheck is the authority.' Claim accurate."
        },
        {
          "claim": "uv's --torch-backend=auto queries the installed CUDA driver, AMD GPU versions, and Intel GPU presence and picks the most-compatible PyTorch index, falling back to CPU when no GPU; it is `uv pip`-interface only and needs uv >=0.5.3.",
          "verdict": "confirmed",
          "reasoning": "uv official docs (docs.astral.sh/uv/guides/integration/pytorch/) state verbatim: 'uv will query for the installed CUDA driver, AMD GPU versions, and Intel GPU presence' and use 'the most-compatible PyTorch index'; 'If no such GPU is found, uv will fall back to the CPU-only index'; 'At present, --torch-backend is only available in the uv pip interface'; and 'features outlined in this guide require uv version 0.5.3 or later.' All four sub-claims confirmed. Minor caveat (not refuting): GitHub issues #14742/#14647/#12357 show real edge-case failures (ignores compute-capability for old GPUs like GTX 1080 Ti; fails on multi-GPU boxes lacking /proc/driver/nvidia/version; may pick highest CUDA version) ? the mechanism works as the claim states but is not bulletproof in practice."
        },
        {
          "claim": "PyTorch publishes no CUDA build for macOS; the macOS PyPI wheel is the MPS-capable one, so cross-platform configs gate CUDA indices to Linux/Windows only and fall back to

PyPI on macOS.",
          "verdict": "confirmed",
          "reasoning": "uv docs state verbatim 'PyTorch doesn't publish CUDA builds for macOS' (repeated across CUDA versions) and show gating markers `sys_platform == 'linux' or sys_platform == 'win32'` that 'limit the PyTorch index to Linux and Windows, falling back to PyPI on macOS.' The 'no CUDA build for macOS' and 'gate to Linux/Windows, PyPI on macOS' parts are directly sourced. The specific 'macOS PyPI wheel is the MPS-capable one' is a true PyTorch fact (macOS arm64 PyPI wheels ship MPS support) but is NOT stated in the cited uv page itself ? it is correct general knowledge rather than a quote from the source. Substance confirmed."
        },
        {
          "claim": "Pinokio (one-click launcher) creates a per-app Python venv and installs the right torch build per GPU/OS: CUDA for NVIDIA, DirectML for AMD-Windows, ROCm for AMD-Linux, MPS/CPU for macOS, with CPU fallback ? the launcher-picks-the-wheel pattern.",
          "verdict": "confirmed",
          "reasoning": "Substantively true but UNDER-sourced by its exact citation. The cited diffusiondoodles substack only says Pinokio 'launches the app in a self-contained, isolated environment' and is 'not virtualisation' ? it does NOT enumerate the per-GPU/OS torch-wheel logic. Corroborating sources confirm the substance: Pinokio's install.js/torch.js branches torch installs by NVIDIA vs AMD and by OS, supports MPS on Mac and ROCm on Linux (with /opt/rocm version detection, GitHub pinokiocomputer/pinokio issue #1087), and the pattern is real. The DirectML-for-AMD-Windows specific is the weakest sub-claim (AMD-Windows torch support is often handled via DirectML/torchruntime but is less uniformly documented in Pinokio recipes). 'Launcher-picks-the-wheel' pattern is accurate. Confirmed on the load-bearing point (Pinokio is a precedent for launcher-driven per-GPU torch install)."
        },
        {
          "claim": "ComfyUI-Desktop is an Electron app that bundles its own Python and auto-configures the env+deps; the Windows portable build ships embedded Python + a prebuilt CUDA torch wheel (one fixed backend per artifact).",
          "verdict": "refuted",
          "reasoning": "The claim conflates two DIFFERENT products under one citation. The cited repo Comfy-Org/desktop is ComfyUI **Desktop**: confirmed Electron (bundles 'Electron, Chromium binaries, and node modules') and it bundles uv and 'will install all the necessary python dependencies with uv and start the ComfyUI server' at RUNTIME ? i.e. it does NOT ship a prebuilt CUDA torch wheel; it resolves torch via uv on first run. The 'embedded Python + prebuilt CUDA torch wheel (one fixed backend per artifact)' description belongs to the SEPARATE 'ComfyUI Windows Portable' artifact (python_embeded, per-GPU prebuilt packages, docs.comfy.org/installation/comfyui_portable_windows) ? not to Comfy-Org/desktop. As worded against its own source, the claim is factually wrong: Desktop does not ship a prebuilt CUDA wheel, and Portable is not built on Electron. Two distinct distribution models are merged into one false statement.",
          "corrected_claim": "ComfyUI ships TWO distinct distributions: (a) ComfyUI Desktop (Comfy-Org/desktop) is an Electron app that bundles its own Python tooling and uses uv to install torch+deps at first run (backend resolved at install time, not pre-frozen); (b) ComfyUI Windows Portable is a separate non-Electron artifact that ships embedded Python (python_embeded) with per-GPU prebuilt torch wheels (one fixed backend per artifact). Both are valid precedents, but for different options: Desktop ~ the uv-bootstrap pattern (option A/C); Portable ~ the prebuilt-per-backend artifact pattern."
        },
        {
          "claim": "Ollama detects the GPU at startup (not strictly install) and silently falls back to CPU when CUDA/ROCm libs are missing/misconfigured ? a UX trap (slow with no loud warning) we explicitly avoid via fell_back/CPU_FALLBACK warnings.",
          "verdict": "uncertain",
          "reasoning": "Partially supported by the cited docs.ollama.com/gpu, but the GENERAL silent-fallback claim is broader than what the page documents. The page only gives ONE specific scenario: 'On linux, after a suspend/resume cycle, sometimes Ollama will fail to discover your NVIDIA GPU, and fallback to running on the CPU.' It does NOT state a general 'silently falls back to CPU whenever CUDA/ROCm libs are missing/misconfigured' nor explicitly characterize the fallback as having 'no loud warning.' That Ollama detects GPU at runtime (server start) rather than at install is true (Ollama has no per-GPU install step ? one binary, runtime detection), and the suspend/resume case IS a silent CPU fallback, so the directional point (runtime detection + can silently land on CPU) is real. But the claim's strong, generalized 'silent CPU fallback on any missing/misconfigured lib, no loud warning' is not established by the source as

worded. The comparative point (our fell_back/CPU_FALLBACK warnings avoid this) is true about OUR code regardless."
        },
        {
          "claim": "faster-whisper selects the best available hardware by default and exposes --device/--device_index to override ? the same auto-with-explicit-override contract as our DeviceContext.",
          "verdict": "confirmed",
          "reasoning": "Confirmed with a precision note on WHERE these surfaces live. The core faster-whisper library exposes WhisperModel(device='auto', device_index=...) ? 'auto' is a supported value selecting the best available hardware. The --device / --device_index FLAGS are CLI surfaces: the whisper-ctranslate2 CLI (built on faster-whisper) documents verbatim 'by default the best hardware available is selected for inference. You can use the options --device and --device_index to control manually the selection,' and faster-whisper-cli exposes the same flags. So the flags belong to the CLI wrappers rather than the bare SYSTRAN/faster-whisper README, but the substance ? auto-by-default with explicit device/device_index override ? is accurate, and the analogy to our DeviceContext (auto resolves; explicit overrides verbatim) holds."
        },
        {
          "claim": "A CUDA-enabled torch PyInstaller bundle is ~4-5GB (CPU-only ~620MB-1.8GB) and the CUDA/cuDNN libs cannot be split, so a single frozen artifact cannot serve both GPU and CPU users without multiple multi-GB downloads.",
          "verdict": "uncertain",
          "reasoning": "The CUDA side and the 'cannot be split' side are confirmed by the cited PyInstaller discussion #8552: CUDA torch ~5GB in conda / ~4GB bundled / ~2.6GB onefile, and the maintainer states 'binary extensions and CUDA/cuDNN shared libs, which cannot be split.' The maintainer also says small (200-300MB) executables require a CPU-only or older-torch build (mutually exclusive with full CUDA). HOWEVER, the specific CPU-only range '~620MB-1.8GB' does NOT appear in the cited discussion ? no CPU-only size figure is given there. That number is therefore unsourced from the citation (it is a plausible directional figure ? CPU-only torch is much smaller ? but not verifiable from the cited URL). Net: the load-bearing conclusion (single frozen artifact can't serve both GPU and CPU cheaply; CUDA bundles are multi-GB and unsplittable) is solid; the precise CPU-only number is unconfirmed."
        }
      ],
      "overall_notes": "VERDICT SUMMARY: 8 confirmed, 1 refuted (claim 8 / ComfyUI), 2 uncertain (claim 10 / Ollama, claim 12 / PyInstaller CPU size). The five local-code claims (1-5) are ALL fully verified against the cited file:line ranges ? they are accurate and load-bearing. The recommendation under test (uv --torch-backend=auto bootstrap + separate WASB env + loud CPU_FALLBACK warnings + avoid PyInstaller single bundle) rests primarily on claims 1-7, which are solid, so the recommendation is well-founded.\n\nWEAKNESSES IN THE EVIDENCE (none fatal to the recommendation):\n1. Claim 8 (ComfyUI Desktop) is REFUTED as worded: it conflates ComfyUI Desktop (Electron + uv-installs-torch-at-runtime ? does NOT ship a prebuilt CUDA wheel) with ComfyUI Windows Portable (a separate non-Electron artifact that DOES ship embedded Python + per-GPU prebuilt wheels). This matters because the recommendation cites ComfyUI as precedent for option E (the prebuilt single-backend artifact). The correct reading actually STRENGTHENS the recommendation: ComfyUI Desktop is itself a uv-bootstrap precedent (option A/C), and only the Portable variant is the prebuilt-per-backend pattern. Fix the doc to name the two variants separately.\n2. Claim 10 (Ollama) overstates the source: docs.ollama.com/gpu documents only ONE silent-CPU-fallback scenario (Linux suspend/resume), not a general 'silent fallback on any missing/misconfigured CUDA/ROCm lib.' The directional point (runtime GPU detection that can silently land on CPU) is real and the contrast with our loud CPU_FALLBACK warning is valid, but soften the generalization.\n3. Claim 12 (PyInstaller) CPU-only figure '~620MB-1.8GB' is NOT in the cited discussion #8552 (only CUDA ~4-5GB + 'cannot be split' are sourced there); the precise CPU number is unverified. The load-bearing conclusion (no single frozen artifact for both GPU+CPU) stands regardless.\n4. Minor sourcing gaps (substance still correct): claim 7's 'macOS PyPI wheel is MPS-capable' is true general knowledge but not a quote from the uv page; claim 8 Pinokio's per-GPU/OS wheel logic is real but under-documented by the thin substack citation (corroborated elsewhere ? Pinokio install.js branching, ROCm version detection); claim 9's --device/--device_index flags live on the faster-whisper CLI wrappers (whisper-ctranslate2 / faster-whisper-cli), not the bare library README, though WhisperModel(device='auto') is in the core lib.\n\nADDITIONAL RISK NOT FLAGGED IN THE CLAIMS (worth adding to the doc): uv --torch-backend=auto has documented edge-case failures (astral-sh/uv issues #14742, #14647, #12357) ? it ignores CUDA compute-capability for older GPUs (e.g. GTX 1080 Ti), can fail on multi-GPU boxes

lacking /proc/driver/nvidia/version, and may pick the highest supported CUDA version rather than the most-compatible one. For the non-technical persona this is exactly where a first-run bootstrap could pick a broken wheel; the launcher should validate the resulting torch.cuda.is_available() post-install and surface option D (CPU-default) as the fallback, which the recommendation already proposes.\n\nLICENSING NOTE (project guardrail check): uv (Apache-2.0/MIT) and the torch CPU/CUDA wheels are compatible with the repo's MIT/Apache/BSD-only policy; nothing in the verified claims introduces a viral-license dependency. faster-whisper (MIT) and the upstream WASB env are unaffected by this packaging recommendation."
    }
  },
  {
    "topic": "launcher-ux-wrapper",
    "research": {
      "topic": "Launch UX for a downloadable, batteries-included local web-service app for a non-tech coach/parent in India (Windows-first): how the user starts the engine and reaches the UI ? weighed against locked decision D10 (\"self-hosted webserver the user spins up, NOT a desktop app\")",
      "options": [
        {
          "name": "(a) Command + open-browser (the baseline: a double-clickable .bat/.cmd that runs the server and opens http://127.0.0.1:8000)",
          "how_it_works": "Today the engine starts via `python -m backend.main --server --port 8000` (backend/main.py:556 main(), :562 --server) and binds 127.0.0.1:8000 (per LOCAL_APPLIANCE_ARCHITECTURE.md:14,30 citing main.py:638). Grep confirms there is NO webbrowser/tray/auto-open in main.py today. The minimal friendly layer is a packaged `Start KhelSutra.bat` (and a Start-Menu/desktop shortcut the installer drops) that: launches the server, polls GET /api/health (exists, main.py:127 per appliance doc) until ready, then calls Python's stdlib `webbrowser.open('http://127.0.0.1:8000')`. No new GUI framework, no WebView2, no extra license surface. This is literally D10's 'webserver the user spins up' with one quality-of-life line added.",
          "pros": [
            "Smallest possible delta from what ships today; zero new heavy dependencies and zero new license surface (stdlib webbrowser is in the std lib).",
            "Cleanest fit to D10's literal wording ? it IS a self-hosted server the user starts; we only auto-open the tab.",
            "No WebView2/Chromium runtime to install (Tauri needs WebView2; on <Win11 the installer must fetch it ? Windows-installer friction we avoid).",
            "Survives the hard packaging problem: the .bat just activates whatever runtime layout we ship (incl. the separate WASB conda env) and runs the server ? no freezing of torch/CUDA.",
            "Trivial to localize the few user-facing strings (a console line / a tiny landing page) into kn/hi consistent with the FRIENDLY_AND_MULTILINGUAL trilingual SPA work."
          ],
          "cons": [
            "A black console window stays open; closing it kills the server with no graceful 'Quit' ? confusing for a non-tech user (looks broken / scary).",
            "No restart-on-crash, no 'is it running?' affordance, no clean shutdown ? bad first impression vs a tray icon.",
            "If port 8000 is busy the server fails (often silently) ? the most common runtime issue reported for sidecar/local-server apps; the .bat alone gives no recovery UX.",
            "Double-clicking a .bat triggers Windows SmartScreen / AV warnings for an unsigned script; non-tech users in India bail at scary prompts."
          ],
          "maturity": "mature"
        },
        {
          "name": "(b) Tiny native launcher: pystray system-tray app that starts the server + opens the browser + offers Open/Restart/Quit (the RECOMMENDED option)",
          "how_it_works": "A small Python launcher (the existing `indexer` console entry can grow a `--launch`/tray mode) uses pystray to put a KhelSutra icon in the Windows system tray. On start it: spawns the engine server as a child process, waits for GET /api/health, auto-opens the default browser to 127.0.0.1:8000, and exposes a right-click menu: Open KhelSutra, Restart, View logs, Language (en/kn/hi), Quit. The UI is still the khelsutra web/ SPA served over localhost ? the browser is the renderer, exactly per D10. pystray supports Windows/macOS/Linux with full feature parity (per docs). It owns the server lifecycle (kills the child on Quit), can pick a

```
free port if 8000 is busy, and shows a balloon on crash. No WebView2/Chromium bundled. NOTE:
pystray is LGPL-3.0 ? dynamic import is generally compatible with our MIT/Apache/BSD code
guardrail, but it is NOT a permissive license, so flag for owner/counsel sign-off (or substitute
an MIT tray lib).",
          "pros": [
            "Satisfies D10's INTENT precisely: the artifact is still a self-hosted webserver
rendered in the user's own browser ? the tray is just a friendly on/off switch, not a desktop UI
shell (no embedded webview, no SPA fork).",
            "Solves every (a) pain point: graceful Quit, Restart, visible 'it's running' state,
log access, auto-port-selection, crash balloon ? the things a non-tech coach needs.",
            "Tiny footprint and no Chromium/WebView2 runtime to install ? avoids the biggest
Windows first-run friction Tauri/Electron carry.",
            "Cleanly wraps the hard packaging problem: the tray process just orchestrates
'activate env -> run server', so the dual-torch WASB conda env and system ffmpeg stay external
runtime config, never frozen.",
            "Cross-platform (Win/mac/Linux) with one codebase, matching the engine's existing
console-script + multi-OS posture.",
            "Tray menu is a natural home for the en/kn/hi language toggle and a 'first-run
wizard' launch."
          ],
          "cons": [
            "pystray is LGPL-3.0 ? a license-guardrail snag that needs an explicit dynamic-link
OK or an MIT alternative (e.g. an MIT tray shim / infi.systray); must be resolved before
adoption.",
            "Slightly more code + a per-OS packaging story (still far less than Tauri/Electron),
and tray behavior has platform quirks (Linux desktop-environment variance).",
            "Adds a process-lifecycle layer to test (start/health-poll/port-fallback/kill) ?
needs its own small test + observability per the project's launch-quality bar.",
            "On Windows, an unsigned launcher .exe still hits SmartScreen unless code-signed
(shared with every option that ships a binary)."
          ],
          "maturity": "mature"
        },
        {
          "name": "(c) Electron/Tauri shell wrapping the SPA, pointed at the local server (an
embedded-webview desktop app)",
          "how_it_works": "Ship a Tauri (Rust + system WebView2 on Windows) or Electron (bundled
Chromium) desktop app whose window renders the khelsutra web/ SPA, with the engine FastAPI
server spawned as a 'sidecar' child process (the documented Tauri+FastAPI+PyInstaller pattern).
Tauri binaries are small (~3-10MB) but require the WebView2 runtime; Electron bundles ~100-150MB
of Chromium. The sidecar pattern is proven but the article-documented failure modes are: silent
failure when port 8000 is taken, 5-10s startup, Windows UTF-8 crashes, and target-triple sidecar
naming. Crucially the Python sidecar is normally frozen with PyInstaller.",
          "pros": [
            "Most 'polished app' feel ? single window, native menus, no browser chrome;
appealing for a consumer launch.",
            "Tauri binaries are tiny and the shell licenses (MIT/Apache) are clean.",
            "One window owns everything; no stray console; obvious Quit."
          ],
          "cons": [
            "Directly contradicts D10's letter ('NOT a desktop app') AND its intent: an
embedded-webview window IS a desktop app and tends to fork the UI/origin away from the 'browser
pointed at localhost' model the owner locked. DESKTOP_APP_PLAN.md was explicitly SUPERSEDED by
D10 (LOCAL_APPLIANCE_ARCHITECTURE.md:39).",
            "WebView2 dependency: preinstalled only on Win11/recent Win10; on older Windows the
Tauri installer must download it (or bundle a +180MB fixed runtime) ? real first-run friction
for non-tech users on varied Indian hardware.",
            "Collides with the hard packaging problem: the sidecar pattern assumes a
PyInstaller-frozen Python, but PyInstaller+torch+CUDA balloons to 3-5GB and CUDA libs 'cannot be
split' (PyInstaller discussions), and Transformer Lab REJECTED PyInstaller precisely because it
can't modify the env post-install (our separate WASB conda env is the same blocker).",
            "Electron's Chromium bloat is the opposite of the 'slim app for huge-video laptops'
thesis; both shells add a large surface to sign/notarize/maintain across OSes.",
            "Highest build + maintenance cost for the least alignment with the locked decision."
          ],
```

```
          "maturity": "mature"
        },
        {
          "name": "(d) One-click installer (Inno Setup) that lays down the runtime + a Start-
Menu shortcut ? pairs WITH (a) or (b), not an alternative to them",
          "how_it_works": "An Inno Setup (open-source, permissive) .exe installer that: drops
the engine + a managed Python runtime (vendored or via uv/Astral standalone or Miniconda for the
WASB env), installs/locates system ffmpeg, optionally fetches the WASB weights from GCS on first
run, registers a Start-Menu + desktop shortcut that invokes the (a) batch or (b) tray launcher,
and (optionally) writes a winget manifest. This is the DISTRIBUTION layer ? it answers 'how do
they get it without git clone / build-from-source' (the stated goal) ? and it composes with
whichever launch mechanism (a/b) is chosen. The Transformer Lab precedent (Electron + Miniconda
installed to an app-specific dir, install.sh pulling pinned deps, defensive conda handling) is
the closest real-world analog to our dual-env reality.",
          "pros": [
            "Directly delivers the project GOAL ? a downloadable, batteries-included app with no
git clone / no build-from-source.",
            "Inno Setup is open-source and license-clean; supports non-admin installs, single-
EXE online distribution, and custom (Pascal) scripting for the GCS-weights fetch + env setup.",
            "A Start-Menu shortcut is the canonical, non-scary 'open the app' affordance for a
non-tech Windows user ? far better than asking them to run a command.",
            "Handles the hard packaging problem honestly: install a managed Miniconda/uv env
(Transformer Lab pattern) so the dual-torch WASB env + system ffmpeg are provisioned at install
time, NOT frozen ? sidesteps the PyInstaller+CUDA 3-5GB dead-end.",
            "First-run wizard naturally lives here (or in the launched app) ? set Gemini key /
pick GPU-or-cloud / point at videos."
          ],
          "cons": [
            "Not a launch-UX by itself ? it must be paired with (a) or (b) to actually start the
server + open the UI; on its own it only installs.",
            "Big download or a long first-run dependency fetch (torch/CUDA wheels need --index-
url, deliberately not in pyproject deps; weights pulled from GCS) ? needs a clear progress UI so
users don't think it hung.",
            "Unsigned installers trip SmartScreen/AV; a code-signing cert (and budget) is needed
for a clean non-tech experience.",
            "Windows-first today; mac/Linux need their own installer story later (out of scope
for the India/Windows beachhead)."
          ],
          "maturity": "mature"
        }
      ],
      "load_bearing_claims": [
        {
          "claim": "D10 locks delivery as 'a self-hosted webserver + UI the user spins up (NOT a
desktop app)... the user points at local storage + GPU/CPU OR cloud locations and the UI drives
the tool', and explicitly SUPERSEDES DESKTOP_APP_PLAN.md.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:39"
        },
        {
          "claim": "The engine today starts via a console entry main() with a --server flag
(python -m backend.main --server --port 8000); there is NO browser auto-open, no system tray,
and no launcher in main.py ? grep for webbrowser/tray/--open returns only def main() and the
--server arg.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/main.py:556,562"
        },
        {
          "claim": "The engine server is hard-bound to 127.0.0.1:8000, so the browser-on-
localhost model is the structural reality the launch UX must wrap.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/docs/.../LOCAL_APPLIANCE_ARCHITECTURE.md:14,30 (citing backend/main.py:638)"
        },
```

```
        {
          "claim": "GET /api/health exists and returns {status, sport, default_segmenter}, so a
launcher can poll readiness before opening the browser.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:47
(citing backend/main.py:127)"
        },
        {
          "claim": "The hard packaging problem is real and load-bearing: native WASB runs in a
SEPARATE conda env (torch 1.11+cu113) while the main engine uses torch 2.6+cu124; ffmpeg/ffprobe
are system deps; WASB weights live in a GCS bucket ? so a single frozen binary cannot represent
the runtime.",
          "confidence": "high",
          "source": "orchestrator-provided KEY FACTS + DESKTOP_APP_PLAN.md:37 (de-WSL native
runner) and LOCAL_APPLIANCE_ARCHITECTURE.md:96 (torch-1.11 conda env makes the image non-
trivial)"
        },
        {
          "claim": "PyInstaller-bundling PyTorch with CUDA produces 3-5GB executables and
CUDA/cuDNN shared libs 'cannot be split'; small frozen torch apps are CPU-only/older-CUDA ?
making PyInstaller a poor fit for our GPU app.",
          "confidence": "high",
          "source": "https://github.com/orgs/pyinstaller/discussions/8552 and
https://discuss.pytorch.org/t/.../194822"
        },
        {
          "claim": "Transformer Lab (Electron + Python/PyTorch API server) explicitly REJECTED
PyInstaller because users install plugins that add arbitrary Python libs, so they needed a
modifiable post-install Python env; they ship an isolated Miniconda installed to an app-specific
dir with pinned deps and defensive conda handling.",
          "confidence": "high",
          "source": "https://lab.cloud/blog/packaging%20python/"
        },
        {
          "claim": "The Tauri+FastAPI sidecar pattern is proven, but its most common runtime
failure is silent backend-start failure when port 8000 is already in use; startup is 5-10s with
model load; Windows UTF-8 crashes are a known gotcha.",
          "confidence": "high",
          "source": "https://aiechoes.substack.com/p/building-production-ready-desktop"
        },
        {
          "claim": "Tauri on Windows requires the WebView2 runtime: preinstalled on Windows 11
(and recent Win10), but on older Windows the Tauri installer must download/install it, or bundle
a fixed runtime (+~180MB).",
          "confidence": "high",
          "source": "https://v2.tauri.app/distribute/windows-installer/ and
https://v2.tauri.app/reference/webview-versions/"
        },
        {
          "claim": "pystray supports Windows/macOS/Linux with full feature parity for icon +
menu + Quit, but is licensed LGPL-3.0 ? not in the project's MIT/Apache/BSD code guardrail, so
dynamic-link compatibility needs explicit sign-off or an MIT substitute.",
          "confidence": "high",
          "source": "https://github.com/moses-palmer/pystray (license) +
https://pystray.readthedocs.io/en/latest/usage.html (platform support)"
        },
        {
          "claim": "Inno Setup is an open-source Windows installer builder supporting single-EXE
distribution, non-admin installs, and Pascal scripting (usable to fetch GCS weights / set up the
env); winget can ingest INNO/EXE/MSI installer types.",
          "confidence": "high",
          "source": "https://jrsoftware.org/isinfo.php + https://learn.microsoft.com/en-
us/windows/package-manager/winget/"
        },
        {
```

```
          "claim": "Mature local-AI apps (Open WebUI/Ollama) treat first-run as: open localhost in a browser, create an account, then enter provider URL + API key in settings, and pull models from an in-app browser ? a precedent for an in-UI (not OS-installer) first-run wizard for the Gemini key / compute choice.",
          "confidence": "medium",
          "source": "https://docs.openwebui.com/getting-started/quick-start/connect-a-provider/"
        },
        {
          "claim": "Compute placement is already a setting surfaced from GET /api/capabilities (truly-local / cloud-serving), so the 'pick GPU-or-cloud' wizard step maps onto an existing engine seam rather than net-new config.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:35,90-100 (D6 + compute matrix); engine backend/config/presets.py PROFILE_PRESETS"
        }
      ],
      "recommendation": "Adopt (b)+(d): a tiny system-tray LAUNCHER that owns the server lifecycle and auto-opens the browser, distributed by a one-click Inno Setup installer. This SATISFIES D10's intent rather than violating it ? the rendered product stays the khelsutra SPA in the user's own browser at 127.0.0.1:8000 (no embedded webview, no UI fork); the tray is only a friendly on/off switch + status light + language toggle, i.e. exactly 'a self-hosted webserver the user spins up' made non-scary. A launcher is NOT a desktop app in D10's sense; option (c) (Electron/Tauri embedded-webview shell) IS, and it should be rejected: it contradicts D10's letter (the superseded DESKTOP_APP_PLAN), adds a WebView2/Chromium runtime burden on varied Indian Windows hardware, and collides head-on with the hard packaging problem because its sidecar pattern presumes a PyInstaller freeze that blows up to 3-5GB with CUDA and can't host our separate WASB conda env.\\n\\nSequencing (smallest-first, evidence-graduated, matching repo norms): Slice 1 ? ship option (a) immediately: have the `indexer` console entry optionally health-poll then webbrowser.open after --server (a few stdlib lines, zero new deps, zero license risk) so the auto-open habit lands first. Slice 2 ? wrap it in the (b) pystray tray (Open/Restart/View logs/Language/Quit, free-port fallback for the documented port-8000-busy failure, crash balloon, graceful child-process kill), FIRST resolving the pystray LGPL-3.0 guardrail question (get a dynamic-link OK or swap to an MIT tray lib). Slice 3 ? the (d) Inno Setup installer that provisions a managed env the Transformer-Lab way (isolated Miniconda/uv in an app-specific dir for the dual-torch + WASB env, locate/ship LGPL ffmpeg, fetch WASB weights from GCS on first run with a progress UI), drops a Start-Menu shortcut to the tray launcher, and (optionally) a winget manifest. Budget a code-signing cert so SmartScreen/AV don't scare non-tech users off the .exe.\\n\\nFirst-run wizard: put it IN the web UI (the Open WebUI/Ollama precedent), launched on the tray app's first open, with 3 steps grounded in existing seams ? (1) Gemini key: paste-or-skip, stored locally, with the consent copy that ?1280px chunks leave the machine (never a training source per the guardrail); (2) Compute: pick Local GPU / Local CPU / Cloud, rendered from GET /api/capabilities so it only offers what the box can honestly do (truly-local vs cloud-serving presets) and disables options the hardware can't back; (3) Videos: point at a local folder (or bucket/Drive later). Keep all three steps en/kn/hi-localized to stay consistent with the trilingual SPA work, and emit structured logs at each step per the project's launch-observability bar.",
      "fit_for_this_app": "This engine is uniquely ill-suited to the 'freeze it into one .exe' instinct, which is why the launcher+installer split is the right call. (1) Dual torch envs: native WASB needs torch 1.11+cu113 in a SEPARATE conda env while the engine runs torch 2.6+cu124 ? a single PyInstaller freeze cannot hold both, and PyInstaller+torch+CUDA is independently a 3-5GB 'CUDA libs can't be split' dead-end; the Transformer Lab precedent (reject PyInstaller, ship an isolated Miniconda that stays modifiable post-install) maps 1:1 onto our reality. (2) torch/torchvision are deliberately NOT in pyproject deps because their CUDA/CPU wheels need --index-url (PEP621 can't express it) ? so the INSTALLER, not a frozen bundle, is the only honest place to resolve the right wheel for the user's GPU/CPU; ffmpeg stays a system dep (ship an LGPL build) and WASB weights are pulled from the GCS bucket on first run. (3) The decoupled-compute seam (M1-M8: deployment.profile/compute_target on AppConfig, PROFILE_PRESETS truly-local/cloud-serving/cpu-mock, StorageBackend + ComputeBackend registries, startup.py per-profile checks) already turns 'GPU-or-cloud' into a SETTING surfaced by GET /api/capabilities ? so the wizard's compute step is config wiring, not net-new. (4) D10 + the live LOCAL_APPLIANCE_ARCHITECTURE design already commit to browser-on-localhost (127.0.0.1:8000, edge-gate/firewall as the tenancy boundary); a tray launcher preserves that origin model exactly while an Electron/Tauri shell would fork it. (5) The SPA lives in khelsutra/web (Vite/React, already en+kn+hi) ? the launcher/installer must serve THAT bundle (bundling-the-UI is the known
```

cross-repo concern), and the wizard/tray strings must stay trilingual to match. (6) Every added dependency must clear the MIT/Apache/BSD guardrail: Inno Setup, Tauri, pywebview, webbrowser(stdlib) are clean; pystray is LGPL-3.0 and is the one flag to resolve before shipping option (b). Paid Gemini stays inference-only with explicit consent copy, never a training source ? the wizard's key step must say so."
    },
    "verdict": {
      "claims_checked": [
        {
          "claim": "D10 locks delivery as 'a self-hosted webserver + UI the user spins up (NOT a desktop app)... the user points at local storage + GPU/CPU OR cloud locations and the UI drives the tool', and explicitly SUPERSEDES DESKTOP_APP_PLAN.md.",
          "verdict": "confirmed",
          "reasoning": "Verified verbatim at C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:39. D10 reads: 'Delivery = a self-hosted webserver + UI the user spins up (NOT a desktop app); the user points at local storage + GPU/CPU OR cloud locations and the UI drives the tool ... supersedes DESKTOP_APP_PLAN.md; one artifact (this appliance) runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings.' Wording and the supersede claim match exactly. (Note: DESKTOP_APP_PLAN.md lives in the ENGINE repo docs/, status 'DRAFT ? nothing is built', so what is being superseded is an unbuilt scoping doc.)"
        },
        {
          "claim": "The engine today starts via a console entry main() with a --server flag (python -m backend.main --server --port 8000); there is NO browser auto-open, no system tray, and no launcher in main.py.",
          "verdict": "confirmed",
          "reasoning": "backend/main.py:556 def main(); :562 --server arg; :563 --port default 8000; pyproject.toml:77 'indexer = \"backend.main:main\"' console entry. Grep of main.py for webbrowser|tray|--open|pystray|webview returned NO matches. The cited line numbers in the claim (556/562) are correct. No auto-open/tray/launcher exists. Confirmed."
        },
        {
          "claim": "The engine server is hard-bound to 127.0.0.1:8000, so the browser-on-localhost model is the structural reality the launch UX must wrap.",
          "verdict": "confirmed",
          "reasoning": "backend/main.py:735 is `uvicorn.run(app, host=\"127.0.0.1\", port=args.port)` ? host is a hard-coded literal (not configurable), port defaults to 8000 (:563). So the substance (hard-bound 127.0.0.1, default 8000) is true in current code. CAVEAT: the cited source line 'backend/main.py:638' is STALE ? line 638 is now a db_path assignment; the binding moved to line 735 as the engine grew. The fact holds; only the line citation drifted (the khelsutra doc warns line numbers drift as the engine ships PRs)."
        },
        {
          "claim": "GET /api/health exists and returns {status, sport, default_segmenter}, so a launcher can poll readiness before opening the browser.",
          "verdict": "confirmed",
          "reasoning": "backend/main.py:135-146: @app.get('/api/health') def health() returns {'status':'ok','sport':sport,'default_segmenter':...}, no auth ('Cheap liveness/readiness probe (no auth)'). Shape matches exactly. A pre-open readiness poll is feasible. (Cited line :127 vs actual :135 is a minor drift; the doc itself notes it was 'corrected 2026-06-21'.) Confirmed."
        },
        {
          "claim": "The hard packaging problem is real: native WASB runs in a SEPARATE conda env (torch 1.11+cu113) while the main engine uses torch 2.6+cu124; ffmpeg/ffprobe are system deps; WASB weights live in a GCS bucket ? so a single frozen binary cannot represent the runtime.",
          "verdict": "confirmed",
          "reasoning": "Substance confirmed with ONE numeric correction. WASB conda env = 'Python 3.8, torch 1.11.0+cu113' (docs/TRACKNET_WSL_SETUP.md:15) ? matches. ffmpeg/ffprobe are NOT pip deps (absent from requirements.txt/pyproject.toml; scripts/doctor.py checks them on PATH) ? system deps, confirmed. WASB weights in GCS: deploy/cloudrun/README.md:15 and deploy/gcp/README.md:183-184 reference gs://<bucket>/models/wasb_badminton_best.pth.tar ? confirmed. CORRECTION: the main engine's documented torch is 'torch>=2.12.0 ... cu124' (pyproject.toml:62, README), NOT 'torch 2.6+cu124' as stated. The dual-incompatible-torch + system-ffmpeg + remote-weights conclusion (a single frozen binary cannot represent the runtime)

holds regardless of the exact minor version.",
          "corrected_claim": "Native WASB runs in a separate conda env (Python 3.8, torch 1.11.0+cu113) while the main engine targets torch>=2.12.0 + cu124 (not 2.6); ffmpeg/ffprobe are system deps and WASB weights live in a GCS bucket (gs://<bucket>/models/wasb_badminton_best.pth.tar) ? so a single frozen binary cannot represent the runtime."
        },
        {
          "claim": "PyInstaller-bundling PyTorch with CUDA produces 3-5GB executables and CUDA/cuDNN shared libs 'cannot be split'; small frozen torch apps are CPU-only/older-CUDA.",
          "verdict": "confirmed",
          "reasoning": "PyInstaller discussion #8552 / issue #8551 and PyTorch forums confirm: torch+CUDA conda install ~5GB, torch lib in PyInstaller ~4GB; most of the size is binary extensions and CUDA/cuDNN shared libs 'which cannot be split'; the only size-reduction path is excluding CUDA (i.e. CPU-only). This directly supports the claim and the 'GPU app is a poor fit for PyInstaller' conclusion."
        },
        {
          "claim": "Transformer Lab (Electron + Python/PyTorch API server) explicitly REJECTED PyInstaller because users install plugins adding arbitrary Python libs; they ship an isolated Miniconda in an app-specific dir with pinned deps and defensive conda handling.",
          "verdict": "confirmed",
          "reasoning": "lab.cloud/blog packaging post confirms each element: 'an Electron app ... communicates with an API that handles ML tasks on top of our Python Engine'; rejected PyInstaller because 'users ... install arbitrary Python libraries. So we needed an accessible Python environment that we could modify after it's installed'; they 'install our OWN conda in a Transformer Lab specific path even if the user already has conda'; 'deactivate conda three times (!)'; 'pin ALL package libraries using piptools'. All specifics match."
        },
        {
          "claim": "The Tauri+FastAPI sidecar pattern's most common runtime failure is silent backend-start failure when port 8000 is in use; startup is 5-10s with model load; Windows UTF-8 crashes are a known gotcha.",
          "verdict": "confirmed",
          "reasoning": "aiechoes.substack.com 'Building Production-Ready Desktop LLM Apps' confirms verbatim: 'Port conflicts are the most common issue. If port 8000 is already in use, the backend fails to start silently'; 'Startup time is 5-10 seconds with model loading'; 'UTF-8 encoding on Windows. Python's default console encoding causes crashes when llama.cpp outputs progress indicators with emojis or special characters' (fixed via PYTHONUTF8=1 / PYTHONIOENCODING). All three sub-claims confirmed. (Source is a single blog/anecdote, but it is exactly the cited source and the failure modes are well-corroborated by the FastAPI/Tauri issue tracker.)"
        },
        {
          "claim": "Tauri on Windows requires WebView2: preinstalled on Windows 11 (and recent Win10); on older Windows the installer must download/install it or bundle a fixed runtime (+~180MB).",
          "verdict": "confirmed",
          "reasoning": "Official v2.tauri.app/distribute/windows-installer: 'On Windows 10 (April 2018 release or later) and Windows 11, the WebView2 runtime is distributed as part of the operating system.' Installer options: downloaded bootstrapper (default, needs internet), embedded bootstrapper (+~1.8MB), offline installer (+~127MB), fixed version (+~180MB). So 'preinstalled on Win11 and recent Win10; older Windows needs download/bundle; fixed runtime +~180MB' is accurate. 'Older Windows' = pre-April-2018 Win10 / Win7."
        },
        {
          "claim": "pystray supports Windows/macOS/Linux with full feature parity for icon + menu + Quit, but is LGPL-3.0 ? outside the project's MIT/Apache/BSD guardrail, so dynamic-link compatibility needs sign-off or an MIT substitute.",
          "verdict": "confirmed",
          "reasoning": "License + guardrail-conflict half is FULLY confirmed: pystray is LGPL-3.0 (GitHub COPYING.LGPL), which is outside the project's ratified MIT/Apache/BSD-only code/weights guardrail (CLAUDE.md), so it genuinely needs a dynamic-link OK or an MIT swap ? this is the load-bearing point and it is correct. PARTIAL OVERSTATEMENT in the supporting detail: it does NOT have 'full feature parity' across all three OSes ? pystray docs state 'Menus

```
are not supported on X' (Linux/Xorg gives only a default action), and AppIndicator/macOS have
default-action/radio limitations. Windows and macOS have full menu+Quit support; Linux is
degraded. Since the launcher targets Windows (Inno installer), the practical recommendation is
unaffected, but the literal cross-platform-parity phrasing is wrong.",
          "corrected_claim": "pystray supports Windows/macOS/Linux for a tray icon + menu + Quit
(icon.stop), with FULL menu support on Windows and macOS but documented Linux limitations (no
menus on Xorg; AppIndicator lacks a default action). It is LGPL-3.0 ? outside the project's
MIT/Apache/BSD guardrail ? so dynamic-link use needs explicit sign-off or an MIT substitute."
        },
        {
          "claim": "Inno Setup is an open-source Windows installer builder supporting single-EXE
distribution, non-admin installs, and Pascal scripting (usable to fetch GCS weights / set up the
env); winget can ingest INNO/EXE/MSI installer types.",
          "verdict": "confirmed",
          "reasoning": "jrsoftware.org/isinfo: Inno Setup is 'an open-source installation
builder' (source on GitHub), 'Supports creation of a single EXE', 'Extensive support for both
administrative and non administrative installations', 'Integrated Pascal scripting engine ...
for advanced run-time install and uninstall customization' (so it can run custom logic like
fetching weights). winget manifest InstallerType supported values include inno, exe, msi
(microsoft/winget-pkgs schema). All confirmed. Minor nuance: Inno is free even for commercial
use but the author 'requests' a commercial license purchase ? it is permissively usable,
consistent with 'open-source'."
        },
        {
          "claim": "Mature local-AI apps (Open WebUI/Ollama) treat first-run as: open localhost
in a browser, create an account, then enter provider URL + API key in settings, and pull models
from an in-app browser ? a precedent for an in-UI first-run wizard.",
          "verdict": "confirmed",
          "reasoning": "Confirmed for the load-bearing precedent (the claim is tagged medium).
Open WebUI docs: open localhost in browser, configure a provider under Admin Settings ->
Connections by entering a URL + API key, models then auto-detected. Account creation on first
login is a known Open WebUI behavior (the first account becomes admin). The 'pull models from an
in-app browser' detail is more an Ollama-integration feature than something this specific page
substantiates ? partially supported but not central. Overall the precedent (an in-UI, not OS-
installer, first-run flow for provider URL + key) is sound."
        },
        {
          "claim": "Compute placement is ALREADY a setting surfaced from GET /api/capabilities
(truly-local / cloud-serving), so the wizard step maps onto an existing engine seam rather than
net-new config.",
          "verdict": "refuted",
          "reasoning": "GET /api/capabilities DOES NOT EXIST in the engine. Grep across the
badminton repo for 'capabilities' hits only artifacts/appliance_design.json; the registered API
routes are /api/config, /api/health, /api/process, /api/jobs/{id}, /api/jobs|videos/{id}/cost,
/api/highlights, /api/query, /api/compile ? no /api/capabilities. The khelsutra doc itself
REFUTES this: line 49 lists 'GET /api/capabilities' under 'Absent (net-new) [verified ? grep
returns none]', line 113 lists it under 'Net-new', and D6 (line 35) tags it '[design]'. What
DOES exist is PROFILE_PRESETS in backend/config/presets.py (truly-local/cloud-serving/cpu-mock)
? config presets, NOT an HTTP capabilities endpoint. So the 'pick GPU-or-cloud' step rests on
presets that exist, but the specific seam named (GET /api/capabilities) is unbuilt ? the claim's
'[verified]/already surfaced' framing is false; it is net-new work.",
          "corrected_claim": "Compute placement maps onto existing engine CONFIG presets
(backend/config/presets.py PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock), but GET
/api/capabilities does NOT exist ? it is explicitly net-new ([design]) per the appliance doc
(lines 35/49/113). The wizard step needs a new endpoint to surface those presets to the UI."
        }
      ],
      "overall_notes": "Verified against current code (engine repo on branch
feat/serving-m9-auth, up to date with origin; khelsutra on master, up to date ? could not ff-
pull khelsutra due to 'multiple branches' but it is current with origin/master, so reads are
fresh). 10 of 13 claims CONFIRMED; 2 confirmed-with-a-correction; 1 REFUTED.\n\nThe single load-
bearing REFUTATION is claim 13: GET /api/capabilities does NOT exist ? the appliance doc's own
§3/§4 list it as net-new/[design], and grep confirms zero hits in the engine. The
recommendation's first-run wizard step 2 ('Compute: rendered from GET /api/capabilities') and
Settings screen therefore depend on building a new endpoint, not wiring an existing seam.
```

PROFILE_PRESETS (truly-local/cloud-serving/cpu-mock) DO exist in backend/config/presets.py, so the underlying config is real; only the HTTP surface is missing. This does not break the recommendation but downgrades 'maps onto existing seam' to 'needs a small net-new endpoint over existing presets'.\n\nTwo corrections that do not change the recommendation's conclusion:\n- Claim 5: main-engine torch is 'torch>=2.12.0 + cu124' (pyproject.toml:62), not 'torch 2.6'. The dual-incompatible-torch packaging problem (WASB env = torch 1.11+cu113) is fully real either way.\n- Claim 10: pystray is LGPL-3.0 (the guardrail-conflict point is correct and load-bearing), but 'full feature parity across Windows/macOS/Linux' is overstated ? Linux/Xorg has no menus per pystray docs. Irrelevant to a Windows-first Inno launcher, but the phrasing is wrong.\n\nStale-but-true line citations (the engine ships PRs between sessions, so doc line numbers drift): main.py:638 for the 127.0.0.1 binding is now main.py:735; /api/health cited :127 is now :135. Substance of claims 3 and 4 unaffected. This matches the project's own '[verified] without a file:line is a hypothesis' lesson ? I re-verified each engine fact at its current file:line.\n\nExternal-fact verifications (claims 6-12) all checked out against their cited sources (PyInstaller discussions, Transformer Lab blog, the aiechoes Tauri/FastAPI post, official Tauri Windows-installer docs, pystray GitHub/readthedocs, jrsoftware/winget docs, Open WebUI docs). Note claims 8 and 12 each rest on a single blog/anecdotal source (medium-strength evidence), though their specifics are corroborated by issue trackers.\n\nNet: the core thesis of the recommendation ? that a tray launcher + Inno installer satisfies D10 while a PyInstaller-frozen Tauri/Electron shell collides with the real dual-torch/CUDA packaging problem ? is well-supported by the verified evidence. The two action items the verification surfaces: (1) treat the /api/capabilities endpoint as net-new build work in Slice 3 / the wizard, not an existing seam; (2) the pystray LGPL-3.0 guardrail gate is correctly flagged and genuinely needs owner sign-off or an MIT tray substitute before Slice 2.\n\nFiles inspected (all absolute): C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py (135-146, 556-572, 728-735); .../pyproject.toml (55-77); .../backend/config/presets.py (27-74); .../docs/TRACKNET_WSL_SETUP.md:15; .../docs/DESKTOP_APP_PLAN.md (3,39,50-51,63,105); .../deploy/cloudrun/README.md:15, .../deploy/gcp/README.md:183-184; C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md (12,14,35,39,49,90-100,113,133)."
    }
  },
  {
    "topic": "prior-art-deepdive",
    "research": {
      "topic": "How comparable LOCAL AI/CV/video apps ship \"batteries-included\" ? distribution, GPU/torch + ffmpeg + model-weights handling, install UX, and updates ? and which patterns fit our downloadable, self-hosted webserver+UI badminton highlight indexer for a non-tech Windows user.",
      "options": [
        {
          "name": "Pinokio-style: one-click launcher that provisions per-app isolated envs (conda/venv) + GPU libs + ffmpeg",
          "how_it_works": "Pinokio is an MIT-licensed 'AI browser'/launcher. One click downloads the target app, then 'automatically downloads the app, sets up Python, Conda, virtual environments, GPU libraries and all required dependencies, then launches a local web interface in your browser.' Everything is isolated under ~/pinokio (built-in Conda, pip, npm, git, ffmpeg). It even 'installs the latest version of FFMPEG to support new features, including torchcodec APIs', and a declarative `venv` directive creates/activates per-app venvs. Each app's python deps are app-isolated, so two apps needing different torch/CUDA versions coexist ? the exact shape of our WASB-conda(torch1.11/cu113) vs engine(torch2.6/cu124) split.",
          "pros": [
            "Solves our SIGNATURE problem natively: multiple isolated Python envs with different torch/CUDA versions side by side ? mirrors wasb_runner.py's `conda activate wasb` (base.py:74,162) vs the main engine env",
            "Bundles ffmpeg automatically (our system-dep pain) and GPU libraries; user never touches a terminal",
            "MIT license = redistributable; we could even author a Pinokio 'install script' (JSON/JS) for our app as a low-effort distribution channel without owning the launcher",
            "Launches directly into a local web UI in the browser ? matches owner decision D10 (self-hosted webserver+UI, NOT a desktop app; LOCAL_APPLIANCE_ARCHITECTURE.md:39)"
          ],
          "cons": [
            "Requires the user to install Pinokio first (one extra dependency / brand), not a single self-branded installer",

```
            "We don't control the shell's UX, update cadence, or error surfaces ? debugging a
non-tech user's broken install is harder when the env manager is third-party",
            "Pinokio scripts are an emerging/niche format; smaller community than
Docker/Electron; long-term maintenance risk",
            "Still needs us to author + maintain a correct install script that fetches weights
from GCS and wires the two envs"
          ],
          "maturity": "emerging"
        },
        {
          "name": "ComfyUI-Desktop-style: Electron/Tauri shell + bundled `uv` that installs
torch on first run + NSIS installer + auto-update",
          "how_it_works": "ComfyUI's official desktop app (GPL-3.0) ships as an Electron app via
an NSIS installer (Windows) / DMG (macOS) to %LOCALAPPDATA%\\Programs\\ComfyUI. It does NOT
embed Python directly ? it bundles `uv` and 'on startup installs all the necessary python
dependencies with uv and starts the ComfyUI server', using pinned `.compiled` requirements. The
user picks a models/outputs folder (config.json + extra_models_config.yaml). It 'automatically
updates with stable releases of ComfyUI, ComfyUI-Manager (pip), and the uv executable'.
Crucially, `uv` is the one packaging tool that CAN express the torch `--index-url` problem our
pyproject.toml header says PEP 621 cannot: `[[tool.uv.index]]` + `[tool.uv.sources]` with
`explicit=true` route torch/torchvision to https://download.pytorch.org/whl/{cpu,cu124} per-
platform.",
          "pros": [
            "`uv` directly solves the torch CUDA/CPU `--index-url` blocker called out in
pyproject.toml:8-11,58-65 ? it pins torch to the right index declaratively, per-platform,
reproducibly",
            "First-run dependency install keeps the downloaded artifact small and lets the SAME
installer adapt to CPU-only vs NVIDIA boxes (resolves the 'typical enthusiast laptop has no
discrete GPU' crux in DESKTOP_APP_PLAN.md:14,46)",
            "Auto-update of app + deps + the package manager itself is a proven, low-friction
update channel for non-tech users",
            "User-chosen models/weights folder is exactly how we'd land the GCS
wasb_badminton_best.pth.tar weights once, then run offline",
            "A thin shell that just spawns our existing `python -m backend.main --server` +
opens 127.0.0.1:8000 reuses 100% of the engine, no re-architecture"
          ],
          "cons": [
            "ComfyUI-desktop's code is GPL-3.0 ? we can copy the PATTERN but must NOT vendor its
shell into our MIT/Apache/BSD-only bundle (licensing guardrail); use a clean shell (Tauri
MIT/Apache, or a plain installer) instead",
            "Electron is heavy (~100-150MB Chromium); Tauri (MIT/Apache, system webview,
~3-10MB) is the license- and size-cleaner shell per DESKTOP_APP_PLAN.md:62",
            "First-run `uv` install needs network + several GB of torch wheels ? a slow/failed
first launch is the top non-tech support risk on Indian uplinks",
            "Still leaves our HARD case unsolved by itself: `uv` gives ONE env; the WASB
conda(torch1.11) sidecar is a second, incompatible env. The real fix is finishing
native_wasb_runner.py so there is only ONE torch ? then `uv` suffices",
            "Building/codesigning a desktop installer (Todesktop/NSIS) is net-new surface to
maintain"
          ],
          "maturity": "mature"
        },
        {
          "name": "Embedded-Python portable ZIP (Fooocus / A1111 / ComfyUI-portable / Ollama-
style single installer)",
          "how_it_works": "Fooocus and ComfyUI-portable ship a ~1GB+ ZIP with an embedded Python
(`python_embeded`) and pre-installed torch; the user extracts and double-clicks `run.bat`, which
downloads model weights on first run and opens a localhost web UI. AUTOMATIC1111 is the
cautionary opposite: its 'one-click' still needs the user to pre-install Python 3.10.6 + git,
then `git clone`, then `webui-user.bat` builds a venv and pip-installs torch (5-10 min first
run) ? too many manual steps for non-tech users. Ollama is the gold standard for install UX: a
~5MB OllamaSetup.exe installs without admin rights, auto-starts a background service, auto-
detects NVIDIA CUDA, pulls weights on demand to %USERPROFILE%\\.ollama, and updates by running a
new installer over the old one (models preserved).",
          "pros": [
```

```
            "Embedded Python = zero 'install Python/git' steps, the #1 thing that breaks A1111
for non-tech users",
            "Double-click run.bat -> browser UI is the simplest possible mental model and
matches our 'spin up a local web service' goal",
            "First-run weight download (Fooocus, Ollama) is exactly how we'd fetch wasb weights
from GCS ? ship the code, not the GBs",
            "Ollama's 'run new installer over old, models preserved' is a dead-simple update
story a non-tech user understands",
            "Fully offline after first run ? fits the privacy/no-upload default in
DESKTOP_APP_PLAN.md D1"
          ],
          "cons": [
            "A pre-baked torch+CUDA ZIP locks ONE CUDA version; mismatched driver = silent GPU
fallback to CPU (hours per clip on a no-GPU laptop). Ollama avoids this by shipping its own CUDA
runtime + CPU fallback ? we'd need the same",
            "Big download (multi-GB if torch is pre-bundled) is painful on Indian uplinks; the
uv/first-run-install approach downloads less up front",
            "ZIP-extract portable apps have no real updater (you re-download) unless you add one
(ComfyUI-portable uses an update/ batch + pygit2)",
            "Fooocus/A1111 are GPL-3.0 ? pattern only, do not vendor",
            "Does NOT solve the dual-env WASB problem; same caveat as the uv option"
          ],
          "maturity": "mature"
        },
        {
          "name": "Frigate-style: Docker image bundling ffmpeg + ML detectors + web UI,
distributed via image tags / Home-Assistant add-on",
          "how_it_works": "Frigate (the closest functional analog: ffmpeg + ML object detectors
+ FastAPI-ish web UI) ships as a Docker image that bundles its own ffmpeg (at /usr/lib/ffmpeg;
v7) and supports pluggable detectors (OpenVINO default, NVIDIA TensorRT in the `-tensorrt` image
variant, Coral). Models are NOT pre-bundled in recent versions ? users drop an ONNX model into
/config/model_cache and Frigate converts it on first boot. Distribution = Docker tags (`stable`,
`stable-tensorrt`) on GHCR + a Home Assistant add-on; updates = re-pull the image or automate
with Watchtower. Image variants per accelerator solve the 'which GPU' problem by selecting the
right tag.",
          "pros": [
            "Bundling ffmpeg INSIDE the artifact (vs a system dep) is the cleanest fix for our
ffmpeg/ffprobe pain ? a container or portable bundle carries an LGPL ffmpeg build
(DESKTOP_APP_PLAN.md D4: openh264/NVENC, no GPL x264)",
            "Per-accelerator image variants (CPU/OpenVINO vs TensorRT) is a clean way to ship
the right GPU stack without one bloated artifact ? maps to our truly-local vs cpu-mock presets
(presets.py)",
            "Docker tag `stable` + Watchtower is a robust, well-understood update channel for
the server/cloud profile",
            "Drop-in model_cache + first-boot conversion mirrors how we'd stage GCS weights",
            "Containerizing also unblocks our reserved Cloud Run+GPU serving path
(LOCAL_APPLIANCE_ARCHITECTURE.md:96,100 ? 'no Dockerfile exists' is currently a gap)"
          ],
          "cons": [
            "Docker on Windows for a NON-TECH user is a non-starter: Frigate itself says Windows
is unofficial, runs only via WSL2+Docker Desktop, and 'getting the GPU and/or Coral passed to
Frigate may be difficult or impossible' ? GPU passthrough on Windows is the exact thing our user
can't debug",
            "Docker Desktop install + WSL2 + licensing prompts is far more friction than
Ollama's 5MB exe",
            "The WASB torch-1.11 conda env makes our worker image 'non-trivial'
(LOCAL_APPLIANCE_ARCHITECTURE.md:96) ? a multi-stage/multi-env image is real work",
            "Best fit for the SERVER/cloud profile, NOT the non-tech-Windows-desktop profile ?
two different delivery shapes"
          ],
          "maturity": "mature"
        },
        {
          "name": "LM Studio / Jan / GPT4All-style: self-contained desktop app with a bundled
inference engine + built-in model hub + local OpenAI-compatible server",
```

```
          "how_it_works": "These bundle the inference runtime (llama.cpp / MLX) INSIDE the app
so there is no separate Python/torch to manage. LM Studio (0.4+) splits into a headless daemon
(`llmster`) + GUI, exposes an OpenAI-compatible server, and manages swappable 'LM Runtimes'
(e.g. llama.cpp engine versioned independently, updated via Ctrl+Shift+R) so the engine updates
without reinstalling the app. Jan is a Tauri app bundling llama.cpp, serving localhost:1337.
GPT4All ships an OS installer + in-app model hub (HuggingFace browse/download to local disk),
local server on :4891. All download weights on demand via a GUI hub and run fully offline
after.",
          "pros": [
            "The GUI model hub (browse -> download -> ready) is the gold standard for getting
weights onto a non-tech user's disk ? directly applicable to fetching our GCS wasb weights with
a progress bar instead of a CLI",
            "Decoupling a headless daemon from the GUI (LM Studio's llmster) matches our
architecture exactly: GUI/SPA in khelsutra + headless `python -m backend.main --server` engine;
the SPA is just a client of 127.0.0.1:8000",
            "Versioning the inference RUNTIME separately from the app (LM Studio 'LM Runtimes')
is a clean model for shipping a new WASB/owned-model engine without a full reinstall",
            "Tauri (Jan) is the license-clean (MIT/Apache) shell choice for us",
            "Local OpenAI-compatible server pattern shows users accept 'app spins up a localhost
service' ? exactly D10"
          ],
          "cons": [
            "Their core trick ? bundle a single C++ engine (llama.cpp) with no Python ? does NOT
map to us: our engine IS a large Python/FastAPI + torch CV stack, not a single static binary; we
can't make the torch problem disappear the way they make Python disappear",
            "Most polished examples are LLM chat apps, not ffmpeg video pipelines ? the ffmpeg +
multi-GB-video I/O dimension is unaddressed by them",
            "Building a true native model-hub GUI is more work than reusing our existing React
SPA + a download endpoint",
            "Doesn't address our dual-env WASB issue at all"
          ],
          "maturity": "mature"
        }
      ],
      "load_bearing_claims": [
        {
          "claim": "Our signature packaging problem is real and in-code: the WASB shuttle
detector runs in a SEPARATE conda env via WSL ? `wasb_runner.py` literally builds commands
`conda activate {self.cfg.conda_env}` with `conda_env` defaulting to 'wasb' (lines 36,74,162) ?
while the main engine uses a different torch. An in-process escape hatch
(`native_wasb_runner.py`) exists but is not production-verified.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/wasb_runner.py:36,74,162; base.py:46-54;
native_wasb_runner.py"
        },
        {
          "claim": "`uv` can express the exact torch CUDA/CPU `--index-url` constraint that our
pyproject.toml header says PEP 621 cannot: define a PyTorch index in pyproject and route
torch/torchvision to it via `[tool.uv.sources]` with `explicit=true`; backends follow
https://download.pytorch.org/whl/{cpu,cu124,...}.",
          "confidence": "high",
          "source": "https://docs.astral.sh/uv/guides/integration/pytorch/ ;
pyproject.toml:8-11,58-65"
        },
        {
          "claim": "Pinokio (MIT-licensed) one-click 'sets up Python, Conda, virtual
environments, GPU libraries and all required dependencies' in app-isolated envs and 'installs
the latest version of FFMPEG', then launches a local web UI ? i.e. it natively handles multiple
isolated torch/CUDA envs + ffmpeg, the shape of our problem.",
          "confidence": "high",
          "source": "https://github.com/pinokiocomputer/pinokio ;
https://github.com/pinokiocomputer/pinokio/blob/main/LICENSE"
        },
        {
```

```
          "claim": "ComfyUI's official desktop app is GPL-3.0, ships via NSIS (Win)/DMG (mac)
Electron, bundles `uv` to install Python deps on first startup (not an embedded Python), uses
pinned `.compiled` requirements, lets the user pick a models folder, and auto-updates the app +
ComfyUI-Manager + uv on stable releases.",
          "confidence": "high",
          "source": "https://github.com/Comfy-Org/desktop"
        },
        {
          "claim": "Ollama ships a ~5MB OllamaSetup.exe that installs without admin rights,
auto-starts a background service, auto-detects NVIDIA CUDA, pulls model weights on demand to
%USERPROFILE%/.ollama, runs offline after, and updates by running the new installer over the old
(models preserved).",
          "confidence": "high",
          "source": "https://docs.ollama.com/windows ;
https://github.com/ollama/ollama/releases"
        },
        {
          "claim": "Fooocus and ComfyUI-portable ship a ~1GB+ ZIP with embedded Python and pre-
installed torch; extract + double-click run.bat downloads weights on first run and opens a
localhost web UI. Fooocus is GPL-3.0.",
          "confidence": "high",
          "source": "https://github.com/lllyasviel/Fooocus ;
https://github.com/lllyasviel/Fooocus/blob/main/LICENSE"
        },
        {
          "claim": "AUTOMATIC1111's 'one-click' is NOT batteries-included for non-tech users: it
requires pre-installing Python 3.10.6 + git, `git clone`, then webui-user.bat builds a venv and
pip-installs torch (~5-10 min first run); third-party launchers exist precisely because the
official one omits Python/git.",
          "confidence": "high",
          "source": "https://github.com/AUTOMATIC1111/stable-diffusion-webui ;
https://github.com/AUTOMATIC1111/stable-diffusion-webui/discussions/8020"
        },
        {
          "claim": "Frigate bundles its own ffmpeg inside the Docker image (at /usr/lib/ffmpeg,
v7), ships per-accelerator image variants (default OpenVINO/CPU, `-tensorrt` for NVIDIA), no
longer pre-bundles detection models (drop ONNX into /config/model_cache, converted on first
boot), and distributes via Docker tags (`stable`) on GHCR + a Home Assistant add-on; updates via
image re-pull or Watchtower.",
          "confidence": "high",
          "source": "https://docs.frigate.video/frigate/installation/ ;
https://docs.frigate.video/configuration/object_detectors/ ;
https://docs.frigate.video/frigate/updating/"
        },
        {
          "claim": "Frigate does not officially support Windows; it runs only via WSL2 + Docker
Desktop, and 'getting the GPU and/or Coral devices properly passed to Frigate may be difficult
or impossible' on Windows ? i.e. Docker+GPU on Windows is exactly what a non-tech user cannot
debug.",
          "confidence": "high",
          "source": "https://docs.frigate.video/frigate/installation/ ;
https://github.com/blakeblackshear/frigate/discussions/4375"
        },
        {
          "claim": "LM Studio (0.4+) decouples a headless daemon (`llmster`) from the GUI behind
an OpenAI-compatible HTTP server and versions its inference engine separately as swappable 'LM
Runtimes'; Jan is a Tauri app bundling llama.cpp serving 127.0.0.1:1337; GPT4All ships an OS
installer + in-app HuggingFace model hub + local server on :4891 ? all bundle the engine and
download weights on demand via a GUI.",
          "confidence": "high",
          "source": "https://lmstudio.ai/docs/app ; https://lmstudio.ai/blog/0.4.0 ;
https://github.com/janhq/jan ; https://docs.gpt4all.io/gpt4all_desktop/quickstart.html"
        },
        {
          "claim": "Owner decision D10 sets the delivery shape: a self-hosted webserver + UI the
```

```
user spins up (NOT a desktop app), pointing at local storage + GPU/CPU OR cloud locations; this
supersedes the prior DESKTOP_APP_PLAN.md.",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:39
(D10)"
        },
        {
          "claim": "There is currently NO Dockerfile in the engine, and the WASB torch-1.11
conda env makes a containerized worker 'non-trivial' ? a noted gap blocking the Cloud Run+GPU
serving path.",
          "confidence": "high",
          "source":
"C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:49,96,100"
        }
      ],
      "recommendation": "Adopt a layered, profile-matched strategy, not one installer. (1)
FIRST, finish `native_wasb_runner.py` (the in-process torch path that retires the WSL/conda
'wasb' sidecar seen at wasb_runner.py:74,162). Almost every clean packaging story below
collapses to ONE torch environment ? so removing the second incompatible env is the highest-
leverage prerequisite, and it's already designed-for (base.py:46-54). (2) For the NON-TECH
WINDOWS user, copy the Ollama+ComfyUI-desktop pattern: a small self-branded installer (license-
clean Tauri shell, or even a plain NSIS/portable launcher ? NOT the GPL ComfyUI/Fooocus code)
that bundles `uv`, on first run uses `uv` with `[[tool.uv.index]]`/`[tool.uv.sources]` to
install torch/torchvision from the correct CUDA-or-CPU `--index-url` (the precise thing
pyproject.toml says PEP 621 can't do), bundles an LGPL ffmpeg build (openh264/NVENC, no GPL x264
per D4), downloads the GCS WASB weights on first run with a progress bar (Ollama/GPT4All model-
hub UX), then launches `python -m backend.main --server` and opens 127.0.0.1:8000 to the
existing khelsutra React SPA. Update like Ollama: run a new installer over the old, weights
preserved. (3) For the SERVER / cloud-serving profile, build the Frigate-style Docker image
(bundled ffmpeg + per-accelerator variant) ? which ALSO closes the 'no Dockerfile' gap blocking
Cloud Run+GPU. (4) As a near-zero-effort additional channel, publish a Pinokio install script
(MIT, app-isolated envs + ffmpeg + GPU libs out of the box) so technically-curious users get
one-click install without us owning a launcher. Do NOT make Docker-on-Windows the non-tech path:
Frigate itself shows Docker+GPU passthrough on Windows is exactly what that user cannot debug.",
      "fit_for_this_app": "Our app is unusual vs the LLM crowd (Ollama/LM Studio/Jan/GPT4All):
they bundle ONE static C++ engine (llama.cpp) so 'Python' disappears, whereas we ship a
Python+FastAPI+torch CV pipeline + ffmpeg + cloud-fallback. So the LLM apps teach UX (GUI
weight-hub, headless-daemon-behind-OpenAI-server, runtime versioning) but not the core mechanic.
The directly-applicable mechanics come from the SD/diffusion + Frigate camp: (a) `uv` is the one
tool that resolves the torch `--index-url` blocker our own pyproject.toml:8-11 documents as un-
expressible ? adopt it. (b) Pinokio (MIT) and ComfyUI-portable already run multiple isolated
torch/CUDA envs side by side, which is the literal shape of our wasb_runner.py conda('wasb')-vs-
engine split ? but the cleaner long-term move is to finish native_wasb_runner.py so there's ONE
torch. (c) Frigate is our closest functional analog (ffmpeg+ML+web UI) and gives the bundled-
ffmpeg-inside-the-artifact + per-accelerator-variant + Docker-tag-update patterns, AND building
that image closes our verified 'no Dockerfile' gap (LOCAL_APPLIANCE_ARCHITECTURE.md:49,96) for
Cloud Run+GPU. The delivery SHAPE must honor owner D10 (self-hosted webserver+UI the user spins
up, not a desktop app, compute/storage as settings) and the dual local/cloud requirement ? which
is why the recommendation splits into a non-tech Windows installer profile and a Docker server
profile rather than one artifact. Licensing guardrail forces shell choice: ComfyUI-
desktop/Fooocus are GPL-3.0 and Pinokio is MIT ? copy GPL patterns but only vendor
MIT/Apache/BSD (Tauri shell, uv, LGPL ffmpeg, MIT WASB weights). Engine reuse is total: a thin
shell just spawns the existing `python -m backend.main --server` / `python -m backend.worker
{infer|train}` and the existing presets.py truly-local/cloud-serving profiles select compute."
    },
    "verdict": {
      "claims_checked": [
        {
          "claim": "Signature packaging problem is real and in-code: WASB runs in a SEPARATE
conda env via WSL ? wasb_runner.py builds 'conda activate {self.cfg.conda_env}' with conda_env
defaulting to 'wasb' (lines 36,74,162) ? while the main engine uses a different torch. An in-
process escape hatch (native_wasb_runner.py) exists but is not production-verified.",
          "verdict": "confirmed",
          "reasoning": "Verified against source. wasb_runner.py:36 sets conda_env='wasb'
default; line 74 (healthcheck) and line 162 (run_predict) both build commands beginning 'conda
```

activate {self.cfg.conda_env}'. The base.py ABC documents the WSL/conda de-risk goal. native_wasb_runner.py exists. 'Not production-verified' is DEFENSIBLE but needs nuance: the native runner's own docstring claims it was '? Validated 2026-06-20 on an RTX 2070 (via WSL): native streaming vs cached WSL CSV matched 1194/1195 frames' ? i.e. a parity check HAS been run, but khelsutra LOCAL_APPLIANCE_ARCHITECTURE.md:45,132 still treats it as 'not production-verified' pending an owner byte-parity gate on a real GPU box and notes trajectory_hybrid._build_native_runner still RAISES (only fusion_hybrid wires it). So 'not production-verified' holds at the production-wiring level, but the flat claim hides that a real-GPU parity run already happened.",
          "corrected_claim": "Confirmed in-code (wasb_runner.py:36/74/162). Caveat: native_wasb_runner.py is not merely an untested 'escape hatch' ? its docstring records a 2026-06-20 RTX 2070 parity run (1194/1195 frames); it remains 'not production-verified' only because the owner byte-parity gate isn't signed off and trajectory_hybrid._build_native_runner still raises (only fusion_hybrid builds it)."
        },
        {
          "claim": "uv can express the torch CUDA/CPU --index-url constraint via a PyTorch index in pyproject + [tool.uv.sources] with explicit=true; backends follow download.pytorch.org/whl/{cpu,cu124,...}.",
          "verdict": "confirmed",
          "reasoning": "Verified against the official uv PyTorch integration guide. It uses [[tool.uv.index]] with name/url/explicit=true (recommended so the index is only used for torch/torchvision), and [tool.uv.sources] routing torch/torchvision to a named index, with backend URLs of the form download.pytorch.org/whl/{cpu,cu118,cu126,cu128,cu130,rocm,...}, optionally per-platform via markers. pyproject.toml:8-11,58-65 does indeed state PEP 621 cannot express --index-url. The only minor staleness: the guide's current CUDA examples lead with cu126/cu128/cu130 rather than cu124, but cu124 is a valid backend URL and the mechanism is exactly as described."
        },
        {
          "claim": "Pinokio (MIT) one-click sets up Python, Conda, virtual envs, GPU libraries and all dependencies in app-isolated envs, installs the latest FFMPEG, then launches a local web UI.",
          "verdict": "confirmed",
          "reasoning": "License is MIT (verified on the GitHub repo). The marketing substance ? 'sets up Python, Conda, virtual environments, GPU libraries and all required dependencies, then launches a local web interface in your browser' and 'installs the latest version of FFMPEG to support new features incl. torchcodec APIs' ? is confirmed verbatim, but it comes from Pinokio's site/program docs, NOT the cited GitHub README (the README itself only describes isolated venv/Conda/Pip/NPM management under ~/pinokio and does not contain that exact wording). Adversarial caveat: multiple open issues (#908/#909/#910 ffmpeg post-link script failures, #1052 'installs an outdated version of ffmpeg') show the FFmpeg-out-of-the-box claim is not always reliable in practice ? relevant to leaning on it as a 'near-zero-effort' channel.",
          "corrected_claim": "True in substance and license (MIT). Citation nuance: the quoted wording is from Pinokio's site/program docs, not the GitHub README cited. Reliability caveat: open issues report Pinokio's FFmpeg install failing or being outdated, so 'ffmpeg out of the box' is not bulletproof."
        },
        {
          "claim": "ComfyUI desktop is GPL-3.0, ships via NSIS (Win)/DMG (mac) Electron, bundles uv to install Python deps on first startup (not embedded Python), uses pinned .compiled requirements, lets the user pick a models folder, and auto-updates the app + ComfyUI-Manager + uv on stable releases.",
          "verdict": "confirmed",
          "reasoning": "Verified point-for-point against the Comfy-Org/desktop repo: license GPL-3.0; NSIS installer for Windows and DMG for macOS, Electron app; bundles uv and installs python deps with uv at startup (not embedded python); references assets/requirements/*.compiled pinned requirements; prompts the user to select a location for models/inputs/outputs/custom_nodes/workflows; automatically updates with stable releases of ComfyUI, ComfyUI-Manager (pip), and the uv executable. Every sub-claim checks out."
        },
        {
          "claim": "Ollama ships a ~5MB OllamaSetup.exe that installs without admin rights, auto-starts a background service, auto-detects NVIDIA CUDA, pulls weights on demand to %USERPROFILE%/.ollama, runs offline after, and updates by running the new installer over the old

(models preserved).",
          "verdict": "uncertain",
          "reasoning": "Mixed. CONFIRMED by docs: installs without Administrator (installs to home dir), runs a background service, stores models/config at %HOMEPATH%\\.ollama (overridable via OLLAMA_MODELS), and the installer 'helps you keep up to date.' REFUTED/STALE: the '~5MB' size is no longer reliable ? the GitHub releases page shows Windows binaries of 260MB?1.3GB, there is an open issue #8005 'Why is OllamaSetup.exe so large', and Softpedia (June 2026) lists 1.3GB; '~5MB no dependencies' reflects older versions before GPU runners were bundled. NOT EXPLICITLY DOCUMENTED on the Windows page: 'auto-detects NVIDIA CUDA' (docs only list a driver-version requirement), 'runs offline after' (not stated), and 'models preserved on update' (not stated, though plausible since models live in a separate user-dir). Given the size claim is likely stale and three sub-claims are unverified, default to uncertain.",
          "corrected_claim": "Ollama's Windows installer installs without admin, runs a background service, stores models under %HOMEPATH%\\.ollama, and updates via re-running the installer ? but it is NOT ~5MB in current releases (Windows assets are 260MB?1.3GB; the small-installer era predates bundled GPU runners), and 'auto-detects CUDA', 'runs offline after', and 'models preserved on update' are reasonable but not explicitly stated in the Windows docs."
        },
        {
          "claim": "Fooocus and ComfyUI-portable ship a ~1GB+ ZIP with embedded Python and pre-installed torch; extract + double-click run.bat downloads weights on first run and opens a localhost web UI. Fooocus is GPL-3.0.",
          "verdict": "confirmed",
          "reasoning": "Verified for Fooocus: GPL-3.0 license; Windows distribution is a 7z archive (~2.5GB for 2.5.0, comfortably '~1GB+'); includes embedded python (.\\python_embeded\\python.exe); run.bat launches and 'will automatically download models' on first run; UI is a Gradio localhost web UI. The '~1GB+ ZIP' is conservative (it is ~2.5GB 7z). ComfyUI-portable's embedded-python/portable ZIP pattern is well-established and consistent; the Fooocus half is fully verified, so the claim stands."
        },
        {
          "claim": "AUTOMATIC1111's 'one-click' is NOT batteries-included: it requires pre-installing Python 3.10.6 + git, git clone, then webui-user.bat builds a venv and pip-installs torch (~5-10 min first run); third-party launchers exist because the official one omits Python/git.",
          "verdict": "confirmed",
          "reasoning": "Verified against the repo's Automatic Installation on Windows: step 1 'Install Python 3.10.6 (Newer version of Python does not support torch), checking Add Python to PATH'; step 2 'Install git'; step 3 git clone the repo; step 4 run webui-user.bat (which creates the venv and pip-installs dependencies incl. torch). The prerequisites Python 3.10.6 + git are explicit and separate. The '~5-10 min first run' is an order-of-magnitude characterization (not a quoted number, but a fair description of the first-run pip/torch download). Third-party launchers existing 'because' the official omits Python/git is a reasonable causal framing consistent with the prerequisites."
        },
        {
          "claim": "Frigate bundles its own ffmpeg (at /usr/lib/ffmpeg, v7), ships per-accelerator variants (default OpenVINO/CPU, -tensorrt for NVIDIA), no longer pre-bundles detection models (drop ONNX into /config/model_cache, converted on first boot), distributes via Docker tags (stable) on GHCR + a HA add-on; updates via image re-pull or Watchtower.",
          "verdict": "confirmed",
          "reasoning": "Verified across Frigate docs/discussions: the container provides ffmpeg binaries under /usr/lib/ffmpeg (bundles 5.0 and 7.0; path /usr/lib/ffmpeg/7.0/bin/ffmpeg); per-accelerator image variants exist (stable for amd64+RPi, stable-tensorrt for NVIDIA, stable-rocm for AMD, jetson/rockchip community variants); ONNX models cached in /config/model_cache and converted/optimized on first boot (e.g. TensorRT engine build is a one-time per-model step); distributed via Docker tags on GHCR and as a Home Assistant add-on; updated by re-pulling the image (Watchtower is a standard pattern). Minor imprecision: 'default OpenVINO/CPU' loosely describes the default 'stable' tag (which ships CPU detectors including OpenVINO support); the variant naming is by accelerator, not literally 'OpenVINO' as the default tag name. Substance holds."
        },
        {
          "claim": "Frigate does not officially support Windows; runs only via WSL2 + Docker Desktop, and getting GPU/Coral devices passed to it 'may be difficult or impossible' on

```
Windows.",
          "verdict": "confirmed",
          "reasoning": "Verified against the Frigate installation docs: 'Windows is not
officially supported, but some users have had success getting it to run under WSL or Virtualbox'
and acquiring proper GPU and Coral device access 'may be difficult or impossible.' The claim
says 'WSL2 + Docker Desktop' where the docs say 'WSL or Virtualbox' ? slightly narrower than the
docs, but WSL2+Docker Desktop is the canonical path and the not-officially-supported +
GPU/Coral-passthrough-hard core is verbatim accurate."
        },
        {
          "claim": "LM Studio (0.4+) decouples a headless daemon (llmster) from the GUI behind
an OpenAI-compatible HTTP server and versions its inference engine separately as swappable 'LM
Runtimes'; Jan is a Tauri app bundling llama.cpp serving 127.0.0.1:1337; GPT4All ships an OS
installer + in-app HuggingFace model hub + local server on :4891 ? all bundle the engine and
download weights on demand via a GUI.",
          "verdict": "confirmed",
          "reasoning": "Verified in substance. LM Studio 0.4: llmster is the standalone headless
daemon (LM Studio's core packaged as a daemon), exposes an OpenAI-compatible REST API, and
supports swappable backends (llama.cpp, MLX) ? the 'LM Runtimes' framing is accurate. Jan:
Tauri-based desktop app, bundles llama.cpp, runs an OpenAI-compatible local server on
localhost:1337 ? all confirmed verbatim. GPT4All: ships OS installers (Win/Mac/Linux), has an
in-app model browser/download hub, and exposes a local API server. Minor unverified detail:
GPT4All's exact port :4891 was not stated on the quickstart page checked (it is the documented
default elsewhere, so plausible, not contradicted); LM Studio's OpenAI server default is :1234
(the claim doesn't pin LM Studio's port). These do not undercut the thrust: all three bundle the
engine and pull weights on demand via a GUI."
        },
        {
          "claim": "Owner decision D10 sets the delivery shape: a self-hosted webserver + UI the
user spins up (NOT a desktop app), pointing at local storage + GPU/CPU OR cloud locations; this
supersedes the prior DESKTOP_APP_PLAN.md.",
          "verdict": "confirmed",
          "reasoning": "Verified verbatim at khelsutra docs/LOCAL_APPLIANCE_ARCHITECTURE.md:39
(D10): 'Delivery = a self-hosted webserver + UI the user spins up (NOT a desktop app); the user
points at local storage + GPU/CPU OR cloud locations and the UI drives the tool' with
implication 'supersedes DESKTOP_APP_PLAN.md'. Source attribution (Owner 2026-06-20) and line
number both match. IMPORTANT downstream consequence for the recommendation: D10 supersedes
DESKTOP_APP_PLAN.md ? which is exactly the doc where the 'D4 / no GPL x264 / LGPL ffmpeg'
decision the recommendation cites actually lives (see overall_notes)."
        },
        {
          "claim": "There is currently NO Dockerfile in the engine, and the WASB torch-1.11
conda env makes a containerized worker 'non-trivial' ? a noted gap blocking the Cloud Run+GPU
serving path.",
          "verdict": "refuted",
          "reasoning": "STALE / now false. A Dockerfile DOES exist at deploy/cloudrun/Dockerfile
(engine repo), created 2026-06-21 16:11 in commit #287 ('feat(serving): M6 ? Cloud Run GPU
dispatch shim + worker image'). It is a Cloud Run GPU Job image that installs ffmpeg, builds the
WASB torch-1.11.0+cu113 conda env, pip-installs the app, and sets RALLY_DETECTOR_IMPL=native
with ENTRYPOINT python3 -m backend.worker. The khelsutra doc claiming 'no Dockerfile'
(LOCAL_APPLIANCE_ARCHITECTURE.md:49,96,100,140) was last committed 2026-06-21 15:05 (#55) and
dated 2026-06-20 ? i.e. it was true when written but the Dockerfile landed ~1 hour later. The
deeper engineering point the claim makes IS validated by that same Dockerfile: it confirms the
WASB env is non-trivial ? the image deliberately keeps TWO python stacks (main app + a separate
conda 'wasb' env with py3.8/torch 1.11.0+cu113) because of the framework conflict. So the 'two-
env pain is real' half is true; the 'no Dockerfile / gap still blocking' half is now factually
wrong.",
          "corrected_claim": "A Cloud Run GPU worker Dockerfile now EXISTS
(deploy/cloudrun/Dockerfile, shipped 2026-06-21 in #287/M6); it bundles ffmpeg and builds the
WASB torch-1.11.0+cu113 conda env. The khelsutra 'no Dockerfile' claim is stale (the doc
predates the Dockerfile by ~1 hour). The 'WASB env makes containerization non-trivial' point is
still valid ? the new image confirms it by keeping two separate python stacks ? but it is no
longer an unaddressed blocker."
        }
      ],
```

```
      "overall_notes": "Net: 9 of 12 claims CONFIRMED, 2 UNCERTAIN/nuanced (claim 1 native-WASB 'not production-verified' hides a recorded RTX-2070 parity run; claim 5 Ollama '~5MB' is stale ? current Windows installers are 260MB?1.3GB), and 1 REFUTED (claim 12: a Cloud Run Dockerfile now exists at deploy/cloudrun/Dockerfile, shipped ~1hr after the khelsutra doc was written).\n\nTWO LOAD-BEARING PROBLEMS WITH THE RECOMMENDATION ITSELF (beyond the individual claims):\n\n1. The recommendation's central premise is FALSE in light of the code it cites. Step (1) says 'finish native_wasb_runner.py' to 'retire the WSL/conda wasb sidecar' and collapse to 'ONE torch environment'. But native_wasb_runner.py does NOT remove the second torch env ? it removes only the WSL transport and 'conda activate'. It still shells out via subprocess to a SEPARATE python (self.cfg.python_bin / $WASB_PYTHON) which is the conda 'wasb' env's py3.8 + torch 1.11.0+cu113. The just-shipped deploy/cloudrun/Dockerfile makes this explicit: it deliberately builds TWO python stacks ('a framework conflict otherwise') ? main app torch + the wasb conda env ? and points $WASB_PYTHON at the latter. So 'almost every clean packaging story collapses to ONE torch environment' is the weakest link: the engine's own M6 image proves the two-env split persists even on the native path. The native runner is still worth finishing (it kills the WSL/Windows-only dependency), but NOT for the stated reason of unifying torch envs.\n\n2. The recommendation cites 'LGPL ffmpeg build (openh264/NVENC, no GPL x264 per D4)'. This mis-cites D4 and conflicts with both a superseded plan and current shipped code: (a) In LOCAL_APPLIANCE_ARCHITECTURE.md (the doc whose D10 the recommendation invokes), D4 is 'Embed RallyPicker verbatim' ? NOT an ffmpeg/licensing decision. (b) The actual 'LGPL ffmpeg / openh264 / no static x264' decision lives in DESKTOP_APP_PLAN.md:38 (its own D4) ? the very doc that D10 EXPLICITLY SUPERSEDES. (c) Current production code hardcodes the GPL encoder libx264 (compiler.py:253, rally_slicer.py:142, video.py:42) and the new Cloud Run README states the image 'keeps ffmpeg's default libx264'. So the LGPL/openh264 build is aspirational, not current state, and the 'D4' citation points at a superseded plan, not a live decision. The licensing intent is sound and worth carrying forward, but the recommendation should re-anchor it (and reconcile it with the live libx264 usage + the AV1/Opus royalty direction in COMMERCIALIZATION.md/DECISIONS.md) rather than cite a non-existent/superseded D4.\n\nSmaller notes: claim 3's Pinokio wording is from the Pinokio site, not the cited GitHub README, and Pinokio has open issues where its FFmpeg install fails/is outdated ? a caveat against leaning on it as a 'near-zero-effort' channel. Claim 8's 'default OpenVINO/CPU' is a loose label for Frigate's default 'stable' tag. None of these smaller points undercut the layered, profile-matched strategy's overall direction; the two load-bearing problems above (the one-torch-env premise and the D4 ffmpeg citation) are the ones that need correction before this recommendation is acted on.\n\nRelevant files: C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/wasb_runner.py (lines 36,74,162); .../native_wasb_runner.py (esp. docstring lines 26-34 parity claim, cfg python_bin/$WASB_PYTHON lines 67-97); .../base.py:46-54; .../pyproject.toml:8-11,58-65; .../deploy/cloudrun/Dockerfile (the now-existing Dockerfile + two-env split, lines 8-12,32-57); .../backend/pipeline/stitcher/compiler.py:253 (libx264); .../deploy/cloudrun/README.md:61; C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:39 (D10),49,96,100,140; .../docs/DESKTOP_APP_PLAN.md:38 (the actual ffmpeg D4)."
    }
  },
  {
    "topic": "cloud-one-click",
    "research": {
      "topic": "Running the SAME downloadable rally-indexer app at cloud services with minimal friction: one artifact serving both local and cloud by config (deployment.profile), behind edge auth, with per-video cost accounting and no data leakage",
      "options": [
        {
          "name": "Cloud Run GPU Jobs (already wired) - the primary cloud target",
          "how_it_works": "Control plane runs DEPLOYMENT_PROFILE=cloud-serving (compute_target=cloud_run). get_compute_backend() returns CloudRunCompute ONLY for cloud_run, else None=run inline (backend/compute/__init__.py:47-49). CloudRunCompute.run_infer dispatches a Cloud Run JOB execution of python -m backend.worker infer --video-uri gs:// --out-uri gs:// --done-uri gs:// on an L4, then bucket-polls a done-marker (cloud_run.py:103-132, D16). The dispatched container is the SAME deploy/cloudrun/Dockerfile image, forced INLINE (overrides compute_target=local_gpu + detector_impl=native, cloud_run.py:41-44) so it never re-dispatches. Create via deploy/cloudrun/README.md: gcloud run jobs create rally-worker --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi. Scale-to-zero (D7).",
          "pros": [
            "Already built + mock-tested (M6/#287); only the real L4 build/run (M7) is owner-gated on the $100 budget (D17)",
            "L4 ~$0.0001867/s (~$0.67/hr), per-SECOND billing, scales to zero -> matches the
```

```
per-video cost ledger (M8 backend/billing.py gpu_seconds x rate) exactly",
            "GPU drivers pre-installed reach GPU-ready ~5s; multi-minute video job amortizes the
1-3 min cold start",
            "Reuses the existing async job table + execution_policy.poll_until verbatim",
            "On GCP (sole stack per ADR-013); gs:// is already cloud-serving storage;
deploy/gcp/provision.sh provisions bucket+SA+budget alert",
            "libx264 encode + NVDEC decode-only (D16) protects the byte-determinism/parity
guarantee"
          ],
          "cons": [
            "Cold start 1-3 min on scale-to-zero - bad for interactive single-clip demo, fine
for batch",
            "CUDA+conda WASB image is large/slow; CI does NOT build it (proven with a fake
client)",
            "WASB-C3 unresolved: badminton weights provenance NOT cleared for commercial sharing
- staged at runtime, resolve before public non-owner serving",
            "A FAILED job writes no marker -> dispatcher times out to clean rc 4, not a precise
failure (follow-up)",
            "Real prices + worker usage-emission into the ledger are a follow-up; placeholder
pricebook = $0 today"
          ],
          "maturity": "mature"
        },
        {
          "name": "BYO-cloud GPU VM via deploy/gcp/create_gpu_vm.sh (+teardown.sh) - the warm /
owner path",
          "how_it_works": "Stand up a persistent GCE GPU VM with bash
deploy/gcp/create_gpu_vm.sh (L4 default, sweeps asia-south1->asia-southeast1 for capacity, bakes
Ops-Agent + rally-worker SA + cloud-platform scope, driver-ready DLVM image). Engine runs there
with the truly-local profile (compute_target=local_gpu, detector_impl=native) over bucket or
local disk. Provider-agnostic (deploy/digitalocean/gpu_setup.sh is the reused Linux-GPU recipe).
Teardown: RALLY_GCP_ZONE=<zone> bash deploy/gcp/teardown.sh delete (deletes, not stops).",
          "pros": [
            "Already scripted + idempotent; one command up, one down; Ops Agent (Observability)
baked in",
            "No cold start - warm GPU for back-to-back jobs; the working real-GPU verification
path for the owner",
            "Truly the SAME engine + profile as local: just 'truly-local on a rented box', zero
code branch",
            "Provider-agnostic recipe - no hard provider lock-in at the script level"
          ],
          "cons": [
            "Always-on billing (~$0.67/hr L4 + ~$34/mo/200GB pd-ssd if disk orphaned) - NOT
scale-to-zero; ledger does not auto-meter idle",
            "Manual lifecycle: forgetting teardown is the #1 cost leak",
            "GPUS_ALL_REGIONS quota gate (often 0; L4 often stocked-out -> zone sweep)",
            "No multi-tenant isolation; single-box",
            "Provisioning-credential leak risk if a cloud token is pasted in chat/committed
(authenticate on-box, least-privilege, rotate)"
          ],
          "maturity": "mature"
        },
        {
          "name": "Render / Railway one-click 'Deploy to Render' button - CONTROL PLANE / CPU-
only modes only",
          "how_it_works": "Add a render.yaml Blueprint + a 'Deploy to Render' button
(render.com/deploy) to the README; a user clicks, reviews, approves, and Render provisions on
THEIR account from the blueprint. Railway offers the same one-click-from-template / GitHub /
Dockerfile. For THIS app they host only the FastAPI control plane + CPU-only profiles
(segmenter=gemini cloud-brain, segmenter=motion, detector_impl=stub) and dispatch the actual GPU
detect+stitch to Cloud Run Jobs or a BYO GPU VM.",
          "pros": [
            "Lowest-friction one-click UX for a non-technical coach: a README button, no gcloud,
no Docker knowledge",
            "render.yaml/Railpack auto-detect a FastAPI ASGI app; set autoDeploy:false on the
```

```
button blueprint",
            "Good fit as the cheap always-light CONTROL PLANE fronting the gate and enqueuing
GPU jobs elsewhere",
            "Railway usage-based; Render managed infra with healthchecks/disks"
          ],
          "cons": [
            "NEITHER Render NOR Railway supports GPU -> cannot run real WASB; only
Gemini/motion/stub CPU modes",
            "Pulls compute OFF GCP - tension with ADR-013 unless used purely as a CPU front door
dispatching back to GCP",
            "Two-vendor sprawl + a second cost/observability surface outside the M8 ledger + GCP
budget alerts",
            "Downloadable-app story is self-host-the-webserver (D10), so a hosted-PaaS button is
an ADDITIONAL mode, not the core",
            "Per-video Gemini cost on a CPU box has no GPU ledger line - weaker bill visibility"
          ],
          "maturity": "mature"
        },
        {
          "name": "Fly.io one-click + GPU - REJECT for GPU; only viable as a CPU front door",
          "how_it_works": "Fly deploys from a Dockerfile via flyctl; historically offered GPU
Machines (A10/L40S/A100) billed per-second with scale-to-zero, could run the worker image at the
edge.",
          "pros": [
            "Full Docker control, global edge, per-second GPU billing resembling Cloud Run's",
            "scale-to-zero Machines"
          ],
          "cons": [
            "BLOCKER: Fly.io GPUs are DEPRECATED and unavailable after Aug 1 (2026 search) - do
not build the GPU path on it",
            "Off-GCP -> conflicts with ADR-013; fragments billing/observability",
            "Requires hand-written Dockerfile + flyctl (more friction than a README button)",
            "No advantage over Cloud Run Jobs for our GCP-co-located bucket + ledger"
          ],
          "maturity": "risky"
        },
        {
          "name": "Edge auth = Cloudflare Access (D9/M9) verifying the Cf-Access JWT - MANDATORY
before any public exposure",
          "how_it_works": "The engine has ZERO app-layer auth and CORS allow_origins=['*']
(backend/main.py:51); safety is structural ONLY because it binds 127.0.0.1:8000 (main.py:735)
and the edge gate is the sole authenticated ingress. Put cloudflared Tunnel (outbound-only, no
inbound ports) + Cloudflare Access (email-OTP allowlist, free <=50 users) in front. Backend
trusts Cf-Access-Authenticated-User-Email for scoping but MUST verify the Access JWT signature
(Cf-Access-Jwt-Assertion header / CF_Authorization cookie) with PyJWT (auth extra already in
pyproject) against the team JWKS at https://<team>.cloudflareaccess.com/cdn-cgi/access/certs,
checking aud + RS256, so a direct hit to the raw Cloud Run URL cannot spoof the identity header
(D9). On Cloud Run, lock ingress to internal-and-cloud-load-balancing.",
          "pros": [
            "Reuses the site's EXISTING Cloudflare - one identity system, $0 at alpha, no
inbound ports, home IP never exposed",
            "D9 already locks the requirement: verify the JWT signature AND restrict ingress -
defends the real header-spoof attack",
            "PyJWT FastAPI dependency is a well-trodden pattern (Cloudflare's own FastAPI
tutorial); auth extra already declared",
            "Per-tenant scoping rides on user_id columns that already exist (v4); M9 just wires
extraction"
          ],
          "cons": [
            "Header-trust without signature verification is exploitable if ingress is open ->
BOTH controls non-negotiable",
            "Access signing key rotates ~every 6 weeks -> JWKS must be re-fetched/cached, not
pinned",
            "M9 is owner/counsel-gated (DPA/consent) - code can land default-OFF but public
signup waits on counsel",
```

```
            "cloudflared must run as a service on the box (one-time owner setup)"
          ],
          "maturity": "mature"
        },
        {
          "name": "One-artifact, config-selected local<->cloud (the spine - the recommended
mechanism, not an alternative)",
          "how_it_works": "Ship ONE pip package (indexer console script) + ONE worker Docker
image. The pure 3-stage core (DETECT->WINDOW->STITCH) never branches on profile (D1). A single
switch deployment.profile in {truly-local | cloud-serving | cpu-mock} (backend/config/presets.py
PROFILE_PRESETS) applies a coherent preset bundle over EXISTING knobs (compute_target,
detector_impl, storage.backend, skip_ai_handoff). Precedence: defaults < preset < config.json <
env (DEPLOYMENT_PROFILE/K_SERVICE) < worker --set. get_compute_backend (__init__.py:47-49)
returns CloudRunCompute for cloud_run else None=inline; StorageBackend registry
(local/gcs/s3/gdrive) makes bytes location a config choice with local=identity. Textbook twelve-
factor build-once-deploy-many. Auto-detect SUGGESTS, user CONFIRMS - cloud spend never silent
(D15).",
          "pros": [
            "Already implemented + tested M1-M8: the seam is real, default-OFF, profile='' ==
today's behaviour byte-for-byte",
            "Same image runs inline on a GPU VM AND as the dispatched Cloud Run Job - proven by
the forced-inline override",
            "Profile is user-switchable anytime as a HEADLINE feature (D13): 'your machine or
our cloud - your call'",
            "Per-profile startup checks fail/warn fast (backend/config/startup.py M5) so a
misconfigured cloud profile errors loudly before any spend",
            "cpu-mock profile gives a $0 deterministic CI/regression path and a no-key local
demo"
          ],
          "cons": [
            "Detection byte-identical only on the SAME GPU; parity guaranteed at WINDOW+STITCH,
not detector-CSV bytes (D5) - advertise 'equivalent', not 'bit-identical'",
            "torch/torchvision are NOT pip-expressible (need --index-url) and WASB is a SEPARATE
conda/torch-1.11 stack -> the downloadable app still needs a bootstrap (image or guided
installer) for the GPU path",
            "The web SPA lives in khelsutra/web - bundling the UI into the one artifact is a
cross-repo packaging task still to do",
            "gdrive-api OAuth backend (D14 Northstar) is net-new (M12)"
          ],
          "maturity": "emerging"
        }
      ],
      "load_bearing_claims": [
        {
          "claim": "The compute backend is config-selected by ONE switch: get_compute_backend
returns CloudRunCompute only for compute_target=='cloud_run', else None which runs the pipeline
inline - the literal local<->cloud seam",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/compute/__init__.py:42-49"
        },
        {
          "claim": "The dispatched Cloud Run Job runs the SAME worker image forced INLINE
(overrides compute_target=local_gpu + detector_impl=native) so the one artifact never re-
dispatches - proving one image serves both roles",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/compute/cloud_run.py:39-44,103-132"
        },
        {
          "claim": "deployment.profile applies a coherent preset bundle (truly-local / cloud-
serving / cpu-mock) over existing knobs; cloud-serving=cloud_run+native+gcs, truly-
local=local_gpu+local; auto-detect SUGGESTS (K_SERVICE->cloud-serving, CI->cpu-mock,
cuda->truly-local) but never auto-applies (D15)",
          "confidence": "high",
```

```
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/config/presets.py (PROFILE_PRESETS, suggest_profile)"
        },
        {
          "claim": "The engine has zero app-layer auth and CORS allow_origins=['*'] and binds
host=127.0.0.1 - so the EDGE gate (Cloudflare Access), not the app, is the tenancy boundary",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py:51,735"
        },
        {
          "claim": "D9 mandates Cloudflare Access AND verifying the Cf-Access JWT signature AND
locking Cloud Run ingress to the CF proxy, specifically so a direct hit to the Cloud Run URL
cannot spoof the identity header",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/docs/COMPUTE_DECOUPLED_SERVING/README.md (D9, line 36)"
        },
        {
          "claim": "Cloudflare Access JWT is validated in Python with PyJWT against the team
JWKS at https://<team>.cloudflareaccess.com/cdn-cgi/access/certs checking aud + RS256; the
signing key rotates ~every 6 weeks so keys must be refreshed",
          "confidence": "high",
          "source": "https://developers.cloudflare.com/cloudflare-one/tutorials/fastapi/ +
https://developers.cloudflare.com/access/setting-up-access/validate-jwt-tokens"
        },
        {
          "claim": "Cloud Run NVIDIA L4 is ~$0.0001867/sec (~$0.67/hr), billed per-second,
scales to zero with no idle cost, needs >=4 CPU + >=16Gi, and reaches GPU-ready in ~5s with
drivers pre-installed",
          "confidence": "high",
          "source": "https://cloud.google.com/run/pricing +
https://docs.cloud.google.com/run/docs/configuring/jobs/gpu"
        },
        {
          "claim": "Render does NOT support GPU workloads and neither does Railway; among
Render/Railway/Fly only Fly offered GPUs - but Fly.io GPUs are DEPRECATED and unavailable after
Aug 1",
          "confidence": "high",
          "source": "https://render.com/articles/fastapi-deployment-options +
https://techsy.io/en/blog/railway-vs-render-vs-fly-io + https://fly.io/docs/gpus/"
        },
        {
          "claim": "'Deploy to Render' is a README button backed by a render.yaml Blueprint that
provisions on the USER's account on click; advise autoDeploy:false on the button blueprint",
          "confidence": "high",
          "source": "https://render.com/docs/deploy-to-render +
https://render.com/docs/infrastructure-as-code"
        },
        {
          "claim": "create_gpu_vm.sh is the single scripted BYO GPU path (L4 default, zone-
sweeps asia-south1->asia-southeast1, bakes Ops-Agent + rally-worker SA), and teardown.sh delete
is the cost-stop (a stopped VM still bills disk ~$34/mo)",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/deploy/gcp/create_gpu_vm.sh + deploy/gcp/teardown.sh"
        },
        {
          "claim": "Per-job cost ledger (M8) computes cost_usd = sum(usage
gpu_seconds,wall_seconds,egress_gb x rate) + gemini_usd from a versioned pricebook, USD internal
/ INR display - but ships default-OFF ($0 placeholder pricebook) and worker usage-emission is a
follow-up",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/billing.py +
COMPUTE_DECOUPLED_SERVING/README.md (M8, line 123)"
        },
```

```
        {
          "claim": "WASB pretrained badminton weights have uncleared commercial-sharing
provenance (WASB-C3): NOT baked into the image, staged at runtime via $WASB_WEIGHTS, resolution
required before any public non-owner serving",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/deploy/cloudrun/Dockerfile:14-16,53-56"
        },
        {
          "claim": "Cloud-serving free-tier data-for-reels trade (D10): train ONLY on attested
ADULT footage + DERIVED data with raw auto-delete; minors->reel-yes-training-NO; paid tier=no-
train; counsel-gated to M9 - the data-leak/consent guardrail",
          "confidence": "high",
          "source": "C:/Users/avidu/Projects/badminton-highlight-
indexer/docs/COMPUTE_DECOUPLED_SERVING/README.md (D10, line 37)"
        },
        {
          "claim": "India's DPDP Act (enforcement phase 2026) makes consent the primary lawful
basis, requires deletion once business purpose ends, treats under-18 as children needing
verifiable parental consent, and permits cross-border transfer except to restricted countries",
          "confidence": "medium",
          "source": "https://www.cookieyes.com/blog/india-digital-personal-data-protection-act-
dpdpa/ + https://ksandk.com/data-protection-and-data-privacy/data-retention-and-deletion-under-
indias-dpdp-rules/"
        },
        {
          "claim": "Twelve-factor build-once-deploy-many config-in-env is the standard pattern
for one image serving local and cloud - swap a backing service via config only, no code change",
          "confidence": "high",
          "source": "https://aws.amazon.com/blogs/containers/developing-twelve-factor-apps-
using-amazon-ecs-and-aws-fargate/"
        }
      ],
      "recommendation": "Make ONE artifact, TWO official run modes, and resist hosted-PaaS GPU
buttons. (1) THE SPINE - already built (M1-M8, default-OFF): ship one pip package (indexer
console script) + one worker Docker image (deploy/cloudrun/Dockerfile), and let
deployment.profile pick everything. backend/compute/__init__.py:47-49 already makes
local<->cloud a single config switch (cloud_run->CloudRunCompute, else inline); the dispatched
Cloud Run Job runs the SAME image forced inline (cloud_run.py:41-44). Textbook twelve-factor;
nothing new is needed to claim 'local AND cloud from one artifact' - only finishing M7 (real L4
run) and wiring worker usage-emission into the M8 ledger. (2) DOWNLOADABLE-APP 'push my workload
to cloud' MODE: in the self-hosted UI Settings, surface profile as a toggle that calls
suggest_profile() to PROPOSE a default but requires explicit confirm (D15 - cloud spend never
silent). 'Push to cloud' = flip to cloud-serving (compute_target=cloud_run + storage=gcs); the
local control plane then dispatches GPU detect+stitch to a Cloud Run Job and bucket-polls. Show
the live per-video cost estimate (M8, INR) BEFORE confirming, and a hard per-video cap (open
owner question in LOCAL_APPLIANCE_ARCHITECTURE section 10). (3) CLOUD TARGET = Cloud Run GPU
Jobs, NOT Render/Railway/Fly for GPU: Render and Railway have no GPU; Fly's GPUs are deprecated
after Aug 1. Cloud Run L4 (~$0.67/hr, per-second, scale-to-zero, ~5s GPU-ready) is the only
option that ALSO matches per-video billing and ADR-013 (GCP-only). Keep a 'Deploy to Render'
button as a SECONDARY CPU-only control-plane distribution (Gemini/motion/stub modes that
dispatch GPU work back to GCP) - lowest-friction one-click for a non-technical coach, but never
the GPU path. (4) BYO-cloud: keep create_gpu_vm.sh/teardown.sh as the warm no-cold-start path
for power users/owner; surface the always-DELETE rule in a UI teardown affordance. (5) NON-
NEGOTIABLE before any public exposure: Cloudflare Access (cloudflared tunnel, no inbound ports)
+ PyJWT verification of the Cf-Access JWT signature (aud + JWKS refreshed for the ~6-week
rotation) + Cloud Run ingress=internal-and-cloud-load-balancing - both header-trust AND
signature-verify, because header-only trust is spoofable (D9). (6) NO DATA LEAK: truly-local
needs no DPA; cloud-serving must honor D10 (adults-only training, derived-data, raw auto-delete,
paid=no-train) and a retention sweep (originals are never deleted today) - all counsel-gated to
M9 under DPDP/GDPR. Sequence: finish M7 (real L4 + DEPLOY_REPORT) -> land M9 edge-auth (default-
OFF) -> wire ledger usage-emission + per-video cap -> add the UI profile toggle with cost-
confirm -> optional Render control-plane button last.",
      "fit_for_this_app": "Maps onto YOUR engine precisely. (a) torch dual-env:
deploy/cloudrun/Dockerfile ALREADY encodes the two-stack problem - a main app env (torch
```

2.6+cu124-class) plus a separate conda 'wasb' env (py3.8 torch 1.11+cu113) that the native runner shells out to via $WASB_PYTHON - so the cloud path inherits the exact split the local GPU box uses (mirrors deploy/digitalocean/gpu_setup.sh); for the DOWNLOADABLE app the same split is why GPU mode needs a bundled image or guided installer rather than pure pip (torch/torchvision deliberately NOT pip deps - need --index-url). (b) ffmpeg: a system dep, installed in the image (ffmpeg + DejaVu fonts for the watermark) and required by the local console; stitch runs co-located with detect on Cloud Run (D4/D16) so its CPU/wall/egress meter into one job. (c) GCS weights: wasb_badminton_best.pth.tar lives in the bucket and is staged at runtime via $WASB_WEIGHTS (WASB-C3 - uncleared commercial provenance, so NEVER baked in and resolved before public serving); the StorageBackend gcs registry + deploy/gcp/provision.sh already provision the bucket+SA. (d) FastAPI+SPA: the engine FastAPI loop is reused unchanged behind the gate; CORS '*' + 127.0.0.1 bind (main.py:51,735) means the appliance is a reverse-proxied single origin (khelsutra LOCAL_APPLIANCE_ARCHITECTURE D1/D2) and the SPA (khelsutra/web, en+kn+hi) is bundled cross-repo - the 'push to cloud' toggle is one Settings screen rendered from /api/capabilities. (e) non-tech India coach on Windows: today's verified real-GPU path is owner Windows+WSL2 (detector_impl=auto); for a coach, 'push my workload to cloud' lets them avoid GPU setup entirely - confirm-first, INR cost shown, Cloudflare Access email-OTP login (free <=50 users), no inbound ports on their network. Every new packaging dependency stays MIT/Apache/BSD; Gemini stays inference-only never a training source; DPDP under-18 = reel-yes/training-no (D10)."
    },
    "verdict": {
      "claims_checked": [
        {
          "claim": "The compute backend is config-selected by ONE switch: get_compute_backend returns CloudRunCompute only for compute_target=='cloud_run', else None which runs the pipeline inline - the literal local<->cloud seam",
          "verdict": "confirmed",
          "reasoning": "Verified directly in backend/compute/__init__.py:47-49: `target = ...compute_target; if target != 'cloud_run': return None` and only the cloud_run branch lazily constructs CloudRunCompute (lines 52-61). The module docstring states None means 'run the worker's existing in-process path (today's behaviour, byte identical).' worker/__main__.py cmd_infer (lines 166-168) calls get_compute_backend(config) and dispatches remotely only when it returns non-None, else falls through to the inline pipeline. Cited line range 42-49 is slightly off (the actual switch is lines 47-49; 38-49 is the function incl. docstring) but the substance is exact."
        },
        {
          "claim": "The dispatched Cloud Run Job runs the SAME worker image forced INLINE (overrides compute_target=local_gpu + detector_impl=native) so the one artifact never re-dispatches - proving one image serves both roles",
          "verdict": "confirmed",
          "reasoning": "backend/compute/cloud_run.py:39-44 defines _DEFAULT_CONTAINER_OVERRIDES = ('deployment.compute_target=local_gpu', 'indexing.detector_impl=native') and run_infer (lines 103-132) appends them to the dispatched argv via --set. The dispatched container runs `python -m backend.worker infer ... --set deployment.compute_target=local_gpu ...`; in cmd_infer get_compute_backend then returns None (target != cloud_run) so it runs inline and cannot recursively re-dispatch. The single deploy/cloudrun/Dockerfile ENTRYPOINT is `python3 -m backend.worker`, the same image both dispatcher-side container and the run unit. Mechanism fully matches."
        },
        {
          "claim": "deployment.profile applies a coherent preset bundle (truly-local / cloud-serving / cpu-mock) over existing knobs; cloud-serving=cloud_run+native+gcs, truly-local=local_gpu+local; auto-detect SUGGESTS (K_SERVICE->cloud-serving, CI->cpu-mock, cuda->truly-local) but never auto-applies (D15)",
          "verdict": "confirmed",
          "reasoning": "backend/config/presets.py PROFILE_PRESETS: truly-local -> {compute_target: local_gpu, storage.backend: local, skip_ai_handoff: True}; cloud-serving -> {compute_target: cloud_run, detector_impl: native, storage.backend: gcs}; cpu-mock -> {compute_target: cpu_mock, detector_impl: stub, storage.backend: local}. suggest_profile() returns 'cloud-serving' on K_SERVICE, 'cpu-mock' on CI truthy, 'truly-local' on probe_cuda+CUDA present, else ''. Module + function docstrings state it only SUGGESTS and never auto-applies (D15). All sub-claims match exactly."
        },
        {

```
          "claim": "The engine has zero app-layer auth and CORS allow_origins=['*'] and binds
host=127.0.0.1 - so the EDGE gate (Cloudflare Access), not the app, is the tenancy boundary",
          "verdict": "uncertain",
          "reasoning": "Partly confirmed, partly stale. CORS allow_origins=['*'] (main.py:51,
with allow_credentials=False) and uvicorn host='127.0.0.1' (main.py:735) are exact. BUT 'zero
app-layer auth' is only true in the DEFAULT-OFF posture: the M9 work (PR #290, present on this
branch) wires backend/api/auth.py into main.py:294 (edge_auth.resolve_tenant on /api/process).
When security.edge_auth_enabled is True the app ITSELF verifies the Cf-Access JWT signature in-
Python (RS256 + JWKS) and 401s on a missing/spoofed header. So once enabled the tenancy boundary
is enforced BOTH at the edge AND in-app (deliberately, per D9 anti-spoof) ? not 'edge, not the
app.' The claim's framing holds for local/default but understates the enabled state."
        },
        {
          "claim": "D9 mandates Cloudflare Access AND verifying the Cf-Access JWT signature AND
locking Cloud Run ingress to the CF proxy, specifically so a direct hit to the Cloud Run URL
cannot spoof the identity header",
          "verdict": "confirmed",
          "reasoning": "docs/COMPUTE_DECOUPLED_SERVING/README.md D9 (line 36) reads verbatim:
'Edge auth = Cloudflare Access ... Lock the Cloud Run ingress to the CF proxy + verify the Cf-
Access JWT signature (so a direct hit to the Cloud Run URL can't spoof the identity header).'
All three mandates and the anti-spoof rationale match exactly."
        },
        {
          "claim": "Cloudflare Access JWT is validated in Python with PyJWT against the team
JWKS at https://<team>.cloudflareaccess.com/cdn-cgi/access/certs checking aud + RS256; the
signing key rotates ~every 6 weeks so keys must be refreshed",
          "verdict": "confirmed",
          "reasoning": "Cloudflare's official 'Validate JWTs' docs confirm the JWKS endpoint
{team}/cdn-cgi/access/certs, RS256, aud/iss checks, and explicitly: 'By default, Access rotates
the signing key every 6 weeks. Previous keys remain valid for 7 days after rotation.' The
FastAPI tutorial confirms PyJWT usage. The repo's backend/api/auth.py implements exactly this
(PyJWT lazy import, PyJWKSet from the team JWKS, algorithms=['RS256'], audience=aud, issuer
check, require exp/iss/aud). It caches JWKS with a 600s TTL, which comfortably handles the
~6-week rotation. Confirmed."
        },
        {
          "claim": "Cloud Run NVIDIA L4 is ~$0.0001867/sec (~$0.67/hr), billed per-second,
scales to zero with no idle cost, needs >=4 CPU + >=16Gi, and reaches GPU-ready in ~5s with
drivers pre-installed",
          "verdict": "confirmed",
          "reasoning": "Google Cloud Run pricing/docs confirm: L4 ~$0.0001867/sec (Tier-1, no
zonal redundancy) ? $0.67/hr, billed per-second; scales to zero; L4 requires >=4 CPU and
>=16GiB; and per docs.cloud.google.com/run/docs/configuring/jobs/gpu (and the services GPU doc)
'instances with an attached L4 GPU with drivers pre-installed start in approximately 5 seconds.'
One nuance: total cold-start for a real workload includes model/container load (~tens of
seconds), which the project's design already accepts as a 1-3 min async cold-start (D7) ? but
the literal '~5s GPU-ready, drivers pre-installed' is accurate. The cited jobs-GPU URL is
valid."
        },
        {
          "claim": "Render does NOT support GPU workloads and neither does Railway; among
Render/Railway/Fly only Fly offered GPUs - but Fly.io GPUs are DEPRECATED and unavailable after
Aug 1",
          "verdict": "confirmed",
          "reasoning": "Render.com (the PaaS) has GPU only as an open Feature Request ? no GPU
instances offered. Railway has no GPU support as of 2026 (CEO 'cannot disclose explicit GPU
plans'). Fly.io GPUs ARE deprecated; the Fly community post states they 'will be deprecated as
of July 31, 2026' (unavailable after Aug 1). The claim says 'after Aug 1' without a year; given
the current date is June 2026, Aug 1 reads as 2026, which is correct. (Caveat: if read as Aug 1
2025 it would be wrong ? but in context it is right.) Substance confirmed."
        },
        {
          "claim": "'Deploy to Render' is a README button backed by a render.yaml Blueprint that
provisions on the USER's account on click; advise autoDeploy:false on the button blueprint",
          "verdict": "confirmed",
```

```
          "reasoning": "Render's official deploy-to-render docs confirm the button is backed by
a render.yaml Blueprint, that clicking it lets users 'review the services to be deployed on
their Render account and click approve,' and that 'For a service that is meant to be deployed
via a Deploy to Render button, it is strongly advised to set autoDeploy: false.' All three sub-
claims match the source verbatim."
        },
        {
          "claim": "create_gpu_vm.sh is the single scripted BYO GPU path (L4 default, zone-
sweeps asia-south1->asia-southeast1, bakes Ops-Agent + rally-worker SA), and teardown.sh delete
is the cost-stop (a stopped VM still bills disk ~$34/mo)",
          "verdict": "confirmed",
          "reasoning": "deploy/gcp/create_gpu_vm.sh: GPU='l4' default (line 23); ZONES sweep
asia-south1-{a,b,c} then asia-southeast1-{a,b,c} (line 28); attaches SA rally-worker@... with
cloud-platform scope (lines 22,45); bakes ops-agent-startup.sh via --metadata-from-file=startup-
script (line 46); header calls it 'THE one way to spin up a rally GPU box.'
deploy/gcp/teardown.sh defaults ACTION to 'delete' (line 23) and its header states a stopped VM
still bills its disk ('pd-ssd ~$0.17/GB/mo -> a 200 GB boot disk ~$34/mo'). Minor inconsistency:
create_gpu_vm.sh provisions a 120GB pd-balanced disk, not 200GB pd-ssd, so the actual disk bill
for this VM differs from the $34/mo example ? but the $34/mo figure is the script's own
documented illustration and the always-DELETE rule is exactly as claimed."
        },
        {
          "claim": "Per-job cost ledger (M8) computes cost_usd = sum(usage
gpu_seconds,wall_seconds,egress_gb x rate) + gemini_usd from a versioned pricebook, USD internal
/ INR display - but ships default-OFF ($0 placeholder pricebook) and worker usage-emission is a
follow-up",
          "verdict": "confirmed",
          "reasoning": "backend/billing.py: compute_cost_usd sums
GPU_SECONDS/WALL_SECONDS/EGRESS_GB x rate plus GEMINI_USD verbatim; to_inr() does the INR
display conversion; module docstring states cost stored in USD, displayed in INR, and 'Default-
OFF: with the seeded placeholder-v0 pricebook (every rate 0.0) every cost is 0.0.' database.py
_migrate_to_v6 (v6) adds the cost columns + a versioned `pricebook` table and seeds
placeholder-v0 at 0.0. README M8 row (line 123, PR #288) explicitly marks
'(ledger+pricebook+/cost default-OFF); real prices + usage-emission owner/follow-up.' All sub-
claims confirmed."
        },
        {
          "claim": "WASB pretrained badminton weights have uncleared commercial-sharing
provenance (WASB-C3): NOT baked into the image, staged at runtime via $WASB_WEIGHTS, resolution
required before any public non-owner serving",
          "verdict": "confirmed",
          "reasoning": "deploy/cloudrun/Dockerfile:14-16 carries an explicit WASB-C3 warning:
'the pretrained badminton weights are academic-data-trained ? provenance NOT cleared for
commercial sharing. They are NOT baked into this image; stage them at runtime ... point
$WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner serving.' Lines 53-56 set
WASB_WEIGHTS to a runtime path and the Dockerfile COPY steps never copy weights in. External
check: the nttcom/WASB-SBDT repo is MIT for CODE, but there is no explicit license/terms
statement on the pretrained weights themselves, so the 'uncleared commercial-sharing provenance'
concern is well-founded. Confirmed."
        },
        {
          "claim": "Cloud-serving free-tier data-for-reels trade (D10): train ONLY on attested
ADULT footage + DERIVED data with raw auto-delete; minors->reel-yes-training-NO; paid tier=no-
train; counsel-gated to M9 - the data-leak/consent guardrail",
          "verdict": "confirmed",
          "reasoning": "README D10 (line 37) reads verbatim: 'Free-tier data-for-reels trade ...
paid tier = no-train ... train ONLY on attested ADULT footage (minor footage -> reel yes,
training pool NO ? DPDP/GDPR need verifiable parental consent); train on DERIVED data + auto-
delete raw; ToS counsel-reviewed; live training-on-uploads gated to M9. Truly-local needs no
DPA.' Every clause matches."
        },
        {
          "claim": "India's DPDP Act (enforcement phase 2026) makes consent the primary lawful
basis, requires deletion once business purpose ends, treats under-18 as children needing
verifiable parental consent, and permits cross-border transfer except to restricted countries",
```

```
          "verdict": "uncertain",
          "reasoning": "Substantive provisions confirmed: DPDP defines a child as anyone under
18 and requires verifiable parental consent (Section 9); consent + limited legitimate uses is
the lawful-basis core; storage-limitation/erasure once purpose ends is an obligation; and
Section 16 uses a negative-list (transfer allowed except to countries the government restricts;
no restricted list notified as of mid-2026). HOWEVER the 'enforcement phase 2026' framing is
imprecise: DPDP Rules 2025 were notified with a phased transition; through 2026 it is 'soft
enforcement' (guidance/warnings), and HARD enforcement of fiduciary obligations (incl. parental
consent) is widely placed at ~May 2027 (end of the 18-month window). So the legal substance is
right but '2026 enforcement' overstates the timeline. Author already marked this medium
confidence."
        },
        {
          "claim": "Twelve-factor build-once-deploy-many config-in-env is the standard pattern
for one image serving local and cloud - swap a backing service via config only, no code change",
          "verdict": "confirmed",
          "reasoning": "12factor.net/config confirms the principle directly: 'Config varies
substantially across deploys, code does not'; 'The twelve-factor app stores config in
environment variables'; env vars are 'easy to change between deploys without changing any code';
and 12factor.net/backing-services treats backing services as attached resources swappable via
config. This is precisely the build-once/deploy-many pattern the recommendation invokes for one
image serving local and cloud. (The claim's cited AWS-Fargate blog is a secondary source, but
the canonical 12factor source corroborates it.) Confirmed."
        }
      ],
      "overall_notes": "Verified against code on branch feat/gcp-ops-agent-and-dryrun-guide
(note: the git-status snapshot showed feat/serving-m9-auth, but the working checkout is a
different branch; backend/api/auth.py and the M8/M9 work are present here). git pull --ff-only
failed (the local branch tracks a ref not on the remote) so I reviewed read-only on the local
checkout ? flagging it may lag remote.\n\nVERDICT SUMMARY: 12 of 14 claims CONFIRMED; 2
UNCERTAIN (Claim 4 and Claim 13). The core engineering spine of the recommendation is sound and
well-evidenced.\n\nThe two soft spots, both of which moderately weaken the recommendation's
framing rather than its substance:\n\n1. Claim 4 ('zero app-layer auth ... the EDGE gate, not
the app, is the tenancy boundary'). This is true only in the DEFAULT-OFF state. M9 (PR #290, on
this branch) wires backend/api/auth.py into backend/main.py:294 so that, when
security.edge_auth_enabled is True, the APP itself verifies the Cf-Access JWT signature
(RS256/JWKS) and 401s on a missing/spoofed header. That is by design (D9 anti-spoof) ? but it
means the recommendation's point (5) is the more accurate statement: the production posture is
BOTH edge ingress-lock AND in-app signature verify, not 'edge, not app.' Anyone repeating Claim
4 verbatim about the enabled state would be wrong.\n\n2. Claim 13 ('India's DPDP Act
(enforcement phase 2026)'). The substantive provisions (under-18 = child needing verifiable
parental consent; consent-centric lawful basis; erasure on purpose-end; negative-list cross-
border) are all correct. But 'enforcement phase 2026' overstates timing: 2026 is soft
enforcement/transition; hard enforcement of fiduciary obligations is widely placed at ~May 2027.
This does not change the recommendation's conclusion (counsel-gate D10/M9 before any public
cloud-serving) ? if anything it gives slightly more runway ? but the date label is
imprecise.\n\nMinor precision notes (do not change verdicts): (a) Claim 1's cited line range
42-49 is loose ? the actual switch is __init__.py:47-49. (b) Claim 10's '$34/mo' is
teardown.sh's own illustrative figure for a 200GB pd-ssd disk, whereas create_gpu_vm.sh actually
provisions 120GB pd-balanced, so the real per-VM disk bill for this script differs; the always-
DELETE cost rule is nonetheless exactly as claimed. (c) Claim 7's '~5s GPU-ready' is the
GPU/driver-ready time only; full workload cold-start (container+model load) is longer, which the
design already accepts under D7.\n\nRelevant files verified (absolute):\n-
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/compute/__init__.py (claim 1)\n-
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/compute/cloud_run.py (claim 2)\n-
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/worker/__main__.py (claims 1,2 ?
worker is a package, NOT backend/worker.py as a source implied)\n-
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/presets.py (claim 3)\n-
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py:51,294,735 (claims 4,5)\n-
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/api/auth.py (claims 4,5,6 ? exists
on this branch; the earlier note that it didn't was a stale snapshot)\n-
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/billing.py +
backend/infrastructure/database.py:292-336,624-660 (claim 11)\n-
C:/Users/avidu/Projects/badminton-highlight-indexer/deploy/cloudrun/Dockerfile:14-16,53-56
(claims 2,12)\n- C:/Users/avidu/Projects/badminton-highlight-indexer/deploy/gcp/create_gpu_vm.sh
```

+ teardown.sh (claim 10)\n- C:/Users/avidu/Projects/badminton-highlight-indexer/docs/COMPUTE_DECOUPLED_SERVING/README.md (D9 line 36, D10 line 37, M8 line 123, M9 line 124) (claims 5,12,11)\n\nOne source-URL correction: docs.cloud.google.com/run/docs/configuring/jobs/gpu is the valid Cloud Run JOBS GPU doc and corroborates the ~5s figure; the project uses Cloud Run Jobs (per D16), so the jobs doc ? not the services doc ? is the right citation."
    }
  }
]

=== CRITIC ===
{
  "gaps": [
    "APP-DATA HOME + DATA MIGRATION is named as an open question but the plan never RESOLVES it, and it is the single most load-bearing gap. ALL mutable state is CWD-relative (load_dotenv() at backend/main.py:10 takes no path; config.json via DEFAULT_CONFIG_PATH=Path('config.json') at loader.py:32; DB at output.output_dir; static mount at main.py:71-72 'backend/api/static'). doctor.py writes config.json to REPO_ROOT (scripts/doctor.py:54) while the server reads it from CWD ? these DISAGREE the moment the launcher's CWD != REPO_ROOT. No plan exists for (a) picking %APPDATA%/~/.local/share root, (b) THREADING it through DEFAULT_CONFIG_PATH + load_dotenv path + OutputConfig.output_dir + the static mount, or (c) MIGRATING an existing CWD-rooted install into it on upgrade. Without this, every distribution channel ships a footgun where launching from the OS menu silently creates an empty config/DB and the user 'loses' their index.",
    "DB SCHEMA MIGRATION ON UPGRADE is unaddressed. database.py has a real PRAGMA user_version ladder (currently at version 6, migrations at lines 183/210/222/242/290/337). A downloadable app that auto-updates will open an OLDER user's DB with NEWER code ? the plan never covers forward-migration safety, backup-before-migrate, or the DOWNGRADE case (user installs v2 app, downgrades to v1 binary, opens a user_version=7 DB). No 'export/backup my data before upgrade' story exists. This is table stakes for any shippable app and is entirely missing.",
    "SECRETS HANDLING for the Gemini key is hand-waved. GET /api/config (main.py:144-146) returns the ENTIRE global_config object unauthenticated with NO field redaction ? if any key/credential field ever lands in AppConfig it leaks to any localhost process/browser tab. The plan repeatedly says 'key goes in env/.env, never config' but never specifies WHERE the wizard writes it (a plaintext .env in the app-data dir? OS keychain?), how it is protected at rest, or how it is scrubbed on uninstall. 'Store locally' (launcher-ux research) is not a secrets design.",
    "UNINSTALL / CLEANUP is never mentioned anywhere. A batteries-included installer that drops a managed Miniconda/uv env, GCS-fetched WASB weights (hundreds of MB), an LGPL ffmpeg binary, a SQLite DB, output reels, and a plaintext Gemini key has NO uninstall story ? what gets removed, what user data is preserved, whether the secret is scrubbed. Inno Setup/winget need an explicit uninstaller manifest + a 'keep my videos/reels?' prompt.",
    "TELEMETRY / APP-LEVEL OBSERVABILITY for the distributed binary is absent. The repo has per-video emit_indexer_report and the M8 cost ledger, but there is NO crash-reporting, NO 'phone home on failure' (or an explicit decision NOT to, with the privacy rationale), and NO way for a non-technical user to send a diagnostic bundle. The owner's own 'launch quality bar' memo demands first-class observability across every NEW code path (installer/bootstrap/torch-fetch/weight-fetch) ? none of that new code is exercised by the current offline CI, so failures in the field are invisible.",
    "WINDOWS-SPECIFIC HAZARDS are barely touched despite the target persona being a non-tech India coach on Windows. Missing: (a) MAX_PATH/long-path failures when the managed conda env + nested torch/CUDA dirs blow past 260 chars under %LOCALAPPDATA%; (b) CRLF/encoding when the wizard writes config.json/.env; (c) Windows Defender/SmartScreen quarantining an unsigned .exe or PyInstaller onefile (AV false-positives are mentioned only in passing for onefile); (d) the documented port-8000-busy case has a free-port fallback only in the tray-launcher slice, not the bare `indexer --server` path; (e) spaces/non-ASCII in Windows usernames breaking conda/uv paths.",
    "AUTO-UPDATE mechanics are asserted ('update like Ollama: run a new installer over the old') but never designed: how the running tray app detects a new version, whether it stops the server cleanly before replacing files, version-pinning the GCS WASB weights against the new engine, and (critically) reconciling auto-update with the DB-migration gap above. 'Run installer over old' silently assumes the data dir survives ? which contradicts the still-unresolved CWD-vs-app-home question.",
    "FIRST-RUN WIZARD is described as living in the SPA across multiple research topics but the engine has NONE of the endpoints it needs. GET /api/capabilities (the wizard's compute-step source) DOES NOT EXIST ? confirmed by grep: main.py exposes only /api/config, /api/health,

/api/process, /api/jobs, /api/query, /api/compile, /api/highlights, uploads. The wizard plan is built on three net-new endpoints (/api/capabilities, /api/videos, /api/reels) that are unbuilt and owner-gated. The plan treats the wizard as 'config wiring, not net-new' ? that is FALSE for the capabilities endpoint it depends on.",
    "SPA<->ENGINE VERSION SYNC has no mechanism. khelsutra/web/package.json version is '0.0.0' and the client hardcodes const BASE='/api' (client.ts:7) with NO API-version header, NO /api/version handshake, and NO contract test between the bundled SPA and the engine it ships with. The plan flags 'version skew' as a risk but proposes no concrete versioned-contract test, no engine-advertised API version, and no build-time pin recorded in the wheel (dist/ is gitignored, so there is NO auditable record of which SPA shipped in a given engine build).",
    "TRAINING GATING / LONG-RUNNING JOB UX is unsolved for a web app. worker train is an OFFLINE CLI; the in-process FastAPI job worker is a ThreadPoolExecutor whose jobs DO NOT survive a restart (main.py:646 fail_stale_jobs sweeps them to 'failed'). A multi-hour GPU training run cannot run as one of these ephemeral in-process jobs, yet the plan never specifies how training is launched from the web UI, how progress is streamed, how it is cancelled, or how it survives the tray app being quit. The honesty fence (gate_verdict hardcodes ship=False at harness.py:177 ? confirmed) must also be surfaced in-UI, which has no design.",
    "ANNOTATION ('rally annotation if needed', an explicit GOAL) has no in-app path. The only annotator is the external VLC 3.0.x Lua plugin in a sibling repo. The plan acknowledges this but does NOT decide between bundling VLC (heavyweight, version-pinned to 3.0.x) vs building the 'HTML5 <video>' in-browser annotator ? leaving a stated product goal with no implementation path and no owner decision."
  ],
  "missing_topics": [
    "Concurrency / single-instance enforcement: nothing prevents two launches (tray double-click + a stale `indexer --server`) from opening the SAME SQLite DB. WAL is on (database.py:41) but the in-process executor + fail_stale_jobs sweep (main.py:646) assume ONE writer ? a second process would sweep the first's live jobs to 'failed'. No PID/lockfile or single-instance guard is planned.",
    "Disk-space / quota management for a long-lived desktop install: output reels, scratch fetches of bucket inputs, and cached weights accumulate. The repo HAS cache-hygiene tooling (tools/cleanup_caches.py, disk guard per the memory index) but the packaging plan never wires it into the app (no UI surface, no auto-purge default, no first-run disk-space check before a GCS weight download).",
    "Weight integrity + supply-chain: the recommendation says 'verify a pinned SHA-256' for the WASB download but no plan exists for WHERE the canonical hash is stored, what happens on mismatch (refuse vs warn), or signing. Combined with WASB-C3 (provenance UNRESOLVED, ship-blocking ? Dockerfile lines 14-16 + harness.py:177 hardcode ship=False), auto-fetching these weights into a PUBLIC download likely VIOLATES the MIT/Apache/BSD-only guardrail until C3 is cleared ? the plan never resolves whether the public app may fetch them at all.",
    "scipy is an UNDECLARED serving dependency (transitive via supervision only) ? flagged in the verification section but never tracked as a concrete pre-packaging task; a freeze that prunes unreferenced transitives or a supervision bump silently breaks inference. Must become an explicit pyproject dep before any freeze/bundle.",
    "Built-artifact CI lane: today's 5 CI lanes test SOURCE on Python 3.13 + CPU torch; NO lane builds and smoke-tests the frozen/bundled artifact (hidden imports, data files like backend/assets/fonts, namespace-package resolution since backend has no __init__.py). Packaging regressions are invisible until an owner manually runs the binary. No plan to add this lane or to exclude bootstrap code from the 70% coverage floor.",
    "Default-segmenter contradiction with offline-first: README config defaults default_segmenter to gemini, but a zero-config local download must default to the truly-local preset (skip_ai_handoff=True). The plan never specifies the SHIPPED config.json / DEPLOYMENT_PROFILE default, so the out-of-box experience could surprise-bill users on Gemini or 0-rally on WASB-without-key.",
    "CORS posture in the packaged single-origin app: main.py:64-67 defaults allow_origins=['*'] with credentials off. For a localhost appliance this is mostly moot, but a DNS-rebinding attack against 127.0.0.1:8000 + the unauthenticated GET /api/config (full config dump) is a real local-exploit vector the plan never addresses (no Host-header check, no localhost-origin allowlist by default).",
    "Conda/Anaconda ToS is NOT yet clean in the shipped reference image: deploy/cloudrun/Dockerfile uses Miniconda3 + `conda tos accept ... pkgs/main`/`pkgs/r` (lines ~33-35), exactly the Anaconda-Distribution-ToS exposure the plan flags. The recommended micromamba+conda-forge fix is NOT implemented anywhere yet and isn't tracked as a blocking task for the WASB env in any artifact.",
    "Time-to-first-result / model-download UX on Indian bandwidth: a multi-hundred-MB weight

fetch + torch wheel (CUDA wheels are ~2-2.5GB) on first run needs resumable downloads, a CDN/mirror decision, and a clear progress UI ? mentioned in passing but with no concrete resumability/mirror plan or a size budget per tier.",
    "Localization quality gate: kn/hi catalogs are AI-DRAFTED with native-speaker review PENDING (flagged in the digest). Shipping the trilingual UI to real non-test users before review is a trust risk with no gating decision in the packaging plan.",
    "Backup/restore of the user's index + labelled corpus: a BYO-training app accumulates irreplaceable human labels (rally CSVs) and a trained-model corpus. No export/import/backup feature is planned, and the MD5 video<->label join means re-encoding breaks it ? a data-loss landmine with no safeguard."
  ],
  "unverified_or_refuted": [
    "REFUTED ? 'no Dockerfile exists' (still asserted in khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:49,96,135,140 as '[verified]'): a Cloud Run GPU Dockerfile DOES exist at deploy/cloudrun/Dockerfile (shipped in #287/M6). The prior-art-deepdive research already caught this, but the live appliance DESIGN DOC still encodes the stale claim in four places including its DoD ? recommendations built on 'feasibility-check a worker image before promising Cloud Run' are arguing a settled point.",
    "WEAKENED ? Nuitka is described as 'Apache-2.0' in the frozen-binaries fit narrative but the refuted note correctly states current Nuitka 4.x is AGPL-3.0+ (with a binary runtime exception). Any recommendation text that lists Nuitka as 'Apache-2.0 (compatible)' (it appears that way in the verification-section implications) is wrong and must be corrected to AGPL-3.0+/with-exception before it informs a licensing decision.",
    "UNVERIFIED ? 'uv pip install --torch-backend=auto into the tool's venv' is presented as the spine of the lead distribution channel, but it is also stated that --torch-backend=auto does NOT work in uv tool install/uvx (only uv pip / project mode). The proposed installer script reaches into the tool's venv to run uv pip ? this cross-mode hack is asserted to work but is not verified against a real uv version, and uv's tool-env layout is not a stable public contract. High risk the lead channel's central mechanic doesn't work as written.",
    "UNVERIFIED ? 'WASB-SBDT is MIT, so weights are license-clean' is repeatedly used to justify shipping/fetching wasb_badminton_best.pth.tar, but the CODE-LEVEL truth (Dockerfile:14-16, harness.py:175) is that the CHECKPOINT provenance is C3-UNRESOLVED (academic-data-trained) and ship is hardcoded False. The MIT CODE license does NOT clear the WEIGHTS ? conflating them weakens every 'fetch weights at first run, license-clean' recommendation. Do not let MIT-code stand in for weight provenance.",
    "UNVERIFIED ? 'imageio-ffmpeg lacks ffprobe and is a GPL build' is stated as fact to reject it; plausible but not grounded in a file/source in this pass. Since it drives the ffmpeg-delivery recommendation, it should be verified before being treated as decisive.",
    "UNVERIFIED ? the claim that the truly-local native-WASB config 'that produced 22 rallies on an L4' is reproducible is contradicted by the noted prior cold-start config-drift that yielded 0 rallies. The exact reproducible config/command is called a 'missing piece' yet a 'native' packaged profile is proposed to bake it in. Shipping that profile on an unverified config risks a 0-rally out-of-box failure.",
    "WEAKENED ? 'pystray is the tray lib' is recommended while simultaneously noting pystray is LGPL-3.0, which trips the MIT/Apache/BSD-only guardrail. The recommendation hedges ('resolve the license question first') but still sequences a slice around pystray; an LGPL dependency should be rejected up front per the non-negotiable guardrail, not provisionally adopted."
  ],
  "contradictions": [
    "DISTRIBUTION CHANNEL vs DUAL-ENV REALITY: the distribution-channels topic leads with 'uv tool install' as the primary channel, but the dual-python-env and frozen-binaries topics both conclude NO single-venv channel can host both torch ABIs (engine torch 2.6+cu124 vs WASB torch 1.11+cu113). The lead channel therefore only ever delivers the LIGHT tier; the plan presents uv-first as if it solves distribution when it explicitly cannot deliver the headline WASB-quality CUJ.",
    "D10 'SELF-HOSTED WEBSERVER, NOT A DESKTOP APP' vs the LAUNCHER+INSTALLER+SYSTEM-TRAY recommendation: launcher-ux argues a tray app 'satisfies D10's intent', but a Start-Menu shortcut + tray icon + auto-open-browser + Inno Setup installer IS a desktop application by any normal-user definition. The plan relitigates a superseded decision (DESKTOP_APP_PLAN) under a euphemism instead of getting an explicit owner re-ratification.",
    "BIND ADDRESS: the containerization topic CORRECTLY states the server must bind 0.0.0.0 inside a container (confirmed: main.py:756 hardcodes uvicorn.run(host='127.0.0.1')), while the cloud-one-click and completed-CUJs topics treat the 127.0.0.1 bind as a SECURITY INVARIANT that must be preserved. These are not reconciled ? the same hardcoded line is simultaneously 'a required change' and 'a must-keep default,' with no config knob proposed to make bind address a

setting gated by the edge-auth profile.",
    "'BATTERIES-INCLUDED' vs CLOUD-SERVING SELF-CONTAINMENT: the config topic states cloud-serving in a downloaded app is NOT self-contained (needs the user's own GCP project + a deployed Cloud Run image + ADC), yet the cloud-one-click topic markets a 'push my workload to cloud' toggle and D13 'your machine or our cloud, switch anytime.' Either there is a hosted Khelsutra service (no StorageBackend/ComputeBackend client exists for it today) or the cloud half is BYO-GCP ? the plan markets (a) while only (b) is built.",
    "WEIGHTS LICENSE-CLEAN vs SHIP-BLOCKED: weights-and-ffmpeg-bundling recommends auto-fetching WASB weights at first run as 'license-clean (MIT)', while verification and the prior-art topic both state WASB-C3 provenance is UNRESOLVED and gate_verdict hardcodes ship=False for any non-owner serving. A PUBLIC download that auto-fetches those weights directly contradicts the non-negotiable weights-licensing guardrail; the plan cannot both 'fetch at first run' and 'respect C3' until C3 is cleared.",
    "CPU-DEFAULT vs PRESERVE-EVERY-CUJ: multiple topics recommend shipping a CPU/Gemini default and gating WASB behind an opt-in heavy tier, but the stated GOAL is 'preserve every already-completed end-to-end CUJ' ? and the verified best-quality CUJ (CUJ-6/7) IS native WASB on GPU. A CPU-only default bundle SILENTLY DROPS a completed CUJ; the plan does not square 'preserve every CUJ' with 'default has no WASB.'"
  ],
  "followups": [
    "DECIDE THE APP-DATA ROOT NOW and write the one-PR change: introduce a resolved app-home (env override + OS default), and thread it through loader.DEFAULT_CONFIG_PATH, the load_dotenv(path=...) call at main.py:10, OutputConfig.output_dir, the db path, and the static mount at main.py:71-72. Unify doctor.py's REPO_ROOT config write (scripts/doctor.py:54) with the server's read so --setup and the server agree. This is the prerequisite for uninstall, auto-update, and migration ? sequence it first.",
    "Make the bind address and CORS origin profile-driven config (not hardcoded): replace uvicorn.run(host='127.0.0.1') at main.py:756 with a config value defaulting to 127.0.0.1, only widening to 0.0.0.0 when an edge-auth profile is active, and add a Host-header / localhost-origin guard so the unauthenticated GET /api/config dump (main.py:144) is not reachable via DNS-rebinding.",
    "AUDIT GET /api/config for secret leakage and add field redaction NOW, before any packaging ? it returns the entire AppConfig unauthenticated. Add a test asserting no credential-shaped field is ever serialized, so a future config field can't silently leak the Gemini key.",
    "Resolve WASB-C3 (or formally scope it OUT of v1): get counsel sign-off on whether the checkpoint may be redistributed/auto-fetched into a PUBLIC build. Until resolved, the only honest v1 is LIGHT-tier (Gemini/motion/stub) ? make that the explicit, recorded v1 scope and ship the WASB path only to the owner. gate_verdict (harness.py:177) and the Dockerfile note already encode ship=False; the packaging plan must inherit that fence.",
    "Build the three net-new endpoints the wizard/console depend on as a single tracked engine slice BEFORE the packaging push: GET /api/capabilities (honest hardware/segmenter probe), GET /api/videos (list, today only single-row get_video), GET /api/reels/{video_id} (reload-survival). Confirmed absent in main.py. Add an /api/version handshake in the same slice and a contract test pinning the bundled SPA against it.",
    "Replace Miniconda + `conda tos accept` in deploy/cloudrun/Dockerfile (lines ~33-35) with micromamba + conda-forge for the WASB env, and record the decision in an ADR; this is the license-clean path for BOTH the cloud image and any local managed env, and it's currently unimplemented.",
    "Add scipy as an explicit [project.dependencies] entry in pyproject.toml before any freeze/bundle (currently transitive via supervision only) and add an automated license-scan CI lane (pip-licenses/reuse) given the guardrail is human-only today ? a single PR closes both latent gaps.",
    "Add a built-artifact smoke CI lane: build the chosen artifact (uv tool install / onedir freeze / container), then run `indexer --help` and boot the server + hit /api/health, so packaging regressions (hidden imports, namespace-package resolution with no backend/__init__.py, missing data files) are caught in CI rather than by the owner manually.",
    "Define the DB migration + backup contract for upgrades: on app start, back up the SQLite file before running the user_version ladder (database.py), refuse to open a DB whose user_version exceeds the binary's known max (downgrade guard), and add an explicit 'export my data' affordance. Pair this with the auto-update design so 'run installer over old' is provably safe.",
    "Get an explicit owner re-ratification of D10 vs the tray-launcher recommendation in writing, and resolve the pystray LGPL question by choosing an MIT/BSD tray library (or stdlib webbrowser-only) up front rather than provisionally ? the guardrail forbids LGPL deps, full stop.",

```
    "Write a single 'PACKAGING_PLAN.md' tracked project doc (per the repo's tracked-project-doc
convention) that fixes the v1 SCOPE decisions: which tier is v1 (recommend LIGHT/Gemini),
shipped default profile/config.json, whether training + annotation are in v1 (recommend deferred
given ship=False + VLC dependency), and the explicit out-of-scope list ? so the contradictions
above are decided once, not relitigated per channel."
  ]
}

=== CONTEXT ===

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer  (Python, FastAPI +
SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing),
C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-
collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included"
Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-
from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies ->
highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION
if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed
end-to-end CUJ.

KEY FACTS the orchestrator already verified:
- pyproject.toml: hatchling build, namespace package (no backend/__init__.py),
packages=["backend","training"], console script: indexer = backend.main:main. requires-python
>=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-
url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT),
reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai
is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker
{infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter
{gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda,
torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps.
Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile +
compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-
serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry);
backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job);
backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi).
The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.
- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be
MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW
packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a
"self-hosted webserver + UI the user spins up" (NOT a desktop app).
khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.
Always ground claims in file:line or a doc path. Read the actual files; do not assume.
```

2. assistant (2026-06-21T14:08:26.567Z)

```
I'll ground the key claims in the actual files before synthesizing, since the plan hinges on
specific file:line facts and several flagged contradictions.
```

3. user (2026-06-21T14:08:29.146Z)

```
feat/serving-m12-gdrive-api
7835f34 feat(gcp): always-install Ops Agent on GPU VMs + a cloud-GPU dry-run guide (#291)
```

4. assistant (2026-06-21T14:08:32.849Z)

Branch differs from the snapshot (now on feat/serving-m12-gdrive-api). Let me verify the load-bearing file:line facts the plan depends on, since the engine ships PRs between sessions.

5. user (2026-06-21T14:08:36.125Z)

```
Already up to date.
---ROUTES---
74:@app.get("/", response_class=HTMLResponse)
144:@app.get("/api/config")
148:@app.get("/api/health")
281:@app.post("/api/process", status_code=202)
332:@app.get("/api/jobs/{job_id}/cost")
347:@app.get("/api/videos/{video_id}/cost")
358:@app.get("/api/jobs/{job_id}")
391:@app.get("/api/highlights/{video_id}/{filename}")
415:@app.post("/api/query")
426:@app.post("/api/compile")
```

6. user (2026-06-21T14:08:43.287Z)

```
===config endpoint===
@app.get("/api/config")
def get_config():
    return global_config

@app.get("/api/health")
def health():
    """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
    (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
    confirm the engine is reachable and which detector is wired. Never raises."""
    sport = getattr(global_config, "sport", None)
    indexing = getattr(global_config, "indexing", None)
    return {
        "status": "ok",
        "sport": sport,
        "default_segmenter": getattr(indexing, "default_segmenter", None),
    }

===uvicorn bind / load_dotenv / static===
backend/main.py:8:from dotenv import load_dotenv
backend/main.py:10:load_dotenv()
backend/main.py:58:# CORS for the web UI. allow_origins='*' with allow_credentials=True is
rejected by browsers per the
backend/main.py:64:    allow_origins=_cors_origins_from_env(),
backend/main.py:71:os.makedirs("backend/api/static", exist_ok=True)
backend/main.py:72:app.mount("/static", StaticFiles(directory="backend/api/static"),
name="static")
backend/main.py:76:    index_path = "backend/api/static/index.html"
backend/main.py:80:    return "<h3>Sports Highlight Indexer Server is Running. Please create
index.html in backend/api/static/</h3>"
backend/main.py:756:        uvicorn.run(app, host="127.0.0.1", port=args.port)
backend/config/loader.py:30:DEFAULT_CONFIG_PATH = Path("config.json")
backend/config/loader.py:112:    cfg_path = Path(path) if path is not None else
DEFAULT_CONFIG_PATH
backend/config/loader.py:202:    cfg_path = Path(path) if path is not None else
DEFAULT_CONFIG_PATH
===native wasb python/weights===
backend/pipeline/detectors/native_wasb_runner.py:17:    (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` /
``WASB_PYTHON`` / ``WASB_SCRATCH``), then
backend/pipeline/detectors/native_wasb_runner.py:68:    python_bin: str = "python"             #
the `wasb` conda env python (invoked by path, no activate)
backend/pipeline/detectors/native_wasb_runner.py:69:    weights_path: str = ""                 #
REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
backend/pipeline/detectors/native_wasb_runner.py:89:            python_bin=env("WASB_PYTHON") or
w.get("python_bin", "python"),
```

```
backend/pipeline/detectors/native_wasb_runner.py:90:            weights_path=env("WASB_WEIGHTS")
or w.get("weights_path", "") or "",
backend/pipeline/detectors/native_wasb_runner.py:120:            res =
self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
backend/pipeline/detectors/native_wasb_runner.py:160:            return False, ("WASB weights
path is empty ? set WASB_WEIGHTS (or indexing.wasb.weights_path).")
backend/pipeline/detectors/native_wasb_runner.py:173:            res =
self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
backend/pipeline/detectors/native_wasb_runner.py:175:            return False, f"python binary
not found: {self.cfg.python_bin!r} (set WASB_PYTHON)."
backend/pipeline/detectors/native_wasb_runner.py:180:            return False,
f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or out).strip()}"
backend/pipeline/detectors/native_wasb_runner.py:215:        argv = [self.cfg.python_bin,
"wasb_infer.py",
backend/pipeline/detectors/native_wasb_runner.py:239:            logger.error("python binary not
found: %r (set WASB_PYTHON).", self.cfg.python_bin)
backend/pipeline/stitcher/compiler.py:253:            "-c:v", "libx264",
backend/utils/video.py:26:        reencode: If True, re-encodes with libx264 (slower but frame-
accurate).
backend/utils/video.py:42:        cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf",
"23", "-c:a", "aac"]
```

7. assistant (2026-06-21T14:08:47.117Z)

```
Important corrections to the digest. `GET /api/config` returns `global_config` directly
(confirmed leak vector), CORS now uses `_cors_origins_from_env()` (not hardcoded `*`), bind is
hardcoded `127.0.0.1` at line 756. Let me verify a few more contested facts: the gate_verdict,
DB migration ladder version, presets defaults, and whether auth.py is wired.
```

8. user (2026-06-21T14:08:52.038Z)

```
===gate ship===
1:"""Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
17:  (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
143:    """Pre-flight ship-gate.
151:    ship=True only when all non-C3 blocks clear (honesty first).
174:    reasons.append("ship-gate F1@tIoU target NOT YET CALIBRATED ? provisional; see issue
#172")
175:    reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
regardless of F1")
176:    # ship remains False until C3 resolved + other gates (by design).
177:    return {"ship": False, "floor_passed": floor_passed, "sign_test": sign, "blockers":
reasons}
===db version===
69:        ``user_version`` stays un-bumped ? and the next open would re-run the same
88:            # Versioned migrations live here. PRAGMA user_version is the schema
91:            version = cursor.execute("PRAGMA user_version").fetchone()[0]
94:                self._migrate_to_v1(cursor)
97:                self._migrate_to_v2(cursor)
100:                self._migrate_to_v3(cursor)
103:                self._migrate_to_v4(cursor)
106:                self._migrate_to_v5(cursor)
109:                self._migrate_to_v6(cursor)
110:            # future: if version < 7: cursor.execute("ALTER TABLE ..."); PRAGMA user_version
= 7
155:    def _migrate_to_v1(self, cursor):
183:        cursor.execute("PRAGMA user_version = 1")
185:    def _migrate_to_v2(self, cursor):
210:        cursor.execute("PRAGMA user_version = 2")
212:    def _migrate_to_v3(self, cursor):
222:        cursor.execute("PRAGMA user_version = 3")
224:    def _migrate_to_v4(self, cursor):
242:        cursor.execute("PRAGMA user_version = 4")
244:    def _migrate_to_v5(self, cursor):
```

```
280:            CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
===presets defaults===
backend/config/presets.py:4:config knobs, so the truly-local / cloud-serving / cpu-mock modes
don't drift apart one
backend/config/presets.py:13:(``config.json`` ``deployment.profile`` or the
``DEPLOYMENT_PROFILE`` env var); with no
backend/config/presets.py:27:PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-
mock")
backend/config/presets.py:33:    "truly-local": "local_gpu",
backend/config/presets.py:40:# ``indexing.{detector_impl,skip_ai_handoff}`` and
``storage.backend``. Later milestones extend
backend/config/presets.py:44:    "truly-local": {
backend/config/presets.py:45:        "deployment": {"profile": "truly-local", "compute_target":
"local_gpu"},
backend/config/presets.py:48:        "indexing": {"skip_ai_handoff": True},
backend/config/presets.py:54:        "indexing": {"detector_impl": "native", "skip_ai_handoff":
False},
backend/config/presets.py:60:        "indexing": {"detector_impl": "stub", "skip_ai_handoff":
True},
===auth wired===
43:from backend.api import auth as edge_auth  # M9: Cloudflare Access edge-auth (default-OFF)
150:    """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
290:    # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local single-
user,
291:    # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
294:        owner_id = edge_auth.resolve_tenant(request.headers, global_config)
295:    except edge_auth.EdgeAuthError as e:
296:        raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
568:    here ? the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted
===cors fn===
def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
    """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-separated),
    default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
    does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to its
origin."""
    src = os.environ if env is None else env
    raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
    origins = [o.strip() for o in raw.split(",") if o.strip()]
    return origins or ["*"]
```

CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by brow fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so credenti off. The origin list is configurable via the env var above (default ["*"]) so a hosted depl lock it to its origin; the middleware is fixed at startup.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=_cors_origins_from_env(),
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
===scipy declared===
===cloudrun conda===
11:#   * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as a
subprocess
23:    CONDA_DIR=/opt/conda \
33:RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
35:    && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
```

```
https://repo.anaconda.com/pkgs/main || true \
36:    && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37:    && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
52:# The native runner finds the WASB env + repo by env (no `conda activate` needed); weights
come in at
54:ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
```

9. assistant (2026-06-21T14:08:56.971Z)

```
Confirmed: gate ship=False hardcoded (harness.py:177), DB at user_version=6, auth.py IS wired at
main.py:294 (default-OFF), CORS env-configurable defaulting `["*"]`, scipy undeclared,
Miniconda+`conda tos accept` still in Dockerfile. Now let me verify the worker train job model,
capabilities endpoint absence, and the SPA dist + version facts.
```

10. user (2026-06-21T14:09:02.251Z)

```
===capabilities/videos/reels endpoints===
backend/main.py:347:@app.get("/api/videos/{video_id}/cost")
===job worker model===
backend/api/job_worker.py:21:from concurrent.futures import ThreadPoolExecutor
backend/api/job_worker.py:26:# Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1
on purpose: a single
backend/api/job_worker.py:30:_job_executor: Optional[ThreadPoolExecutor] = None
backend/api/job_worker.py:33:def _get_executor() -> ThreadPoolExecutor:
backend/api/job_worker.py:37:        _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
backend/main.py:664:    swept = db_instance.fail_stale_jobs()
backend/main.py:666:        logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous
run ? failed.")
===scipy import in serving===
backend/features/rally_seq.py
backend/eval/rally_seq_proto.py
backend/eval/tolerance_metrics.py
backend/features/rally_seq.py
===SPA===
assets
index.html
7:const BASE = "/api";
33:  const url = `${BASE}${path}`;
===doctor config write===
7:# root (requirements.txt, config.json). Resolve the root from this file's
9:REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
22:    requirements_path = os.path.join(REPO_ROOT, "requirements.txt")
45:    """Initialize a default config.json if it doesn't exist.
51:        from backend.config import save_default_config
57:    config_path = os.path.join(REPO_ROOT, "config.json")
63:        saved_path = save_default_config(config_path)
64:        print(f"[SUCCESS] config.json initialized with default settings at {saved_path}.")
66:        print(f"[ERROR] Failed to create config.json via loader: {e}")
```

11. assistant (2026-06-21T14:09:08.739Z)

```
All key facts confirmed: `/api/capabilities`, `/api/videos` (list), `/api/reels` are absent; job
worker is ThreadPoolExecutor(max_workers=1) with fail_stale_jobs sweep at main.py:664; scipy
used in serving (rally_seq.py) but undeclared; SPA dist exists with BASE="/api"; doctor.py
writes config.json to REPO_ROOT (line 57) while server reads CWD `Path("config.json")`
(loader.py:30) ? the disagreement is real. I have enough verified ground to synthesize.
```

12. user (2026-06-21T14:11:47.815Z)

```
Structured output provided successfully
```

13. assistant (2026-06-21T14:11:59.949Z)

I've delivered the structured packaging plan. Summary of what I verified against the live code (branch `feat/serving-m12-gdrive-api`, up to date) and how it shaped the plan:

Corrections to the digest, confirmed in code:
- `GET /api/config` returns `global_config` directly (backend/main.py:144-146) ? a real unauthenticated full-config leak vector ? made P0 redaction a blocking prerequisite.
- CORS is no longer hardcoded `*`; it's `_cors_origins_from_env()` defaulting to `["*"]` (M9). Bind is hardcoded `host="127.0.0.1"` at main.py:756 ? bind+CORS must become config/profile-driven.
- `auth.py` IS wired (default-OFF) at main.py:294, contradicting "zero app auth" in the enabled state.
- `gate_verdict` hardcodes `ship=False` (harness.py:177) ? WASB-C3 unresolved ? weights are owner-only until counsel clears.
- DB is at `user_version=6` with a real migration ladder ? upgrade needs backup + downgrade-guard.
- scipy used in serving (backend/features/rally_seq.py) but undeclared ? P0 task.
- `/api/capabilities`, `/api/videos` (list), `/api/reels` confirmed ABSENT ? the wizard depends on net-new endpoints (P1).
- doctor.py writes config.json to REPO_ROOT (:57) while server reads CWD `Path("config.json")` (loader.py:30) ? the app-data-home unification is the load-bearing first fix.
- deploy/cloudrun/Dockerfile still uses Miniconda + `conda tos accept` (lines 33-37) ? micromamba migration tracked.

The plan resolves the critic's biggest gaps (app-data home, secrets, uninstall, DB migration, single-instance, built-artifact CI) as concrete P0?P2 work, decides every flagged contradiction once (v1 = LIGHT tier; CUJ-6/7 owner-only; bind config-gated; weights owner-only until C3), and keeps every increment small, default-OFF, and reversible per the project's working style.